```
#!/bin/sh
####!/bin/bash   ### switched so it's also valid in busybox (ash) shell
#   script header with basic functions
# Author: joe brendler 19 Jul 2014
#         24 June 2025 - legacy header v1.0.0 moved to /usr/sbin/ and consolidated as scr
ipt_header_joetoo v0.1.0
#         10-28 January 2026 - major update, reorganization in v0.2.0

toc() # table of contents (this fn just calls
{ # call summarize_header
summarize_header; printf "\n"
j_msg -5 "That's all I have to say about that"
}
# @usage source /usr/sbin/script_header_joetoo
# @deps package dependencies are managed by the dev-util/script_header_joetoo ebuild
# @req linux shell
# @note all functions in script_header_joetoo are designed to work in any POSIX shell
# @note other script header modules are available as USE flag selections in the dev-util/
script_header_joetoo package
#-----[ metadata tagging key ]-------------------------------------------------
# @note
# @note ${BGon}'summarize_header()'${Boff} extracts metadata from files like script_heade
r_joetoo
# @rule the '@xxx' in lines starting '# @xxx' is a metadata tag
# @rule the function declaration itself, e.g. POSIX_func_name(), is a top-level tag
# @rule func_name() is followed by a comment providing a top-level description
# @rule a section of tags like these below follows the fn's closing brace }
# @rule metadata lines should be concise and less than 80 columns if possible
# @cont if more than one line must be used, it can be continued w @cont
# @rule as a minimum, every function should have a @usage tag
# @rule most functions should also have arg adn vars tags
# @rule tags should generally appear in this order:
# @cont usage opts args ret dep var rule note (with cont used where needed)
# @rule @usage describes syntax, options, and arguments for a function
# @rule @opts define positional parameters serving as command options
# @rule @args define positional parameters serving as arguments
# @rule @vars names external global variables required or modified by the function
# @rule @ret identifies value(s) returned and/or emitted
# @rule @deps lists function, script, or package dependencies
# @rule @req specifies prerequisites or state requirements
# @rule @rule outlines logic constraints or mandatory behaviors
# @rule @note provides administrative or general notes for user or maintainer
# @rule @warn provides administrative warning for user or maintainer
# @rule @ex provides a usage example
# @rule @cont identifies a continuation of the previous metadata tag

# @note =====[ Section 0: Global Instructions ]==========================================
=
# @note
# @note use set -u to provide "nounset" protection for the parent script/shell and all su
bsequent functions
# ensure these variables have a bound state
# (else bash will complain after set -u is active when sourcing this header in your login
 shell)
#: "${PROMPT_DIRTRIM:=}"
#: "${PROMPT_COMMAND:=}"
#: "${BASH_COMPLETION:=}"
#: "${BASH_COMPLETION_VERSINFO:=}"
#set -u
# as of 20260128 - not quite ready for this yet

# @note =====[ Section 1: Global Variables  ]============================================
=
# @vars SCRIPT_ENV: script paths and environment state (logFile, verbosity, TRUE/FALSE)
# @vars MSG_STRINGS: standardized error messages (E_NOTROOT, E_BAD_ARGS) and UI tokens (n
o_msg)
# @vars REGEX_POSIX: portable BRE patterns (W0/W1 whitespace, P0/P1 printable, C0/C1 cont
rol)
# @vars ANSI_SGR: "precooked" SGR styling (RED, GRN, BOLD) and CSI control initiator
# @vars DEC_GRAPHICS: DEC Special Graphics codes (H_wall, V_wall) and set toggles (SO/SI)
```

```
# @note Variables use CAPITALIZED names; ANSI sequences are binary "precooked" for printf
 '%s' compatibility

BUILD="0.0.0 (19620207)"  #redefine in base script
ROOT_UID=0       # Only users with $UID 0 have root privileges
script_header_installed_dir="/usr/sbin"
#script_header_installed_dir=/home/joe/myUtilities/dev-util/script_header_joetoo
#script_header_installed_dir=/home/joe/myUtilities/dev-util/script_header_joetoo/testing

non_interactive_header="${script_header_installed_dir%/}/script_header_joetoo_noninteract
ive"

# define default logFile (used by all logging functions)
#   this is a global variable calling scripts SHOULD define themselves,
#   so this is really just a safeguard - not a mandate on where to log
# note: using id -u here to test if the user is root, b/c this is the top
#   of the header where no functions are defined yet, but once defined,
#   isroot can do the same sort of thing
if [ "$(id -u)" -eq 0 ]; then
  logFile=/var/log/script_header_joetoo.log
else
  # Fallback to home dir or /tmp if user sourcing header is not root
  logFile="${HOME:-/tmp}/script_header_joetoo.log"
fi

# use id -un to assign a default value for the global $user variable
#   this is a global variable calling scripts SHOULD define themselves,
#   so this is really just a safeguard - not a mandate on where to log
user=$(id -un)

#  Error message used by various scripts
E_NOTROOT="Must be root to run this script"
E_ROOT="Please run this script as user, not root"
E_BAD_ARGS="Improperly formatted command line argument"
E_BAD_OPT="Invalid option"
E_BAD_COLOR="Invalid color argument"

# pseudo-boolean values for most shells which have none
TRUE=0    # will evaluate to be logically true in a boolean operation
FALSE=""  # will evaluate to be logically false in a boolean operation
  # play with test function:
  # test() { [ $1 ] && echo "$1 is true" || echo "$1 is false"; }
# in comparison, exit status code must be tested with math, not logic
  # play with test function:
  # result=1; [ "$result" ] && echo true || echo false  {status 1 == failure; but this wi
ll return "true"}
  # to test an exit status, use [ "$result" -eq 0 ]

# no_msg is a space overwritten by a backspace, used handle_result to send null message w
ith a non-null string
no_msg=" \b"

#-----[ defined logging/display severity/verbosity levels ]----------
# keyword=val Severity  Description
emerg=0   #   Emergency System is unusable. Complete failure or kernel panic.
alert=1   #   Alert     Immediate action required. E.g., corrupted database or loss of pr
imary connection.
crit=2    #   Critical  Critical conditions. Hardware failure or primary application fail
ure.
err=3     #   Error     Error conditions. Application errors or non-critical service inte
rruptions.
warning=4 #   Warning   Warning conditions. Potential issues like low disk space or high
CPU usage.
notice=5  #   Notice    Normal but significant. Significant events like a successful syst
em startup/shutdown.
info=6    #   Info.     Informational messages. Normal operations like user logins or sys
tem status.
debug=7   #   Debug     Debug-level messages. Detailed information typically only used du
ring troubleshooting.
```

```
# @note keyword=val Severity  Description
# @note emerg=0     Emergency System is unusable. Complete failure or kernel panic.
# @note alert=1     Alert     Immediate action required. E.g., corrupted database or loss
 of primary connection.
# @note crit=2      Critical  Critical conditions. Hardware failure or primary applicatio
n failure.
# @note err=3       Error     Error conditions. Application errors or non-critical servic
e interruptions.
# @note warning=4   Warning   Warning conditions. Potential issues like low disk space or
 high CPU usage.
# @note notice=5    Notice    Normal but significant. Significant events like a successfu
l system startup/shutdown.
# @note info=6      Info.     Informational messages. Normal operations like user logins
or system status.
# @note debug=7     Debbug    Debug-level messages. Detailed information typically only u
sed during troubleshooting.

# check set null vs unset and assign default verbosity and loglevel
loglevel="${loglevel:-"${notice}"}"
#  but only if not yet assigned by calling program
verbosity="${verbosity:-"${notice}"}"  # Set default (2) if verbosity is UNSET or NULL
# synchronized deprecation strategy for "$VERBOSE" (redundant to any verbosity -gt 0)
# 20260126 - transitioned to computed $VERBOSE retained only for compatibility
# if legacy VERBOSE is *unset, initialize it from verbosity
if [ -z "${VERBOSE+set}" ];
then  # VERBOSE is unset - initialize IAW verbosity
  if [ "$verbosity" -eq 0 ]; then VERBOSE=$FALSE; else VERBOSE=$TRUE; fi
else  # VERBOSE is set - override if inconsistent IAW verbosity
  case "$VERBOSE" in
    $TRUE  )
      if [ "$verbosity" -eq 0 ]; then
        j_msg -3 "Inconsistent value for VERBOSE (resetting IAW verbosity)"
        VERBOSE=$FALSE
      fi
      ;;
    $FALSE )
      if [ "$verbosity" -gt 0 ]; then
        j_msg -3 "Inconsistent value for VERBOSE (resetting IAW verbosity)"
        VERBOSE=$TRUE
      fi
      ;;
    *      )
      j_msg -3 "invalid value for VERBOSE (resetting IAW verbosity)";
      if [ "$verbosity" -eq 0 ]; then VERBOSE=$FALSE; else VERBOSE=$TRUE; fi
      ;;
  esac
fi

# moved this here so it can be used below (and in app_configure when called by sourcing s
cripts)
is_utf8_terminal()  # (POSIX) returnn 0 if terminal can print utf-8, 1 otherwise
{ # UTF-8 bytes for U+2713 CHECK MARK, expressed as octal escapes
  # but written in a way editors won't reinterpret
  _u=$(printf '\342\234\223')
  [ "$_u" = "$(printf '\342\234\223')" ]
}

# determine if the user's terminal supports utf-8
# and ensure EMOJI_CAPABLE is defined accordingly
EMOJI_CAPABLE="$FALSE"  # this is set automatically accordint to system capability
is_utf8_terminal && EMOJI_CAPABLE="$TRUE"
EMOJI_DISABLE="$FALSE"  # this can be set manually by a user .conf, to opt out of emojis

#-----[ initialize ascii control characters for UI ]-------------------------------------

_init_ascii_controls() # (POSIX) precook the set of emoji considered most "active" for op
timizatoin
{ iac_ret=0; SED_LOT_FILTER=""
  # initialize global ASCII controls
  BEL=$(printf '\007')  # h:0x07, d:07;  only when you literally need bells and whistles
```

```
  BS=$(printf  '\010')  # h:0x08, d:08;  oves cursor one column left; does not erase
  HT=$(printf  '\011')  # h:0x09, d:09;  equiv \t (horizontal tab)
  LF=$(printf  '\012')  # h:0x0A, d:10;  (linefeed)
  VT=$(printf  '\013')  # h:0x0B, d:11   (vertical tab; rarely used)
  FF=$(printf  '\014')  # h:0x0C, d:12;  equiv \f (form feed; page break in printers)
  CR=$(printf  '\015')  # h:0x0D, d:13;  equiv \r (carriage return; moves cursor to colum
n 0)
  ESC=$(printf '\033')  # h:01Bx, d:27;  see CSI, ESCon, ESG0 (introduces ANSI/VT control
 sequences)
  US=$(printf  '\037')  # h:0x1F, d:31;  Unit Separator (an ASCII "file separator"; ideal
 delimiter -never appears in normal text)
  DEL=$(printf '\177')  # h:0x7F, d:127; delete (destructive delete on paper tape; now a
control character)
  EOF=$(printf '\032')  # h:0x1A, d:26;  End of File (aka SUB, CTRL-Z; but note: the byte
 itself is not an EOF character in Unix)
  SO=$(printf  '\016')  # h:0x0E, d:14;  Shift Out (switch to G1 character set (e.g., lin
e-drawing)
  SI=$(printf  '\017')  # h:0x0F, d:15;  Shift In (switch back to G0 character set)

  SED_LOG_FILTER="
    s/${ESC}\[[0-9;]*[mK]//g;
    s/[${SO}${SI}${BEL}${ESC}]//g;
    s/[${BS}${HT}${LF}${VT}${FF}${CR}${DEL}]/ /g;
  "
  return $iac_ret
}
_init_ascii_controls

#-----[ standard multiples of byte sizes ]---------------------------
KiB=1024
MiB=$(( KiB * 1024 ))
GiB=$(( MiB * 1024 ))
TiB=$(( GiB * 1024 ))
PiB=$(( TiB * 1024 ))

KB=1000
MB=$(( KB * 1000 ))
GB=$(( MB * 1000 ))
TB=$(( GB * 1000 ))
PB=$(( TB * 1000 ))
#-----[ standard time intervals ]-----------------------------------
SEC=1
MIN=$(( 60 * SEC ))
HOUR=$(( 60 * MIN ))
DAY=$(( 24 * HOUR ))
WEEK=$(( 7 * DAY ))

#-----[ regex character class patterns ]-----------------------------
W0="[[:space:]]*"     # POSIX regex match for zero+ whitespaces
W1="[[:space:]]${W0}" # POSIX regex match for one+ whitespaces ([[:space:]]+ is not porta
ble)
P0="[[:print:]]*"     # POSIX regex match for zero+ printable characters (incl. whitespac
e)
P1="[[:print:]]${P0}" # POSIX regex match for one+ printable characters (incl. whitespace
)(+ not portable)
G0="[[:graph:]]*"     # POSIX regex match for zero+ printable characters (excl. whitespac
e)
G1="[[:graph:]]${G0}" # POSIX regex match for one+ printable characters (excl. whitespace
)(+ not portable)
A0="[[:alnum:]]*"     # POSIX regex match for one+ alphanumeric characters
A1="[[:alnum:]]${A0}" # POSIX regex match for one+ alphanumeric characters(+ not portable
)
C0="[[:cntrl:]]*"     # POSIX regex match for zero+ control characters
C1="[[:cntrl:]]${C0}" # POSIX regex match for one+ control characters
F1='^[_[:alpha:]][_[:alnum:]]*'   # regex match for one+ characters allowed in POSIX func
tion name
BF1='^[_[:alpha:]][[:alnum:]_-]*' # regex match for one+ characters allowed in Bash funct
ion name
# The definitions above conform to the "lowest common denominator"
# Basic Regular Expression (BRE) standards rather than using
```

```
# Extended Regular Expressions(ERE), to make them more portable
# (e.g. will work with any sed, because default is BRE and would
#  require use of an extra -E flag to work if we defined to a ERE standard)

#-----[ raw hex Unicode/Emojis ]-------------------------------------------------------
-----
# source the 2026-0303-refactored script_header_joetoo_unicode
. /usr/sbin/script_header_joetoo_unicode

#-----[ Global Active Emoji Registry ]----------------------------------------
# format: "index:name:hexvalue:log_translation" (Space or Newline separated)
# index (1-31) maps to the bitmask field [10-14] extracted in emit
# name is used for the bin_<name> variables and log filtering
# hexvalue is the asset by referneced by the source
#   codepoint of the variable by the same name
# log_translation is an "ascii-art" or "ascii-moji"
# IMPORTANT: this assignment must FOLLOW the raw hex definitions sourced above,
#   and it must PRECEDE the emit_utf8_from_hex and _precook_ui_assets function definition
s below

# start with zero-width unicode characters
ACTIVE_ZERO_WIDTH_REGISTRY="
1${US}VS16${US}$VS16${US}[VS16]
2${US}VS15${US}$VS15${US}[VS15]
3${US}ZWJ${US}$ZWJ${US}[ZWJ]
4${US}ZWNJ${US}$ZWNJ${US}[ZWNJ]
5${US}ZWS${US}$ZWS${US}[ZWS]
6${US}WJ${US}$WJ${US}[WJ]
7${US}SHY${US}$SHY${US}[SHY]
8${US}CGJ${US}$CGJ${US}[CGJ]
9${US}LRO${US}$LRO${US}[LRO]
10${US}RLO${US}$RLO${US}[RLO]
11${US}LRE${US}$LRE${US}[LRE]
12${US}RLE${US}$RLE${US}[RLE]
13${US}PDF${US}$PDF${US}[PDF]
14${US}LRI${US}$LRI${US}[LRI]
15${US}RLI${US}$RLI${US}[RLI]
16${US}FSI${US}$FSI${US}[FSI]
17${US}PDI${US}$PDI${US}[PDI]
18${US}FITZ_1_2${US}$FITZ_1_2${US}[SKIN1]
19${US}FITZ_3${US}$FITZ_3${US}[SKIN2]
20${US}FITZ_4${US}$FITZ_4${US}[SKIN3]
21${US}FITZ_5${US}$FITZ_5${US}[SKIN4]
22${US}FITZ_6${US}$FITZ_6${US}[SKIN5]
"

# active emoji registry
ACTIVE_EMOJI_REGISTRY="
1${US}cross_mark${US}$cross_mark${US}!!
2${US}check_mark${US}$check_mark${US}Ok
3${US}check_mark_button${US}$check_mark_button${US}Ok
4${US}no_entry${US}$no_entry${US}[NO_ENTRY]
5${US}prohibited${US}$prohibited${US}[PROHIBITED]
6${US}explosion${US}$explosion${US}[EXPLOSION]
7${US}collision${US}$collision${US}[COLLISION]
8${US}bomb${US}$bomb${US}[BOMB]
9${US}hammer${US}$hammer${US}[WORK]
10${US}hammer_and_wrench${US}$hammer_and_wrench${US}[PROCESS]
11${US}wrench${US}$wrench${US}[WRENCH]
12${US}gear${US}$gear${US}[GEAR]
13${US}toolbox${US}$toolbox${US}[TOOLBOX]
14${US}face_thinking${US}$face_thinking${US}[THINKING]
15${US}magnifying_glass_left${US}$magnifying_glass_left${US}[SEARCH]
16${US}magnifying_glass_right${US}$magnifying_glass_right${US}[SEARCH]
17${US}face_grin${US}$face_grin${US}[:D]
18${US}face_beam${US}$face_beam${US}[:D]
19${US}face_grimace${US}$face_grimace${US}[:S]
20${US}face_frown${US}$face_frown1${US}[:(]
21${US}confetti${US}$confetti${US}[SUCCESS]
22${US}trophy${US}$trophy${US}[WIN]
```

```
23${US}uSUN${US}$uSUN${US}[CLEAR]
24${US}uCLOUD${US}$uCLOUD${US}[IDLE]
25${US}uRAIN${US}$uRAIN${US}[WARN]
26${US}uTHUNDER${US}$uTHUNDER${US}[FAIL]
27${US}uSNOW${US}$uSNOW${US}[HALT]
28${US}uWIND${US}$uWIND${US}[FAST]
29${US}tree_evergreen${US}$tree_evergreen${US}[WOODS]
30${US}uBOOT${US}$uBOOT${US}[TRAIL]
31${US}uMOUNTAIN${US}$uMOUNTAIN${US}[SUMMIT]
32${US}uSUN_ALT${US}$uSUN_ALT${US}[CLEAR]
33${US}uRAIN_WET${US}$uRAIN_WET${US}[WET]
34${US}uSNOW_ICE${US}$uSNOW_ICE${US}[ICE]
35${US}uFOG${US}$uFOG${US}[FOG]
36${US}uWIND_EXP${US}$uWIND_EXP${US}[WIND]
37${US}uCAMP${US}$uCAMP${US}[SHELTER]
38${US}compass${US}$compass${US}[NAV]
39${US}uBEAR${US}$uBEAR${US}[CAUTION]
"

idx_of_emoji() # (POSIX) returns the registry index of a named emoji
{ _ioe_ret=1; _ioe_name="${1:-}"; _ioe_idx=""
  [ -z "$_ioe_name" ] && return 1
  # scan the registry for the name in the second field
  _ioe_idx=$(echo "$ACTIVE_EMOJI_REGISTRY" | while read -r _line; do
    # find _line; strip longest ${US}* to get idx
    case "$_line" in *"${US}${_ioe_name}${US}"*) printf '%s' "${_line%%${US}*}";
      exit 0 ;; # exit the subshell with success
    esac; done)
  _ioe_ret=$?
  if [ -n "$_ioe_idx" ]; then printf '%s\n' "$_ioe_idx";else _ioe_ret=1; fi
  unset -v _ioe_name _ioe_idx; return $_ioe_ret ; }
# @usage idx_of_emoji <name>
# @args 1: emoji_name (e.g. "explosion", "hammer")
# @vars ACTIVE_EMOJI_REGISTRY
# @ret 0: success (prints index), 1: not found
# @deps ACTIVE_EMOJI_REGISTRY
# @ex idx_of_emoji explosion

name_of_emoji() # (POSIX) returns the registry name of an indexed emoji
{ _noe_ret=1; _noe_idx_in="${1:-}"; _noe_name=""
  [ -z "$_noe_idx_in" ] && return 1
  # scan the registry for the index in the first field
  _noe_name=$(echo "$ACTIVE_EMOJI_REGISTRY" | while read -r _line; do
    # find _line; strip shortest *${US} to get remainder then strip longest ${US}* to get
 name
    case "$_line" in "${_noe_idx_in}${US}"*) _noe_rem="${_line#*${US}}"
        printf '%s' "${_noe_rem%%${US}*}"; exit 0 ;;   # exit the subshell with success
    esac; done)
  _noe_ret=$?
  if [ -n "$_noe_name" ]; then printf '%s\n' "$_noe_name"; else _noe_ret=1; fi
  unset -v _noe_idx_in _noe_name _noe_rem; return $_noe_ret ; }
# @usage name_of_emoji <idx>
# @args 1: emoji_idx (0-31)
# @vars ACTIVE_EMOJI_REGISTRY
# @ret 0: success (prints name), 1: not found
# @deps ACTIVE_EMOJI_REGISTRY
# @ex name_of_emoji 9

#-----[ Global Active Box UI Registry ]---------------------------------------
# format: "index:name:hexvalue:log_translation" (Space or Newline separated)
# name is used for the bin_<name> variables and log filtering
# hexvalue is the asset by referneced by the source
#   codepoint of the variable by the same name
# log_translation is an "ascii-art" equivalent
# IMPORTANT: this assignment must FOLLOW the raw hex definitions sourced above,
#   and it must PRECEDE the emit_utf8_from_hex and _precook_ui_assets function definition
s below
ACTIVE_UI_BOX_REGISTRY="
1${US}uH_light${US}$uH_light${US}--
2${US}uV_light${US}$uV_light${US}|
```

```
3${US}uUL_light${US}$uUL_light${US}+
4${US}uUR_light${US}$uUR_light${US}+
5${US}uLL_light${US}$uLL_light${US}+
6${US}uLR_light${US}$uLR_light${US}+
7${US}uT_down_light${US}$uT_down_light${US}+
8${US}uT_up_light${US}$uT_up_light${US}+
9${US}uT_right_light${US}$uT_right_light${US}+
10${US}uT_left_light${US}$uT_left_light${US}+
11${US}uCROSS_light${US}$uCROSS_light${US}+
12${US}uH_heavy${US}$uH_heavy${US}==
13${US}uV_heavy${US}$uV_heavy${US}#
14${US}uUL_heavy${US}$uUL_heavy${US}#
15${US}uUR_heavy${US}$uUR_heavy${US}#
16${US}uLL_heavy${US}$uLL_heavy${US}#
17${US}uLR_heavy${US}$uLR_heavy${US}#
18${US}uH_double${US}$uH_double${US}==
19${US}uV_double${US}$uV_double${US}#
20${US}uUL_double${US}$uUL_double${US}#
21${US}uUR_double${US}$uUR_double${US}#
22${US}uLL_double${US}$uLL_double${US}#
23${US}uLR_double${US}$uLR_double${US}#
24${US}uARC_UL${US}$uARC_UL${US}(
25${US}uARC_UR${US}$uARC_UR${US})
26${US}uARC_LL${US}$uARC_LL${US}(
27${US}uARC_LR${US}$uARC_LR${US})
28${US}uT_down_hH_vL${US}$uT_down_hH_vL${US}+
29${US}uT_up_hH_vL${US}$uT_up_hH_vL${US}+
30${US}uT_right_vH_hL${US}$uT_right_vH_hL${US}+
31${US}uT_left_vH_hL${US}$uT_left_vH_hL${US}+
"

#-----[ raw values for emit bitmask definition ]----------------------------------------
--------------------------

# @note Messaging/Logging = semantic meaning + severity + destination
# @note joetoo consolidated logging/messaging protocol uses a bitmask
# @cont to enable calling functions to provide all relevant information
# @cont in a single positional parameter $1; after it is injested and
# @cont the bits are extracted, $1 can be discarded, and the remainder
# @cont of $@ is the payload to be emitted
#
# @note single-bit flags for boolean behaviors
# @cont  (orthogonal, expressive, easy to combine)
# @note bit(s)
# @note [0]      STDOUT        - emit to stdout
# @note [1]      STDERR         - mirror (or redirect?) output to stderr
# @note [2]      LOG          - output to a file
# @note [3]      INTERPRET      - use '%b' to INTERPRET backslash escapes
# @note [4]      NEWLINE        - append a newline at the end of the message
# @note [5]      TIMESTAMP      - prepend a timestamp (on log messages)
# @note [6]      CONDITIONAL    - respect verbosity IAW severity
# @note [7]      FORCE          - force print regardless of verbosity
# @note [8]      SILENCE        - suppress, override severity, even if FORCE
# @note NOTE: FORCE and SILENCE ara part of a POLICY layer,
# @note  ( not part of the severyity/verbosity [CONDITIONAL] system;
# @note    FORCE overrides severity/verbosity [CONDITIONAL] rules,
# @note    as does SILENCE, which also overrides even FORCE)
# @note  ( Override priority: SILENCE > FORCE > CONDITIONAL > unconditional )
# @note [6-8] 8:SILENCE; 7:FORCE; 6:CONDITIONAL
# @note   000       unconditional print (no verbosity logic)
# @note   100       conditional print (respect verbosity)
# @note   010       forced print (ignore verbosity)
# @note   110       severity labeling + forced print
# @note   *01       always suppress (even if FORCE is set)
# @note [9]      EMOJI          - single-bit flat to turn on emoji prefix
# @note [10-15]  EMOJI_INDEX    - (6 bit index into ACTIVE_EMOJI_REGISTRY
# @note   see list at ACTIVE_EMOJI_REGISTRY="..."
# @note [16]     RESERVED (future)
# @note [17]     PREFIX         - single-bit flag to turn on prefix
# @note [18]     PREFIX_BOLD    - intensity
```

```
# @note [19-21]  PREFIX_COLOR   - (3 bit RGB)(21:R, 20:G, 19:B)
# @note   000 (0)   black normally, but here means severity-implied color
# @note   001 (1)   blue
# @note   010 (2)   green
# @note   011 (3)   cyan
# @note   100 (4)   red
# @note   101 (5)   magenta
# @note   110 (6)   yellow
# @note   111 (7)   white
# @note [22-23]  RESERVED (future)
# @note [24-26]  SEVERITY (3 bits)
# @note   000 (0)   emerg
# @note   001 (1)   alert
# @note   010 (2)   crit
# @note   011 (3)   err
# @note   100 (4)   warning
# @note   101 (5)   notice
# @note   110 (6)   info
# @note   111 (7)   debug
# @note [26-30]  RESERVED (future)
# @note [31]     DO NOT USE (sign bit)
#

# binary bitmask bit mapping IAW scheme above
STDOUT_OFFSET=0
STDERR_OFFSET=1
LOG_OFFSET=2
INTERPRET_OFFSET=3
NEWLINE_OFFSET=4
TIMESTAMP_OFFSET=5
CONDITIONAL_OFFSET=6
FORCE_OFFSET=7
SILENCE_OFFSET=8
PREMOJI_OFFSET=9
PREMOJI_IDX_OFFSET=10
PREFIX_OFFSET=17
PREFIX_BOLD_OFFSET=18
PREFIX_COLOR_OFFSET=19
SEVERITY_OFFSET=24

# single-bit binary values
STDOUT=$((1 << STDOUT_OFFSET))
STDERR=$((1<< STDERR_OFFSET ))
LOG=$((1<< LOG_OFFSET ))
INTERPRET=$((1<< INTERPRET_OFFSET ))
NEWLINE=$((1<< NEWLINE_OFFSET ))
TIMESTAMP=$((1<< TIMESTAMP_OFFSET ))
CONDITIONAL=$((1<< CONDITIONAL_OFFSET ))
FORCE=$((1<< FORCE_OFFSET ))
SILENCE=$((1<< SILENCE_OFFSET ))
PREMOJI=$((1<< PREMOJI_OFFSET ))
PREFIX=$((1<< PREFIX_OFFSET ))
PREFIX_BOLD=$((1<< PREFIX_BOLD_OFFSET ))

# field length and base mask values for multi-bit fields
PREFIX_COLOR_LEN=3
PREMOJI_IDX_LEN=6
SEVERITY_LEN=3
PREFIX_COLOR_BASE_MASK=$(( (1 << PREFIX_COLOR_LEN) - 1 ))
PREMOJI_IDX_BASE_MASK=$(( (1 << PREMOJI_IDX_LEN) - 1 ))
SEVERITY_BASE_MASK=$(( (1 << SEVERITY_LEN) - 1 ))

# multi-bit field (mask) binary values
PREFIX_COLOR_MASK=$(( PREFIX_COLOR_BASE_MASK << PREFIX_COLOR_OFFSET ))
PREMOJI_IDX_MASK=$(( PREMOJI_IDX_BASE_MASK << PREMOJI_IDX_OFFSET ))
SEVERITY_MASK=$(( SEVERITY_BASE_MASK << SEVERITY_OFFSET ))

# NOTE: these global constant values are used to facilitate setting of the
#   bitmask by calling functions.  This essentially makes the api
#   "self documenting" while this is integer math, it is usefull to
```

```
#   think of each of these as a single-bit binary number, 1 or 0,
#   shifted into its bit-position to attain the value above.
# Note that the assignment SILENCE=0; SILENCE=256 is equivalent
#   to SILENCE=0; SILENCE$(( SILENCE | ( 1 << 8 ) ))
#
# To avoid confusion, the names applied inside the function are prefixed
#   with '_' to inidcate their local scope. THESE local-scoped variables
#   *ARE *ACTUAL* BOOLEAN values, in which 1=true; 0=false, and as such
#   they cannot be directly tested --> [ "$CONDITIONAL" ] does NOT work,
#   it MUST be done like this [ "$CONDITIONAL" -eq 1 ]
#
# DO NOT CONFUSE these with joetoo's pseudo-boolean type only ever assigned
#   one of TRUE=0, FALSE="", so that they CAN be directly tested like
#   FLAGGED=$TRUE; [ $FLAGGED ] will always behave as expected


#-----[ globals supporting terminal io ]-----------------------------------------------
-----------------------------------------
#  these functions must be defined before the variable assignments that will depend on va
lues they "cook:
#
#   see http://en.wikipedia.org/wiki/ANSI_escape_code
#   see also http://ascii-table.com/ansi-escape-sequences.php

emit_utf8_from_hex() # (POSIX) encode a unicode codepoint (hex) into UTF-8 bytes as octal
 escapes
{ _euh_input="$1"
  # use a while loop to process each '\Uxxxx' or '\uxxxx' in the input string
  while [ ! -z "$_euh_input" ]; do
    # isolate the first hex block and the remainder
    _euh_current="${_euh_input%%\\*}" # this is likely empty if starting with \
    [ -z "$_euh_current" ] && _euh_input="${_euh_input#\\}"   # strip leading \ from left
    # get the hex digits of the leading code point
    _euh_hex="${_euh_input%%\\*}"  # strip longest match for \* from right - leaving e.g.
 U2639 now that first \ removed above
    _euh_hex="${_euh_hex#[Uu]}"    # strip shortest match for a U or u from the left - le
aving e.g. 2639
    # calculate the remainder for the next iteration
    _euh_remainder="${_euh_input#*\\}"  # strip shortest match for *\ from the left - lea
ving remainder e.g. UFE0F (VS16)
    # because above returns an unmodified string it it contains no \ (i.e. was already ha
ndled),
    #   so set it '' if that is the case, to prevent infinite loop on last codepoint
    if [ "$_euh_input" = "$_euh_remainder" ] ; then _euh_input=""; else _euh_input="\\$_e
uh_remainder"; fi
    # if _euh_input="" now, it is because we are either done or didn't have input to star
t with (return 0)
    [ -z "$_euh_hex" ] && continue # dont do 16# math with ""; skip and let loop complete
 naturally
    _euh_dec=$((16#$_euh_hex))
    # each tier shifts and masks to extract the payload bits for each UTF-8 byte
    # shift: 6 bits per continuation byte; 12 bits for 3-byte sequences; 18 bits for 4-by
te sequences
    # OR masks 0xC0 = 1100 0000; 0x80 = 1000 0000; 0xE0 = 1110 0000; 0xF0 = 1111 0000
    # AND mask: 0x3F = 0011 1111 (read the 6 least significant bits of a continuation byt
e)
    # 0x0-0x7F - single-byte (ASCII)(7bits)
    # printf '\\%03o' renders a 3digit octal number
    # 0xxxxxxx
#    if [ $_euh_dec -le 0x7F ]; then
    if [ $_euh_dec -le 127 ]; then
      printf '\\%03o' "$_euh_dec"
    # 0x80-0x7FF - two byte unicode sequence (5 + 6 = 11 bits)
    # 110xxxxx 10xxxxxx
#    elif [ $_euh_dec -le 0x7FF ]; then
    elif [ $_euh_dec -le 2047 ]; then
      # shift: 6, past LSB byte 1, OR with C0 to read 5 bit payload of byte 1 with 110 x
lead bits
      # then AND mask w 3F to read 6 LSB payload of cont. byte, with 10 led bits
      printf '\\%03o\\%03o' \
```

```
        $(( (_euh_dec >> 6)         | 0xC0 )) \
        $(( (_euh_dec & 0x3F)       | 0x80 ))
    # 0x800-0xFFFF - three byte unicode sequence (4 + 6 + 6 = 16 bits)
    # 1110xxxx 10xxxxxx 10xxxxxx
#    elif [ $_euh_dec -le 0xFFFF ]; then
    elif [ $_euh_dec -le 65535 ]; then
      # shift: 12, past LSB bytes 1 and 2, payload bits, OR with E0 to read 4 bit payload
 of byte 1 with 1110 x lead bits
      # then shift 6, past LSB byte 1, AND mask w 3F to read 6 LSB payload of cont byte 1
, with 10 led bits
      # then AND mask w 3F to read 6 LSB payload of cont byte 2, with 10 led bits
      printf '\\%03o\\%03o\\%03o' \
        $(( (_euh_dec >> 12)        | 0xE0 )) \
        $(( ((_euh_dec >> 6)&0x3F)  | 0x80 )) \
        $(( (_euh_dec & 0x3F)       | 0x80 ))
    # 0x10000-? - four byte unicode sequence (3 + 6 + 6 + 6 = 21 bits)
    # 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
    else
      # shift: 18, past LSB bytes 1-3, payload bits, OR with F0 to read 3 bit payload of
byte 1 with 11110 x lead bits
      # then shift 12, past LSB byte 1-2, AND mask w 3F to read 6 LSB payload of cont byt
e 1, with 10 led bits
      # then shift 6, past LSB byte 1, AND mask w 3F to read 6 LSB payload of cont byte 2
, with 10 led bits
      # then AND mask w 3F to read 6 LSB payload of cont byte 3, with 10 led bits
      printf '\\%03o\\%03o\\%03o\\%03o' \
        $(( (_euh_dec >> 18)        | 0xF0 )) \
        $(( ((_euh_dec >> 12)&0x3F) | 0x80 )) \
        $(( ((_euh_dec >> 6)&0x3F)  | 0x80 )) \
        $(( (_euh_dec & 0x3F)       | 0x80 ))
    fi
  done
  unset -v _euh_input _euh_current _euh_hex _euh_remainder _euh_dec
}
# @usage emit_utf8_from_hex <hex_sequence>
# @args $1: string of one or more hex codepoints (e.g., '\U2600\UFE0F' or '1F427')
# @vars _euh_input, _euh_current, _euh_hex, _euh_remainder, _euh_dec
# @ret emits one or more strings of octal escapes (e.g., \342\230\200\357\270\217) to std
out
# @rule processes input in a loop to handle multiple escaped codepoints in a single strin
g
# @rule uses shell bit-shifting arithmetic to calculate UTF-8 byte sequences
# @rule supports 1, 2, 3, and 4-byte UTF-8 encodings
# @rule unsets all localized variables starting with _euh_ before exiting
# @note use command like emoji_safe $uSUN_new | od -t x1
# @cont  => 0000000 e2 98 80 ef b8 8f
# @cont  to see bytes confirming proper output
# @cont  (noting emoji_safe calls emit_utf8_from_hex)

_precook_registry() # (POSIX) helper to precook the assets in one registry "$1"
{ pr_ret=0; _pr_REGISTRY="$1"
  # precook binary multi-byte binary unicode characters AND append to SED_LOG_FILTER for
  #   filtering log output - this is a series of substitution lines like
  #   s/$bin_emoji/$trans/g so that unicode characters in the logging stream are replaced
 by ascii-art
  # These are not index-constrained, but use the same 4-field parsing as emojis
  ### for _pr_entry in $_pr_REGISTRY; do  ## (replace with while read to protect vs word
splits
  while IFS= read -r _pr_entry; do
   _pr_bin=""  # (re)initialize
   # strip shortest match for *${US} from the left - removing index
    _pr_rem="${_pr_entry#*${US}}"
    # strip longest match for ${US}* from the right - leaving name
    _pr_name="${_pr_rem%%${US}*}"
    # strip shortest match for *${US} from the left - removing name
    _pr_rem="${_pr_rem#*${US}}"
    # strip longest match for ${US}* from the rith - leaving $name = hex
    _pr_hex="${_pr_rem%%${US}*}"
    # Note: pseudo-boolean TRUE=0; FLASE="" at global scope
    _VALID="$TRUE"; _tmp="$_pr_hex"
```

```
    while [ ! -z "$_tmp" ]; do
      case "$_tmp" in
        # at least partial valid codepoint matches literal-\ followed by u-or-U followed
by 4xhex ..
        \\[Uu][0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f]*)   # matches '\Uhhhh'
           _tmp="${_tmp#\\[Uu][0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f]}"  # strip the
 match part
           case "$_tmp" in [0-9A-Fa-f]*) _tmp="${_tmp#?}";; esac              # check/str
ip optional digit 5
           case "$_tmp" in [0-9A-Fa-f]*) _tmp="${_tmp#?}";; esac              # check/str
ip optional digit 6
           ;;
        *)
            echo "DEBUG: setting _VALID false because _tmp [$_tmp] is not valid"
            _VALID="$FALSE"
            break
            ;;
      esac
      # if got here w/o fail-out-break, then weve striped exactly one '\Uhhhhxx' (4-6 dig
it) block
    done  # multi-codepoint loop
    [ "$_VALID" ] || { pr_ret=$(( pr_ret | 1 )); continue; }  # invalid codepoint
    # strip longest match for *${US} from the left - leaving ascii-art (trans)
    _pr_trans="${_pr_entry##*${US}}"
    # capture the output of the pure-shell octal byte generator emit_utf8_from_hex
    #   engine directly into the binary variable
    # assign e.g. bin_name="bin_hammer"
    bin_name="bin_${_pr_name}"
    # assign octal byte sequences for hex input
    bin_val=$(emit_utf8_from_hex "$_pr_hex") || pr_ret=$(( pr_ret | 2 ))  # emit_utf8_fro
m_hex failure
#echo "pr DEBUG TRIPWIRE: bin_val [$bin_val]" >&2

    # assign interpreted octal bytes to the var e.g. bin_hammer=$(printf '%b' "\360\237\2
24\250")
#    printf -v "$bin_name" '%b' "$bin_val"           # I love this, but it is not POSIX
    # interpret octal escapes; store entire UTF-8 byte string in $1 (or printf %b failure
    set -- "$(printf '%b' "$bin_val")" || pr_ret=$(( pr_ret | 4 ))
    # assign only $1 (never $@) to avoid splitting or globbing, or eval failure
    eval "$bin_name=\$1" || pr_ret=$(( pr_ret | 8 ))
#echo "pr DEBUG TRIPWIRE: bin_name [$bin_name]" >&2
    # now add this emoji's rule to the precooked log filtering sed script
    eval "_pr_bin=\$bin_${_pr_name}"
#echo "pr DEBUG TRIPWIRE: _pr_bin [$_pr_bin]" >&2
    # now that both _pr_bin and _pr_trans are defined, create SED_LOG_FILTER
    #   to support emit filtering of its logging output stream
    #   (i.e. do the heavy lifting here when the registry is "cooked" upon sourcing of th
e header
    #    so the dependent functions can operate efficiently, in real time)
   if [ -z "$_pr_bin" ] || [ -z "$_pr_trans" ]; then
      # failed to make a bin_ or don't have a trans for it ...
      continue
    else
      # append rule: s/binary_emoji/text_translation/g;
      SED_LOG_FILTER="${SED_LOG_FILTER}s/$_pr_bin/$_pr_trans/g; "
    fi
    # now that _pr_bin _pr_name and _pr_hex are defined, create _PATTERN_WIDTH_1 and _PAT
TERN_WIDTH_2
    #   to support utf8_string_width and get_msg_len
    #   (i.e. do the heavy lifting here when the registry is "cooked" upon sourcing of th
e header
    #    so the dependent functions can operate efficiently, in real time)
    # use the first codepoint in the hex string to determine width
    # extract the first hex block (e.g., '\U1F300') and convert to decimal
    # (ensure the hex is clean and use arithmetic expansion for decimal conversion)
    _pr_cp_hex="${_pr_hex#\\[Uu]}"
    _pr_cp_hex="${_pr_cp_hex%%\\*}" # handle multi-codepoint strings by isolating the fir
st
    # guard: ensure _pr_cp_hex is not empty and contains only hex digits
    # ( this prevents ash from throwing a syntax error on $((16#...)) )
```

```
    case "$_pr_cp_hex" in
        [0-9A-Fa-f]*)                # double-check (_VALID above should have caught)
            _pr_cp_dec=$((16#$_pr_cp_hex))
            ;;
        *)  # fall back to a safe neutral or trigger an error
            _pr_cp_dec=0
            pr_ret=$(( pr_ret | 16 ))  # malformed codepoint
            ;;
    esac
    # translate codepoint range to character column-width
    if [ "$_pr_cp_dec" -ge 768 ] && [ "$_pr_cp_dec" -le 879 ]; then _pr_width=0
# combining marks
    elif [ "$_pr_cp_dec" -eq 65039 ] || [ "$_pr_cp_dec" -eq 8205 ]; then _pr_width=0
# VS16, ZWJ
    elif [ "$_pr_cp_dec" -ge 9728 ] && [ "$_pr_cp_dec" -le 10175 ]; then _pr_width=2
# misc symbols/dingbats
    elif [ "$_pr_cp_dec" -ge 127744 ] && [ "$_pr_cp_dec" -le 129775 ]; then _pr_width=2
# emojis/pictographs
    else
        # default for box-drawing (U+2500 range) and most others is 1
        _pr_width=1
#echo "*** pr tripwire: $_pr_name with _pr_cp_dec: [$_pr_cp_dec] fell into the else defau
lt width: [$_pr_width]" >&2

    fi
#echo "pr debug: _pr_name: [$_pr_name]" >&2
#echo "pr debug: _pr_hex: [$_pr_hex]" >&2
#echo "pr debug: _pr_cp_dec: [$_pr_cp_dec]" >&2
#echo "pr debug: _pr_width: [$_pr_width]" >&2
    # append the binary character to the corresponding pattern variable with | bookending
    # ensure a leading | and a trailing | so that *"|$_usw_char|"* always matches
    # note: these are delimiters, not logical OR pipes, for the pattern to be matched as
above
    case "$_pr_width" in
        0) _PATTERN_WIDTH_0="${_PATTERN_WIDTH_0:-|}$_pr_bin|" ;;
        1) _PATTERN_WIDTH_1="${_PATTERN_WIDTH_1:-|}$_pr_bin|" ;;
        2) _PATTERN_WIDTH_2="${_PATTERN_WIDTH_2:-|}$_pr_bin|" ;;
    esac
   # careful: POSIX heredoc must be at the left edge
  done <<EOF
$_pr_REGISTRY
EOF

  # one more precook task - pre-calculate index numbers of the emoji used to represent se
verity levels
  #   so emit() doesn't have to look them up and can stay efficient
  idx_severity_01_emoji=$(idx_of_emoji "cross_mark")            # emerg|alert
  idx_severity_23_emoji=$(idx_of_emoji "explosion")             # crit|err
  idx_severity_4_emoji=$(idx_of_emoji "no_entry")               # warn
  idx_severity_5_emoji=$(idx_of_emoji "face_thinking")          # notice
  idx_severity_6_emoji=$(idx_of_emoji "face_grin")              # info
  idx_severity_7_emoji=$(idx_of_emoji "magnifying_glass_left")  # debug

  unset -v _pr_rem _pr_name _pr_hex
  unset -v _pr_entry _pr_trans _pr_bin
  return $pr_ret
}

_precook_ui_assets() # (POSIX) precook the set of emoji considered most "active" for opti
mizatoin
{ cua_ret=0
  # Note: all ascii control precooking moved up to _init_ascii_controls
  # precook standalone UI assets (Box drawing)
  # (list contains the variables we want to 'bake' into bin_ equivalents

  # INITIALIZE/RESET global patterns to prevent doubling on re-sourcing
  _PATTERN_WIDTH_0=""
  _PATTERN_WIDTH_1=""
  _PATTERN_WIDTH_2=""
  SED_LOG_FILTER=""
```

```
  #----[ pass 0: Zero Width Registry ]----------------------------------------
  _precook_registry "$ACTIVE_ZERO_WIDTH_REGISTRY" || cua_ret=1

  #----[ pass 1: UI Box Drawing Registry ]------------------------------------
  _precook_registry "$ACTIVE_UI_BOX_REGISTRY" || cua_ret=1

  #----[ pass 2: Emoji Registry ]---------------------------------------------
  _precook_registry "$ACTIVE_EMOJI_REGISTRY" || cua_ret=1


  unset -v _cua_idx _cua_rem _cua_name _cua_hex
  unset -v _cua_entry _cua_trans _cua_octal _cua_bin
  return $cua_ret
}
# @usage _precook_ui_assets
# @vars BEL, BS, HT, LF, VT, FF, CR, ESC, US, DEL, EOF, SO, SI (global ASCII)
# @vars bin_<name>: baked binary UTF-8 characters for the active emoji set
# @ret cua_ret (0 for success, 1 for hex conversion failure)
# @deps emit_utf8_from_hex, printf
# @note precooks unicode characters to binary to optimize emit and log filtering performa
nce

# call the _precook_ui_assets function to pre-bake the set of emojis considered "active"
# (this is primarily to enable binary optimization of the real-time log-filering that emi
t has to do
#  to provide clean log output when EMOJI_CAPABLE is true and EMOJI_DISABLE is not)
_precook_ui_assets
# order matters - this call must precede the variable assignments depending on the availa
bility
# of values set by it (e.g. ESC, so CSI can be set below and then all the colors
# depending on that

# --- TRIPWIRE: Post-Cook Audit ---
#_audit_var="check_mark"
#eval "_audit_val=\$$_audit_var"
#case "$_audit_val" in
#    *[!0-9a-fA-F]*)
#        printf "!!! TRIPWIRE: %s is ALREADY binary: [%s]\n" "$_audit_var" "$_audit_val"
>&2 ;;
#    *)
#        printf "--- TRIPWIRE: %s is still hex: [%s]\n" "$_audit_var" "$_audit_val" >&2 ;
;
#esac
#unset -v _audit_var _audit_val

# order matters - this function must precede the variable assignments depending
#   on its availability (like longone= )
emoji_safe() # (POSIX) use u_echo() to display one unicode char $1
{ [ ! $# -eq 1 ] && die "one arg allowed/required"
  [ "$EMOJI_CAPABLE" ] && [ ! "$EMOJI_DISABLE" ] || return 0
  _es_hex="$1"
  _es_oct=$(emit_utf8_from_hex "$_es_hex")
  LC_CTYPE=en_US.utf8 printf '%b' "$_es_oct"
  unset -v _es_oct _es_hex
}
# @usage emoji_safe <hex_codepoint>
# @args $1: hex codepoint for a single emoji
# @vars EMOJI_CAPABLE: global flag; must be TRUE to enable output
# @vars EMOJI_DISABLE: global flag; if TRUE, suppresses output
# @ret emits binary UTF-8 character to stdout if enabled; otherwise null
# @deps emit_utf8_from_hex, printf


#-----[ ASCII Control Characters ( [:control:] )]-----------------------------
#Octal  Hex     Abbr    Description
#000    00      NUL     Null
#001 - 006      01-06   SOH-ACK Start of Heading to Acknowledge
#007    07      BEL     Bell (audible alert)
```

```
#010    08      BS      Backspace
#011    09      HT      Horizontal Tab (\t)
#012    0A      LF      Line Feed / Newline (\n)
#013    0B      VT      Vertical Tab
#014    0C      FF      Form Feed (\f)
#015    0D      CR      Carriage return (\r)
#016 - 032      0E-1A   SO-SUB  Shift Out to Substitute; note S0 & S1 ar "precooked" belo
w
#033    1B      ESC     Escape (Start of your ${CSI})
#034 - 037      1C-1F   FS-US   Separators
#177    7F      DEL     Delete

# potentially useful in scripts --
# NOTE: precooked variables for these controls is now done inside the
# _init_ascii_controls function which is defined and called above

NUL='\000'            # only for matching  (do not precook, and do not try to print)
                      # NUL cannot be stored in a shell variable as a raw byte

# (POSIX) CSI - define the control sequence initiator: ascii "\e[", octal "\033[", or hex
 "\x1b["
# which is used for Parameterized Control (e.g., cursor positioning, setting colors, clea
ring screen)
# The sequence is always followed by numbers and a final letter (e.g., \e[31m for red tex
t)
# LEGACY: assigning CSI='\033[' stores the characters of the "Literal Escape Sequence" in
 a string
#   requires an interpreter (echo -e or printf %b) to convert the '\' '0' '3' '3'
#   strings into a real control character
# UPGRADED: precooking the ESC above and assigning CSI="${ESC}[") stores the "Raw ANSI Co
ntrol Sequence"
# The subshell executes and emits the actual non-printing Escape Byte (Hex 1B)
# This is "precooked" -- it works in printf '%s' because the byte is already binary
# Note: using octal \033 rather than hex \x1b because octal remains the gold standard
#   for portable shell scripts... specifically, POSIX explicitly requires support for
#   octal escapes (\ddd). It does not require support for hexadecimal escapes (\x),
#   Many strictly POSIX shells (like dash or certain minimalist versions of ash) will
#   treat \x1b as literal text, and while some specific BusyBox or Alpine Linux versions
#   of ash might have expanded printf to include \x, this is a non-standard extension;
#   relying on it (\x1b) makes the script less portable
CSI="${ESC}["

#-----[ Select Graphics Rendition on/off ]-----------------------------------------
BOLD="1"          # bold on
UL="4"            # underline on
BLINK="5"         # slow blink on
BLINKFAST="6"     # fast blink on
REVERSE="7"       # image: inverse (reverse video)
ULoff="24"        # underline off
BLINKoff="25"     # blink off
SGRoff="0"        # Bold off (reset all SGR (e.g. blink, underline)
#-----[ Set Text Color, Foreground ]-----------------------------------------------
BLACK="30"        # foreground black
RED="31"          # foreground red
GREEN="32"        # foreground green
YELLOW="33"       # foreground yellow
BLUE="34"         # foreground blue
MAG="35"          # foreground magenta (it's like fucia)
CYAN="36"         # foreground cyan
WHITE="37"        # foreground white
FG_DEFAULT="39"   # foreground to terminal default
#FG_SPEC="38"     # foreground to specified (e.g., 2 for RGB or 5 for a 256-color palette)
# For example, ${CSI}38;2;255;165;${SGRoff}m sets the foreground to orange
# NOT USED HERE. used in RGB_FG() in script_header_joetoo_extended
#-----[ Set Background Color ]-----------------------------------------------------
BACKoff="40"      # background black
RBACK="41"        # background red
GBACK="42"        # background green
YBACK="43"        # background yellow
BBACK="44"        # background blue
```

```
MBACK="45"       # background magenta
CBACK="46"       # background cyan
WBACK="47"       # background white
BG_DEFAULT="49"  # foreground to terminal default
#BG_SPECB="38"   # background to specified (e.g., 2 for RGB or 5 for a 256-color palette)
# For example, ${CSI}48;2;0;0;${SGRoff}m sets the background to black
# NOT USED HERE. used in RGB_BG() in script_header_joetoo_extended

#-----[ My Favorite Colors (terminate with ${Boff} ]---------------------------------
Ron="${CSI}${RED}m"
Gon="${CSI}${GREEN}m"
Yon="${CSI}${YELLOW}m"
Bon="${CSI}${BLUE}m"
Mon="${CSI}${MAG}m"
Con="${CSI}${CYAN}m"
Won="${CSI}${WHITE}m"
BRon="${CSI}${RED};${BOLD}m"
BGon="${CSI}${GREEN};${BOLD}m"
BYon="${CSI}${YELLOW};${BOLD}m"
BBon="${CSI}${BLUE};${BOLD}m"
BMon="${CSI}${MAG};${BOLD}m"
BCon="${CSI}${CYAN};${BOLD}m"
BWon="${CSI}${WHITE};${BOLD}m"
RVon="${CSI}${REVERSE}m"
Boff="${CSI}${SGRoff}m"          # Bold off (reset all SGR (e.g. blink, underline)
#-----[ Here setting background as well ]---------------------------------------------
BBonY="${CSI}${BLUE};${YBACK};${BOLD}m"

#-----[ for "localized" use ]---------------------------------------------------------
# when differentiation from above is needed, such as the "command preview"
# technique used in demonstrate_header
_BRon="${BRon}" ; _BGon="${BGon}" ; _BYon="${BYon}" ; _BBon="${BBon}"
_BMon="${BMon}" ; _BCon="${BCon}" ; _BWon="${BWon}" ; _Boff="${Boff}"
_Ron="${Ron}" ; _Gon="${Gon}" ; _Yon="${Yon}" ; _Bon="${Bon}"
_Mon="${Mon}" ; _Con="${Con}" ; _Won="${Won}"
#-----[ for "theme" engines ]---------------------------------------------------------
# primary use: construct "theme engine" for the demonstration suite
# matching common regex-based syntax highlighting rules
_cmd_color="${BBon}"   # command
_var_color="${BRon}"   # variable (e.g. $var)
_quote_color="${BYon}" # quotation (e.g. "..." or '...'
_opt_color="${BMon}"   # option (e.g. -v)
_cmt_color="${Con}"    # comment
_data_color="${Mon}"   # data (e.g. 0.5)
_ctl_color="${Gon}"    # control keyword (e.g. if, while, do, case ...)
_func_color="${BGon}"  # functions (e.g. myFunction()
_op_color="${Gon}"     # operators (e.g. &&, ||, |, >)

#-----[ global 3-bit RGB color codes ]----------------------------------------------
# see emit below, which uses these to colorize message prefix symbols
#   but note that emit interprets 000 as no color, therefor apply severity level coloring
RGB_X=0   # black/none. 0 is posix equiv to 0b000 which isn't posix
RGB_B=1   # blue.    1 is posix equiv to 0b001 which isn't posix
RGB_G=2   # green.   2 is posix equiv to 0b010 which isn't posix
RGB_C=3   # cyan.    3 is posix equiv to 0b011 which isn't posix
RGB_R=4   # red.     4 is posix equiv to 0b100 which isn't posix
RGB_M=5   # magenta. 5 is posix equiv to 0b101 which isn't posix
RGB_Y=6   # yellow.  6 is posix equiv to 0b110 which isn't posix
RGB_W=7   # white.   7 is posix equiv to 0b111 which isn't posix

#-----[ globals supporting Box Drawing methods ]-------------------------------------
# See: https://en.wikipedia.org/wiki/Box-drawing_character
# Note: this is from Digital Equipment Corporation (DEC) command language (DCL) experianc
e
# I had at Nordson Corporation (Amherst, OH) Robotics EE Research department in 1982
# (though that was actual VT102 terminals connected by serial lines to a Vax-11/70 mainfr
ame)
# DEC was the pioneer that developed the VT100 terminal in the late 1970s. DCL programs f
or the VT100
# used of escape sequences (\e or ESC) to control cursor movement, colors, and special ch
```

```
aracter sets,
# and this became the industry standard. Most modern terminal emulators (like xterm, Kons
ole, or the
# console in BusyBox) still implement compatibility with these original VT series termina
ls
#
# Define the escape sequence initiator (ESCon) not to be confused with the CSI defined ab
ove
# with a Single Shift Unlock (SSU) followed by a character set selection.
# The SSU is: ascii "\e(", octal "\033(", or hex "\x1b(" ,  and appending character "0" m
akes ESCon
# a Character Set Selection that tells the terminal to switch the current font mapping
# (the G0 character set) from the standard ASCII font to the DEC Special Graphics font.
# (VT100-compatible terminals maintain four character sets, known as G0, G1, G2, and G3 -
#  G0 is the primary character set. It's what the terminal uses by default for all incomi
ng
#  characters; G1-G3 are secondary sets)
# ESCon and ESCoff are thus SSU sequences that permanently map G0 to another G-set until
# another sequence shifts it again

# ESC(C Sequences (Explicit G0 Selection)
ESCon="${ESC}(0"   # Maps G0 to DEC Special Graphics
ESCoff="${ESC}(B"  # Maps G0 to Standard ASCII
# ESC)C Sequence (Explicit G1 Selection)
ESG1="${ESC})0"    # Maps G1 to DEC Special Graphics
# Shift In/Out Control Characters (Fast G0/G1 Toggling)
# NOTE: precooked variables for SO/SI controls is now done inside the
# _init_ascii_controls function which is defined and called above
# note: migrated all of these from legacy string assignments (e.g. ESCon="${ESC}(0")
#   to subshell printf generation of the actual binary escape code bytes
#   (i.e. precooked and don't need the -e for echo or the %b vs %s for printf)

# Graphics Character Codes (Using portable octal escapes)
H_wall='\161'      # Horizontal line (0x71)
V_wall='\170'      # Vertical line (0x78)
UL_wall='\154'     # Upper-Left corner (0x6C)
UR_wall='\153'     # Upper-Right corner (0x6B)
LL_wall='\155'     # Lower-Left corner (0x6D)
LR_wall='\152'     # Lower-Right corner (0x6A)

# note - below are unicode (do not need SI/SO)
# Single-Line T-junctions & Intersections (UTF-8 Octal)
T_down='\342\224\254'       # â\224¬ (0x252C)
T_up='\342\224\264'         # â\224´ (0x2534)
T_right='\342\224\234'      # â\224\234 (0x251C)
T_left='\342\224\244'       # â\224¤ (0x2524)
CROSS='\342\224\274'        # â\224¼ (0x253C)

# Double-Line segments (UTF-8 Octal)
H_wall_dbl='\342\225\220'     # â\225\220 (0x2550)
V_wall_dbl='\342\225\221'     # â\225\221 (0x2551)
UL_wall_dbl='\342\225\224'    # â\225\224 (0x2554)
UR_wall_dbl='\342\225\227'    # â\225\227 (0x2557)
LL_wall_dbl='\342\225\232'    # â\225\232 (0x255A)
LR_wall_dbl='\342\225\235'    # â\225\235 (0x255D)

# Double-Line T-junctions & Intersections (UTF-8 Octal)
T_down_dbl='\342\225\246'     # â\225¦ (0x2566)
T_up_dbl='\342\225\251'       # â\225© (0x2569)
T_right_dbl='\342\225\240'    # â\225  (0x2560)
T_left_dbl='\342\225\243'     # â\225£ (0x2563)
CROSS_dbl='\342\225\254'      # â\225¬ (0x256C)

# Note: these should NOT be precooked like the control sequences above
#   b/c that would convert them to binary representation of letters (e.g. \161 => q)
#   and we need to presesrve the octal code so it can be interpreted as graphic
#   after the interpreter (e.g. echo -e or printf '%b') has received the
#   control sequence telling it to do so
# The Rule: Never precook DEC Graphics because sed and grep cannot
#   distinguish them from standard alphabet letters
```

```
# Note also: the unicode emojis below are NOT like that - they are represented
#   by specific byte-sequences that occupy space in the utf8 map reserved
#   exclusively for emoji - there is no standard ascii character that shares
#   these values -- every byte of every emoji has bit 8 = 1, so its value
#   is always > 128 and thus out of range for ascii



# @note =====[ Section 2: Terminal & Cursor Control Functions ]==========================
============================
color() # (POSIX) set FG $1, optional [bold $2], optional [BG $3]
{ # @usage color FG [BOLD] [BG]
  # @args $1: FG code (30-37, 39); $2: Bold (0/1); $3: BG code (40-47, 49)
  # @ret Raw ANSI escape sequence (precooked CSI...m)
  # @rule Validates inputs using case; returns E_BAD_COLOR and exit 1 on invalid codes
  # @ex color 31 1 44  # returns Red on Blue, Bold
  # @note Use $(color) with no args to reset to terminal default (equiv to Boff)
    _FG=""    # Foreground (3x)
    _BG=""    # Background (4x)
    _B=""     # Bold (0/1)
    _V_FG="${1:-39}"  # Default _FG is 39 (terminal default)
    _V_B="${2:-0}"    # Default Bold is 0 (Normal)
    _V_BG="${3:-49}"  # Default _BG is 49 (terminal default)
    # validate foreground (3x) or set default 39; set bold appropriately
    case "$_V_FG" in
        3[0-7])  # foreground is set
            _FG="$_V_FG" ;
            # validate bold (0/1)
            case "$_V_B" in
                1) _B=";1" ;;
                0) _B="" ;;
                *) echo "$E_BAD_COLOR: Bold attr ($_V_B)(0/1)"; return 1 ;;
            esac  # _B
            ;;
        39)     # unset foreground for default
            _FG="" ;
            # validate bold (0/1) - omit leading ; since no $1
            case "$_V_B" in
                0|1) _B="$_V_B" ;;
                *) echo "$E_BAD_COLOR: Bold attr ($_V_B)(0/1)"; return 1 ;;
            esac  # _B
            ;;  # allow unset args $(color) to equate to ${Boff}
        *) echo "$E_BAD_COLOR: Foreground ($_V_FG)(3x)"; return 1 ;;
    esac # _FG

    # validate background (4x) or set default (unset)
    case "$_V_BG" in
        4[0-7]) _BG=";$_V_BG" ;;
        49) _BG="" ;;  # allow unset _BG to imply terminal default
        *) echo "$E_BAD_COLOR: Background ($_V_BG)(4x)"; return 1 ;;
    esac
    # output the raw ANSI escape sequence
    printf "%s%s%s%s%s" "${CSI}" "${_FG}" "${_B}" "${_BG}" "m"
    # Clean up local variables to maintain POSIX hygiene
    unset -v _FG _BG _B _V_FG _V_B _V_BG
}

termwidth() # (POSIX plus) calculate and output the width of the terminal
{ # @usage termwidth
  # @ret Integer representing number of terminal columns
  # @rule Priority: 1. $COLUMNS env; 2. tput cols; 3. stty size; 4. Default (80)
  # prefer environment variable; if null try tput; or fall back to tput
  _cols="$COLUMNS"
  [ -z "$_cols" ] && _cols=$(tput cols 2>/dev/null)
  # if _cols is still unassigned, fall back to stty carefully
  if [ -z "$_cols" ]; then
    # if /dev/tty exists and is available, force its use, to avoid "inapprpriate io contr
ol" errors in
    #   non-interactive control structures like:
    #   "while read -r var; do ... right_status $?...; done <<<$( ... )" )
```

```
    # double-wrap the input redirection and assignment so that the outer sub-shell can st
ill redirect the
    #   inner-shell's error exit status if the input redirection fails OR if the assignme
nt fails
    _cols=$( (stty size < /dev/tty) 2>/dev/null | cut -d' ' -f2)
  fi  # _cols check
  # fall back to default (standard) 80 cols if none of above worked
  isint "$_cols" || _cols=80  # a bit more robust than printf '%s' "${_cols:-80}"
  printf '%s' "$_cols"
  unset -v _cols
}
# Maintainer note: further explanation of the need for the double-wrapper
# guarding against a number of reasons redirection might fail rather than the command --
#   1 - /dev/tty might not exist [ ENOENT (2) "No such file or directory" ]
#   2 - it might not be available [ ENXIO (6) "No such device or address" ]
# note that the shell processes redirection before it executes the command, so
#   stderr is redirected for the **command's exit status (open /dev/null for write 2>)
#   but if input redirection for the command (open /dev/stty for read <) is from /dev/tty
 that does not exist,
#   or is not available, that error is not sent to redirected stderr (it is not the resul
t of executing the command) --
#   instead, it is provided on non-redirected stderr, as the *shell's exit status
# both ENOENT and ENXIO above I have experienced in
#   dev-util/mkinitramfs's init script running in busybox
#   other possibilities this double wrap will protect against include --
#   EACCES (13) "Permission denied"
#   EBUSY  (16) "Device or resource busy"
#   ENOTTY (25) "Inappropriate ioctl for device"  # I have experienced (see above)
#   EPERM  (1)  "Operation not permitted"
#   EINTR  (4)  "Interrupted system call"

termheight() # (POSIX plus) calculate and output the height of the terminal
{ # @usage termheight
  # @ret Integer representing number of terminal lines
  # @rule Priority: 1. $LINES env; 2. tput rows; 3. stty size; 4. Default (25)
  # prefer environment variable; if null try tput; or fall back to tput
  _rows="$LINES"
  [ -z "$_rows" ] && _rows=$(tput lines 2>/dev/null)
  # if _rows is still unassigned, fall back to stty carefully
  if [ -z "$_rows" ]; then
    # if /dev/tty exists and is available, force its use, to avoid "inapprpriate io contr
ol" errors in
    #   non-interactive control structures like:
    #   "while read -r var; do ... right_status $?...; done <<<$( ... )" )
    # double-wrap the input redirection and assignment so that the outer sub-shell can st
ill redirect the
    #   inner-shell's error exit status if the input redirection fails OR if the assignme
nt fails
    _rows=$( (stty size < /dev/tty) 2>/dev/null | cut -d' ' -f1)
  fi  # _rows check
  # fall back to default (standard) 25 rows if none of above worked
  isint "$_rows" || _rows=25  # a bit more robust than printf '%s' "${_rows:-25}"
  printf '%s' "$_rows"
  unset -v _rows
}
# Maintainer note: see notes under termwidth above for further explanation of the need fo
r the double-wrapper

vt_init() # (POSIX) Initialize G0/G1 character sets for DEC SI/SO graphics
{ # @usage vt_init
  # @note maps G0 to ASCII and G1 to DEC Special Graphics
  # @cont this enables the use of SO (\016) and SI (\017) for fast toggling
  # @cont between modes, which is faster than ESCon/off
  # ( ensures standard ASCII ('B') is mapped to G0 (primary display set)
  #   maps DEC Special Graphics ('0') to G1 (secondary set)
  #   this is what prepares the terminal for the fast SI/SO switching )
  printf "%b%b" "$ESCoff" "$ESG1"
}

#---[ Simple Cursor State Control ]----------------------------------
```

```
SCP() # (ANSI) save the current cursor position
{ printf '%b' "${CSI}s" ; }
# @usage SCP
# @ret 0 on success (sequence emitted)
# @note Positions are terminal-internal; not stored in shell variables

RCP() # (ANSI) restore the cursor to the saved position
{ printf '%b' "${CSI}u" ; }
# @usage RCP
# @ret 0 on success

HCU() # (ANSI) Hide the cursor (Note: the trailing character is lowercase L)
{ printf '%b' "${CSI}?25l" ; }
# @usage HCU
# @ret 0 on success

SCU() # (ANSI) Show the cursor
{ printf '%b' "${CSI}?25h" ; }
# @usage SCU
# @ret 0 on success

CLR() # (ANSI) Clear stdout
{ printf '%b' "${CSI}2J" ; }
# @usage CLR
# @ret 0 on success

EL() # (ANSI) Erase line
{ printf '%b' "${CSI}K" ; }
# @usage EL
# @ret 0 on success

#---[ Cursor Absolute Positioning ]-----------------------------------
HVP() # (ANSI) move cursor to position row=$1, col=$2 (both default to 1 if omitted)
{ _row="${1:-1}"; _col="${2:-1}"; if isint "$_row" && isint "$_col"; then
    printf '%b' "${CSI}${_row};${_col}f" ; unset -v _row _col; return 0;
  else unset -v _row _col; return 1; fi }
# @usage HVP [row] [col]
# @args $1: Row (default 1); $2: Column (default 1)
# @ret 0 if moved; 1 if args are not integers
# @rule Validates input via isint()

CUP() # (ANSI) move cursor to position row=$1, col=$2 (both default to 1 if omitted)
{ _row="${1:-1}"; _col="${2:-1}"; if isint "$_row" && isint "$_col"; then
    printf '%b' "${CSI}${_row};${_col}H" ; unset -v _row _col; return 0;
  else unset -v _row _col; return 1; fi }
# @usage CUP [row] [col]
# @args $1: Row (default 1); $2: Column (default 1)
# @ret 0 if moved; 1 if args are not integers
# @rule Validates input via isint

#---[ Cursor Relative Movement ]-----------------------------------
CUU() # (ANSI) Move the cursor up ($1 cells)
{ _reps="${1:-1}"; if isint "$_reps"; then
    printf '%b' "${CSI}${_reps}A" ; unset -v _reps; return 0;
  else unset -v _reps; return 1; fi; }
# @usage CUU [count]
# @args $1: Number of cells to move (default 1)
# @ret 0 if moved; 1 if arg is not an integer

CUD() # (ANSI) Move the cursor down ($1 cells)
{ _reps="${1:-1}"; if isint "$_reps"; then
     printf '%b' "${CSI}${_reps}B" ; unset -v _reps; return 0;
  else unset -v _reps; return 1; fi; }
# @usage CUD [count]
# @args $1: Number of cells to move (default 1)
# @ret 0 if moved; 1 if arg is not an integer

CUF() # (ANSI) Move the cursor fwd/right ($1 cells)
{ _reps="${1:-1}"; if isint "$_reps"; then
     printf '%b' "${CSI}${_reps}C" ; unset -v _reps; return 0;
```

```
  else unset -v _reps; return 1; fi; }
# @usage CUF [count]
# @args $1: Number of cells to move (default 1)
# @ret 0 if moved; 1 if arg is not an integer

CUB() # (ANSI) Move the cursor back/left ($1 cells)
{ _reps="${1:-1}"; if isint "$_reps"; then
     printf '%b' "${CSI}${_reps}D" ; unset -v _reps; return 0;
  else unset -v _reps; return 1; fi; }
# @usage CUB [count]
# @args $1: Number of cells to move (default 1)
# @ret 0 if moved; 1 if arg is not an integer

CPR() # (POSIX) [r|c|s] - Cursor Position Report
{ cpr_ret=0; _cpr_opt="$1"; _cpr_pos=""; _cpr_old_stty=""; _cpr_QUERY="${CSI}6n"
  # save state and disable canonical mode
  # (switch to "cbreak" mode (to read without a newline)
  # (if this fails, we likely don't have a TTY (exit status 1) ; cannot continue)
  _cpr_old_stty=$(stty -g 2>/dev/null) || { cpr_ret=$?; return $cpr_ret; }
  stty -icanon -echo 2>/dev/null || cpr_ret=2
  # query terminal (CSI 6n)
  # (use in=/dev/tty to ensure it will work even if stdin is redirected)
  printf '%b' "$_cpr_QUERY" > /dev/tty 2>/dev/null || cpr_ret=$((cpr_ret + 4))
  # read response: format is "^[[row;colR"
  # (read until 'R' using 'dd' because POSIX 'read' can't change delimiters)
  # (use in=/dev/tty to ensure it will work even if stdin is redirected)
  _cpr_pos=$(dd if=/dev/tty count=1 2>/dev/null) || cpr_ret=$((cpr_ret + 8))
  # restore terminal immediately
  stty "$_cpr_old_stty" 2>/dev/null || cpr_ret=$((cpr_ret + 16))
  # strip leading ESC[ and trailing R
  _cpr_pos=${_cpr_pos#*[}; _cpr_pos=${_cpr_pos%R}
  # split row/col using positional parameters (the POSIX "array")
  _cpr_old_ifs="$IFS"; IFS=';'; set -- ${_cpr_pos:-0;0}; IFS="$_cpr_old_ifs"
  # global-ish persistence for QCR/QCC (not localized with prefix)
  CPR_ROW=$1
  CPR_COL=$2
  # handle command line options
  case "$_cpr_opt" in
    [rR]) echo "$CPR_ROW" ;;
    [cC]) echo "$CPR_COL" ;;
    [sS]) : ;;
    *)    echo "$CPR_ROW $CPR_COL" ;;
  esac

  unset -v _cpr_opt _cpr_pos _cpr_old_stty _cpr_old_ifs _cpr_QUERY
  return $cpr_ret
}
# @usage CPR [r|c|s]
# @args $1: report option: 'r' (row), 'c' (col), 's' (silent), null (both)
# @vars CPR_ROW, CPR_COL (set globally for use by QCR/QCC)
# @ret emitted: row, col, or both, based on $1
# @ret status: 0 (success) or failure bitmask: 1=init, 2=config, 4=query, 8=read, 16=rest
ore
# @deps stty, printf, dd
# @rule uses stty -icanon; terminal state is saved and restored via stty -g
# @note reliable on modern systems (Linux 2.6+ / BSD)
# @cont to be 100% robust against high-latency fragmentation, dd read
# @cont could be wrapped in a 'while' loop until 'R' is received
# @warn while CPR should be reliable in real terminals, it will likely still fail
# @cont in pipelines, cron, systemd units, non-interactive SSH commands,
# @cont while read loops fed by here-strings, subshells without a TTY, or
# @cont any context where /dev/tty is not available
# @note see "stty --help" (fascinatingly arcane detail on tty driver)

QCR() { CPR s; echo "$CPR_ROW"; }
# @usage QCR
# @vars CPR_ROW (updated via CPR s)
# @ret emitted: current cursor row index
# @deps CPR
```

```
QCC() { CPR s; echo "$CPR_COL"; }
# @usage QCC
# @vars CPR_COL (updated via CPR s)
# @ret emitted: current cursor column index
# @deps CPR

box_esca() # (POSIX) DEC box drawing usingw ESCon/off (Explicit G0 mapping)
{ # @usage box_esca "text"
  # @args $1: String to be boxed
  # @note Uses Explicit G0 mapping (slower but state-safe)
  # @note text is displayed inside a single-wall box
  be_ret=0
  _be_text="$1"
  _be_len=$(get_msg_len "${_be_text}")   # the visible length of text determines width of
 box
  _be_tw=$(termwidth)
  [ "$_be_len" -gt "$_be_tw" ] && { j_msg -3 "error: box_esca handles only one-line messa
ges, but \$1 is too big"; return 1; }
  # use fixed width printf to create proper length string of spaces | tr into wall segmen
t
  _be_h_line=$(printf "%*s" "$((_be_len + 2))" "" | tr ' ' "${H_wall}")
  # safely handle each part in a separate bucket to ensure proper interpretation (%s for
the text)
  # top line
  printf "%b%b%s%b%b\n" "$ESCon" "$UL_wall" "$_be_h_line" "$UR_wall" "$ESCoff" || be_ret=
2
  # content
  printf "%b%b %b%s%b %b%b\n" "$ESCon" "$V_wall" "$ESCoff" "$_be_text" "$ESCon" "$V_wall"
 "$ESCoff" || be_ret=3
  # bottom
  printf "%b%b%s%b%b\n" "$ESCon" "$LL_wall" "$_be_h_line" "$LR_wall" "$ESCoff" || be_ret=
4
  unset -v _be_text _be_len _be_h_line _be_tw
  return $be_ret
}

box_shift() # (POSIX) DEC box drawing using SO/SI toggling
{ # @usage box_shift "text"
  # @args $1: String to be boxed
  # @note Uses SO/SI toggling (faster)
  # @note text is displayed inside a single-wall box
  bs_ret=0
  _bs_text="$1"
  _bs_len=$(get_msg_len "${_bs_text}")   # the visible length of text determines width of
 box
  _bs_tw=$(termwidth)
  [ "$_bs_len" -gt "$_bs_tw" ] && { j_msg -3 "error: box_esca handles only one-line messa
ges, but \$1 is too big"; return 1; }
  # use fixed width printf to create proper length string of spaces | tr into wall segmen
t
  _bs_h_line=$(printf "%*s" "$((_bs_len + 2))" "" | tr ' ' "${H_wall}")
  # safely handle each part in a separate bucket to ensure proper interpretation (%s for
the text)
  # Top Line
  printf "%b%b%s%b%b\n" "$SO" "$UL_wall" "$_bs_h_line" "$UR_wall" "$SI" || bs_ret=2
  # Content
  printf "%b%b %b%s%b %b%b\n" "$SO" "$V_wall" "$SI" "$_bs_text" "$SO" "$V_wall" "$SI" ||
bs_ret=3
  # Bottom
  printf "%b%b%s%b%b\n" "$SO" "$LL_wall" "$_bs_h_line" "$LR_wall" "$SI" || bs_ret=4
  unset -v _bs_text _bs_len _bs_h_line _bs_tw
  return $bs_ret
}

u_box_msg() # (POSIX) $1=msg, $2=width, $3=center, $4=color, $5=style
{ _ubm_ret=0; _ubm_msg="$1"; _ubm_tw=0; _ubm_target_w=0; _ubm_center=0
  _ubm_color_seq=0; _ubm_buffer=0; _ubm_r_margin=0; _ubm_l_margin=0
  _ubm_h_line=0; _ubm_line=""

  _ubm_tw=$(termwidth)
```

```
  # defaults: width=tw/2, center=tw/2
  [ "$2" -gt 0 ] 2>/dev/null && _ubm_target_w=$2 || _ubm_target_w=$(( _ubm_tw / 2 ))
  [ "$3" -gt 0 ] 2>/dev/null && _ubm_center=$3   || _ubm_center=$(( _ubm_tw / 2 ))
  _ubm_color_seq="${4:-$Boff}"

  case "$5" in
    2 ) # Double Line
        _ubm_H_wall="$bin_uH_double"; _ubm_V_wall="$bin_uV_double"; _ubm_UL_wall="$bin_uU
L_double"
        _ubm_UR_wall="$bin_uUR_double"; _ubm_LL_wall="$bin_uLL_double"; _ubm_LR_wall="$bi
n_uLR_double" ;;
    3 ) # Heavy/Bold
        _ubm_H_wall="$bin_uH_heavy"; _ubm_V_wall="$bin_uV_heavy"; _ubm_UL_wall="$bin_uUL_
heavy"
        _ubm_UR_wall="$bin_uUR_heavy"; _ubm_LL_wall="$bin_uLL_heavy"; _ubm_LR_wall="$bin_
uLR_heavy" ;;
    4 ) # Rounded
        _ubm_H_wall="$bin_uH_light"; _ubm_V_wall="$bin_uV_light"; _ubm_UL_wall="$bin_uARC
_UL"
        _ubm_UR_wall="$bin_uARC_UR"; _ubm_LL_wall="$bin_uARC_LL"; _ubm_LR_wall="$bin_uARC
_LR" ;;
    * ) # Light (Default)
        _ubm_H_wall="$bin_uH_light"; _ubm_V_wall="$bin_uV_light"; _ubm_UL_wall="$bin_uUL_
light"
        _ubm_UR_wall="$bin_uUR_light"; _ubm_LL_wall="$bin_uLL_light"; _ubm_LR_wall="$bin_
uLR_light" ;;
  esac

  _ubm_buffer=4
  _ubm_l_margin=$(( _ubm_center - (_ubm_target_w / 2) ))
  _ubm_r_margin=$(( _ubm_tw - _ubm_target_w + _ubm_buffer ))
  _ubm_r_wall=$(( _ubm_l_margin + _ubm_target_w ))
  [ "$_ubm_l_margin" -lt 0 ] && _ubm_l_margin=0

  _ubm_h_line=$(full_repeat "$_ubm_H_wall" "$_ubm_target_w")

  # top border
  CUF "$_ubm_l_margin"
  printf "%b%b%b%b%b\n" "$_ubm_color_seq" "$_ubm_UL_wall" "$_ubm_h_line" "$_ubm_UR_wall"
"$Boff"

  # body - using the redirection pattern to avoid subshell issues if needed
  # though piped here is fine as we only unset local variables at the end
  smart_wrap_n 0 0 "$_ubm_r_margin" "$_ubm_msg" | \
  while read -r _ubm_line || [ -n "$_ubm_line" ]; do
      CUF "$_ubm_l_margin"
      printf "%b%b%b " "$_ubm_color_seq" "$_ubm_V_wall" "$Boff"
      printf "%b" "$_ubm_line"
      printf '\r'; CUF "${_ubm_r_wall}"
      printf " %b%b%b\n" "$_ubm_color_seq" "$_ubm_V_wall" "$Boff"
  done
  _ubm_ret=$?

  # bottom border
  CUF "$_ubm_l_margin"
  printf "%b%b%b%b%b\n" "$_ubm_color_seq" "$_ubm_LL_wall" "$_ubm_h_line" "$_ubm_LR_wall"
"$Boff"

  unset -v _ubm_msg _ubm_tw _ubm_target_w _ubm_center _ubm_color_seq
  unset -v _ubm_buffer _ubm_r_margin _ubm_l_margin _ubm_h_line _ubm_line
  unset -v _ubm_H_wall _ubm_V_wall _ubm_UL_wall _ubm_UR_wall _ubm_LL_wall _ubm_LR_wall
  return $_ubm_ret
}

box_msg() # (POSIX) $1=msg, $2=width, $3=center, $4=color, $5=style
{ bm_ret=0; _bm_msg="$1"; _bm_tw=0;  _bm_target_w=0; _bm_center=0
  _bm_color_seq=0; _bm_buffer=0; _bm_r_margin=0; _bm_l_margin=0
  _bm_h_line=0; _bm_line=""

  _bm_tw=$(termwidth);
```

```
  # defaults: width=tw/2, center=tw/2
  [ "$2" -gt 0 ] 2>/dev/null && _bm_target_w=$2 || _bm_target_w=$(( _bm_tw / 2 ))
  [ "$3" -gt 0 ] 2>/dev/null && _bm_center=$3   || _bm_center=$(( _bm_tw / 2 ))
  _bm_color_seq="${4:-$Boff}"
  case "$5" in
    2 ) _bm_H_wall="$H_wall_dbl"; _bm_V_wall="$V_wall_dbl"; _bm_UL_wall="$UL_wall_dbl";
        _bm_UR_wall="$UR_wall_dbl"; _bm_LL_wall="$LL_wall_dbl"; _bm_LR_wall="$LR_wall_dbl
";
        _bm_SO=""; _bm_SI="";;
    * ) _bm_H_wall="$H_wall"; _bm_V_wall="$V_wall"; _bm_UL_wall="$UL_wall";
        _bm_UR_wall="$UR_wall"; _bm_LL_wall="$LL_wall"; _bm_LR_wall="$LR_wall";
        _bm_SO="$SO"; _bm_SI="$SI"; vt_init ;;
  esac

  _bm_buffer=4   # safety margin to help prevent words from exploding out of the box
                   # _bm_buffer must be an even number -ge 4 (integer divisible to split
on each side)

    # calculate the "right_margin" that smart_wrap_n should use to wrap at "_bm_wrap_at"
    # if it uses 0 for left margin
    _bm_l_margin=$(( _bm_center - (_bm_target_w / 2) ))      # where we actually draw the
 left wall
    _bm_r_margin=$(( _bm_tw - _bm_target_w + _bm_buffer ))   # where we smart_wrap
    _bm_r_wall=$(( _bm_l_margin + _bm_target_w ))            # where we actually draw the
 right wall
    [ "$_bm_l_margin" -lt 0 ] && _bm_l_margin=0              # clamp to 0 in case negativ
e
    _bm_h_line=$(full_repeat "$_bm_H_wall" "$_bm_target_w")  # -2 = corners +2 = pad

    # top border
    CUF "$_bm_l_margin"
    printf "%b%b%b%b%b%b%b\n" "$_bm_color_seq" "$_bm_SO" "$_bm_UL_wall" "$_bm_h_line" "$_
bm_UR_wall" "$_bm_SI" "$Boff"
    # body - piped instead of array or here-string for POSIX compliance
    # (capture wrapped lines using smart_wrap_n as the output engine)
    # (smarw_wrap_n does not finish with a newline, so use the
    #  || [ -n in case read wont take the input without a delimiter)
    smart_wrap_n 0 0 "$_bm_r_margin" "$_bm_msg" | \
    while read -r _bm_line || [ -n "$_bm_line" ]; do
        CUF "$_bm_l_margin"
        printf "%b%b%b%b%b " "$_bm_color_seq" "$_bm_SO" "$_bm_V_wall" "$_bm_SI" "$Boff" #
 left wall
        printf "%b" "$_bm_line"                  # pre-wrapped text
        # ignore the gap, use absolute ref from left
        printf '\r'; CUF "${_bm_r_wall}"
        printf " %b%b%b%b%b\n" "$_bm_color_seq" "$_bm_SO" "$_bm_V_wall" "$_bm_SI" "$Boff"
 # Right wall
    done
    bm_ret=$?

    # bottom border
    CUF "$_bm_l_margin"
    printf "%b%b%b%b%b%b%b\n" "$_bm_color_seq" "$_bm_SO" "$_bm_LL_wall" "$_bm_h_line" "$_
bm_LR_wall" "$_bm_SI" "$Boff"

    unset -v _bm_msg _bm_tw _bm_target_w _bm_center _bm_color_seq
    unset -v _bm_buffer _bm_r_margin _bm_l_margin _bm_h_line _bm_line
    unset -v _bm_H_wall _bm_V_wall _bm_UL_wall _bm_UR_wall _bm_LL_wall _bm_LR_wall _bm_SO
 _bm_SI
    return $bm_ret
}
# @usage box_msg "$message" "$width" "$center_point" "$color_sequence"
# @vars _bm_ (prefixed local scope)
# @deps termwidth, full_repeat, CUF, smart_wrap_n
# @rule width and center_point must be numeric or null for defaults


test_colors() # (POSIX) print the joetoo RGB name of each eponymous color
{ _rgb_color_names="BLACK RED GREEN YELLOW BLUE MAG CYAN WHITE"
  _h=$(termheight); _w=$(termwidth); _current_row=$(( (_h / 2) - 4 )); _c_idx=0
```

```
  # ensure start row is within screen bounds
  if [ "$_current_row" -lt 1 ]; then _current_row=1; fi
  for _name in $_rgb_color_names; do
    _fg_code=$(( 30 + _c_idx ))
    _bg_code="${BG_DEFAULT}"
    # special case: if text is BLACK (30), set background to WHITE (47)
    if [ "$_fg_code" -eq "${BLACK}" ]; then _bg_code="${WBACK}"; fi;
    _esc_start=$(color "$_fg_code" 0 "$_bg_code")
    _colored_text="${_esc_start}${_name}${Boff}"
    # centering logic (use the raw name length)
    _col=$(( (_w / 2) - (${#_name} / 2) ))
    CUP "$_current_row" "$_col"
    printf '%b' "$_colored_text"
#    printf '%b' "$(CUP "$_current_row" "$_col") $_colored_text"
    _current_row=$(( _current_row + 1 ))
    _c_idx=$(( _c_idx + 1 ))
  done
  CUP "$_h" 1
#  printf '%b' "$(CUP "$_h" 1)"
unset -v _rgb_color_names _h _current_row _c_idx _name _fg_code
unset -v _bg_code _esc_start _esc_reset _colored_text _w _col
}
# @usage test_colors
# @note validates SGR color rendering and vertical centering logic
# @note special case: forces white background for BLACK text to ensure visibility
# @deps color, termheight, termwidth, CUP

test_terminal() # (POSIX} full-suite validation of cursor manipulation and terminal dimen
sions
{
    # clear screen and show dimensions
    CLR
    _W=$(termwidth)
    _H=$(termheight)
    # test absolute positioning (CUP) - print an X in each corner
    CUP 1 1; printf "X"
    CUP 1 "$_W"; printf "X"
    CUP "$_H" 1; printf "X"
    CUP "$_H" "$_W"; printf "X"
    # test centering
    _msg="Terminal: ${_W}x${_H}"
    _msg_len=${#_msg}
    _start_col=$(( (_W / 2) - (_msg_len / 2) )) # start message half-its-length left of c
enter
    _start_row=$(( (_H / 2) - 10 ))             # start far enough above middle to leave
room for colors
    CUP "$_start_row" "$_start_col"; SCP
    printf '%s' "_msg_len: $_msg_len"; RCP; CUD; SCP
    printf '%s\n' "$_msg"
    # run test_colors()
    test_colors
    # test save/restore cursor (SCP/RCP)
    CUP $((_start_row + 2)) "$_start_col"
    printf "Saving cursor position..."
    SCP
    # move away ...
    CUP 1 5 ; printf "[Moving]"
    # restore cursor to saved position and finish
    RCP; printf " ...Restored!"
    # test repeat by drawing a screen-wide line of "=" chars
    CUP $((_start_row + 4)) 1
    repeat "=" "$_W"
    # move cursor to bottom for prompt
    CUP $(( _H - 2 )) 1; printf "\nTest Complete. Press Enter"
    read _junk < /dev/tty
    unset -v _W _H _msg _msg_len _start_col _start_row _junk
}
# @usage test_terminal
# @note tests: CLR (clear), CUP (positioning), SCP/RCP (save/restore), and repeat
# @deps CLR, termwidth, termheight, CUP, SCP, RCP, repeat, test_colors
```

```
# @note =====[ Section 3: Core Validation Functions ]=================================
==========================

isint() # (POSIX) tests if $1 is a decimal integer
{ [ $# -eq 0 ] && return 1  # guard clause: check if an argument was provided
  _val=$1
  _test_val="${_val#-}"  # Remove leading dash (negative # ok; test the rest)
  case "$_test_val" in
    "" | *[!0-9]* ) unset -v _val _test_val; return 1 ;;
    *        ) unset -v _val _test_val; return 0 ;;
  esac ; }
# @usage isint "input"
# @args $1: the string to be validated as a decimal integer
# @ret 0 if valid integer; 1 if not
# @rule accepts leading '-' for negative values; rejects all non-digit characters
# @note mutually exclusive with isfloat_posix and ishexint in this suite
# @rule handles negative signs via parameter expansion ${1#-}
# @ex isint "-42"  # returns 0

ishexint() # (POSIX) recognizes hexadecimal integers per shell standards
{ # @usage ishexint "input"
  [ $# -eq 0 ] && return 1          # guard: return false if no argument is provided
  _val=$1;   _test_val="${_val#-}"  # parameter expansion: removes a leading dash if pres
ent
                                    # this allows the function to recognize negative hex
constants
  case "$_test_val" in
    0[xX][0-9a-fA-F]*)              # 1: Check for the required POSIX shell arithmetic pr
efix (0x or 0X)
                                    #    followed by at least one valid hex digit ([0-9a-
fA-F]*)
      _test_val="${_test_val#??}"   # +  Strip the prefix using parameter expansion
                                    #    #?? removes the first two characters (the '0x' o
r '0X')
      case "$_test_val" in          # +  Validate the remaining payload.
        "" | *[!0-9a-fA-F]*)        #    a. Reject if the payload is empty (just "0x") or
 contains non-hex characters
          unset -v _val _test_val; return 1
          ;;
        *)                          #    b. Success: the string is a valid hexadecimal co
nstant
          unset -v _val _test_val; return 0
          ;;
      esac ;;
    *)                              # 2: If it doesn't start with 0x/0X, it is not recogn
ized as hex in this scope
      unset -v _val _test_val
      return 1
      ;;
  esac
}
# @usage ishexint "input"
# @args $1: the string to be validated as a hexadecimal constant
# @ret 0 if valid hex (0x/0X); 1 if not
# @rule requires 0x or 0X prefix; handles optional leading '-'
# @note aligns with POSIX shell $((...)) arithmetic constant requirements
# @ex ishexint "0xAF12" # returns 0

isfloat_posix() # (POSIX) tests if $1 is a floating point decimal number
{ [ $# -eq 0 ] && return 1  # guard clause: check if an argument was provided
  _val=$1
  _test_val="${_val#-}"  # Remove leading dash (negative # ok; test the rest)
  # reject what we confirm in NOT float, of what is left accept only what has exactly one
 .; reject the rest
  case "$_test_val" in
    "" | "." | *[!0-9.]* ) unset -v _val _test_val; return 1 ;;    # reject if empty, jus
t a dot, or contains non-numeric/non-dot chars
    *"."*"."*            ) unset -v _val _test_val; return 1 ;;    # eject if there is mo
re than one dot
```

```
    *"."*                ) unset -v _val _test_val; return 0 ;;    # *success. must conta
in exactly one dot; no overlap with isint
    *                    ) unset -v _val _test_val; return 1 ;;
  esac
}
# @usage isfloat_posix "value"
# @args $1: value to test
# @ret 0 if valid float; 1 if not
# @rule requires exactly one decimal point; rejects scientific notation (e.g. 1e10)
# @rule fails on multiple dots or non-numeric characters
# @note aligns with bc "scale" semantics; distinct from isint
# @note does not support scientific notation (e.g. 1e10)

isnumeric() # (POSIX) tests if $1 is any valid numeric type (Dec, Hex, or Float)
{ isint "$1" || ishexint "$1" || isfloat_posix "$1" ; }
# @usage isnumeric "input"
# @args $1: the string to be tested
# @deps isint, ishexint, isfloat_posix
# @ret 0 if input matches any supported POSIX numeric type; 1 if not
# @note wrapper function for the strict POSIX validation suite

isroot() # (POSIX) silent check for root UID
{ # @usage isroot || echo "not root"
  # @ret 0 if root; 1 if not
  # @note silent boolean check; does not emit messages or exit the shell
  [ "$(id -u)" -eq "${ROOT_UID:-0}" ]
}

checkroot() # (POSIX) run as root, of course (${var:-0} means default to 0)
{ if [ "$(id -u)" -ne "${ROOT_UID:-0}" ]; then j_msg -3 "${E_NOTROOT}"; echo; j_msg -3 "e
xiting process [$$]"; exit 1; fi; }
# @usage checkroot
# @req root privileges
# @rule enforces root UID; exits process [$$] with status 1 on failure
# @ret 0 if root
# @deps j_msg -3

checknotroot() # (POSIX) run as not root, of course (${var:-0} means default to 0)
{ if [ "$(id -u)" -eq "${ROOT_UID:-0}" ]; then j_msg -3 "${E_ROOT}"; echo; j_msg -3 "exit
ing process [$$]"; exit 1; fi; }
# @usage checknotroot
# @req non-root user account
# @rule prevents execution as root; exits process [$$] with status 1 on failure
# @ret 0 if not root
# @deps j_msg -3

# ")(

checkboot() # (POSIX) check fstab for boot/efi mountpoints and verify mounted status
{ _ck_fstab="/etc/fstab"; _ck_targets="/boot /efi /boot/efi"
  _ck_i=0; _ck_status=0
  for _ck_tgt in $_ck_targets; do
    # check if target is a defined (non-commented) mount point in fstab
    if grep -v "^${W0}#" "$_ck_fstab" | grep -q "${W1}${_ck_tgt}${W1}"; then
      # defined, so next verify it is currently mounted
      if ! findmnt -nlo TARGET,SOURCE "$_ck_tgt" >/dev/null 2>&1; then
        # set the ith most significant bit of status
        _ck_status=$(( _ck_status | ( 1 << _ck_i ) ))
      fi
    fi
    _ck_i=$((_ck_i + 1))
  done
  unset -v _ck_fstab _ck_targets _ck_tgt _ck_i
  return "$_ck_status"
}
# @usage checkboot
# @ret 0: all fstab-defined boot targets are mounted
# @ret bit 1 set: /boot should mounted but is not
# @ret bit 2 set: /efi should be mounted but is not
# @ret bit 3 set: /boot/efi should be mounted but is not
```

```
# @ex return 3 (binary 11) means both /boot /efi should be mounted, but neither is
# @deps grep findmnt
# @rule aggreagaate check for /boot, /efi, and /boot/efi

checkshell() # reset colors if this shell is not interactive
{ [ -z "$PS1" ] && . ${non_interactive_header}; }
# @usage checkshell
# @req non_interactive_header variable must be defined in Section 1
# @rule sources a fallback header if PS1 is null (non-interactive shell)
# @note used to prevent ANSI escape "garbage" in cron jobs or piped output

validate_logfile()  # validate existence and ownership of logFile
{ vl_ret=0; j_msg -7 "in validate_logfile"
  _logFile_owner_uid=""; _logFile_group_gid=""
  _user_uid=""; _user_gid=""
  _msg=""
  j_msg -5n "looking for logFile [${logFile}]"
  if [ ! -f "${logFile}" ] ; then
      echo_e_n " (${BRon}not found${Boff})"
      right_status 1
      _msg="logFile [${logFile}] not found\n"
      _msg="${_msg}${BRon}Please run (as root) ${BGon}touch ${logFile}; chown ${user}:${u
ser} ${logFile}${Boff}"
      j_msg -3 "${_msg}"
      unset -v _logFile_owner_uid _logFile_group_gid _user_uid _user_gid _msg
      return 1
  else
      echo_e_n " (${BGon}found${Boff})"
      right_status 0
      j_msg -5n "checking logFile ownership"
      _logFile_owner_uid=$(ls -n "${logFile}" | awk '{print $3}')
      _logFile_group_gid=$(ls -n "${logFile}" | awk '{print $4}')
      _user_uid=$(id -u "${user}")
      _user_gid=$(id -g "${user}")
      if [ "${_logFile_owner_uid}:${_logFile_group_gid}" = "${_user_uid}:${_user_gid}" ]
; then
          echo_e_n " (${BGon}${_logFile_owner_uid}:${_logFile_group_gid}${Boff})"
          right_status 0
      else
          echo_e_n "(${BRon}${_logFile_owner_uid}:${_logFile_group_gid}${Boff})"
          right_status 1
          _msg="bad ownership on logFile [${logFile}]\n"
          _msg="${_msg}${BRon}Please run (as root) ${BGon}chown ${user}:${user} ${logFile
}${Boff}"
          j_msg -3 "${_msg}"
          vl_ret=1
      fi # ownership
  fi # existence
  unset -v _logFile_owner_uid _logFile_group_gid _user_uid _user_gid _msg
  return $vl_ret
}
# @usage validate_logfile
# @req logFile and user must be set apriori
# @ret 0: exists and owned by $user; 1: missing or bad ownership
# @deps j_msg -5 j_msg -5n echo_e_n right_status j_msg -3 message
# @vars logFile (path); user (owner name)
# @note no arrays in ash/busybox, so don't use ${FUNCNAME[0]}
# @note in ash, the =+ operator works for arithmetic but not text concatenation
# @rule identifies owner/group UIDs via ls and awk for POSIX compatibility
# @rule emits colored "corrective" instructions to stderr on failure# @req logFile and us
er must be set apriori

# @note =====[ Section 4: String & Layout Utilities ]=================================
==========================

strip_ansi() # (POSIX) Strip ANSI escape sequences from stdin
{ # Works with standard sed to remove color/UI codes for clean logging
 sed "s/${CSI}[0-9;]*[mK]//g" ; }
# @usage strip_ansi "string" OR echo "string" | strip_ansi
# @args $*: the string containing ANSI sequences
```

```
# @ret String with all \033[...m sequences removed
# @note essential for calculating true "visible" string length
# @note regex pattern in "${CSI}[0-9;]*[mK]" is std for matching nearly all SGR (Select G
raphic Rendition) parameters
# @note global CSI=$(printf "${ESC}[") is defined at the top of this header

_translate_escapes() # (POSIX) translate layout control chars to spaces, preserve ESC
{ # explicitly target "layout breakers", not the entire [:control:] class --
  # \010 = BS (backspace)
  # \011 = HT (tab)
  # \012 = LF (newline)
  # \013 = VT (vertical tab)
  # \014 = FF (form feed)
  # \015 = CR (carriage return)
  tr '\010\011\012\013\014\015' ' ' ; }
# @usage _translate_escapes
# @ret Cleaned string with layout-breakers converted to spaces
# @rule targets BS, HT, LF, VT, FF, CR (octal 010-015)
# @rule preserves ESC (octal 033) to maintain ANSI styling functionality

utf8_string_width() # (POSIX) compute display width of a UTF-8 string
{ usw_ret=0
  _usw_string=$1; _usw_str_width=0
  # iterate until string is empty
  while [ ! -z "$_usw_string" ]; do
    # tier 2 optimization - detect any "run" of ascii chars and handle them quickly
    LC_ALL=C _usw_prefix="${_usw_string%%[! -~]*}"
    if [ ${#_usw_prefix} -gt 1 ] ; then
      _usw_run_length=${#_usw_prefix}
      _usw_str_width=$(( _usw_str_width + _usw_run_length ))
      _usw_string=${_usw_string#"${_usw_prefix}"}; continue
    fi
    # read first byte (POSIX 'X idiom = 1st char [byte] of string X)
    _usw_b1=$(printf '%d' "'$_usw_string")
    # determine UTF-8 sequence byte-length of the current character, by examining the fir
st byte only
    # if these are non-ascii, then they are the actual binary bytes of which the multi-by
te
    # unicode characters consist, placed into the message with a method like
    # _msg="${_msg} $(emoji_safe $hammer)" or simpler by _msg="${_msg} $(printf '%s' $bin
_hammer)"
    # tier 1 optimization - if byte 1 is ascii, increment count and return quicky
    if   [ $_usw_b1 -lt 128 ]; then  # ascii: ( bit 7 = 0 0-127) handle quickly (incremen
t; drop; continue)
       _usw_n=1; _usw_str_width=$((_usw_str_width + 1)); _usw_string=${_usw_string#?}; co
ntinue
    elif [ $_usw_b1 -lt 224 ]; then _usw_n=2     # utf-8 ( bits 765 = 110 192-223)
    elif [ $_usw_b1 -lt 240 ]; then _usw_n=3     # utf-8 ( bits 7654 = 1110 224-239)
    elif [ $_usw_b1 -lt 248 ]; then _usw_n=4     # utf-8 ( bits 76543 = 11110 240-247)
    else _usw_n=1   # treat as single 1-col char # invalid (248-255)( there are no 5- or
6-byte UTF-8 sequences)
    fi                                           # also     bits 76 = 10 marks a continua
tion byte
                                                 #          (so, 128-191 also not allowed
 in 1st byte)
    # extract the full UTF-8 "grapheme cluster"
    # moved this _usw_char= here, to enable tier 5 optimizaton and slice/drop from string
 further below
    # %. is precision; with s it is length; so '%.*s' substitutes the first arg for *
    #   and uses the result to format the second arg in the printf output
    #   (thus if _usw_n=4 printf '%.4s' "$_usw_string" prints 4 bytes)
    #   (the fact that it is s and not b means the result is the byte sequence, NOT the c
ooked binary character)
    # start by extracting the first full character identified by _usw_n bytes above
    _usw_char=$(printf '%.*s' "$_usw_n" "$_usw_string")
    # preserve the remainder to look ahead for evidence of a continuing grapheme cluster
    _usw_rem="${_usw_string#$_usw_char}"
    # now consume the full grapheme cluster from the remainder
    # this loop eats all modifiers that "stick" to the base character visually
    while [ -n "$_usw_rem" ]; do
```

```
        case "$_usw_rem" in
            "${bin_ZWJ}"*)   # zero-width-joiner
                _usw_char="${_usw_char}${bin_ZWJ}"
                _usw_rem="${_usw_rem#$bin_ZWJ}"
                # after ZWJ, consume the next full codepoint;
                # determine character byte-length by looking at first byte (as above)
                _next_b1=$(printf '%d' "'$_usw_rem" 2>/dev/null)
                if   [ "$_next_b1" -lt 128 ]; then _next_n=1     # ascii: ( bit 7 = 0 0-1
27)
                elif [ "$_next_b1" -lt 224 ]; then _next_n=2     # utf-8 ( bits 765 = 110
 192-223)
                elif [ "$_next_b1" -lt 240 ]; then _next_n=3     # utf-8 ( bits 7654 = 11
10 224-239)
                elif [ "$_next_b1" -lt 248 ]; then _next_n=4     # utf-8 ( bits 76543 = 1
1110 240-247)
                else _next_n=1   # treat as single 1-col char # invalid (248-255)( there
are no 5- or 6-byte UTF-8 sequences)
                fi                                               # also     bits 76 = 10 mark
s a continuation byte
                                                        #          (so, 128-191 also not allowed
 in 1st byte)
                _next_char=$(printf '%.*s' "$_next_n" "$_usw_rem")
                _usw_char="${_usw_char}${_next_char}"
                _usw_rem="${_usw_rem#$_next_char}"
                ;;

            # these (next unicode character) matches are all 3-byte zero width, non-spaci
ng characters
            "${bin_VS15}"*|"${bin_VS16}"*|"${bin_ZWNJ}"*|"${bin_ZWS}"*|"${bin_WJ}"*|"${bi
n_CGJ}"*|"${bin_LRI}"*|"${bin_RLI}"*|"${bin_FSI}"*|"${bin_PDI}"*)
                _mod=$(printf '%.*s' 3 "$_usw_rem")   # print the next 3 bytes from remai
nder
                _usw_char="${_usw_char}${_mod}"       # append them to the current charac
ter to form full grapheme
                _usw_rem="${_usw_rem#$_mod}"          # strip them from remainder before
moving on
                ;;

            #  these (next unicode character) matches are all 4-byte zero width, non-spac
ing skin-tone characters
            "${bin_FITZ_1_2}"*|"${bin_FITZ_3}"*|"${bin_FITZ_4}"*|"${bin_FITZ_5}"*|"${bin_
FITZ_6}"*)
                _mod=$(printf '%.*s' 4 "$_usw_rem")   # print the next 3 bytes from remai
nder
                _usw_char="${_usw_char}${_mod}"       # append them to the current charac
ter to form full grapheme
                _usw_rem="${_usw_rem#$_mod}"          # strip them from remainder before
moving on
                ;;

            # these (next unicode character)(soft hyphen and bidi controls) are all 2-byt
e
            "${bin_SHY}"*|"${bin_LRO}"*|"${bin_RLO}"*|"${bin_LRE}"*|"${bin_RLE}"*|"${bin_
PDF}"*)
                _mod=$(printf '%.*s' 2 "$_usw_rem")   # print the next 3 bytes from remai
nder
                _usw_char="${_usw_char}${_mod}"       # append them to the current charac
ter to form full grapheme
                _usw_rem="${_usw_rem#$_mod}"         # strip them from remainder before m
oving on
                ;;
            *) break ;;
        esac
    done

    # tier 5 optimization - use precomputed character width look-up
    #   (heavy lifting done in advance by _precook_registry)
    # cannot use case "$_usw_char" in "$_PATTERN_WIDTH_0") ... here,
    # because in posix case, the pattern is not interpreted as logic
    # so the | caracters inserted by _precook_registry do not function
```

```
    # as logical OR among alternative patterns - they are just literal |
    # characters considered to be part of the pattern itself
   _usw_cw=""
    case "$_PATTERN_WIDTH_0" in *"|$_usw_char|"* ) _usw_cw=0 ;; esac
    if [ -z "$_usw_cw" ]; then
        case "$_PATTERN_WIDTH_1" in *"|$_usw_char|"* ) _usw_cw=1 ;; esac
    fi
    if [ -z "$_usw_cw" ]; then
        case "$_PATTERN_WIDTH_2" in *"|$_usw_char|"* ) _usw_cw=2 ;; esac
    fi
    # if tier 5 lookup matched, skip UTF-8 decode entirely
    if [ ! -z "$_usw_cw" ]; then
      _usw_str_width=$((_usw_str_width + _usw_cw))
      _usw_string="${_usw_string#"$_usw_char"}"
      continue
    fi
    # tier 3 optimization - in-line the utf8_char_width calculation rather than call func
tion
    # extract the full UTF-8 character (n bytes)
    # ( .n = precision so , '%.*s' gets the 1st arg # of bytes out of 2nd arg str )
    #  Decode enough of the codepoint to check ranges
    #    We only need the numeric value, not full Unicode decoding.
    #    Extract up to 4 bytes (missing bytes become empty).
    # strip one byte and print decimal value of first remaining
    _usw_b2=$(printf '%d' "'${_usw_string#?}" 2>/dev/null)
    # strip two bytes and print decimal value of first remaining
    _usw_b3=$(printf '%d' "'${_usw_string#??}" 2>/dev/null)
    # strip three bytes and print decimal value of first remaining
    _usw_b4=$(printf '%d' "'${_usw_string#???}" 2>/dev/null)
    # reconstruct codepoint from multi-byte payload bits (minimal UTF-8 decode)
    if   [ $_usw_b1 -lt 224 ]; then
      # 2-byte: 110xxxxx 10xxxxxx
      _cp=$(( ((_usw_b1 & 0x1F) << 6) | (_usw_b2 & 0x3F) ))
    elif [ $_usw_b1 -lt 240 ]; then
      # 3-byte: 1110xxxx 10xxxxxx 10xxxxxx
      _cp=$(( ((_usw_b1 & 0x0F) << 12) | ((_usw_b2 & 0x3F) << 6) | (_usw_b3 & 0x3F) ))
    else
      # 4-byte: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
      _cp=$(( ((_usw_b1 & 0x07) << 18) | ((_usw_b2 & 0x3F) << 12) | ((_usw_b3 & 0x3F) <<
6) | (_usw_b4 & 0x3F) ))
    fi
    #  zero-width characters
    # combining marks: U+0300-U+036F
    if [ $_cp -ge 768 ] && [ $_cp -le 879 ]; then _usw_cw=0
    # variation selector VS-16: U+FE0F
    elif [ $_cp -eq 65039 ]; then _usw_cw=0
    # zero-width joiner: U+200D
    elif [ $_cp -eq 8205 ]; then _usw_cw=0
    # emoji (width = 2)
    # misc symbols: U+2600-U+26FF
    elif [ $_cp -ge 9728 ] && [ $_cp -le 9983 ]; then _usw_cw=2
    # dingbats: U+2700-U+27BF
    elif [ $_cp -ge 9984 ] && [ $_cp -le 10175 ]; then _usw_cw=2
    # emoticons: U+1F600-U+1F64F
    elif [ $_cp -ge 128512 ] && [ $_cp -le 128591 ]; then _usw_cw=2
    # misc symbols & pictographs: U+1F300-U+1F5FF
    elif [ $_cp -ge 127744 ] && [ $_cp -le 128511 ]; then _usw_cw=2
    # transport & map: U+1F680-U+1F6FF
    elif [ $_cp -ge 128640 ] && [ $_cp -le 128767 ]; then _usw_cw=2
    # supplemental symbols & pictographs: U+1F900-U+1F9FF
    elif [ $_cp -ge 129280 ] && [ $_cp -le 129535 ]; then _usw_cw=2
    # symbols & pictographs extended-A: U+1FA70-U+1FAFF (un-interested in B,C,D,etc - non
 emoju class chars)
    elif [ $_cp -ge 129648 ] && [ $_cp -le 129775 ]; then _usw_cw=2
    else _usw_cw=1  # default width = 1
    fi
    _usw_str_width=$((_usw_str_width + _usw_cw))         # accumulate total column width
    _usw_string="${_usw_string#"$_usw_char"}"            # drop n bytes from the front
  done
  printf '%s\n' "$_usw_str_width"  # send answer to caller on stdout
```

```
  unset -v _usw_string _usw_str_width _usw_b1 _usw_b2 _usw_b3 _usw_b4
  unset -v _usw_n _usw_char _usw_cw _usw_prefix _usw_run_length _cp
  return $usw_ret
}
# @usage utf8_string_width "string"
# @args  $1 : UTF-8 string whose display width (columns) is computed
# @vars  (none)
# @ret   Prints computed display width (columns) to stdout; returns 0 on success
# @deps  printf; LC_ALL=C for ASCII range tests; POSIX parameter slicing
# @rule  tier 2: detect printable ASCII runs ([ -~]) and count them in bulk
# @rule  tier 1: single ASCII byte fast-path (width=1; drop byte; continue)
# @rule  tier 3: inline UTF-8 decode and width rules (no utf8_char_width call)
# @rule  tier 5: precomputed width table for known box-drawing and emoji
# @rule  Zero-width: combining marks U+0300â\200\223U+036F, VS-16 (U+FE0F), ZWJ (U+200D)
# @rule  Wide: selected emoji and symbol ranges treated as width 2
# @rule  Default width = 1 for all other decoded codepoints
# @note  Uses minimal UTF-8 decode; invalid sequences are treated as width 1
# @note  Designed for POSIX shells; avoids external commands except printf
# @ex    w=$(utf8_string_width "A U+2600 B")  # sun symbol counted as width 2

old_utf8_string_width() # (POSIX) compute display width of a UTF-8 string
{ usw_ret=0
  _usw_string=$1; _usw_str_width=0
  # iterate until string is empty
  while [ ! -z "$_usw_string" ]; do
    # tier 2 optimization - detect any "run" of ascii chars and handle them quickly
    LC_ALL=C _usw_prefix="${_usw_string%%[! -~]*}"
    if [ ${#_usw_prefix} -gt 1 ] ; then
      _usw_run_length=${#_usw_prefix}
      _usw_str_width=$(( _usw_str_width + _usw_run_length ))
      _usw_string=${_usw_string#"${_usw_prefix}"}; continue
    fi
    # read first byte (POSIX 'X idiom = 1st char [byte] of string X)
    _usw_b1=$(printf '%d' "'$_usw_string")
    # determine UTF-8 sequence byte-length of the current character, by examining the fir
st byte only
    # if these are non-ascii, then they are the actual binary bytes of which the multi-by
te
    # unicode characters consist, placed into the message with a method like
    # _msg="${_msg} $(emoji_safe $hammer)" or simpler by _msg="${_msg} $(printf '%s' $bin
_hammer)"
    # tier 1 optimization - if byte 1 is ascii, increment count and return quicky
    if   [ $_usw_b1 -lt 128 ]; then  # ascii: ( bit 7 = 0 0-127) handle quickly (incremen
t; drop; continue)
       _usw_n=1; _usw_str_width=$((_usw_str_width + 1)); _usw_string=${_usw_string#?}; co
ntinue
    elif [ $_usw_b1 -lt 224 ]; then _usw_n=2     # utf-8 ( bits 765 = 110 192-223)
    elif [ $_usw_b1 -lt 240 ]; then _usw_n=3     # utf-8 ( bits 7654 = 1110 224-239)
    elif [ $_usw_b1 -lt 248 ]; then _usw_n=4     # utf-8 ( bits 76543 = 11110 240-247)
    else _usw_n=1   # treat as single 1-col char # invalid (248-255)( there are no 5- or
6-byte UTF-8 sequences)
    fi                                           # also     bits 76 = 10 marks a continua
tion byte
                                                 #          (so, 128-191 also not allowed
 in 1st byte)
    # extract the full UTF-8 character (n bytes)
    # moved this _usw_char= here, to enable tier 5 optimizaton and slice/drop from string
 further below
    # %. is precision; with s it is length; so '%.*s' substitutes the first arg for *
    #   and uses the result to format the second arg in the printf output
    #   (thus if _usw_n=4 printf '%.4s' "$_usw_string" prints 4 bytes)
    #   (the fact that it is s and not b means the result is the byte sequence, NOT the c
ooked binary character)
    _usw_char=$(printf '%.*s' "$_usw_n" "$_usw_string")
    # tier 5 optimization - use precomputed character width look-up
    #   (heavy lifting done in advance by _precook_registry)
    # cannot use case "$_usw_char" in "$_PATTERN_WIDTH_0") ... here,
    # because in posix case, the pattern is not interpreted as logic
    # so the | caracters inserted by _precook_registry do not function
    # as logical OR among alternative patterns - they are just literal |
```

```
    # characters considered to be part of the pattern itself
   _usw_cw=""
    case "$_PATTERN_WIDTH_0" in *"|$_usw_char|"* ) _usw_cw=0 ;; esac
    if [ -z "$_usw_cw" ]; then
        case "$_PATTERN_WIDTH_1" in *"|$_usw_char|"* ) _usw_cw=1 ;; esac
    fi
    if [ -z "$_usw_cw" ]; then
        case "$_PATTERN_WIDTH_2" in *"|$_usw_char|"* ) _usw_cw=2 ;; esac
    fi
    # if tier 5 lookup matched, skip UTF-8 decode entirely
    if [ ! -z "$_usw_cw" ]; then
      _usw_str_width=$((_usw_str_width + _usw_cw))
      _usw_string="${_usw_string#"$_usw_char"}"
      continue
    fi
    # tier 3 optimization - in-line the utf8_char_width calculation rather than call func
tion
    # extract the full UTF-8 character (n bytes)
    # ( .n = precision so , '%.*s' gets the 1st arg # of bytes out of 2nd arg str )
    #  Decode enough of the codepoint to check ranges
    #    We only need the numeric value, not full Unicode decoding.
    #    Extract up to 4 bytes (missing bytes become empty).
    # strip one byte and print decimal value of first remaining
    _usw_b2=$(printf '%d' "'${_usw_string#?}" 2>/dev/null)
    # strip two bytes and print decimal value of first remaining
    _usw_b3=$(printf '%d' "'${_usw_string#??}" 2>/dev/null)
    # strip three bytes and print decimal value of first remaining
    _usw_b4=$(printf '%d' "'${_usw_string#???}" 2>/dev/null)
    # reconstruct codepoint from multi-byte payload bits (minimal UTF-8 decode)
    if   [ $_usw_b1 -lt 224 ]; then
      # 2-byte: 110xxxxx 10xxxxxx
      _cp=$(( ((_usw_b1 & 0x1F) << 6) | (_usw_b2 & 0x3F) ))
    elif [ $_usw_b1 -lt 240 ]; then
      # 3-byte: 1110xxxx 10xxxxxx 10xxxxxx
      _cp=$(( ((_usw_b1 & 0x0F) << 12) | ((_usw_b2 & 0x3F) << 6) | (_usw_b3 & 0x3F) ))
    else
      # 4-byte: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
      _cp=$(( ((_usw_b1 & 0x07) << 18) | ((_usw_b2 & 0x3F) << 12) | ((_usw_b3 & 0x3F) <<
6) | (_usw_b4 & 0x3F) ))
    fi
    #  zero-width characters
    # combining marks: U+0300-U+036F
    if [ $_cp -ge 768 ] && [ $_cp -le 879 ]; then _usw_cw=0
    # variation selector VS-16: U+FE0F
    elif [ $_cp -eq 65039 ]; then _usw_cw=0
    # zero-width joiner: U+200D
    elif [ $_cp -eq 8205 ]; then _usw_cw=0
    # emoji (width = 2)
    # misc symbols: U+2600-U+26FF
    elif [ $_cp -ge 9728 ] && [ $_cp -le 9983 ]; then _usw_cw=2
    # dingbats: U+2700-U+27BF
    elif [ $_cp -ge 9984 ] && [ $_cp -le 10175 ]; then _usw_cw=2
    # emoticons: U+1F600-U+1F64F
    elif [ $_cp -ge 128512 ] && [ $_cp -le 128591 ]; then _usw_cw=2
    # misc symbols & pictographs: U+1F300-U+1F5FF
    elif [ $_cp -ge 127744 ] && [ $_cp -le 128511 ]; then _usw_cw=2
    # transport & map: U+1F680-U+1F6FF
    elif [ $_cp -ge 128640 ] && [ $_cp -le 128767 ]; then _usw_cw=2
    # supplemental symbols & pictographs: U+1F900-U+1F9FF
    elif [ $_cp -ge 129280 ] && [ $_cp -le 129535 ]; then _usw_cw=2
    # symbols & pictographs extended-A: U+1FA70-U+1FAFF (un-interested in B,C,D,etc - non
 emoju class chars)
    elif [ $_cp -ge 129648 ] && [ $_cp -le 129775 ]; then _usw_cw=2
    else _usw_cw=1  # default width = 1
    fi
    _usw_str_width=$((_usw_str_width + _usw_cw))         # accumulate total column width
    _usw_string="${_usw_string#"$_usw_char"}"            # drop n bytes from the front
  done
  printf '%s\n' "$_usw_str_width"  # send answer to caller on stdout
  unset -v _usw_string _usw_str_width _usw_b1 _usw_b2 _usw_b3 _usw_b4
```

```
  unset -v _usw_n _usw_char _usw_cw _usw_prefix _usw_run_length _cp
  return $usw_ret
}
# @usage utf8_string_width "string"
# @args  $1 : UTF-8 string whose display width (columns) is computed
# @vars  (none)
# @ret   Prints computed display width (columns) to stdout; returns 0 on success
# @deps  printf; LC_ALL=C for ASCII range tests; POSIX parameter slicing
# @rule  tier 2: detect printable ASCII runs ([ -~]) and count them in bulk
# @rule  tier 1: single ASCII byte fast-path (width=1; drop byte; continue)
# @rule  tier 3: inline UTF-8 decode and width rules (no utf8_char_width call)
# @rule  tier 5: precomputed width table for known box-drawing and emoji
# @rule  Zero-width: combining marks U+0300â\200\223U+036F, VS-16 (U+FE0F), ZWJ (U+200D)
# @rule  Wide: selected emoji and symbol ranges treated as width 2
# @rule  Default width = 1 for all other decoded codepoints
# @note  Uses minimal UTF-8 decode; invalid sequences are treated as width 1
# @note  Designed for POSIX shells; avoids external commands except printf
# @ex    w=$(utf8_string_width "A U+2600 B")  # sun symbol counted as width 2

get_msg_len() # (POSIX) compute display width of a message string
{ gml_ret=0
  # require at least one argument
  [ $# -lt 1 ] && return 1
  # tier 4 optimization - memoizaton (single entry cache) to speed up repeat-calls
  _gml_str="$*"
  # if the message is an empty string, answer so immediately
  [ -z "$_gml_str" ] && { printf "%s\n" 0; return 0 ; }
  if [ "$_gml_str" = "$gml_last_str" ]; then printf '%s\n' "$gml_last_width"; return; fi
  # expand escapes, strip ANSI SGR sequences
  _gml_clean=$(printf '%b' "$*" | _translate_escapes | strip_ansi)
  # tier 0 optimization - determine if the msg contains any bytes that are non-ascii char
acters ( bit 7 = 1 )
  # (cant use [![;ascii:]] because that misses control characters (0-31, 127)(incl. non-p
rintable ascii),
  #  and _translate_escapes() removes only 6 format-breaking escape sequences;
  #  further, printf '%b' could (re)introduce other control characters when interpreting
"$*"
  # (can't use [! -~] because e.g. ash cannot parse it properly
  # define a pattern that matches any byte > 127 (bit 7 = 1)(128-255/x80-xFF/o200-o377)
  _80=$(printf '\200')
  _FF=$(printf '\377')
  LC_ALL=C; case "$_gml_clean" in
    *["${_80}"-"${_FF}"]*)
      _gml_width=$(utf8_string_width "$_gml_clean");;
    *        )   # all ascii - just use byte count
      _gml_width="${#_gml_clean}";;
  esac
  printf '%s\n' "${_gml_width:-0}"  # emit byte count or 0 if empty
  gml_last_str="$_gml_str"
  gml_last_width="$_gml_width"
  unset -v _gml_str _gml_clean _gml_width _80 _FF
  return $gml_ret
}
# @usage  get_msg_len "${STRING}"
# @args   STRING : message text; may contain escapes and ANSI SGR codes
# @ret    Prints display width of visible characters; returns 0
# @vars   gml_last_str, gml_last_width - globals to persist the single-entry cache
# @deps   utf8_string_width, _translate_escapes, strip_ansi
# @rule   Expands escapes, strips ANSI, then computes UTF-8 display width
# @rule   Uses utf8_string_width() for width 0/1/2 per character
# @rule   tier 0: ASCII-only fast-path; skip UTF-8 width calc when no non-ASCII bytes
# @rule   tier 4: single-entry memoization cache; return last width without recomputation
# @note   Caller may pass multiple words; all are joined by "$*"
# @note   No external commands; POSIX-compliant
# @ex     get_msg_len "A$(printf '\xF0\x9F\x94\xA5')B"  # prints 4


get_longest() # echo length of longest string in input $@
{ _ans=0; for _x in "$@"; do _N=$(get_msg_len "$_x"); [ $_N -gt $_ans ] && _ans=$_N; done
 ;
```

```
  printf '%s' "$_ans"; unset -v _ans _x _N; }
# @note switched from ${#_x} to get_msg_len "$_x" to strip ansi;
# @note unlike get_msg_len, which is a lookahead for word-wrapping decisions,
# @cont get_longest calculates the length of the longest element (like varnames for displ
ay_vaars)
# @cont but to ensure utility outside just single word entries in the column width need b
eing counted
# @cont it was switched from $@ (unquoted) to "$@"
#{ _ans=0; for _x in $@; do [ ${#_x} -gt $_ans ] && _ans=${#_x}; done ; printf '%s' "$_an
s"; unset -v _ans _x; }

repeat() # (POSIX) quickly output a string of char $1 (of len $2)
{ r_ret=0; _r_marker="${1:--}"; _r_reps="${2:-40}";
  ! isint "$_r_reps" && { j_msg -3 "error: $2 (_fr_reps) must be an integer"; return 1; }
  [ "${#_r_marker}" -gt 1 ] && _r_marker=$(printf -- '%s' "${_r_marker%${_r_marker#?}}")
  printf -- "%${_r_reps}s" | tr -- ' ' "$_r_marker" ; r_ret=$?
  unset -v _r_marker _r_reps; return $r_ret ; }
# @usage repeat "marker" "count"
# @args $1 char to repeat; $2 number of times
# @ret (emits) string of repeated characters
# @note helpful for creating variable-length separators or visual borders
# maintainer note: '--' indicates 'end of options' to prevent cmd from interpreting '-' i
n str as opt

full_repeat() # (POSIX) repeat full input string pattern $1 $2 times
{ fr_ret=0; _fr_marker=${1:--}; _fr_reps=${2:-40}
  ! isint "$_fr_reps" && { j_msg -3 "error: $2 (_fr_reps) must be an integer"; return 1;
}
  _fr_i=0; while [ "$_fr_i" -lt "$_fr_reps" ]; do
    printf -- '%b' "$_fr_marker"
    _fr_i=$((_fr_i + 1))
  done ; fr_ret=$?
  unset -v _fr_reps _fr_marker _fr_i
  return $fr_ret
}
# @usage full_repeat "string" "<integer>"
# @args $1: the string or character pattern to be repeated (default: "-")
# @args $2: the number of times to repeat the pattern (default: 40)
# @vars ret: initialized to 0; used to capture exit status of the printf loop
# @ret (emitted) a single string containing the repeated pattern to stdout
# @deps seq: uses the standard sequence generator for the loop range
# @note this is the "string-capable" version of the single-character repeat function
# @note helpful for creating variable-length separators or visual borders
# maintainer note: '--' indicates 'end of options' to prevent cmd from interpreting '-' i
n str as opt
# @note use '%b' (expand backslash) to interpret e.g. \n (like echo -e)

pad() # generate pad for cur(str $1) + sep(str $2) w min(int $3) color(str $4) marker(cha
r $5); filling col(int $6)
{ # initial guard - all six arguments must exist (some can be null to be defaulted)
  [ $# -lt 6 ] && { j_msg -3 "#args: $#; 6 args required: cur, sep, min, color, marker, c
ol"; return 1; }
  # assign localized variables with parameters received or defaults where allowable (seco
ndary guard)
  p_ret=0; _pad_cur="$1"; _pad_sep="${2:- }"; _pad_min="${3:-2}";
  _pad_color="$4"; _pad_marker="${5:-.}"; _pad_col="$6"
  # exercise additional guards --
    # $1: reject null current element
    [ -z "$_pad_cur" ] && { j_msg -3 "arg1 (str cur) cannot be null"; return 1; }
    # $2: remove ansi from marker (will be colored by $color if that is provided)
    _pad_sep=$(printf '%s' "$_pad_sep" | _translate_escapes | strip_ansi)
    # $3: min is defaulted above :- (exists?) in "${3:-2}" ensures that null (or even uns
et) is reassigned
    # $4: initial guard ensures existence, and null is OK for printf,
    #     so, ensure color is a proper ANSI SGR sequence and not printable text (e.g.: ${
CSI}${RED};${BOLD}m = ${BRon})
    if [ -n "$_pad_color" ]; then
      case "$_pad_color" in
        "${CSI}"*m?*)    # reject if there is anything after the 'm'
          j_msg -3 "arg4 (color) contains trailing printable characters"; return 1 ;;
```

```
        "${CSI}"*m) : ;; # Valid sequence, do nothing and continue
        *)               # everything else is malformed or printable text
          j_msg -3 "arg4 (color) must be a valid ANSI sequence (e.g. \${CSI}31m)"; return
 2 ;;
      esac
    fi
    # $5: marker is defaulted above :- (exists?) in "${5:-.}" ensures that null (or even
unset) is reassigned
    # $5: remove ansi from  separator (will be colored by $color if that is provided) and
 truncate
    _pad_marker=$(printf '%s' "$_pad_marker" | _translate_escapes | strip_ansi)
    # truncate marker to a single character
    # ? matches a single char, so ${_pad_marker#? returns all after the first (result)
    # ${_pad_marker%  strips shortest match for (result) from the tail - leaving only the
 first
    _pad_marker="${_pad_marker%${_pad_marker#?}}"
    # $6: reject null column width
    ! isint "$_pad_col" && { j_msg -3 "arg6 (int col) cannot be null"; return 2; }
  # use get_msg_len for ANSI stripping just to get the length of current element
  _pad_cur_len=$(get_msg_len "$_pad_cur")
  _pad_sep_len="${#_pad_sep}"   # already ANSI stripped abpve
  # calculate the pad length
  # (L->R): min + col - (cur_len + sep_len)
  # (R->L): col - (cur_len + sep_len) + min
  _pad_length=$(( _pad_min + _pad_col - (_pad_cur_len + _pad_sep_len) ))
  # generate the full pad as a string
#  j_msg -7p "_pad_cur: $_pad_cur" "$debug"
#  j_msg -7p "_pad_sep: $_pad_sep" "$debug"
#  j_msg -7p "_pad_min: _pad_min" "$debug"
#  j_msg -7p "_pad_color: ${_pad_color}this${Boff}" "$debug"
#  j_msg -7p "_pad_marker: $_pad_marker" "$debug"
#  j_msg -7p "__pad_col: $_pad_col" "$debug"
#  j_msg -7p "_pad_length: $_pad_length" "$debug"
  _pad_string=$(repeat "$_pad_marker" "$_pad_length")
  # output the pad
  printf "%b%s%b" "$_pad_color" "$_pad_string" "$Boff" ; p_ret=$?
  unset -v _pad_cur _pad_sep _pad_min _pad_color _pad_marker _pad_col
  unset -v _pad_cur_len _pad_sep_len _pad_length _pad_string
  return $p_ret
}
# @usage pad "cur" "sep" "min" "color" "marker" "col"
# @req all 6 args must be supplied; see below how some '' vals can default
# @note where . str cur = the current element in fixed width column
# @note ....... str sep = internal separator (e.g. ': ')
# @note ....... int min = the minimum number of pad chars ...... (default 2)
# @note ..... str color = ANSI escape sequence (e.g. '\033[32m')
# @note ... char marker = the character w which to construct pad (default '.')
# @note ....... int col = the columns total fixed width
# @note marker string will be stripped of ANSI and truncated to 1 char
# @deps _translate_escapes, strip_ansi
# @warn failure to properly quote all args may result in unexpected behavior

longone="I had been studying trail guides and maps of the Adirondacks"
longone="${longone} $(emoji_safe $uMOUNTAIN) on and off for years."
longone="${longone} The desire to become a ${BGon}46er${Boff} was not new I had put it on
 my bucket list while I was still"
longone="${longone}  in high school."
longone="${longone}  This new idea on the other hand -- to do it as a through hike"
longone="${longone} $(emoji_safe $uBOOT$tree_evergreen) -- had begun to formulate in my m
ind"
longone="${longone} shortly after I had finished hiking the Appalachian Trail and Long Tr
ail in 2017. In 2018 and 2019"
longone="${longone} I got updated maps and trail guides to study, but my hopes of doing t
his in 2020 were suffocated"
longone="${longone} by COVID-19.  During the spring of 2021, after I got my second round
of COVID vaccination in March,"
longone="${longone} I tried to get serious and finalize my initial plans and preparation.
"
longone="${longone}  I used both Alltrails and Gaia GPS on-line mapping software to desig
n the routes for my draft plan."
```

```
longone="${longone}  I watched online tutorials about how to use these tools for planning
 and real-time navigation,"
longone="${longone} $(emoji_safe $compass)"
longone="${longone}  and I created an initial draft route plan.  I also used that draft r
oute plan to create a spreadsheet"
longone="${longone}  (like I had used on the Appalachian Trail) to estimate achievable da
ily objectives peaks, distances,"
longone="${longone}  water sources, camps, etc.  I knew my plan would change after I star
ted executing it, but I felt much more"
longone="${longone}  comfortable with this plan and a waterproof map in a Ziplock bag in
the cargo pocket of my hiking pants."
longone="${longone}   I also downloaded to my phone the topographic maps and route plans
I needed for the phone’s version"
longone="${longone}  of the Alltrails and Gaia GPS apps, so that I could use the phone as
 a real-time GPS navigation device,"
longone="${longone}  to both record my actual route and to enable me to compare that to m
y plan visually - or to see my"
longone="${longone}  actual track, as I might intentionally deviate from my ${BRon}planne
d${Boff} route."
longone="${longone} $(emoji_safe $uCAMP)"

smart_wrap_n() # (POSIX) wordwrap string $1 current col, $2 = l_margin, remainder = messa
ge words
{ swr_ret=0
  if [ $# -lt 3 ] ; then j_msg -3 "minimum 3 args: curr_col, l_margin, r_margin; remainde
r=msg"; return 1; fi
  if ! isint $1 ; then j_msg -3 "Error: arg \$1 must be numeric starting column/caracter
position"; return 1; fi
  # provide numeric defaults for null/empty inputs
  _sw_current_col=${1:-0}; _sw_l_margin=${2:-0}; _sw_r_margin=${3:-0}
  # revert to 0 if the provided value is a non-integer string
  isint "$_sw_current_col" || _sw_current_col=0
  isint "$_sw_l_margin"    || _sw_l_margin=0
  isint "$_sw_r_margin"    || _sw_r_margin=0
  shift 3
  # (do not translate internal escapes ([:control:] chars like \n, \t, \r, etc.)
  #  b/c want to preserve (e.g. multi-line prompt) line formatting, and
  #  do not strip_ansi b/c want to preserve color codes, etc; only remove those in subshe
ll to get_msg_len)
 _sw_msg=$(printf ’%b’ "$*" | _translate_escapes)   # operate on entire set of args as a
single string "$*"
#    _sw_msg="$*"   # operate on entire set of args as a single string "$*"
  # determine the initial total visible length of the message
  _sw_remaining_len=$(get_msg_len "$_sw_msg")  # Initialize the counter here
  # perform controlled split on the remaining parameters ($@)
  set -f     # Disable globbing to prevent interpretation of "*" as wildcard to be expand
ed, etc.
  # safely convert a positional parameters (either multiple args or a single quoted strin
g) into individual words
  set -- $_sw_msg    # set with $_sw_msg unquoted so each split word becomes a positional
 parameter
  set +f     # re-enable globbing (minimize the "blast radius" of the globbing-disabled s
tate)
  # after shift and reassembly; set a default message argument to "<empty>" if none provi
ded
  [ $# -eq 0 ] && set -- "$@" "<empty>"
  _sw_tw=$(termwidth); isint "$_sw_tw" || _sw_tw=80  # ensure _sw_tw is a save number for
 math
  # clamp to terminal width
  [ "$_sw_l_margin" -ge "$_sw_tw" ] && _sw_l_margin=$(( _sw_tw - 1 ))
  [ "$_sw_r_margin" -ge "$_sw_tw" ] && _sw_r_margin=$(( _sw_tw - 1 ))
  # calculate the "right_status collision zone" so the last wrap keeps us clear of it
  _sw_status_width=6   # e.g. "[ Ok ]" = 6 char
  _sw_buffer=4         # padding for more visually appealing wrapping
  _sw_collision_limit=$(( _sw_tw - _sw_status_width - _sw_buffer ))
  [ "$_sw_collision_limit" -lt 0 ] && _sw_collision_limit=0   # clamp to zero incase nega
tive
  # if r_margin is deep enough to cover the status zone, no special wrap is needed
  if [ "$_sw_r_margin" -ge $(( _sw_status_width + _sw_buffer )) ]; then
    _sw_FORCE_STATUS_WRAP=$FALSE
```

```
  else
    _sw_FORCE_STATUS_WRAP=$TRUE
  fi

  _sw_line_len="$_sw_current_col" # initialize with starting position
  # smartly consider whether to print each word or wrap it to the _sw_l_margin
  _sw_FIRST_WORD=$TRUE       # first word on a line. dont normally wrap, but may need to
slice if it wont fit
  _sw_VERY_FIRST_WORD=$TRUE  # very first word in the stack, dont normally wrap, but may
need to slice if it wont fit
                             # but since start col of first line may not be the same as l
eft margin, check that
                             # before deciding to slice the word
  # (if one word fits between margins, we can print all of it, else we have to slice this
 much of it)
  _sw_slice_len=$(( (_sw_tw - _sw_r_margin ) - _sw_l_margin -1 ))

  while [ $# -gt 0 ]; do
    if [ $# -eq 1 ]; then _sw_LAST_WORD=$TRUE; else _sw_LAST_WORD=$FALSE; fi
    # use collision limit ONLY for last word; otherwise use the requested r_margin
    if [ "$_sw_LAST_WORD" ]; then
      _sw_boundary=$_sw_collision_limit
    else
      _sw_boundary=$(( _sw_tw - _sw_r_margin ))
    fi
    # optimization - calculate if everything left fits on this line
    _sw_space_left=$(( _sw_boundary - _sw_line_len ))
    # first word goes alone; others preceded by space (+1)
    [ "$_sw_FIRST_WORD" ] || _sw_space_left=$(( _sw_space_left - 1 ))
    # if there is enough room for all of it, be done
    if [ "$_sw_remaining_len" -le "$_sw_space_left" ]; then
      [ ! "$_sw_FIRST_WORD" ] && printf " "
      printf "%b" "$*"
      break # done
    fi

    # not room for all of it, so step off a word at a time
    # (step logic - measure only the next word)
    _sw_x_peak="$1"
    _sw_x_peak_len=$(get_msg_len "$_sw_x_peak")
    if [ "$_sw_FIRST_WORD" ]; then
      _sw_needed=$(( _sw_line_len + _sw_x_peak_len ))
    else
      _sw_needed=$(( _sw_line_len + _sw_x_peak_len + 1 ))
    fi

    # if this word fits, print it, else wrap
    if [ "$_sw_needed" -le "$_sw_boundary" ]; then
      # it fits; print it (no longer 1st word if it was before, reset that after shift, b
efore loop done)
      [ ! "$_sw_FIRST_WORD" ] && printf " "
      printf "%b" "$_sw_x_peak"
      _sw_line_len=$_sw_needed  # keep track of what has been rendered
    else
      # it won't fit - so wrap (incl logic for final word if margin is thin)
      if [ "$_sw_VERY_FIRST_WORD" ] && [ ! "$_sw_current_col" -eq "$_sw_l_margin" ]; then
          # this is the very first word in the stack and the margin is not the same as th
e startig column
          # so wrap, advance to margin, and try again as a regular first word on a full l
ine in next loop
          if [ "${#_sw_x_peak}" -le "$_sw_slice_len" ]; then
            # it will fit between margins - print it
            printf "\n%b%b" "$(repeat ' ' "$_sw_l_margin")" "$_sw_x_peak"
          else
            # The current word wont fit between smart_wrap_n's margins, so we have to
            # slice it and put remainder back in the stack of words for next loop
            # use printf with format string '%.*s' (. [precision] for string is lenght) *
 means take arg1
            _sw_slice=$(printf '%.*s' "$_sw_slice_len" "$_sw_x_peak")
            _sw_remainder="${_sw_x_peak#"${_sw_slice}"}"
```

```
            shift                                          # remove the big word from the
 stack
            printf '%b' "$_sw_slice"                       # print the slice
            set -- "$_sw_remainder" "$@"                   # push the remainder back onto
 the stack
            printf '\n%b' "$(repeat ' ' "$_sw_l_margin")"  # indent to left margin
            _sw_line_len=$_sw_l_margin                     # reset how much has been rend
ered
            _sw_FIRST_WORD=$TRUE
            _sw_VERY_FIRST_WORD=$FALSE
            continue     # skip the shift to get the piece we just put in as the next wor
d
          fi
        elif [ "$_sw_FIRST_WORD" ]; then
          # first word on the line, so we are already at the left margin
          # and b/c it wont fit, we have to forc-print it in order to avoid an infinite loo
p;
          # also, to avoid hard-wrapping to the left edge of the screen, I'm going to slice
          # the word, print the part that will fit, and push the remainder back into the st
ack
          # of words to be printed.
            # The current word wont fit between smart_wrap_n's margins, so we have to
            # slice it and put remainder back in the stack of words for next loop
            # use printf with format string '%.*s' (. [precision] for string is lenght) * m
eans take arg1
            _sw_slice=$(printf '%.*s' "$_sw_slice_len" "$_sw_x_peak")
            _sw_remainder="${_sw_x_peak#"${_sw_slice}"}"
            shift                                          # remove the big word from the s
tack
            printf '%b' "$_sw_slice"                       # print the slice
            set -- "$_sw_remainder" "$@"                   # push the remainder back onto t
he stack
            printf '\n%b' "$(repeat ' ' "$_sw_l_margin")"  # indent to left margin
            _sw_line_len=$_sw_l_margin                     # reset how much has been render
ed
            _sw_FIRST_WORD=$TRUE
            _sw_VERY_FIRST_WORD=$FALSE
          continue     # skip the shift to get the piece we just put in as the next word
        elif [ "$_sw_LAST_WORD" ] && [ "$_sw_FORCE_STATUS_WRAP" ] && [ "$_sw_needed" -le "$
_sw_tw" ]; then
          # last word in the stack; thin margin; not enough room (avoid collision with stat
us zone);
          # (wouldn't need wrap otherwise, so force it - wrap, advance to l_margin, print w
ord
          if [ "${#_sw_x_peak}" -le "$_sw_slice_len" ]; then
            # it will fit between margins - print it
            printf "\n%b%b" "$(repeat ' ' "$_sw_l_margin")" "$_sw_x_peak"
          else
            # The current word wont fit between smart_wrap_n's margins, so we have to
            # slice it and put remainder back in the stack of words for next loop
            # use printf with format string '%.*s' (. [precision] for string is lenght) * m
eans take arg1
            _sw_slice=$(printf '%.*s' "$_sw_slice_len" "$_sw_x_peak")
            _sw_remainder="${_sw_x_peak#"${_sw_slice}"}"
            shift                                          # remove the big word from the s
tack
            printf '%b' "$_sw_slice"                       # print the slice
            set -- "$_sw_remainder" "$@"                   # push the remainder back onto t
he stack
            printf '\n%b' "$(repeat ' ' "$_sw_l_margin")"  # indent to left margin
            _sw_line_len=$_sw_l_margin                     # reset how much has been render
ed
            _sw_FIRST_WORD=$TRUE
            _sw_VERY_FIRST_WORD=$FALSE
            continue     # skip the shift to get the piece we just put in as the next word
          fi
        else
          # the word that won't fit is neither the last word in the stack nor the first on
the line,
          # so wrap, advance to l_margin (don't print word; just set first word **
```

```
        # (i.e. let next loop handle it as first word)
        printf "\n%b" "$(repeat ' ' "$_sw_l_margin")"
        _sw_line_len=$_sw_l_margin
        _sw_FIRST_WORD=$TRUE
        continue #  (i.e. don't shift)(** evaluate the same word as FIRST_WORD, on the ne
w line)
      fi  # last word
    fi  # fits?

    # update total message remainder counter by subtraction
    _sw_remaining_len=$(( _sw_remaining_len - _sw_x_peak_len ))
    [ $# -gt 1 ] && _sw_remaining_len=$(( _sw_remaining_len - 1 ))
    shift
    _sw_FIRST_WORD=$FALSE
    _sw_VERY_FIRST_WORD=$FALSE
  done
  swr_ret=$?

  # clean up variables to maintain "local" scope in sourced environments
  unset -v _sw_buffer _sw_collision_limit _sw_current_col
  unset -v _sw_FIRST_WORD _sw_VERY_FIRST_WORD _sw_LAST_WORD
  unset -v _sw_l_margin _sw_line_len _sw_status_width _sw_total_tail_len
  unset -v _sw_trace _sw_tw _sw_x _sw_x_len _sw_x_peak _sw_x_peak_len
  return $swr_ret
}
# @usage smart_wrap_n "$start" "$l_margin" "$r_margin" "$message"
# @deps _translate_escapes, strip_ansi
# @req The function must know, for absolute reliability -- where it is starting and (b) l
_margin and r_margin
# @rule $1 (start) must be supplied as a decimal integer.
# @cont This informs the engine where the cursor is located before it starts printing. (p
utting it there is the call>
# @rule $2 (l_margin) must be set (either to a decimal integer or null value)
# @rule $3 (r_margin) must be set (either to a decimal integer or null value)
# @note (1) This function is a fragment-aware smart-wrapping layout engine designed for p
osix ash compatibility
# @note (2) Primary objective: deterministic word-wrapping to protect the right_status co
llision zone
# @note (3) Secondary objective: visual alignment on the l_margin and wraps inside r_marg
in
# @warn The engine is highly reliable. Onus is on the caller to assign and provide the co
rrect values
# @cont for $1, $2, $3, and consequences of these choices are thus purely the caller's re
sponsibility
# @note smart_wrap_n is the successor to echo_n_long (faster, plus right-margin aware)
# Design and Maintenance Notes:
# use printf '%b' throughout, to handle ANSI color codes and embedded backslashes (e.g.\n
)
# loop uses set -- $* inside set -f/set +f globbing protection to safely force word-split
ting
#   of strings for word-by-word evaluation
# clean scope is maintained by unsetting all _sw_ prefixed variables at exit
# 'set -- ...' (described above) tells the command to stop looking for options;
#   everything that follows is a positional argument (so e.g. "-o" or "--example"
#   won't be seen as command options
# The engine employs a word wrapping algorithm to display its message --
#   it begins wherever the cursor is when called, then -
#   it respects the callers l_margination request for the left edge of every wrapped line
, and
#   it respects the r_margin and space required for right_status on the right margin of t
he
#   last line of the message

ask_pass() # (POSIX) silently read password equivalent to 'read -s -p'
{ # @usage ask_pass "prompt_string"
  # @args $1: prompt string (e.g., "Password: ")
  # @ret (emit to stdout) string: the entered password
  # @ret 0: success; 1: fail
  # @rule uses stderr for the prompt to keep stdout clean for capturing the password
  # @rule uses 'stty -echo' to disable terminal character display
```

```
    ap_ret=0
    printf '%s' "$1" >&2  # use stderr so the prompt goes to the screen
    stty -echo
    read -r _pass || ap_ret=1
    stty echo
    printf "\n" >&2 || ap_ret=1
    printf '%s' "$_pass"
    unset -v _pass
    return $ap_ret
}

alpha_words() # (POSIX) alphabetize the words in a quoted input string
{ aw_ret=0; [ $# -eq 0 ] && return 0  # guard clause; just return 0 success if no args
  for _x in "$@"; do printf '%s\n' "$_x"; done | LC_ALL=en_US.UTF-8 sort -db 2>/dev/null
|| \
    for _x in "$@"; do printf '%s\n' "$_x"; done | sort -db ; aw_ret=$?  # Fallback if UT
F-8 locale is missing
  unset -v _x ; return $aw_ret ; }
# @usage alpha_words "word list"
# @args $* string of words to be sorted
# @ret newline-separated list of alphabetized words
# @rule uses LC_ALL=en_US.UTF-8 sort -db for dictionary-order sorting
# @note sort -d (dictionary): Ignores punctuation and only looks at letters, digits, and
blanks
# @note sort -b (ignore leading blanks) in case string has accidental leading spaces

right_most() # (POSIX) echo the right-most character in a string variable $1
{ rm_ret=0; _rm_str="$1"; [ -z "$_rm_str" ] && return 1; # guard clause; reject empty str
ing
printf -- '%s' "${_rm_str#${_rm_str%?}}" ; rm_ret=$?    # -- prevents printf from treatin
g leading "-" as a flag
unset -v _rm_str; return $rm_ret ; }                       # clean up local variable
# @usage right_most "string"
# @args $1 input string
# @ret the single final character of the input string
# @rule uses POSIX parameter expansion ${str#${str%?}} to extract character
# @note -- prevents printf from treating leading "-" as a flag
# Note: ${_rm_str%?} removes the last character from the string;
#   ${_rm_str#...} tells the shell to "look at the full string and remove the prefix that
 matches ${_rm_str%?}
#   (together, the only thing left is the last character)
# echo ${_rm_str:$((${#_rm_str}-1)):1};        # other way 1 (non-POSIX string slicing; s
lower external echo)
# echo ${_rm_str} | sed "s|${_rm_str:0:-1}||"  # other way 2 (non-POSIX string slicing; s
lower external sed)
# echo ${_rm_str} | rev | cut -c 1 ; }          # other way 3 (non-POSIX rev/cut -c and s
lower external cut/rev)

left_most() # (POSIX) echo the left-most character in a string variable $1
{ lm_ret=0; _lm_str="$1"; [ -z "$_lm_str" ] && return 1
  printf -- '%s' "${_lm_str%${_lm_str#?}}" ; lm_ret=$?
  unset -v _lm_str ; return $lm_ret ; }
# @usage left_most "string"
# @args $1 input string
# @ret the single first character of the input string
# @rule uses POSIX parameter expansion ${str%${str#?}} to extract character

print_center() # (POSIX) print text $2 centered on row $1
{ p_ctr_ret=0; _pc_row="$1"; _pc_text="$2"; _pc_w=$(termwidth)
  _pc_len=${#_pc_text}; _pc_col=$(( (_pc_w / 2) - (_pc_len / 2) ))
  [ "$_pc_col" -lt 1 ] && _pc_col=1  # Prevent negative/zero column
  CUP "$_pc_row" "$_pc_col" ; printf '%s' "$_pc_text" ; p_ctr_ret=$?
  unset -v _pc_row _pc_text _pc_w _pc_len _pc_col ; return $p_ctr_ret ; }
# @usage print_center "row" "text"
# @args $1 row number; $2 text to center
# @deps termwidth CUP
# @rule calculates column based on termwidth and raw string length
# @note does not currently account for ANSI length; use for raw text

# @note =====[ Section 5: Logic & Conversion ]==========================================
```

```
============================

TrueFalse() # (POSIX) echo "True" or "False", depending on truth of arg $1
{ # (T/F Y/N up/down)(1=0=$TRUE [Green], ""=$FALSE [Red])
  # @usage TrueFalse "input"
  # @args $1 value to evaluate (y/n, t/f, u/d, h/l, 0)
  # @ret "True" (exit 0), "False" (exit 1), or "unset" (exit 2)
  # @rule evaluates 0 as True based on global TRUE=0 [cite: 419]
  # @note handles High/Low and Up/Down for hardware-style logic checks
  case $1 in
    [yY]*|[tT]*|[uU]*|[hH]|"$TRUE"|"0" )
       printf '%s' "True"; return 0 ;;
    [nN]*|[fF]*|[dD]*|[lL]|"$FALSE"    )
       printf '%s' "False"; return 1 ;;
    *) printf '%s' "unset"; return 2 ;;
  esac ; }
# Note: "0" is included as part of the "true" line because of the definition TRUE=0 atop
this header
#       "1" is NOT included in the "false" line - return 2 (unset)
#       this is a deliberate choice b/c binary numbering 1/0 is the opposite
#       High/Low and Up/Down are included instead

status_color() # (POSIX) emit escape seq for color per arg $1
{ # (T/F Y/N up/down)(1=0=$TRUE [Green], ""=$FALSE [Red])
  # @usage status_color "input"
  # @args $1 logic value to colorize
  # @ret emits $BGon (green), $BRon (red), or $BWon (white)
  # @rule maps 0 to green and 1 to red for standard exit code visualization
  case $1 in [yY]*|[tT]*|[uU]*|[hH]|"$TRUE"|"0"  ) printf '%b' "${BGon}" ;;
           [nN]*|[fF]*|[dD]*|[lL]|"$FALSE"|"1" ) printf '%b' "${BRon}" ;;
           *) printf '%b' "${BWon}" ;; esac ; }
# Note: "0" is included as part of the "true" line because of the definition TRUE=0 atop
this header
#       "1" is NOT included in the "false" (though that would map to exit code use)
#       this is a deliberate choice b/c binary numbering 1/0 is the opposite
#       High/Low and Up/Down are included instead
# 20260119 - update trying 1 among map to BRon (test if this "breaks" something)

vercomp() # (almost POSIX)(portible w/ coreutils) compare versions return [0:equal|1:grea
terthan|2:lessthan]
{ vc_ret=0; _candidate1="$1"; _candidate2="$2"; _first=""
  if [ "$_candidate1" = "$_candidate2" ]; then
    vc_ret=0    # equal
  else
     _first=$(printf '%b' "$1\n$2" | sort -V | head -n1);
    if [ "$_first" = "$_candidate2" ]; then
      vc_ret=1  # greater
    else
      vc_ret=2  # less
    fi;  # _first
  fi;  # _candidates
  unset -v _candidate1 _candidate2 _first
  return $vc_ret
}
# @usage vercomp "v1" "v2"
# @args $1 candidate version; $2 comparison version
# @ret 0:equal | 1:greater than | 2:less than
# @deps sort -V (coreutils)
# @rule uses sort -V for natural version sorting (e.g., 1.10 > 1.2)

show_result() # (POSIX) show the result of vercomp comparison, in english
{ case $1 in
    0) printf '%b\n' "${BGon}=${Boff}"; return 0;;
    1) printf '%b\n' "${BCon}>${Boff}"; return 0;;
    2) printf '%b\n' "${BRon}<${Boff}"; return 0;;
    *) printf '%b\n' "${BYon}*** Error ***${Boff}"; return 1;;
  esac ; }
# @usage show_result "code"
# @args $1 integer code from vercomp (0, 1, or 2)
# @ret colorized mathematical operator string (=, >, <)
```

```
my_date() # (POSIX) echo date string (DDD MM dd hh:mm:ss TZ yyyy)
{ _date_string=$(date "+%a %b %d %H:%M:%S %Z %Y"); printf '%s\n' "$_date_string" ; unset
-v _date_string; }
# @usage my_date
# @ret string formatted as "Wed Jan 21 12:47:00 EST 2026"
# @note standard POSIX date formatting

my_short_date() # (POSIX) echo shorter date string (yyyymmdd-timehack)
{ _date_string=$(date "+%Y%m%d-%H:%M:%S"); printf '%s\n' "$_date_string" ; unset -v _date
_string; }
# @usage my_short_date
# @ret string formatted as "20260121-12:47:00"
# @note optimized for log file naming or compact timestamps

initialize_vars() # initialize values for vars in $@; incl bool. lv. (info severity)
{ [ $# -lt 1 ] && return 1
  _iv_parmlist=""
  for _iv_item in "$@"; do
#    j_msg -7 "considering whether to include [$_iv_item] in _iv_parmlist"
    case "$_iv_item" in
      # effectively remove these from the list, if user included them
      *loglevel|*verbosity|*VERBOSE|*QUIET|*DEBUG|*EMOJI*)
#        j_msg -7 "NOT including [$_iv_item] in _iv_parmlist"
        continue
        ;;
      *) _iv_parmlist="$_iv_parmlist $_iv_item" ;;        # thus NOT re-initializing thes
e system-wide globals
    esac
  done
  j_msg -7 "initializing vars: $_iv_parmlist"
  for _iv_var_name in $_iv_parmlist
  do
    j_msg -7n "about to initialize $_iv_var_name"
    case $_iv_var_name in
      *.* )   # if echo $_iv_var_name | grep -q "\."; then
        # initialize by type
        _iv_type="${_iv_var_name%.*}"   # strip off chars right of and .
        _iv_var="${_iv_var_name#*.}"    # strip off chars left of and .
        j_msg -7np " of _iv_type: [$_iv_type] ..."
        case ${_iv_type} in
          [bB]*  )
            # (set $FALSE) boolean convention: parameter is "bool.${_iv_var}" where ${_iv
_var} is e.g. "VERBOSE"
            j_msg -7np "  (${_iv_type}) initializing ${_iv_var} FALSE ..."
            eval "${_iv_var}=\"\$FALSE\"" ; right_status $? "$debug"
          ;;
          [yY]*  )
            # (set "no") yes/no convention: parameter is "yn.${_iv_var}" where ${_iv_var}
 is e.g. "EIX"
            j_msg -7np "  (${_iv_type}) initializing ${_iv_var} (no) ..."
            eval "${_iv_var}='no'" ; right_status $? "$debug"
          ;;
          [lLrR]*  )
            # (set "") lonstring convention: parameter is "lv.${_iv_var}" where ${_iv_var
} is e.g. "CMDLINE"
            j_msg -7np "  (${_iv_type}) initializing ${_iv_var} NULL ..."
            eval "${_iv_var}=''" ; right_status $? "$debug"
          ;;
          *      )
            # invalid variable _iv_type
            eval "${_iv_var}=''" ; right_status $? "$debug"
            unset -v _iv_item _iv_parmlist _iv_var_name _iv_type _iv_var
            return 2
          ;;
        esac  # type
        ;;
      "BREAK" )  # elif [ "$_iv_var_name" = "BREAK" ] ; then
        # ignore BREAK
        _iv_type=""
```

```
        j_msg -7np " of _iv_type: [$_iv_type] ..."
        j_msg -7np "  ignoring BREAK ..."
        right_status 0 "$debug"
        ;;
      * )  # else
        # untyped - initialize null
        _iv_type=""
        _iv_var="$_iv_var_name"
        j_msg -7np " of _iv_type: [$_iv_type] ..."
        j_msg -7np "  (${_iv_type}) initializing ${_iv_var} NULL ..."
        eval "${_iv_var}='' " ; right_status $? "$debug"
        ;;
    esac  # _iv_var_name (fi)
  done
  unset -v _iv_item _iv_parmlist _iv_var_name _iv_type _iv_var
  return 0
}
# @usage initialize_vars [type.var_name] ...
# @args $@ list of variables to initialize, optionally prefixed by type (e.g., bool.VAR)
# @ret 0: success; 1: fail (no args); 2: fail (invalid type)
# @deps j_msg right_status
# @rule type prefixes: bool (set $FALSE), yn (set 'no'), lv/rlv (set '')
# @rule handles untyped variables by initializing them to NULL ('')
# @note uses eval for dynamic variable assignment; handles case-insensitive type prefixes
# @note skips 'verbosity' to prevent accidental reset of script state


display_vars() # (POSIX) show vals for vars in $@; incl bool/lv/; $1=longest
{ dv_ret=0
  if ! isint $1 ; then
    j_msg -3  "Error: arg \$1 must be numeric longest expected input var name"
    exit
  fi
  if [ $# -le 1 ] ; then set $@ "<empty>"; fi  # insert placeholder "<empty>" as $2
#  logFile=/var/tmp/_dv_trace.log
  _dv_maxlen=$(($1 + 2))  # pad longest by 2 "." chars (others will have more dots)
  shift  # drop $1 (length of longest item to display in column)
  # loop through list of variable names and pad each accordingly
  for _dv_var_name in $@;  do
    _dv_type="${_dv_var_name%.*}"   # strip off chars right of and .
    _dv_var="${_dv_var_name#*.}"    # strip off chars left of and .
    _dv_pad_len=$(get_msg_len "$_dv_var")  # strip ansi and then count length
    _dv_pad=" $(repeat '.' $(( _dv_maxlen - _dv_pad_len )) )"
    case "$_dv_var_name" in
      *.* )  # display by type
        case ${_dv_type} in
          [bB]*  )    # bool (bool.)
            eval "_dv_key=\$${_dv_var}"
            j_msg -5 "${BCon}${_dv_var}${_dv_pad}: ${BBon}[ $(status_color ${_dv_key})$(T
rueFalse ${_dv_key}) ${BBon}]${Boff}"
            ;;
          [yY]*  )    # yes/no (yn.)
            eval "_dv_key=\$${_dv_var}"
            j_msg -5 "${BCon}${_dv_var}${_dv_pad}: ${BBon}[ $(status_color ${_dv_key})${_
dv_key} ${BBon}]${Boff}"
            ;;
          [lL]*  )    # long variable (lv.) - wrap to full indent
            eval "_dv_key=\$${_dv_var}"
            # │ * longestvarname ..: [
            # │123              4  5678  (_dv_maxlen already padded for two dots)
            _dv_start_col=$(( _dv_maxlen + 8 )); _dv_indent="${_dv_start_col}"
            j_msg -5n "${BCon}${_dv_var}${_dv_pad}: ${BBon}[ ${Boff}"
            smart_wrap_n "${_dv_start_col}" "${_dv_indent}" "" "${_dv_key}"
            j_msg -p " ${BBon}]${Boff}"  # print trailing ] and newline while respecting
messaging policies
            ;;
          [rR]*  )    # REALLY long variable (rlv.)  - wrap to prefix-only (3 char) inden
t
            eval "_dv_key=\$${_dv_var}"
            # │ * longestvarname ..: [
```

```
            # |123              4  5678  (_dv_maxlen already padded for two dots)
            _dv_start_col=$(( _dv_maxlen + 8 )); _dv_indent=3
            j_msg -5n "${BCon}${_dv_var}${_dv_pad}: ${BBon}[ ${Boff}"
            smart_wrap_n "${_dv_start_col}" "${_dv_indent}" "" "${_dv_key}"
            j_msg -p " ${BBon}]${Boff}"  # print trailing ] and newline while respecting
messaging policies
            ;;
          *      )
            # invalid variable _dv_type
            j_msg -3  "  (${_dv_type}) invalid variable type [${_dv_type}] for variable [
${_dv_var_name}]"
            exit
            ;;
        esac
        ;;
      BREAK) j_msg -p "" ;; # BREAK, print newline while respecting messaging policies
      * ) # untyped
        _dv_type=""
        _dv_var="$_dv_var_name"
        eval "_dv_key=\$${_dv_var}"
        _dv_pad=" $(repeat '.' $(( _dv_maxlen - _dv_pad_len )) )"
        j_msg -5 "${BCon}${_dv_var}${_dv_pad}: ${BBon}[ ${Boff}${_dv_key} ${BBon}]${Boff}
"
        ;;
    esac
  done ; dv_ret=$?
  unset -v _dv_maxlen _dv_var_name _dv_type _dv_var _dv_pad _dv_key _dv_start_col _dv_ind
ent
  return $dv_ret
}
# @usage display_vars $longest "type.var1" "type.var2" ...
# @args $1 integer: length of the longest variable name for column alignment
# @args $2+ list of variable names (optionally typed: bool.VAR, yn.VAR, lv.VAR)
# @ret colorized table of variable states emitted to stdout
# @deps isint get_msg_len repeat TrueFalse status_color j_msg -5 j_msg -5n smart_wrap_n
# @deps _translate_escapes, strip_ansi
# @rule type bool: renders [ True/False ] based on global logic
# @rule type yn: renders [ yes/no ] as raw strings
# @rule type lv: renders [ value ] with word-wrap (smart_wrap_n) to variable indent for l
ong strings
# @rule type rlv: renders [ value ] with word-wrap (smart_wrap_n) to prefix-only (3 char)
 for REALLY long strings
# @note uses eval for dynamic symbol resolution; ensures layout-aware padding via repeat

show_config() # (POSIX) show config with get_longest and display_vars
{ sc_ret=0; j_msg -7 "in show_config" ; separator "${PN}-${BUILD}" "(configuration)"
  _longest=$(get_longest ${varlist}) ; [ ! "$QUIET" ] && display_vars ${_longest} ${varli
st} ; sc_ret=$?
  unset -v _longest; return $sc_ret ; }
# @usage show_config
# @vars varlist (list of variables to display)
# @vars PN, BUILD (for separator title)
# @deps j_msg -5 separator get_longest display_vars
# @rule automatically calculates column width via get_longest

# @note =====[ Section 6: Logging/Messaging & UI/Layout Framework ]=====================
==========================================

# @note -----[ Section 6.a.: Logging/Messaging Engine/Protocol ]-------------------------
-------------------------------------


emit() # (POSIX) consolidated msg engine in subshell isolates positional parameters
( emit_ret=0; mask=$1; shift  # first positiona parameter is the bitmask
  _PRINT_OK="$TRUE"; _LOG_OK="$TRUE"  # default actions
  # extract boolean flags using bitwise shift, AND to read the nth most
  #  significant bit. this is the value extracted from the bit location(s)
  #  pointed to by the glogals of similar name above
  # extract single-bit boolean flags
  _STDOUT=$((      ( mask & STDOUT )      >> STDOUT_OFFSET      ))
```

```
  _STDERR=$((      ( mask & STDERR )      >> STDERR_OFFSET      ))
  _LOG=$((         ( mask & LOG )         >> LOG_OFFSET         ))
  _INTERPRET=$((   ( mask & INTERPRET )   >> INTERPRET_OFFSET   ))
  _NEWLINE=$((     ( mask & NEWLINE )     >> NEWLINE_OFFSET     ))
  _TIMESTAMP=$((   ( mask & TIMESTAMP )   >> TIMESTAMP_OFFSET   ))
  _CONDITIONAL=$(( ( mask & CONDITIONAL ) >> CONDITIONAL_OFFSET ))
  _FORCE=$((       ( mask & FORCE )       >> FORCE_OFFSET       ))
  _SILENCE=$((     ( mask & SILENCE )     >> SILENCE_OFFSET     ))
  _PREMOJI=$((     ( mask & PREMOJI )     >> PREMOJI_OFFSET     ))
  _PREFIX=$((      ( mask & PREFIX )      >> PREFIX_OFFSET      ))
  _PREFIX_BOLD=$(( ( mask & PREFIX_BOLD ) >> PREFIX_BOLD_OFFSET ))
  # extract multi-bit fields (using calculated masks and offsets)
  _PREMOJI_IDX=$(( ( mask & PREMOJI_IDX_MASK )   >> PREMOJI_IDX_OFFSET   ))
  _PREFIX_COLOR=$(( ( mask & PREFIX_COLOR_MASK ) >> PREFIX_COLOR_OFFSET  ))
  _SEVERITY=$((    ( mask & SEVERITY_MASK )      >> SEVERITY_OFFSET      ))
#  args="$@"
#  d_do '
#    echo "mask: $mask"
#    echo "args: $args"
#    echo "_STDOUT: $_STDOUT"
#    echo "_STDERR: $_STDERR"
#    echo "_LOG: $_LOG"
#    echo "_INTERPRET: $_INTERPRET"
#    echo "_NEWLINE: $_NEWLINE"
#    echo "_TIMESTAMP: $_TIMESTAMP"
#    echo "_CONDITIONAL: $_CONDITIONAL"
#    echo "_FORCE: $_FORCE"
#    echo "_SILENCE: $_SILENCE"
#    echo "_PREFIX: $_PREFIX"
#    echo "_PREFIX_BOLD: $_PREFIX_BOLD"
#    echo "_PREFIX_COLOR: $_PREFIX_COLOR"
#    echo "_SEVERITY: $_SEVERITY"
#    echo "_PREMOJI: $_PREMOJI"
#    echo "_PREMOJI_IDX: $_PREMOJI_IDX"
#    echo "verbosity: $verbosity"
#    echo "loglevel: $loglevel"
#    echo "_PRINT_OK: $(TrueFalse $_PRINT_OK)"
#    echo "_LOG_OK: $(TrueFalse $_LOG_OK)"
#  ' "$debug"
  # explicitly disable logging if _LOG bit is not set
  if [ "$_LOG" -eq 0 ]; then _LOG_OK="$FALSE"; fi

  # if _SILENCE bit was set (QUIET is true) and not overridden by INTERACTIVE, then turn
off printing (but not logging)
  if [ "$_SILENCE" -eq 1 ] && [ ! "$INTERACTIVE" ]; then _PRINT_OK="$FALSE"; fi

  # apply verbosity and loglevel rules according to policy
  if [ "$_CONDITIONAL" -eq 1 ]; then
    # compare global verbosity to this message's severity (lower severity number = more i
mportant)
    if [ "$_FORCE" -eq 0 ] && [ "$verbosity" -lt "$_SEVERITY" ]; then _PRINT_OK="$FALSE";
 fi
    # compare global loglevel to this message'sseverity (lower severity number = more imp
ortant)
    if [ "$loglevel" -lt "$_SEVERITY" ]; then _LOG_OK="$FALSE"; fi
  fi

  # early exit if _LOG is requested but logFile is not writable
  if [ "$_LOG" -eq 1 ] && [ ! -w "$logFile" ]; then j_msg -3  "Error: logFile is not writ
able"; exit 1 ; fi

  # non-zero bold/color value means prefix, yes
  # determine prefix color (severity-implied or user override)
  if [ "$_PREFIX_COLOR" -eq 0 ]; then
    # no explicit color provided by caller; so derive it from severity
    case "$_SEVERITY" in
      0|1|2|3) prefix_color="${BRon}"  ;;   # emerg/alert/crit/err .. (red bold)
      4)       prefix_color="${BYon}"  ;;   # warning ............... (yellow bold)
#      5|6)     prefix_color="${Won}"   ;;   # notice/info ........... (white)
      5)       prefix_color="${Gon}"   ;;   # notice ................ (green - joetoo std
```

```
 good to know, normal ops)
      6)       prefix_color="${BGon}"  ;;   # info .................. (green bold - joeto
o std need to know, normal ops)
#      7)       prefix_color="${Con}"   ;;   # debug ................. (cyan)
      7)       prefix_color="${BMon}"  ;;   # debug ................. (magenta bold - joe
too std debug)
    esac
  else
    # caller-specified prefix color
    case "$_PREFIX_COLOR" in
      1) prefix_color="${Bon}"  ;; # blue
      2) prefix_color="${Gon}"  ;; # green
      3) prefix_color="${Con}"  ;; # cyan
      4) prefix_color="${Ron}"  ;; # red
      5) prefix_color="${Mon}"  ;; # magenta
      6) prefix_color="${Yon}"  ;; # yellow
      7) prefix_color="${Won}"  ;; # white
      *) prefix_color=""        ;; # invalid explicit color (no prefix color)
    esac
  fi
  # If _PREFIX=1, keep prefix_color and optionally add bold; if _PREFIX=0, disable prefix
_color entirely
  if [ "$_PREFIX" -eq 1 ]; then
    if [ "$_PREFIX_BOLD" -eq 1 ]; then
      prefix_color="${prefix_color%?};${BOLD}m"
    fi
  else
    prefix_color=""
  fi

  # build timestamp (if enabled)
  if [ "$_TIMESTAMP" -eq 1 ]; then ts="$(timestamp)"; else ts=""; fi

  # build prefix symbol (note that space on right is separation from msg
  # (also note that each is precisely 3 char - for predictable spacing)
  case "$_SEVERITY" in
    0|1|2|3) prefix_symbol="!! " ;; # high severity
    4)       prefix_symbol=" ! " ;;
    5|6)     prefix_symbol=" * " ;; # "normal"
    7)       prefix_symbol=" - " ;; # debug
  esac
#    d_do '
#        echo "Prior to _PREFIX - _LOG - _SEVERITY block"
#        echo "_PREFIX......: [$_PREFIX]"
#        echo "_LOG.........: [$_LOG]"
#        echo "_SEVERITY....: [$_SEVERITY]"
#        severity_mark=$(get_severity "$_SEVERITY")
#        echo "severity_mark: $severity_mark"
#        echo "log_prefix...: $log_prefix"
#    ' "$debug"
    # build prefix symbol (null prefix if color=black) (for logging, append severity mark
ing)
  if [ "$_PREFIX" -eq 1 ]; then
    prefix="${prefix_color}${prefix_symbol}${Boff}"
    if [ "$_LOG" -eq 1 ]; then
      # trim spaces and append severity mark only for clean version of prefix to be used
for logging
      #   remove leading spaces
      junk="${prefix_symbol%%[! ]*}"; log_prefix="${prefix_symbol#"$junk"}"
      #   remove trailing spaces
      junk="${log_prefix##*[! ]}"; log_prefix="${log_prefix%"$junk"}"
      #   get severity mark
      severity_mark=$(get_severity "$_SEVERITY")
      #   assemble ansi-free clean prefix for logging
      log_prefix="[${log_prefix}] (${severity_mark})"
    fi
  else
    prefix=""; log_prefix=""
  fi
```

```
  premoji_str=""  # default: no emoji prefix
  # if no explicit emoji was requested, check if severity implies one
  if [ "$_PREMOJI" -eq 1 ]; then
    premoji_idx=$_PREMOJI_IDX  # start with the explicit bits set in the bitmask
    # yes emoji prefix
    if [ "$premoji_idx" -eq 0 ]; then
      # no user-specified emoji, so severity dictates
      case "$_SEVERITY" in
        0|1)   premoji_idx="$idx_severity_01_emoji" ;;  # Emerg, Alert ,...... cross_mark
        2|3)   premoji_idx="$idx_severity_23_emoji" ;;  # Error, Critical .... explosion
        4)     premoji_idx="$idx_severity_4_emoji" ;;   # Warning ............ no_entry
        5)     premoji_idx="$idx_severity_5_emoji" ;;   # Notice ............. face_think
ing
        6)     premoji_idx="$idx_severity_6_emoji" ;;   # Info ............... face_grin
        7)     premoji_idx="$idx_severity_7_emoji" ;;   # Debug .............. magnifying
_glass_left
      esac
    fi  # index is zero
    # look up the name via the authority; if found, resolve the binary variable
#    echo "looking up name of emoji $premoji_idx"
    _name=$(name_of_emoji "$premoji_idx")
 #   echo "result: $?"
 #   echo "_name: $_name"
    if [ $? -eq 0 ]; then
 #     echo "_name: _name"
      eval "_cooked=\$bin_${_name}"
      if [ "$_cooked" ]; then
        # apply padding: space + emoji + space (4 columns total)
        premoji_str=" ${_cooked} "
      fi
    else
        :
#       echo "name was not found (or name_of_emoji inappropriately exited with status oth
er than 0"
    fi  # name found?
  fi  # PREMOJI

  # prepend timestamp (for LOG only) and space only if there actaully is content in ts
  log_prefix="${ts}${ts:+ }${log_prefix}"
  # build the message body
  msg="$*"  # all remaining positional parameters, after bitmask was shifted
  # build the full screen output string (do for log below)
  out="${premoji_str}${prefix}${msg}"

#  d_do '
#      echo "After _PREFIX - block"
#      echo "_PREFIX......: [$_PREFIX]"
#      echo "_PREMOJI.....: [$_PREMOJI]"
#      echo "premoji_idx..: $premoji_idx"
#      echo "premoji_str..: $premoji_str"
#      echo "_LOG.........: [$_LOG]"
#      echo "_SEVERITY....: [$_SEVERITY]"
##      severity_mark=$(get_severity "$_SEVERITY")
#      echo "severity_mark: $severity_mark"
#      echo "log_prefix...: $log_prefix"
#      echo "out.........: $out"
#  ' "$debug"

  # choose printf format based on _INTERPRET
  if [ "$_INTERPRET" -eq 1 ]; then fmt='%b'; else fmt='%s'; fi
  # modify printf format for newline, if needed
  if [ "$_NEWLINE" -eq 1 ]; then out_fmt="$fmt\n"; else out_fmt="$fmt"; fi

  # emit to stdout (if _STDOUT), with _NEWLINE, if selected and enabled
  if [ "$_PRINT_OK" ] && [ "$_STDOUT" -eq 1 ]; then
    printf "$out_fmt" "$out"
  fi

  # mirror to stderr (if _STDERR) if enabled
  if [ "$_PRINT_OK" ] && [ "$_STDERR" -eq 1 ]; then
```

```
    printf "$out_fmt" "$out" >&2
  fi

  # log to file (if _LOG) - note: I moved strip_ansi() and _translate_escapes into
  #   the filter pipeline now built as SED_LOG_FILTER by _precook_UI_assets (so if a newl
ine is needed
  #   it must be added after SED_LOG_FILTER does its job
  if [ "$_LOG_OK" ]; then
    # tier 3 log filtering optimization (ASCII-only short-circuit)
    # case statement with *[! -~]* did not but *"[! -~]"* did, until I applied LC_ALL=C,
then they both broke
    #   *[^ -~]* also did not work, so I converted to this exit status check on grep -q f
or that pattern, which does work
    # use LC_ALL=C so "[! -~]" sees the raw byte array of output and can interpret bytes
    if printf '%s' "${msg}" | LC_ALL=C grep -q '[^ -~]'; then
        # need filtering because there is at least one non-ascii character present in the
 msg
        # to use sed-in-a-pipeline for log filtering, we need to be careful with line-end
ings
        if [ "$_NEWLINE" -eq 1 ]; then
          # standard line - simple pipe to stream edit in SED_LOG_FILTER
          # then append a newline with the grouping | { cat; printf '\n'; } (cat i/o foll
owed by printf)
          # use LC_ALL=C so the filter sees the raw byte array of output and can interpre
t bytes
          LC_ALL=C printf '%s' "${log_prefix}${log_prefix:+ }${msg}" | \
            LC_ALL=C sed "$SED_LOG_FILTER" | { cat; printf '\n'; } >> "$logFile"
        else
          # partial line - also remove newline, i.e. delete $(printf '\n'), the literal b
yte 0x0A
          # use LC_ALL=C so the filter sees the raw byte array of output and can interpre
t bytes
          LC_ALL=C printf '%s' "${log_prefix}${log_prefix:+ }${msg}" | \
            LC_ALL=C sed "$SED_LOG_FILTER" | tr -d "$(printf '\n')" >> "$logFile"
        fi  # NEWLINE?
    else
        if [ "$_NEWLINE" -eq 1 ]; then
          # standard line - all printable append a newline with the grouping | { cat; pri
ntf '\n'; } (cat i/o followed by printf)
          printf '%s' "${log_prefix}${log_prefix:+ }${msg}" | { cat; printf '\n'; } >> "$
logFile"
        else
          # partial line - also remove newline, i.e. delete $(printf '\n'), the literal b
yte 0x0A
          printf '%s' "${log_prefix}${log_prefix:+ }${msg}" | tr -d "$(printf '\n')" >> "
$logFile"
        fi  # NEWLINE?
    fi  # contains non-prinable?
  fi  # _LOG_OK ?

  # emit body is a subshell, so not doing unset -v (all will be discarded on exit)
)
# @usage emit <bitmask> "message text"
# @args  $1: bitmask selecting stdout/stderr/log, prefix, severity, policy bits
# @args  $*: message string (all remaining args)
# @vars  logFile, verbosity, loglevel, INTERACTIVE; bitmask constants STDOUT..SEVERITY_OF
FSET
# @ret   prints and/or logs formatted message; honors severity and policy bits
# @ret   always returns 0 on successful print or suppression (design contract)
# @deps  timestamp(), get_severity(), SED_LOG_FILTER
# @rule  PRINT and LOG decisions are made independently:
# @rule    _PRINT_OK defaults to $TRUE; suppressed by _SILENCE or verbosity rules
# @rule    _LOG_OK defaults to $TRUE; suppressed by missing LOG bit or loglevel rules
# @rule  POLICY OVERRIDE ORDER (highest->lowest):
# @rule    INTERACTIVE (environment) overrides _SILENCE
# @rule    _SILENCE (QUIET environment) suppresses PRINT regardless of FORCE or severity
# @rule    _FORCE (DEBUG environment) bypasses PRINT verbosity checks (but not loglevel)
# @rule    _CONDITIONAL enables severity filtering for PRINT and LOG
# @rule    (unconditional) âalways prints/logs (severity ignored)
# @rule  $verbosity controls PRINT suppression for _CONDITIONAL messages
```

```
# @rule  $loglevel  controls LOG   suppression for _CONDITIONAL messages
# @rule  unconditional messages (e.g. j_msg -u) bypass both $verbosity and $loglevel chec
ks
# @rule  _STDOUT/_STDERR select screan output streams
# @rule  _NEWLINE appends '\n'
# @rule  _LOG writes ANSI-stripped, escape-normalized text to logFile
# @rule  _TIMESTAMP applies only to log output
# @rule  _PREFIX enables prefix_symbol with severity-implied or caller-specified color
# @rule  _PREFIX_COLOR (3-bits) specifies an RGB color _PREFIX_BOLD adds intensity
# @rule  _PREFIX_COLOR=0 uses severity color; nonzero overrides
# @rule  _PREMOJI enables selection of an emoji prefix
# @rule  _PREMOJI_IDX (6-bits) specifies index number in ACTIVE_EMOJI_REGISTRY
# @rule  _PREMOJI_IDX=0 uses severity to select index number in ACTIVE_EMOJI_REGISTRY
# @rule  _SEVERITY (3 bits) controls prefix symbol, color, and filtering behavior
# @note  emit runs in a subshell to isolate positional parameters and avoid
# @cont  polluting caller scope; all local _FLAG vars are true booleans (1/0)
# @note  PRINT and LOG suppression are independent; suppression is not an error
# @note  callers must construct bitmask correctly; emit does not validate it
# @note  $QUIET (_SILENSE) suppresses PRINT only; logging remains active unless $loglevel
 blocks it
# @note  $DEBUG (_FORCE) bypasses PRINT $verbosity but never bypasses loglevel
# @note  unconditional messages (no _CONDITIONAL bit) always print/log unless SILENCE
# @ex    emit "$((STDOUT|NEWLINE|PREFIX|CONDITIONAL|(info<<SEVERITY_OFFSET)))" "$_msg"

_d2b()
{    _d2b_num=$1
    binary_num=$(printf '%s\n' "scale=10; obase=2; $_d2b_num" | bc)
    printf '%s\n' "$binary_num"
}

_view_mask() # (POSIX) (debug utility to extract and view bitmask bit settings
{ vm_ret=0; _vm_mask=$1  # first positiona parameter is the bitmask
  # extract boolean flags using bitwise shift, AND to read the nth most
  #  significant bit. this is the value extracted from the bit location(s)
  #  pointed to by the glogals of similar name above
  # extract single-bit boolean flags
  _vm_STDOUT=$((       ( _vm_mask & STDOUT )      >> STDOUT_OFFSET ))
  _vm_STDERR=$((       ( _vm_mask & STDERR )      >> STDERR_OFFSET ))
  _vm_LOG=$((          ( _vm_mask & LOG )         >> LOG_OFFSET ))
  _vm_INTERPRET=$((    ( _vm_mask & INTERPRET )   >> INTERPRET_OFFSET ))
  _vm_NEWLINE=$((      ( _vm_mask & NEWLINE )     >> NEWLINE_OFFSET ))
  _vm_TIMESTAMP=$((    ( _vm_mask & TIMESTAMP )   >> TIMESTAMP_OFFSET ))
  _vm_CONDITIONAL=$((  ( _vm_mask & CONDITIONAL ) >> CONDITIONAL_OFFSET ))
  _vm_FORCE=$((        ( _vm_mask & FORCE )       >> FORCE_OFFSET ))
  _vm_SILENCE=$((      ( _vm_mask & SILENCE )     >> SILENCE_OFFSET ))
  _vm_PREMOJI=$((      ( _vm_mask & PREMOJI )     >> PREMOJI_OFFSET ))
  # extract multi-bit fields (using calculated masks and offsets)
  _vm_PREMOJI_IDX=$((  ( _vm_mask & PREMOJI_IDX_MASK )   >> PREMOJI_IDX_OFFSET ))
  _vm_PREFIX=$((       ( _vm_mask & PREFIX )             >> PREFIX_OFFSET ))
  _vm_PREFIX_BOLD=$((  ( _vm_mask & PREFIX_BOLD )        >> PREFIX_BOLD_OFFSET ))
  _vm_PREFIX_COLOR=$(( ( _vm_mask & PREFIX_COLOR_MASK )  >> PREFIX_COLOR_OFFSET ))
  _vm_SEVERITY=$((     ( _vm_mask & SEVERITY_MASK )      >> SEVERITY_OFFSET ))
  _vn_args="$@"
  printf '%s\n' "_vm_mask: $_vm_mask"
  printf '%s\n' "bin mask: $(_d2b "$_vm_mask")"
  printf '%s\n' "_vm_args: $_vm_args"
  printf '%s\n' "_vm_STDOUT: $_vm_STDOUT"
  printf '%s\n' "_vm_STDERR: $_vm_STDERR"
  printf '%s\n' "_vm_LOG: $_vm_LOG"
  printf '%s\n' "_vm_INTERPRET: $_vm_INTERPRET"
  printf '%s\n' "_vm_NEWLINE: $_vm_NEWLINE"
  printf '%s\n' "_vm_TIMESTAMP: $_vm_TIMESTAMP"
  printf '%s\n' "_vm_CONDITIONAL: $_vm_CONDITIONAL"
  printf '%s\n' "_vm_FORCE: $_vm_FORCE"
  printf '%s\n' "_vm_SILENCE: $_vm_SILENCE"
  printf '%s\n' "_vm_PREMOJI: $_vm_PREMOJI"
  printf '%s\n' "_vm_PREMOJI_IDX: $_vm_PREMOJI_IDX"
  printf '%s\n' "_vm_PREFIX: $_vm_PREFIX"
  printf '%s\n' "_vm_PREFIX_BOLD: $_vm_PREFIX_BOLD"
  printf '%s\n' "_vm_PREFIX_COLOR: $_vm_PREFIX_COLOR"
```

```
  printf '%s\n' "_vm_SEVERITY: $_vm_SEVERITY"
  unset -v _vm_STDOUT _vm_STDERR _vm_LOG _vm_INTERPRET _vm_NEWLINE _vm_TIMESTAMP _vm_COND
ITIONAL _vm_FORCE
  unset -v _vm_SILENCE _vm_PREFIX _vm_PREFIX_BOLD _vm_PREFIX_COLOR _vm_SEVERITY _vm_PREMO
JI _vm_mask _vn_args
}

_parse_msg_options() # (POSIX) standardized policy engine with -c color override
{ _pmo_sm_bitmask="$1"; shift; _pmo_shift=0;    _pmo_premoji_default=0
  # initialize the premoji defaults based on environment variable capable/disabled state
  [ $EMOJI_CAPABLE ] && [ ! $EMOJI_DISABLE ] && _pmo_premoji_default=$PREMOJI
  # apply the default to the starting bitmask
  _pmo_sm_bitmask=$(( _pmo_sm_bitmask | _pmo_premoji_default ))
  # iterate through args and act on options
  while [ $# -gt 0 ]; do
    case "$1" in
      --) _pmo_shift=$(( _pmo_shift + 1 )); break ;;
      -[0-7a-zA-Z]*)                                          # handle compound ar
gument like -5Ben
          _pmo_cluster="${1#-}"                               # strip '-'
          while [ -n "$_pmo_cluster" ]; do                    # iterate chars fo c
ompound arg
            _pmo_char=${_pmo_cluster%"${_pmo_cluster#?}"}     # posix forkless lef
t_most char
            _pmo_cluster=${_pmo_cluster#?}                    # posix forkless rem
ainder
            case "$_pmo_char" in
              h) unset -v _pmo_sm_bitmask _pmo_lm_bitmask _pmo_shift;
                 return 100 ;;    # trigger caller to emit a one-line help/usage message
              [0-7])                                          # severity policy
                  _pmo_lvl="$_pmo_char"                       # assign severity an
d CLEAR the bitmask
                  _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~(SEVERITY_MASK | STDOUT | STDERR
 | \
                                      PREFIX | PREFIX_BOLD | CONDITIONAL | PREFIX_COLOR_MASK
) ))
                  case "$_pmo_lvl" in                         # set bitmask parts
per sevrity policy
                    0)     _pmo_sm_dest=$STDERR; _pmo_sm_bold=$PREFIX_BOLD; _pmo_sm_cond=
0;            _pmo_sm_col=$RGB_M ;;
                    1)     _pmo_sm_dest=$STDERR; _pmo_sm_bold=0;            _pmo_sm_cond=
0;            _pmo_sm_col=$RGB_M ;;
                    2)     _pmo_sm_dest=$STDERR; _pmo_sm_bold=$PREFIX_BOLD; _pmo_sm_cond=
0;            _pmo_sm_col=$RGB_R ;;
                    3)     _pmo_sm_dest=$STDERR; _pmo_sm_bold=0;            _pmo_sm_cond=
0;            _pmo_sm_col=$RGB_R ;;
                    4)     _pmo_sm_dest=$STDERR; _pmo_sm_bold=$PREFIX_BOLD; _pmo_sm_cond=
0;            _pmo_sm_col=$RGB_Y ;;
                    5)     _pmo_sm_dest=$STDOUT; _pmo_sm_bold=$PREFIX_BOLD; _pmo_sm_cond=
$CONDITIONAL; _pmo_sm_col=$RGB_G ;;
                    6)     _pmo_sm_dest=$STDOUT; _pmo_sm_bold=0;            _pmo_sm_cond=
$CONDITIONAL; _pmo_sm_col=$RGB_G ;;
                    7)     _pmo_sm_dest=$STDOUT; _pmo_sm_bold=$PREFIX_BOLD; _pmo_sm_cond=
$CONDITIONAL; _pmo_sm_col=$RGB_C ;;
                  esac
                  # now update the sm bitmask itself *** PREFIX, _pmo_premoji_default ***
 set, too (implied) for sm only
                  _pmo_sm_bitmask=$(( _pmo_sm_bitmask | _pmo_sm_dest | PREFIX | _pmo_prem
oji_default | _pmo_sm_bold | _pmo_sm_cond \
                                      | (_pmo_sm_col << PREFIX_COLOR_OFFSET) | ((_pmo_lvl &
SEVERITY_BASE_MASK) << SEVERITY_OFFSET))) ;;
              C) # color override: expects arg 3-bit RGB (0-7)
                 if [ -z "$_pmo_cluster" ]; then
                   _pmo_val="$2"; shift; _pmo_shift=$(( _pmo_shift + 1 ))
                 else
                   _pmo_val="$_pmo_cluster"; _pmo_cluster=""
                 fi
                 # clear existing 3-bit color field and then set with new value
                 _pmo_sm_bitmask=$(( (_pmo_sm_bitmask & ~PREFIX_COLOR_MASK) | ((_pmo_val
& PREFIX_COLOR_BASE_MASK) << PREFIX_COLOR_OFFSET) )) ;;
```

```
              c) _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~PREFIX_COLOR_MASK )) ;;  # CLEAR
color to zero (no argument)
              B) _pmo_sm_bitmask=$(( _pmo_sm_bitmask | PREFIX_BOLD )) ;;    # set bold (d
ont need to clear b/c ( 1 | 1 = 1 )[N/A for lm bitmask]
              b) _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~PREFIX_BOLD )) ;;   # clear binar
y PREFIX_BOLD switch (disable bold prefix)
              e) _pmo_sm_bitmask=$(( _pmo_sm_bitmask | INTERPRET )) ;;      # interpret (
like echo -e) [N/A for lm bitmask]
              E) _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~INTERPRET )) ;;     # force-overr
ide INTERPRET to OFF (disable backslash interpretation)
              N) _pmo_sm_bitmask=$(( _pmo_sm_bitmask | NEWLINE )) ;;        # force-overr
ide NEWLINE to ON
              n) _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~NEWLINE )) ;;       # -n meals cl
ear NEWLINE (like echo -n) [N/A for lm bitmask]
              P) _pmo_sm_bitmask=$(( _pmo_sm_bitmask | PREFIX )) ;;         # set PREFIX
(*, !, !! per severity) [N/A for lm bitmask]
              p) _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~PREFIX )) ;;        # clear PREFI
X [N/A for lm bitmask]
              u) # "set unconditional" = clear CONDITIONAL and severity-related bits [N/A
 for lm bitmask]
                 _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~(PREFIX | PREFIX_BOLD | SEVERITY_
MASK | CONDITIONAL | PREFIX_COLOR_MASK) )) ;;
              T) _pmo_sm_bitmask=$(( _pmo_sm_bitmask | TIMESTAMP )) ;;      # T = timesta
mp ON [N/A for lm bitmask]
              t) _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~TIMESTAMP )) ;;     # t = timesta
mp OFF [N/A for lm bitmask]
              M) # override PREFIX with selected emoji: set binary PREMOJI switch and 5-b
it index (0-63) [shift for arg] [N/A for lm bitmask]
                if [ -z "$_pmo_cluster" ]; then
                  _pmo_idx="$2"; shift; _pmo_shift=$(( _pmo_shift + 1 ))
                else
                  _pmo_idx="$_pmo_cluster"; _pmo_cluster=""
                fi
                # clear both PREMOJI and existing 5-bits
                _pmo_sm_bitmask=$(( ( _pmo_sm_bitmask & ~( PREMOJI | PREMOJI_IDX_MASK ) )
 ))
                # SET both with bitwise OR PREMOJI and shifted Index
                _pmo_sm_bitmask=$(( _pmo_sm_bitmask | PREMOJI | ((_pmo_idx & PREMOJI_IDX_
BASE_MASK) << PREMOJI_IDX_OFFSET) )) ;;
              m) # explicit opt-out to disable emoji prefix
                _pmo_premoji_default=0                                      # kill the de
fault so severity logic doesn't re-enable it
                _pmo_sm_bitmask=$(( _pmo_sm_bitmask & ~PREMOJI )) ;;        # clear binar
y PREMOJI switch (disable emoji prefix)
            esac
          done ;;
      -*) die "unknown option: $1" ;;
      *)  break ;;
    esac
    _pmo_shift=$(( _pmo_shift + 1 ))
    shift
  done
  pmo_sm_bitmask=$_pmo_sm_bitmask; pmo_shift=$_pmo_shift
  unset -v _pmo_sm_bitmask _pmo_shift _pmo_lvl _pmo_idx _pmo_cluster _pmo_char _pmo_sm_de
st
  unset -v _pmo_sm_bold _pmo_sm_cond _pmo_sm_col _pmo_val _pmo_premoji_default
}
# @usage _parse_msg_options <initial_bitmask> [options...]
# @opts -[0-7] severity; -C <0-7>/c color; -B/b bold; -M <0-63>/m emoji; -T/t timestamp
# @cont -P/p prefix; -e/E interpret; -N/n newline; -u unconditional; -h help
# @args 1: initial_bitmask; 2+: options (supports clustering)
# @ret 0: success, 100: help request; sets pmo_sm_bitmask/pmo_shift
# @vars pmo_sm_bitmask pmo_shift (modified); EMOJI_CAPABLE EMOJI_DISABLE (req)
# @deps die
# @rule consumes options until '--' or non-option; 'h' triggers immediate return 100
# @note internal logic engine for j_msg policy enforcement

j_msg() # (POSIX) universal messaging entry point
{ jm_ret=0
  # set default bitmasks (default to notice severity, but let _parse_msg_options
```

```
  #   update that and decide about NEWLINE, etc per policy

  # standard mask  (sm) is for short messages (sm) only
  _jm_sm_base=$(( STDOUT | INTERPRET | PREFIX | PREFIX_BOLD | CONDITIONAL | NEWLINE
                | PREMOJI | ( $notice << SEVERITY_OFFSET ) ))
  # long messages cannot go to emit for screen print (use smart_wrap_n for that)
  # so the long message (lm) bitmast does NOT use STDERR or STDOUT destinations
  # and the three-step process used to print long messages starts with using
  # emit to send a prefix to both screen and log, so the second step, where the
  # lm bitmask is used, does NOT include PREFIX/BOLD/COLOR, also b/c it is for the
  # log only,the lm bitmask does NOT include INTERPRET (logs are sanitized)
  _jm_lm_bitmask=$(( CONDITIONAL | ( $notice << SEVERITY_OFFSET ) ))
  # hand off to helper _parse_msg_options
  #   sets bitmask according to severity policy and options set by caller
  #   sets global pmo_bitmask and pmo_shift
  #   returns 100 if option -h received
  _parse_msg_options "$_jm_sm_base" "$@"
  case $? in   # handle signal that option -h was received
    100)
      printf 'usage: j_msg [-0-7] [-C idx|c] [-B|b] [-M idx|m] [-e|E] [-N|n] [-p|P] [-t|T
] [-u] [-h] [--] <message>\n'
      printf '  -0-7 ......: set severity (0:emerg, 1:alert, 2:crit, 3:err, 4:warn, 5:not
e, 6:info, 7:debug)\n'
      printf '  -C <0-7>|c : set/override prefix color index (0-7) or clear (c)\n'
      printf '  -B | -b ...: force (B) or suppress (b) bold prefix\n'
      printf '  -M <0-63>|m: override prefix with emoji index (0-63) or disable emoji (m)
\n'
      printf '  -e | -E ...: enable (e) or disable (E) backslash escape interpretation\n'
      printf '  -N | -n ...: force newline (N) or suppress trailing newline (n)\n'
      printf '  -p | -P ...: suppress (p) or force (P) standard UI prefix symbol\n'
      printf '  -t | -T ...: suppress (t) or force (T) log timestamp\n'
      printf '  -u ........: unconditional (clear prefix/severity/timestamp/color/gating)
\n'
      printf '  -- ........: end of options; treat all subsequent args as message text\n'
      printf '  -h ........: print this help message\n'
      return 0
      ;;
  esac
  # drop the number of arguments _parse_msg_options says were options
  shift "$pmo_shift"
  # set the bitmast returned by _parse_msg_options
  _jm_sm_bitmask=$pmo_sm_bitmask;
  # rotate-scrub arguments (message fragments) to remove any empty ones
  _jm_i=$#; while [ "$_jm_i" -gt 0 ]; do [ -n "$1" ] && set -- "$@" "$1"; shift; _jm_i=$(
(_jm_i - 1)); done
  # combine remainin#   usage: _parse_msg_options <initial_bitmask> [options...]
  _jm_msg="$*"
  # apply stage of global environment variables
  [ "$LOGGING" ] && _jm_sm_bitmask=$(( _jm_sm_bitmask | LOG | TIMESTAMP ))
  [ "$DEBUG" ]   && _jm_sm_bitmask=$(( _jm_sm_bitmask | FORCE ))
  [ "$QUIET" ]   && _jm_sm_bitmask=$(( _jm_sm_bitmask | SILENCE ))

  # If NEWLINE is not set (messaging like echo -n) and *** if length merits it ***, then
smart-wrap
  #   screen output, and ensure there is room for a right_stats on the right side.  Since
 smart_wrap_n
  #   is not needed for log output this is one instance where bifurcated messaging is app
ropriate - use
  #   a special "long message" bitmask to have emit send only to the logFile,  and separa
tely, use
  #   smart_wrap_n for screen output - but since smart_wrap_n has no business handling bi
tmasks and
  #   messaging severity policy, etc, use '_maybe_smart_wrap_n' to determine if policy/en
vironment
  #   conditions indicate this should happen (i.e. send bitmas to _maybe_smart_wrap_n and
 check
  #   its exit status)
  if [ $(( _jm_sm_bitmask & NEWLINE )) -eq 0 ]; then
    _jm_len=$(get_msg_len "$_jm_msg")    # get_msg_len is a unicode-aware message-width c
alculator
```

```
    _jm_tw=$(termwidth)                   # termwidth checks COLUMNS, tput, stty or falls
back to 80
    _jm_prefix_width=0                    # smart_wrap_n will be instructed use this as l_
margin
    # extract PREFIX bit to determine if its standard 3-col width is included
    if [ $(( _jm_sm_bitmask & PREFIX )) -ne 0 ]; then
      _jm_prefix_width=$(( _jm_prefix_width + 3 ))   # e.g. " * " or "!! "
    fi
    # extract PREMOJI_IDX bits to determine if a padded emoji width is included
    _jm_premoji_val=$(( (_jm_sm_bitmask >> PREMOJI_IDX_OFFSET) & PREMOJI_IDX_BASE_MASK ))
    if [ "$_jm_premoji_val" -gt 0 ]; then
      _jm_prefix_width=$(( _jm_prefix_width + 4 ))      # space (1) + emoji (2) + space (
1) = 4
    fi
    _jm_status_width=6                    # this is the width of "[ Ok ]", "[ !! ]", or em
oji equiv, since emoji=2
    _jm_buffer=4                          # this is just a "safety" buffer to ensure there
 is wrap-room
    # determine how long a one-line message can be and still have room for right_status
    _jm_threshold=$(( _jm_tw - ( _jm_prefix_width + _jm_status_width + _jm_buffer ) ))
    if [ "$_jm_len" -gt "$_jm_threshold" ] && [ "$_jm_threshold" -gt 0 ]; then
      # LONG message path - this message would not fit and still have room for right stat
us so - 3 steps
      # (1) use regular bitmast to have emit print a prefix (both log and screen)
      emit "$_jm_sm_bitmask" ""
      # (2a) after _parse_options, update _jm_lm_bitmask
      # first clear severity, then read it from the sm bitmask and assign it to lm bitmas
k
      _jm_lm_bitmask=$(( _jm_lm_bitmask & ~SEVERITY_MASK ))  # clear severity bits
      _jm_SEVERITY=$(( (_jm_sm_bitmask & SEVERITY_MASK) >> SEVERITY_OFFSET ))  # read sev
erity from sm bitmask
      _jm_lm_bitmask=$(( _jm_lm_bitmask | (_jm_SEVERITY << SEVERITY_OFFSET) )) # set new
severity
      [ "$LOGGING" ] && _jm_lm_bitmask=$(( _jm_lm_bitmask | LOG  ))
      [ "$DEBUG" ]   && _jm_lm_bitmask=$(( _jm_lm_bitmask | FORCE ))
      [ "$QUIET" ]   && _jm_lm_bitmask=$(( _jm_lm_bitmask | SILENCE ))
      # (2) use the new lm bitmast to have emit print the message to the log (emit applie
s policy to determine yes/no)
      emit "$_jm_lm_bitmask" "$_jm_msg"
      # (3) use the regular bitmask to ask _maybe_smart_wrap_n (y/n?) is it ok to have sm
art_wrap_n
      #     output to screen? if yes, then semd tp smart_wrap_n, using prefix width for m
argins
      if _maybe_smart_wrap_n "$_jm_sm_bitmask"; then smart_wrap_n "$_jm_prefix_width" "$_
jm_prefix_width" "$_jm_prefix_width" "$_jm_msg"; fi
    else
      # SHORT message path - this message will fit and still have room for right status,
so just have emit handle it
      emit "$_jm_sm_bitmask" "$_jm_msg"
    fi
  else
    # NEWLINE is set so don't need to consider whether to smart_wrap_n - just have emit h
andle it
    emit "$_jm_sm_bitmask" "$_jm_msg"
  fi
  jm_ret=$?
  unset -v _jm_base _jm_sm_bitmask _jm_lm_bitmask _jm_msg _jm_len _jm_tw _jm_threshold
  unset -v _jm_prefix_width _jm_status_width _jm_buffer
  return $jm_ret
}
# @usage j_msg [-0-7] [-C idx|c] [-B|b] [-M idx|m] [-e|E] [-N|n] [-p|P] [-t|T] [-u] [-h]
[--] <message>
# @args $1: (optional) clustered or discrete bitmask override options
# @args $2+: message string (fragments are joined)
# @opts -0..7: severity policy; -C <0-7>/c: color override; -B/b: bold override
# @opts -M <0-63>/m: emoji index override; -e/E: interpret escapes; -N/n: newline policy
# @opts -p/P: prefix policy; -t/T: timestamp policy; -u: unconditional; -h: usage
# @ret 0 on success, 100 for help, or return code of emit
# @deps _parse_msg_options, emit, get_msg_len, termwidth, smart_wrap_n, _maybe_smart_wrap
_n
```

```
# @vars pmo_bitmask, pmo_shift, LOGGING, DEBUG, QUIET
# @rule severity levels [0-7] establish base policy for gate, destination, color, and pre
fix
# @rule lowercase modifiers (c, b, m, n, p, t) allow subtractive overrides of severity de
faults
# @rule uppercase modifiers (C, B, M, N, P, T) allow explicit enabling or overriding of f
eatures
# @rule lower/uppercase e/E is an exception -e enables interpret (like echo -e) -E turns
it off
# @rule if -n is set and message exceeds threshold, output is split:
# @cont   header(emit) -> body(log:emit) -> body(screen:smart_wrap_n)
# @cont   * this is a separation of concerns issue: j_msg is the brain/policy engine
# @cont   * and emit remains a high-performance, low-latency stream-pusher
# @note handles policy-based messaging with automatic terminal wrapping and log sanitizat
ion

# @note -----[ Section 6.b.: UI/Layout tool ]-------------------------------------------
------------------
# @note UI = conceptual user-facing interaction │ Layout = physical arrangement of output

# @note the _msg_n family has two main use cases in layout: (1) print just a prefix
# @note (j_msg -5n ") and print a line-openint for "announce/act/report-result" idiom
# @note right_status and handle_result serve the "report-result" role
# @note right_status and handle_result operate at notice level of severity
# @note but they can be "demoted" to info or debg severity with d_do

_maybe_smart_wrap_n() # (POSIX) logic similar to emit; decide whether a caller should nex
t call smart_wrap_n
( mask=$1; shift  # first positiona parameter is the bitmask
  # extract boolean flags using bitwise shift, AND to read the nth most significant bit
  # this is the value extracted from the bit location(s) pointed to by the glogals of sim
ilar name above
  # _STDOUT, _STDERR, _LOG, _INTERPRET, _NEWLINE, _TIMESTAMP bits are N/A to this decisio
n
  # _PREFIX, _PRIFIX_BOLD, and _PREFIX_COLOR are N/A to this decision
  _CONDITIONAL=$(( ( mask >> CONDITIONAL_OFFSET ) & 1 ))
  _FORCE=$((       ( mask >> FORCE_OFFSET       ) & 1 ))
  _SILENCE=$((     ( mask >> SILENCE_OFFSET     ) & 1 ))
  _SEVERITY=$((    ( mask >> SEVERITY_OFFSET    ) & SEVERITY_BASE_MASK ))
#  d_do '
#    echo "mask: $mask"
#    echo "_CONDITIONAL: $_CONDITIONAL"
#    echo "_FORCE: $_FORCE"
#    echo "_SILENCE: $_SILENCE"
#    echo "_SEVERITY: $_SEVERITY"
#    echo "verbosity: $verbosity"
#    echo "loglevel: N/A for smart-wrapping"
#  ' "$debug"
  # INTERACTIVE mode overrides SILENCE
  [ "$INTERACTIVE" ] && _SILENCE=0
  # otherwise early exit if _SILENCE is set
  [ "$_SILENCE" -eq 1 ] && exit 1
  # apply verbosity rules (unless _FORCE)
  if [ "$_CONDITIONAL" -eq 1 ] && [ "$_FORCE" -eq 0 ]; then
    # compare severity to global verbosity (lower severity number = more important)
    if [ "$_SEVERITY" -gt "$verbosity" ]; then
      exit 2
    fi
  fi
  # this function's body is a subshell, so not doing unset -v (all will be discarded on e
xit)
  # if we made it here, then we dont have a reason NOT to smart_wrap_n
  # note: like emit, this functions body is a subshell,
  #   so unsets are not needed and exit will not terminate the parent
  exit 0
)
# @usage _maybe_smart_wrap_n <bitmask> [args...]
# @args $1: bitmask constructed by caller (policy + severity)
# @args $@: optional args passed through to smart_wrap_n by caller
# @vars verbosity, INTERACTIVE (global policy controls)
```

```
# @ret status: 0 = caller SHOULD invoke smart_wrap_n; non-zero = skip it
# @deps emit semantics (policy mirror), global bitmask constants
# @req caller must supply a valid bitmask with severity and policy bits set
# @rule mirrors emit policy for terminal output: INTERACTIVE overrides
# @rule SILENCE suppresses wrapping unless INTERACTIVE is set
# @rule CONDITIONAL obeys verbosity unless FORCE is set
# @note function body runs in a subshell; exit codes do not affect caller
# @note smart_wrap_n is NOT called here; caller must invoke it on success
# @warn only evaluates terminal-output policy; logging policy is ignored

right_status()  # (POSIX) right-margin indicator of status $1, per severity $2
{ rs_ret=0; _rs_CUF_OK="$TRUE"
  # guard - reject zero args
  [ $# -lt 1 ] && { j_msg -3  "Error: right_status requires integer status code"; return
1; }
  _rs_status=$1; _rs_lvl=${2:-$notice}  # default to notice severity
#echo "DEBUG TRIPWIRE: right_status received arg1: [$1]" >&2
#isint "$1" || echo "DEBUG: CRITICAL - $1 IS NOT AN INTEGER" >&2
  # choose ascii or emoji messages per policy - use _rs_width to know how far from right
margin to print status
  # presently _rs_width=7 in all cases [ Ok ], [ !! ], [ <2-col-emoju> ] are all 6 cols
  #   plus 1 col to ensure a character ON the right margin does now result it terminal ad
ding extra newline
  if [ "$_rs_status" -eq 0 ]; then
    if [ "$EMOJI_CAPABLE" ] && [ ! "$EMOJI_DISABLE" ]; then
      _rs_msg="${BBon}[ ${Boff}$(emoji_safe "$check_mark_button") ${BBon}]${Boff}"
      _rs_width=7
    else
      _rs_msg="${BBon}[ ${BGon}Ok ${BBon}]${Boff}"
      _rs_width=7
    fi
  else
    if [ "$EMOJI_CAPABLE" ] && [ ! "$EMOJI_DISABLE" ]; then
      _rs_msg="${BBon}[ ${Boff}$(emoji_safe "$cross_mark") ${BBon}]${Boff}"
      _rs_width=7
    else
      _rs_msg="${BBon}[ ${BGon}!! ${BBon}]${Boff}"
      _rs_width=7
    fi
  fi
  # get termwidth and calculate status position
  _rs_tw=$(termwidth); isint "$_rs_tw" || _rs_tw=80 # default to 80 just in case {though
termwidth() should do that anyway}
  _rs_move=$(( _rs_tw - _rs_width ))
  [ "$_rs_move" -lt 0 ] && _rs_move=0
  # apply silence policy to cursor movement like emit will do for printing
  if [ "$QUIET" ] && [ ! "$INTERACTIVE" ]; then _rs_CUF_OK="$FALSE"; fi
  # apply severity handling to cursor movement like emit will do for printing
  if [ "$_rs_CUF_OK" ] ; then
    # use the passed severity level to gate the d_do logic
    d_do '
      #move the cursor. absolute start at left edge, then fwd to calculated position; uns
et and return if error
      printf "%b" "\r"
      CUF "$_rs_move" || { unset -v _rs_status _rs_lvl _rs_msg _rs_tw _rs_width _rs_move
_rs_CUF_OK; return 1; }
    ' "$_rs_lvl"
  fi
  # cannot just use j_msg to build bitmask and get emit to print status
  #   b/c policy dictates j_msg applies TIMESTAMP in logging
  # so right_status must build its own bitmask for emit (no PREFIX, no TIMESTAMP, YES NEW
LINE)
  # use the severity-appropriate destination (STDOUT/STDERR)
  [ "$_rs_lvl" -le 4 ] && _rs_dest=$STDERR || _rs_dest=$STDOUT
  # load bitmask as described above
  _rs_bitmask=$(( _rs_dest | NEWLINE | CONDITIONAL | ( _rs_lvl << SEVERITY_OFFSET ) ))
  # apply global envinoment to bitmask so emit can apply policies (standard j_msg logic,
but NO TIMESTAMP)
  [ "$LOGGING" ] && _rs_bitmask=$(( _rs_bitmask | LOG ))
  [ "$DEBUG" ]   && _rs_bitmask=$(( _rs_bitmask | FORCE ))
```

```
  [ "$QUIET" ]   && _rs_bitmask=$(( _rs_bitmask | SILENCE ))
  # call emit (not j_msg)
  emit "$_rs_bitmask" "$_rs_msg"; rs_ret=$?
  unset -v _rs_status _rs_lvl _rs_msg _rs_tw _rs_width _rs_move _rs_CUF_OK _rs_bitmask _r
s_dest; return $rs_ret ; }
# @usage right_status <status_code> [severity_level] || <fail-action>
# @args $1 integer status code (0=Ok, nonzero=failure); $2 severity level (default=notice
)
# @vars LOGGING, DEBUG, QUIET, EMOJI_CAPABLE, EMOJI_DISABLE
# @deps j_msg emit termwidth CUF isint emoji_safe d_do
# @ret 0 on success; 1 on CUF failure
# @rule severity-aware reporting; output suppressed if verbosity < level unless FORCE is
set
# @rule prints to stdout/stderr; uses emit directly to manage bitmask and avoid j_msg pre
fixing
# @cont preserves context-aware logging: e.g. j_msg -5n "doing work"; right_status $? 5
# @rule cursor movement handled via '\r' and CUF, gated by d_do with passed severity
# @note acts as a surveillance proxy: reports observed results of previous commands
# @ex j_msg -6n "creating file"; sudo touch /tmp/file; right_status $? 6
# @note right_status is a canonical UI primitive for the "report-result" step of the
# @cont announce/act/report idiom.

handle_result()  # (POSIX) if status $1 is 0, descriptive result $2 else $3; then right_s
tatus
{ hr_ret=0; _hr_status="$1"; _hr_ok_msg_in="$2"; _hr_err_msg_in="$3"
  _hr_lvl=${4:-$notice}  # default to notice severity
  # guard: reject invalid status code
  if ! isint "$_hr_status" ; then j_msg -3 "error: handle_result requires \$1 to be integ
er exit_status"; return 1; fi
  # pick case and construct the appropriate message
  case "$_hr_status" in
    0 )
      if [ "$_hr_ok_msg_in" = "${no_msg}" ] ; then _hr_msg=""; else _hr_msg=" (${BGon}${_
hr_ok_msg_in:-success!}${Boff})"; fi
      ;;
    * )
      if [ "$_hr_err_msg_in" = "${no_msg}" ] ; then _hr_msg=""; else _hr_msg=" (${BRon}${
_hr_err_msg_in:-error!}${Boff})"; fi
      ;;
  esac
  # cannot just use j_msg to build bitmask and get emit to print status
  #   b/c policy dictates j_msg applies PREFIX when asked to set severity
  # so handle_result must build its own bitmask for emit (no PREFIX, no TIMESTAMP, CONDIT
IONAL, SEVERITY)
  # use the severity-appropriate destination (STDOUT/STDERR)
  # IMPORTANT: No NEWLINE (right_status does that), No PREFIX, No TIMESTAMP
  [ "$_hr_lvl" -le 4 ] && _hr_dest=$STDERR || _hr_dest=$STDOUT
  # load bitmask as described above
  _hr_bitmask=$(( _hr_dest | INTERPRET | CONDITIONAL | ( _hr_lvl << SEVERITY_OFFSET ) ))
  # apply global envinoment to bitmask so emit can apply policies (standard j_msg logic,
but NO TIMESTAMP)
  [ "$LOGGING" ] && _hr_bitmask=$(( _hr_bitmask | LOG ))
  [ "$DEBUG" ]   && _hr_bitmask=$(( _hr_bitmask | FORCE ))
  [ "$QUIET" ]   && _hr_bitmask=$(( _hr_bitmask | SILENCE ))
  # call emit (not j_msg) to emit the descriptive text (if any)
  [ ! -z "$_hr_msg" ] && emit "$_hr_bitmask" "$_hr_msg"
  # now call right_status to render status symbol and close the line on screen and in log
, as applicable
  right_status "$_hr_status" "$_hr_lvl"; hr_ret=$?
  unset -v _rs_status _rs_lvl _rs_msg _rs_tw _rs_width _rs_move _rs_CUF_OK _rs_bitmask _r
s_dest; return $rs_ret ; }
# @usage handle_result <status> <ok_msg> <err_msg> [severity] || <fail-action>
# @args $1 exit status; $2 success text; $3 error text; $4 severity (default=notice)
# @vars LOGGING, DEBUG, QUIET, no_msg
# @deps j_msg emit right_status isint
# @ret returns exit status from right_status ($hr_ret)
# @note severity-aware; uses emit with custom bitmask to append text without timestamp, p
refix, or newline
# @note only constructs the message string for the branch matching the status
# @note preserves context-aware logging: j_msg -5n "task"; handle_result $? "done" "faile
```

```
d" 5
# @note prints inline descriptive text, then calls right_status to close the line
# @ex j_msg -5n "syncing data"; do_sync; handle_result $? "ok" "failed" 5
# @note handle_result is a canonical UI primitive for the "report-result" step of the
# @cont announce/act/report idiom.

# @note -----[ Section 6.c.: Unicode/Emoji UI/Layout tools ]-----------------------------
-------------------------

# is_utf8_terminal moved to top of header to run befure needed in default setting
# emit_utf8_from_hex moved to early in header to participate in precook prep of terminal
io

u_echo() # (POSIX) Wrapper for emit_utf8_from_hex to render multiple codepoints
{ ue_ret=0
  for _ue_h in "$@"; do
    # remove '\U' or '0x' prefixes if present
    _ue_clean="${_ue_h#\\U}"
    _ue_clean="${_ue_clean#0x}"
    # generate octal and print via %b
    _ue_oct=$(emit_utf8_from_hex "$_ue_clean")
    LC_CTYPE=en_US.utf8 printf '%b' "$_ue_oct" || ue_ret=1
  done
  unset -v _ue_h _ue_clean _ue_oct
  return $ue_ret
}
# @usage u_echo <hex_codepoint> [hex_codepoint2 ...]
# @args $@: one or more hex codepoints, with or without '\U' or '0x' prefixes
# @vars ue_ret: localized exit status variable
# @ret emits binary UTF-8 characters to stdout
# @deps emit_utf8_from_hex, printf

# emoji_safe moved to the front of the header after _precook_ui_assets so it is
# available to initialize .e.g longone

bremoji() # (POSIX) use emoji()/uecho() to display one unicode char $1 in brackets
{ [ ! $# -eq 1 ] && die "one arg allowed/required"
  printf "["; emoji_safe "${1}"; printf "]" ; }
# @usage bremoji <hex_codepoint>
# @args $1: hex codepoint for a single emoji
# @ret emits bracketed emoji (e.g., [ð\237\220§]) to stdout
# @deps emoji_safe, printf

bremojis() # (POSIX) use emoji()/uecho() to display chars $@ in brackets
{ printf "["; for each in $@; do emoji_safe "${each}"; done; printf "]" ; }
# @usage bremojis <hex_codepoint> [hex_codepoint2 ...]
# @args $@: sequence of hex codepoints
# @ret emits bracketed sequence of emojis (e.g., [ð\237\220§ð\237\232\200]) to stdout
# @deps emoji_safe, printf


bremojis2() # (POSIX) use emoji()/uecho() to display chars $@ in brackets
{ _bs_payload=""; for each in "$@"; do
  _bs_oct=$(emit_utf8_from_hex "$each")
  _bs_payload="${_bs_payload}${_bs_oct}"
  done
  printf "[ %b ]" "${_bs_payload}"
  unset -v _bs_oct _bs_payload
 }
# @usage bremojis2 <hex_codepoint> [hex_codepoint2 ...]
# @args $@: sequence of hex codepoints
# @ret emits padded bracketed sequence (e.g., [ ð\237\220§ð\237\232\200 ]) to stdout
# @rule aggregates all octal escapes into a single variable for one printf call
# @note high-performance alternative to bremojis via I/O reduction
# @deps emit_utf8_from_hex, printf

#-----[ these need to be refactored and integrated with severity-level messaging ] ------
---
#       (or perhaps absorbed into emit and the existing framework )
```

```
#u_message() {   # use bremoji() and message() to prefix message() with unicode char $1
#[ ! $# -ge 2 ] && die "usage: <unicodeprefix> <message>"
#prefix=$1 ; shift ; bremoji $prefix ; message "$@" ; }

#uE_message() {   # use bremojis() and E_message() to prefix message() with unicode char
$1
#[ ! $# -ge 2 ] && die "usage: <unicodeprefix> <error_message>"
#prefix=$1 ; shift ; bremojis $prefix ; E_message "$@" ; }

#uEx_message() {   # use bremojis() and E_message() to prefix message() with unicode expl
osion
#[ ! $# -ge 1 ] && die "usage: <error_message>"
#prefix=${explosion} ; bremojis ${prefix} ; E_message "$@" ; }

#summarize_my_unicode() { # list and demo functions in header ("source" to employ)
#grep --color=auto "()" ${script_header_installed_dir%/}/script_header_joetoo_unicode | \
#    grep --color=auto -v "grep" | sed 's|^|  |';
## demonstrate u_message
#echo ; message "running ${BGon}u_message ${BMon}"'${hammer}'"${Boff} hello, world"
#u_message ${hammer} "hello, world"
## demonsgrate bremojis
#echo; msg="running ${BGon} bremojis ${BMon}"' ${explosion} ${face_grimace}'
#msg+=" ; ${BGon}E_message ${Boff}\"I think we have a problem here\""
#message "${msg}"
#bremojis ${explosion} ${face_grimace} ; E_message "I think we have a problem here"
## demonstrate uE_message
#echo; msg="running ${BGon} uE_message ${BMon}\${cross_mark} ${Boff}\"testing uEx_error_m
essage\""
#message "${msg}"
#uE_message ${cross_mark} "testing uE_error_message"
## demonstrate uEx_message
#echo; msg="running ${BGon} uEx_message ${Boff}\"testing uEx_error_message\""
#message "${msg}"
#uEx_message "testing uEx_error_message"
## demonstrate emoji
#echo; message "running ${BGon}emoji${Boff} '1F427' ; ${LBon}echo${Boff}"
#emoji "1F427" ; echo
## demonstrate u_echo
#msg="running ${BGon}u_echo${Boff} \""
#msg+='\U2550\U2550\U2550\U2550 \U${biohazard} \U${no_entry} \U2550\U2550\U2550\U2550'
#msg+="\" ; ${LBon}echo${Boff}"
#message "${msg}"
#u_echo "\U2550\U2550\U2550\U2550 \U${biohazard} \U${no_entry} \U2550\U2550\U2550\U2550"
; echo
#echo; message "${BYon}Run ${BGon}emoji_demo ${BYon}to see available unicode emoticons (e
mojis)${Boff}"
#}



get_severity() # (POSIX) map integer $1 severity levels to RFC 5424 keywords and (if $2=[
dD]*) descriptions
{ gs_ret=0
  ! isint $1 && { j_msg -3 "$E_BAD_ARGS [$@]; get_severity requires integer severity leve
l 0-7"; return 1; }
  [ $# -ge 2 ] && case "$2" in [dD]* ) ;; * ) j_msg -3 "invalid arg \$2 [$2]; only [dD]*
allowed"; return 2 ;; esac
  case "$1" in
    0 ) _gs_level="emerg"; _gs_desc="system is unusable" ;;
    1 ) _gs_level="alert"; _gs_desc="immediate action required" ;;
    2 ) _gs_level="crit"; _gs_desc="critical conditions: hardware or primary application
failure" ;;
    3 ) _gs_level="err"; _gs_desc="error conditions: app errors or service interruption"
;;
    4 ) _gs_level="warning"; _gs_desc="warning conditions: potential issues; e.g. resourc
e use vs availability" ;;
    5 ) _gs_level="notice"; _gs_desc="normal but significant events" ;;
    6 ) _gs_level="info"; _gs_desc="information about  normal operations" ;;
    7 ) _gs_level="debug"; _gs_desc="Debug-level details typically only used during troub
```

```
leshooting" ;;
    * ) j_msg -3 "$E_BAD_ARGS [$@]; get_severity requires integer severity level 0-7"; re
t=1 ;;
  esac
  _gs_msg="$_gs_level"
  # maybe also append description, if $2 = d
  case $2 in [dD]* ) _gs_msg="${_gs_msg} - ${_gs_desc}" ;; esac
  printf '%s\n' "$_gs_msg" ; gs_ret=$?

  unset -v _gs_level _gs_desc _gs_msg
  return $gs_ret
}
# @usage get_severity <0-7> [d]
# @args $1 (int) syslog-standard severity level (0-7)
# @args $2 (string) optional; if starts with 'd' or 'D', includes description in output
# @deps isint, j_msg -3
# @rule maps integer severity levels to RFC 5424 keywords and descriptions
# @ret 0 on success (emit keyword/description to stdout); 1 on invalid input

separator()  # (POSIX) NOTICE-level structural output-divider
{ # defalult preface to hostname and title equivalent to +$(basename "$0") but w/o subshe
ll function call
  sep_ret=0; _preface=${1:-$(hostname)}; _title=${2:-${0##*/}}; _tw=$(termwidth)
  # compute number of '-' characters needed to fill the screen
  _reps=$(( _tw - ( ${#_preface} + ${#_title} + 8 ) ))
  [ "$_reps" -lt 0 ] && _reps=0   # clamp to 0 if negative
  # build the colorized message for stdout
  _msg="$(printf '%b---[ %b%s %b%s %b]%s%b' \
      "$BYon" "$BRon" "$_preface" "${Boff}${BCon}" "$_title" \
      "$BYon" "$(repeat '-' "$_reps")" "$Boff")"
  # NOTICE severity, no prefix, don't need to interpret escapes (colors are precooked)
  # log if enabled; respect FORCE and SILENCE
  _bitmask=$(( STDOUT | NEWLINE | CONDITIONAL | ( notice << SEVERITY_OFFSET ) ))
  [ "$LOGGING" ] && _bitmask=$(( _bitmask | LOG | TIMESTAMP ))
  [ "$DEBUG" ] &&  _bitmask=$(( _bitmask | FORCE ))
  [ "$QUIET" ] && _bitmask=$(( _bitmask | SILENCE ))
  emit "$_bitmask" "$_msg"; sep_ret=$?
  unset -v _preface _title _tw _reps _msg _bitmask; return $sep_ret ; }
# @usage separator "preface" "title"
# @args $1 preface string (defaults to hostname)
# @args $2 title string (defaults to script name via ${0##*/})
# @vars LOGGING, DEBUG, QUIET (pseudo-booleans controlling policy)
# @ret emitted: colorized NOTICE-level structural divider to stdout (and log if enabled)
# @ret status: exit status of emit
# @deps emit, termwidth, repeat, global bitmask constants
# @rule NOTICE severity governs conditionality; suppressed if severity > $verbosity unles
s FORCE is set
# @rule applies policy dictated by DEBUG (FORCE) and QUIET (SILENCE)
# @rule logs only when LOGGING is enabled; logged form is ANSI-free and timestamped
# @rule computes terminal width dynamically to draw a full-width horizontal rule
# @note no severity prefix is applied; the visual divider itself serves as the prefix
# @note colorized stdout version uses yellow brackets and dashes, red preface, cyan title
# @note canonical wrapper for NOTICE-level structural output dividers
# @ex separator "$PN-$BUILD" "(configuration)"
# @ex separator "$PN" "(step [3]: print a newline)"

yn_prompt() # (POSIX) set external variable $answer based on reponse to prompt $1
{ yn_ret=0; _yn_ps="$*"; answer=""; printf '%b' "\n\n"; CUU; SCP;
while ! expr "${answer:- }" : '[yYnN]' >/dev/null; do
RCP; repeat ' ' $(termwidth);  ## blank the line
RCP; j_msg -5n "$_yn_ps [Y/n]: " && read -r answer < /dev/tty; done; yn_ret=$?
#RCP; printf " ${BGon}*${Boff} %s" "$_yn_ps [Y/n]: " && read -r answer < /dev/tty; done;
yn_ret=$?
unset -v _yn_ps ; return $yn_ret ; }
# @usage yn_prompt "question text"
# @args $1 string: the question to ask the user
# @vars answer (global): modified by this function
# @ret exit 0 on valid [y/n] input
# @deps CUU SCP RCP termwidth repeat j_msg -5n
# @rule loops until a valid y/n response is received from /dev/tty
```

```
# @note uses terminal escape codes to blank the line during redraw
# @note use "$*" to bind message as a single string (no word-split)
# @note use '%b' (expand backslash) to interpret e.g. \n (like echo -e)

yns_prompt() # (POSIX) set external variable $answer based on reponse to prompt $1
{ yns_ret=0 ; _yns_ps="$*"; response=""; printf '%b' "\n\n"; CUU; SCP;
while ! expr "${response:- }" : '[yYnNsS]' >/dev/null; do
RCP; repeat ' ' $(termwidth);  ## blank the line
RCP; j_msg -5n "$_yns_ps [Yes/no/skip|Yns]: " && read -r response < /dev/tty; done; yns_r
et=$?
#RCP; printf " ${BGon}*${Boff} %s" "$_yns_ps [Yes/no/skip|Yns]: " && read -r response < /
dev/tty; done; yns_ret=$?
unset -v _yns_ps ; return $yns_ret ; }
# @usage yns_prompt "question text"
# @args $1 string: the question to ask the user
# @vars response (global): modified by this function
# @ret exit 0 on valid [y/n/s] input
# @deps CUU SCP RCP termwidth repeat j_msg -5n
# @rule loops until a valid Yes/No/Skip response is received from /dev/tty
# @note use "$*" to bind message as a single string (no word-split)
# @note use '%b' (expand backslash) to interpret e.g. \n (like echo -e)

confirm_continue_or_exit() # prompt() for confirm to continue (answer must be set globall
y)
{ answer="" ; _cce_msg="${BYon}Do you want to continue?" ; yn_prompt "${_cce_msg}"
    case ${answer} in
        [yY]* ) j_msg -5 "Affirmative response from user; continuing" ;;
        [nN]* ) j_msg -3 "Negative response from user; quitting";
                # safeguard: only exit if running as a script, otherwise return 1
                case "$-" in
                    *i*) return 1 ;; # interactive: just return failure
                      *)   exit 1 ;; # script: kill execution
                esac ;;
        * ) j_msg -3 "invalid response to confirm_continue_or_exit"; return 1 ;;
    esac ; unset -v _cce_msg; return 0 ; }
# @usage confirm_continue_or_exit
# @vars answer (global): modified by internal call to yn_prompt()
# @deps yn_prompt j_msg -5 j_msg -3
# @rule interactive shell: returns 1 on "no"; script: exits 1 on "no"
# @note provides a safety gate to prevent accidental batch execution

die() # (POSIX) display reason with j_msg -3 and exit/return with status 1
{ _die_msg="${1:-"unknown fatal error"}" ; j_msg -3 "${BRon}FATAL: ${_die_msg}${Boff}"
  unset -v _die_msg ; SCU  # (re)show cursor in case it was hidden when die was called
  case "$-" in       # use the $- (current shell flags/options)
    *i*) return 1 ;; # interactive: return failure to terminal
    *)   exit 1   ;; # script: exit with error code
  esac; }
# @usage die "error message"
# @args $1 string: the reason for the fatal error
# @ret exit 1 (script) or return 1 (interactive)
# @deps j_msg -3
# @rule detects shell state via $- to avoid closing interactive terminals

leave() # (POSIX) display reason with j_msg -5 and exit/return with status 0
{ _leave_msg="${1:-"normal exit"}" ; j_msg -5 "${BYon}Leaving: ${Boff}${_leave_msg}${Boff
}"
  unset -v _leave_msg ; SCU  # (re)show cursor in case it was hidden when die was called
  case "$-" in       # use the $- (current shell flags/options)
    *i*) return 1 ;; # interactive: return failure to terminal
    *)   exit 1   ;; # script: exit with error code
  esac; }
# @usage leave "message"
# @args $1 string: the reason for leaving
# @ret exit 0 (script) or return 0 (interactive)
# @deps j_msg -5
# @rule detects shell state via $- to avoid closing interactive terminals

progress_inline() # (POSIX) show progress of silent process on next line $1=step, $2=tota
l
```

```
{ pil_ret=0; _p_step=$1; _p_total=$2; _p_tw=$(termwidth); isint "$_p_tw" || _p_tw=80
  _p_percent=$(( (100 * _p_step) / _p_total ))
  # use 1/4 of screen width for the bar to keep room for text
  _p_bar_max=$(( _p_tw / 4 ))
  _p_filled=$(( (_p_bar_max * _p_step) / _p_total ))
  _p_empty=$(( _p_bar_max - _p_filled ))
  # determine which color to use for the "filled" markers and text NN% marker (heat map s
tyle)
  if [ "$_p_percent" -ge 90 ]; then _p_color="${BGon}"    # hot-good
  elif [ "$_p_percent" -ge 80 ]; then _p_color="${Gon}"   # warm-ok
  elif [ "$_p_percent" -ge 50 ]; then _p_color="${Con}"  # luke-warm
  elif [ "$_p_percent" -ge 25 ]; then _p_color="${Yon}"  # luke-warm
  else _p_color="${Ron}"; fi                              # cold

  # print inline with console output by using carriage return \r
  # assuming the process is silent, this stays on one line until the task is done
  # BUCKETS: - 1:markers_done 2:markers_not_done 3:color 4: xx% 5:reset 6:step 7:total
  # this printf command is going to be over 190 characters, and I don't like using "\"
  # to break commands over multiple lines because you cannot safely copy/paste those line
s
  # (e.g. for testing in another terminal), so I will use set to build it up
  # inside a curly brace block to capture one exit status
  #   start with the format string (see BUCKETS above note %b vs %s for ANSI color etc)
  { set -- "\r Working: [%b%b] %b%3d%%%b (%d/%d)"
  #   append the bar segment buckets
  #  set -- "$@" "${_p_color}$(repeat "#" $_p_filled)${Boff}"    # status-changing colore
d progress bar
  set -- "$@" "${Bon}$(repeat "#" $_p_filled)${Boff}"            # single-color (blue) mo
notone progress bar
  #  set -- "$@" "$(repeat "#" $_p_filled)${Boff}"               # uncolored monotone pro
gress bar
  set -- "$@" "${BBon}$(repeat "." $_p_empty)${Boff}"
  #   append the remaining status variables
  set -- "$@" "${_p_color}" "$_p_percent" "${Boff}" "$_p_step" "$_p_total"
  #   execute (double quotes are vital here, to preserve the quoting of the arguments)
  printf "$@" ; } ; pil_ret=$?
  # on 100%, move to next line so the bar remains as a record
  [ "$_p_step" -eq "$_p_total" ] && printf "\n"
  unset -v _p_step _p_total _p_tw _p_percent _p_bar_max _p_filled _p_empty _p_color
  return $pil_ret ; }
# @usage progress_inline $current $total
# @args $1 current step; $2 total steps
# @deps termwidth isint repeat
# @rule renders a horizontal progress bar using 1/4 of terminal width
# @rule implements a heat-map color scale for the percentage text
# @note uses "set --" to build complex printf commands without line continuations
# @note automatically emits newline upon reaching 100% to preserve the bar

sh_countdown() # (POSIX) count-down seconds (arg $1)
{ shc_ret=0; _shc_rem_time="${1:-30}"  # guard; default to 30 sec. if no arg given
  while [ "${_shc_rem_time}" -ge "0" ] ; do
  printf '\r'; # return to left edge
  _msg="Pausing ${BBon}[${Boff}"                 # prefix and "Pausing ["
  _msg="${_msg} ${BGon}$(printf '%02d' "$_shc_rem_time")${Boff}"   # nn
  _msg="${_msg} ${BBon}]${Boff} seconds remaining ...   "          # " ] seconds remainin
g ...   "
  j_msg -n "$_msg"
  [ ! $? -eq 0 ] && shcy_ret=1
  sleep 1; _shc_rem_time=$(( _shc_rem_time - 1 ))
  [ "$_shc_rem_time" -eq 0 ] && break; done || shc_ret=2
  printf '\n'      #clean up the line for the next command
  unset -v _shc_rem_time ; return $shc_ret ; }
# @usage sh_countdown [seconds]
# @args $1 number of seconds to pause (defaults to 30)
# @ret 0 if no error; 1 if print error; 2 in countdown error
# @rule uses \r for in-place counter updates
# @note special note: bb sh cannot use base#number notation (don't prefix w 10#)
# @note breaks immediately at 0 to avoid an unnecessary final sleep cycle

#-----[ Logging ]-------------------------------------------------------------------------
```

```
--------------

timestamp() # (POSIX) echo a simple timestamp (yyyymmdd-hh:mm:ss)
{ printf '%s\n' "$(date +'%Y%m%d-%H:%M:%S')"; }
# @usage timestamp
# @ret string: formatted date/time (e.g., 20260121-13:45:00)
# @note used for unique filenames, log entries, and non-blocking timehacks

loggit() # (POSIX) send a job completion status message to the cron logger
{ [ $# -gt 2 ] && { j_msg -3 "invalid loggit arguments" ; return 1; }
  _loggit_exit_status="${2:-0}"
  _loggit_program="${1:-Unspecified}"
  case "$_loggit_exit_status" in
    0 ) _loggit_priority="notice"; _loggit_message="succeeded.";;
    * ) _loggit_priority="err"; _loggit_message="failed with exit status ${_loggit_exit_s
tatus}.";;
  esac
  logger -p "cron.$_loggit_priority" "$_loggit_program $_loggit_message"
  unset -v _loggit_exit_status _loggit_program _loggit_priority _loggit_message
}
# @usage loggit "program_name" [exit_status]
# @args $1 string: program name; $2 int: exit status (defaults to 0)
# @deps j_msg -3 logger
# @rule maps 0 to 'notice' and non-zero to 'err' for system logging
# @ret exit 0 on success; 1 on invalid arguments
# @note interfaces with the system logger (syslog) under the 'cron' facility

#-----[ Debugging ]-----------------------------------------------------------------------
----------

#~~~~~[ basic debiggomg ]~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~

d_do() # (POSIX) IAW verbosity, execute complex (multi-command) jobs
{
  # guard 1 - check for min. 2 args (word + level)
  [ $# -lt 2 ] && { j_msg -3 "minimum 2 args required <command string> <level>"; return 1
; }
  # guard 2 - check for unset global verbosity
  [ -z "$verbosity" ] && { j_msg -3 "Error: global verbosity not set"; return 2; }
  # extract the last argument and assign to _dd_level POSIX-style
  #   _dd_level will now equal the last argument (equivalent to for _dd_level in "$@"; do
 :; done)
  for _dd_level; do :; done;
  # guard 3 - ensure _dd_level is an integer
  ! isint "$_dd_level"  && { j_msg -3 "Error: Final arg '$_dd_level' must be a decimal ve
rbosity level"; return 3; }
  # guard 4 - check global verbosity thresholds
  if [ "$DEBUG" ] || [ "$verbosity" -ge "$_dd_level" ]; then
    # argument rotation: assign everything except the last arg back to $@
    _dd_arg_count=$(( $# - 1 ))   # this is the key that will leave the last arg at $1 wh
en the while completes
    _i=0
    while [ "$_i" -lt "$_dd_arg_count" ]; do
       _dd_arg="$1"; shift
       set -- "$@" "$_dd_arg"
       _i=$((_i + 1))
    done
    shift # Remove the arg alread assigned to $_dd_level, which is now at $1
    # safe execution - even for 'd_do "$crit"', $#=1 post shift; eval "" does nothing, re
turns 0 (safe)
    if [ $# -eq 1 ]; then
      eval "$1"  # if user passed a single string with ';' or '&&', eval it.
    else
      "$@"       # Otherwise, execute the arguments directly.
    fi  # zero or one command arg?
  else
    :  # if global verbosity level does not exceed the conditional severity _dd_level, ju
st return 0
  fi  # verbosity?
```

```
  unset -v _dd_level _dd_arg_count _i _dd_arg
  return 0
}
# @usage d_do <command string> <level>
# @args last arg: integer severity level; preceding args: the command(s) to execute
# @ret 0: success or no action; 1: severity not set ; 2: verbosity not set
# @deps j_msg -3 isint
# @rule last arg must be integer severity level
# @rule executes command(s) if [ $DEBUG ] or if $verbosity >= (severity) $level
# @rule uses POSIX argument rotation (set --) to separate the level from the command
# @rule executes via eval if a single string is provided, or directly if multiple args re
main
# @note handles complex strings like d_do 'ls | grep txt' "$notice" safely
# @note detects unset global verbosity and missing arguments via guard clauses
# In the very specific scenario where a user calls d_do 1 (only one argument),
#   Guard 1 catches it. If they call d_do "" "$crit" (empty string command),
#   Guard 4 processes it, shift happens, $# becomes 1, and eval "" runs safely


# @note =====[ Section 7: System, Net & LUKS ]=========================================
==============================

milli_sleep() # (almost POSIX)(busybox safe) "sleep" for sub-second delays
{ # @usage milli_sleep <ms>
  # @args $1: delay in milliseconds (default: 100)
  # @deps awk
  # @note uses awk system() to provide sub-second sleep for shells lacking it
    _ms_in="${1:-100}"
    # pass the variable into awk via -v to avoid shell quoting hell
    # (with -v, the shell variable is passed as data, not as part of the code to be inter
preted)
    # awk then handles the division and calls the system sleep.
    awk -v ms="$_ms_in" 'BEGIN { system("sleep " ms / 1000) }'
}

fe() # find ebuild files matching filter "*$1*"
{ [ $# -lt 1 ] && { j_msg -3 "please specify ebuild patter for search filter"; return 1 ;
 }
  _fe_repo_dir="/var/db/repos/"; _fe_filter="*$1*.ebuild"
  printf '%s\n' "${BYon}looking for ebuild(s) matching filter${Boff}: \"${Mon}${_fe_filte
r}${Boff}\""
  _fe_output=$(find "${_fe_repo_dir}" -iname "${_fe_filter}" -type f -print 2>/dev/null)
  if [ -z "${_fe_output}" ] ; then
    j_msg -3 "no files matching ${_fe_filter} found in ${_fe_repo_dir}"
    unset -v _fe_repo_dir _fe_filter _fe_output ; return 1
  else
    printf '%s\n' "${BWon}${_fe_output}${Boff}"
  fi
  unset -v _fe_repo_dir _fe_filter _fe_output
  return 0
}
# @usage fe <pattern>
# @args $1: search pattern for ebuild filenames
# @deps find
# @ret prints matching ebuild paths to stdout; returns 1 if none found
# @note searches recursively within /var/db/repos/

pur() # (POSIX) ping ip address $1 until it is ready (0% loss of 3 packets)
{ pur_ret=0; _pur_target="$1"; SCP; _pur_count=1; _hits=0; _hit_color="${Boff}"; _pur_max
_retries=1000 ;
  while [ "$_hits" -lt 3 ] && [ "$_pur_count" -lt "$_pur_max_retries" ]; do
    if ping -c1 -W1 "$_pur_target" >/dev/null 2>&1; then _hits=$((_hits + 1)); else _hits
=0; fi;
    case "$_hits" in [01]) _hit_color="${BRon}";; 2) _hit_color="${BYon}";; 3) _hit_color
="${BGon}";; esac;
    RCP; printf '%b' "not ready -- try: [${_pur_count}], hits: [${_hit_color}${_hits}${Bo
ff}] ";
    _pur_count=$((_pur_count + 1)); [ "$_hits" -lt 3 ] && sleep 1; done;
  RCP; EL ; # erase from cursor to end of line
  if [ "$_hits" -lt 3 ] ; then
```

```
    RCP; j_msg -3 "timed out"; pur_ret=1
  else
    RCP; j_msg -5 "${BGon}OK${Boff} - ready now"
  fi
  unset -v _pur_target _pur_count _hits _hit_color _pur_max_retries
  return $pur_ret
}
# @usage pur <ip_address>
# @args $1: target IP or hostname
# @deps ping SCP RCP EL
# @ret 0 when 3 consecutive pings succeed; 1 on timeout (1000 retries)
# @note uses SCP/RCP for in-place terminal status updates

swr() # (POSIX) use pur() to check first; then ssh when netcat indicates port is ready
{ swr_ret=0; [ $# -lt 1 ] && return 1 ;   # guard; reject null target
  _swr_target="$1"; _swr_max_retries=10; _swr_count=0; _ssh_port="${2:-22}";  # default t
o port 22 but allow another to be specified as $2
  _yellow_thresh=$(($_swr_max_retries * 33/100)); _red_thresh=$(($_swr_max_retries * 75/1
00));
  pur "$_swr_target";
  if [ ! $? -eq 0 ] ; then
    j_msg -3 "pur() failed or timed out";
    unset -v _swr_target _ssh_port _swr_max_retries _swr_count _swr_color _yellow_thresh
_red_thresh
    return 1;
  fi;
  j_msg -5 "confirming $1 has started sshd on port $_ssh_port ..." ; SCP;
  # use netcat to verify port is open on target (one hit is sufficient)
  while [ "$_swr_count" -lt "$_swr_max_retries" ] && ! nc -z -w 1 "$_swr_target" "$_ssh_p
ort" 2>/dev/null; do
    RCP; EL;
    # set colors with POSIX logic
    if [ "$_swr_count" -le "$_yellow_thresh" ]; then
      _swr_color="${BGon}"
    elif [ "$_swr_count" -le "$_red_thresh" ]; then
      _swr_color="${BYon}"
    else
      _swr_color="${BRon}"
    fi
      printf "checking port %s -- try: [%b] " "$_ssh_port" "${_swr_color}${_swr_count}${B
off}";
    _swr_count=$((_swr_count + 1)); sleep 1; done;
  # test nc one last time to ensure the loop exited for success, not timeout
  if nc -z -w 1 "$_swr_target" "$_ssh_port" 2>/dev/null; then
    RCP; EL; j_msg -5 "${BGon}OK${Boff} - $_swr_target sshd is ready"
    ssh -p "$_ssh_port" "$_swr_target"
    swr_ret=0
  else
    RCP; EL; j_msg -3 "sshd port $_ssh_port on $_swr_target not responding after $_swr_ma
x_retries tries"
    swr_ret=1
  fi
  unset -v _swr_target _ssh_port _swr_max_retries _swr_count _swr_color _yellow_thresh _r
ed_thresh
  return $swr_ret
}
# @usage swr <target> [port]
# @args $1: target IP/host; $2: SSH port (default 22)
# @deps pur nc ssh SCP RCP EL
# @ret 0 if SSH connection succeeds; 1 if pur() fails or port never opens
# @rule calls pur() first to ensure basic network reachability
# @note color-coded retry counter (green -> yellow -> red) based on retry thresholds

maybe_format_warning() # (POSIX) issue a warning if INTERACTIVE is $TRUE
{ if [ $INTERACTIVE ] ; then format_warning "$@" || return 1; fi; return 0 ; }

format_warning() # (POSIX) issue a warning that the operation will destroy target device
{ fw_ret=0
  [ -z "$1" ] && { j_msg -3 "\$1 is null but must not be; Specify target_device"; return
1; }
```

```
  _fw_target_device="$1"
  : "${PN:=$(hostname)}"
  separator "$PN" "(format warning)"
  answer=""
  _fw_msg1="${BMon}*   ${BRon}Warning:${BYon} this process will destroy all data on devic
e"
  _fw_msg1="${_fw_msg1} [ ${BRon}${_fw_target_device}${BYon} ]${Boff}   ${BMon}*${Boff}"
  _fw_msg2="${BMon}*   ${BYon}Warning:${BRon} this process will destroy all data on devic
e"
  _fw_msg2="${_fw_msg2} [ ${BYon}${_fw_target_device}${BRon} ]${Boff}   ${BMon}*${Boff}"
  # estimate j_msg -5 function left side padding
  # adjust for non-printing chars and j_msg -5 padding
  _fw_msg_width=$(get_msg_len "$_fw_msg1")
  _fw_msg3="   ${BWon}Are you sure you want to continue?"
  echo
  printf '  %s' "${BMon}"; repeat '*' "${_fw_msg_width}"; printf '%s\n' "${Boff}"
  printf '  %s\n' "$_fw_msg1"
  printf '  %s\n' "$_fw_msg2"
  printf '  %s\n' "$_fw_msg1"
  printf '  %s' "${BMon}"; repeat '*' "${_fw_msg_width}"; printf '%s\n' "${Boff}"
  yn_prompt "$_fw_msg3"
  case ${answer} in
    [yY]* ) j_msg -5 "Affirmative response from user; continuing"; fw_ret=0 ;;
    [nN]* ) j_msg -3 "Negative response from user; quitting"; fw_ret=0 ;;
    * ) j_msg -3 "invalid response to format_warning"; fw_ret=1 ;;
  esac
  unset -v _fw_msg1 _fw_msg2 _fw_msg_width _fw_msg3
  return $fw_ret ; }
# @usage format_warning
# @vars PN _fw_target_device answer (globals)
# @ret 0: user confirmed; 1: user denied or invalid input
# @deps separator get_msg_len repeat yn_prompt j_msg -5 j_msg -3
# @req requires _fw_target_device to be set globally or in caller scope
# @warn this function will return 1 to trigger a die or exit in the caller
# @note uses get_msg_len to strip non-printing chars

wipe_device() # (LINUX) use dd to fill target device $1 with random data
{ wd_ret=0
  [ -z "$1" ] && { j_msg -3 "\$1 is null but must not be. Specify target_device"; return
1 ; }
  _wd_target_device="$1"
  [ -b "$_wd_target_device" ] || die "wipe_device: [$_wd_target_device] is not a block de
vice"
  checkroot  # must be root
  # if INTERACTIVE does not exist or is not set in the callers context, then re-assign w/
 default
  #   := operator is "assign" vs :- which is "substitute" for current command only
  : "${INTERACTIVE:=$TRUE}"

  # note that dd will exit with error 1 (disk full) if it does not know how much to write
,
  #   so calculate how much using blockdev commands (sys-apps/util-linux) --
  # raw size in bytes
  _wd_total_bytes=$(blockdev --getsize64 "$_wd_target_device")
  # physical sector size (usually 4096 or 512)
  _wd_sector_size=$(blockdev --getpbsz "$_wd_target_device")
  # calculate the largest multiple of _wd_sector_size that is -le 4MB
  #   and perfectly divides _wd_total_bytes
  _wd_preferred_bs=$((4 * 1024 * 1024))
  # determine initial block size (align to _wd_sector_size)(integer division rounds)
  if [ "$_wd_total_bytes" -lt "$_wd_preferred_bs" ]; then
    _wd_bs=$(( (_wd_total_bytes / _wd_sector_size) * _wd_sector_size ))
  else
    _wd_bs=$(( (_wd_preferred_bs / _wd_sector_size) * _wd_sector_size ))
  fi
  # safety - ensure _wd_bs is at least 1 sector
  [ "$_wd_bs" -gt 0 ] || _wd_bs=$_wd_sector_size
  # loop - work downward to find the largest _wd_bs (bytes) that perfectly divides _wd_to
tal_bytes
  while [ "$_wd_bs" -gt "$_wd_sector_size" ]; do
```

```
    if [ $((_wd_total_bytes % _wd_bs)) -eq 0 ]; then break; fi
    _wd_bs=$((_wd_bs - _wd_sector_size))
  done

  # final _wd_bs SHOULD now a perfect divisor of _wd_total_bytes and multiple of _wd_sect
or_size
  #   however in rare circumstances (virtual disk, etc.), the disk might not be perfectly
  #   build with integer number of sectors... so detect that
  if [ $((_wd_total_bytes % _wd_bs)) -eq 0 ]; then
    _wd_count=$((_wd_total_bytes / _wd_bs))
    _wd_EXPECT_DISK_FULL=$FALSE
  else
    j_msg -4 "Warning: Partition size not a multiple of sector size. Expecting 'No space
left'"
    # round up the number of sector-sized blocks to write
    _wd_count=$(( (_wd_total_bytes + _wd_bs - 1) / _wd_bs ))
    _wd_EXPECT_DISK_FULL=$TRUE
  fi

  # build the dd command in readable lines - retirect stderr to the tempfile so kill -USR
1 can read it
  _wd_my_cmd="dd if=/dev/urandom of=$_wd_target_device bs=$_wd_bs count=$_wd_count"
  # add iflag=fullblock to tell dd to wait until it has accumulated the full requested bl
ock size from /dev/urandom
  # before proceeding, preventing partial reads and ensuring a continuous transfer (i.e.
guard against short supply of entropy)
  _wd_my_cmd="${_wd_my_cmd} iflag=fullblock"

  # conditionally show the configuration derived above (display_vars operates at NOTICE s
everity)
  _wd_varlist="_wd_target_device _wd_total_bytes _wd_sector_size _wd_preferred_bs"
  _wd_varlist="${_wd_varlist} _wd_bs _wd_count bool._wd_EXPECT_DISK_FULL lv._wd_my_cmd"
  separator "wipe_device" "(configuration)"
  _wd_longest=$(get_longest ${_wd_varlist})
  display_vars ${_wd_longest} ${_wd_varlist}

  # build _wd_msg, in readable lines, to explain and prompt
  printf '\n'
  j_msg -5 "${BYon}about to run ${BBon}[${Boff}${_func_color}$_wd_my_cmd${BBon}]${Boff}"
  j_msg -5 "${BCon}(you can safely skip this step if you have done it before)${Boff}"
  _wd_msg="${BWon}Continue? (y: yes; s: skip; n: exit program)${Boff} "
  if [ "$INTERACTIVE" = "$TRUE" ]; then
    yns_prompt "$_wd_msg"
    _wd_res="$response"
  else
    _wd_res="y"
  fi

  # handle response to prompt, with pattern matching for portability
  case "$_wd_res" in
    [yY]* )
      # be very sure
      maybe_format_warning "$_wd_target_device"
      handle_result $? "proceeding" "" || die "failed format warning somehow"
      echo
      # OK - run it (in background, capturing its PID)
      # capture start time
      _wd_start_time=$(date +%s)
      # disable monitor mode (job control) for background process
      # (to silence the asynchronous "[1]+ Done" messages that break the UI)
      # (yes, it feels backwards set -X enables X, set +X disables X)
      set +m
      # launch inside curly braces  to silence start-up message
#      eval "$_wd_my_cmd &"; _wd_pid=$!
      { eval "$_wd_my_cmd &" ; } >/dev/null 2>&1 ; _wd_pid=$!
      # give dd some breathing room before monitoring it (instead of sleep .2, poll for /
proc/$pid)
      _wd_timeout=3000  # milliseconds - using milli_sleep function defined in script_hea
der_joetoo
      until { [ -f "/proc/$_wd_pid/status" ] && [ -f "/proc/$_wd_pid/io" ] ; } || [ $_wd_
```

```
timeout -eq 0 ] ; do
        milli_sleep 100
        _wd_timeout=$(( _wd_timeout - 100 ))
      done
      # if timeout expired, die
      if [ $_wd_timeout -eq 0 ]; then
        die "${BYon}Timed out waiting for /proc/$_wd_pid/status and /proc/$_wd_pid/io${Bo
ff}"
      fi
      # announce start and warn about possible delays if device is slow, etc.
      j_msg -4 "${BYon}Started wipe of $_wd_target_device (pid=$_wd_pid). This may take a
 while...${Boff}"
      j_msg -4 "If the device is slow or has issues, progress updates may be delayed."
      j_msg -4 "${BYon}Note${Boff}: if you see \"(D state)\" right of progress bar,"
      j_msg -4 "dd has entered the kernel's D state (Uninterruptible Sleep), and this"
      j_msg -4 "usually indicates a hardware issue (e.g. old/flakey/slow USB stick)."
      j_msg -4 "Investigate/abort (CTRL-C and/or re-seat the device) if you see no write"
      j_msg -4 "progress and this condition persists"
      echo

      # now monitor and display progress
      _wd_bytes_complete=0
      HCU   # hide the cursor to prevent flicker during progress bar updates
      # use kill -0 as a safety check so the loop doesn't hang if dd dies
      while kill -0 "$_wd_pid" 2>/dev/null; do
        if [ -f "/proc/$_wd_pid/status" ]; then
          # check for Hardware Hang (D state) (likely old/small/flakey USB stick block de
vice)
          _wd_state=$(grep "State:" "/proc/$_wd_pid/status" | awk '{print $2}')
          # avoid reading cancelled_write_bytes: (which would also match grep "^write_byt
es:")
          _wd_bytes_complete=$(grep "^${W0}write_bytes:" /proc/$_wd_pid/io | awk '{print
$2}')
          _wd_curr_time=$(date +%s)
          _wd_elapsed=$(echo "$_wd_curr_time - $_wd_start_time" | bc)

          d_do '
            _wd_varlist="_wd_state _wd_bytes_complete _wd_start_time _wd_curr_time _wd_el
apsed MiB"
            _wd_longest=$(get_longest $_wd_varlist)
            echo
            display_vars $_wd_longest $_wd_varlist
        ' "$debug"

          # guard the calculation -
          #   bc will fail on division by zero (MiB defined in script_header_joetoo), or
          #   if no bytes written yet (_wd_bytes_complete = "" rather than 0)
          if [ -n "$_wd_bytes_complete" ] && [ "$_wd_elapsed" -gt 0 ]; then
            _wd_elapsed_time=$(date -ud "@$_wd_elapsed" +%T)
            _wd_rate=$(echo "scale=2; $_wd_bytes_complete / $_wd_elapsed / $MiB" | bc)
            _wd_stats="$_wd_elapsed_time @ ${_wd_rate} MB/s"
          else
            # While elapsed is 0, we can't calculate a rate; show pending status
            _wd_stats="00:00:00 @ ... MB/s"
          fi

          if [ "$_wd_state" = "D" ]; then _wd_DSTATE=$TRUE; else _wd_DSTATE=$FALSE; fi
# D state check
        fi   # /proc/$pid/status exists

        # update progress for integer results only
        if isint "$_wd_bytes_complete"; then _wd_INT=$TRUE; else _wd_INT=$FALSE; fi
        # debugging output visible only if verbosity -ge "$debug"
        d_do '
          _wd_varlist="_wd_pid _wd_bytes_complete bool._wd_INT _wd_elapsed _wd_elapsed_ti
me _wd_rate"
          _wd_varlist="${_wd_varlist} _wd_stats bool._wd_DSTATE"
          _wd_longest=$(get_longest $_wd_varlist)
          echo
          display_vars $_wd_longest $_wd_varlist
```

```
        ' "$debug"
        if [ $_wd_INT ] ; then
          # print the status update right of the progress bar *first b/c 100% progress ad
ds a newline*
          # dynamically calculated offset for this screen and this block device; append s
tatus info sufficiently offset
          # x=" Working: [##################################.]  100% ("
          # but the ### part is max 1/4 termwidth, so it is variable
          # x=" Working: []  100% (/)"; echo ${#x} ==> 22 is the non-variable part
          #   also include room for (done/total) fraction ( max = 2 * total) plus 2 extra
 pad
          _wd_offset=$(( 22 + ( $(termwidth) / 4 ) + ( 2 * ${#_wd_total_bytes} ) + 2 ))
          SCP  # save cursor position where progress_inline completed this step
          printf '\r'; CUF $_wd_offset   # use dynamic offset from left edge of screen
          printf '%s' "${BBon}[${Boff} ${_wd_stats} ${BBon}]${Boff}"  # print [stats]
          # print D state warning if in D state
          if [ $_wd_DSTATE ] ; then printf ' %s' "${BRon}(D state)${Boff}"; fi
          EL   # erase to end of line, to keep it clean if rate is reduced for example
          RCP  # restoe cursor to where progress_inline left it
          # print the status update right of the progress bar *first b/c 100% progress ad
ds a newline*
          # if 100% but in D state, do not add newline
          if ! { [ "$_wd_bytes_complete" = "$_wd_total_bytes" ] && [ $_wd_DSTATE ] ; }; t
hen
            # save cursor position, write progress bar, Erase to end of Line (clean up e.
g. typing); resore cursor position
            SCP; progress_inline "$_wd_bytes_complete" "$_wd_total_bytes" ; EL; RCP
          fi  # print progress bar?
        fi  # _wd_bytes_complete is integer
        sleep .5   # was 2 seconds; shorter sleep for more responsive UI
      done
      SCU   # ok to show the cursor again

      # wait for dd to finish and capture its exit status code
      wait "$_wd_pid"
      wd_ret=$?
      set -m  # re-enable monitor mode (job control) after background process is done
      case $wd_ret in
        # exit codes from the dd cmd above are 0: success, 1: general error (disk full, I
/O error, permission denied)
        #   or over 100 for shell errors (e.g. 126 dd not executable, or 127 dd not found
 ..)
        #   and exit code 130 means the user hit CTRL-C to quit
        0   )
          # success. Now that it has finished successfully, update the progress bar (whic
h will add a newline)
          progress_inline "$_wd_total_bytes" "$_wd_total_bytes"
          ;;
        1   )
          # "disk full" error. ignore if expecting it
          # note: _wd_EXPECT_DISK_FULL is a psequdo-boolean like globally defined TRUE=0;
 FALSE=""
          if [ "$_wd_EXPECT_DISK_FULL" ]; then
            progress_inline "$_wd_total_bytes" "$_wd_total_bytes"   # Success
          else
            die "general failure; dd exit code [$wd_ret]"
          fi ;;
        126 ) die "dd not executable; dd exit code [$wd_ret]" ;;
        127 ) die "dd not found; dd exit code [$wd_ret]" ;;
        130 ) die "process interrupted by user; dd exit code [$wd_ret]" ;;
        137 ) die "killed (OOM or kill -9); dd exit code [$wd_ret]" ;;
        138 ) die "bus error (SIGBUS); dd exit code [$wd_ret]" ;;
        *   ) die "unrecognized failure; dd exit code [$wd_ret]" ;;
      esac
      ;;
    [nN]* ) die "aborting, as instructed" ;;
    [sS]* ) j_msg -5 "skipping wipe of $_wd_target_device, as instructed" ;;
    *     ) die "invalid response to prompt for dd command"
  esac
```

```
# Unset localized variables
  unset -v _wd_total_bytes _wd_sector_size _wd_preferred_bs _wd_bs _wd_count
  unset -v _wd_EXPECT_DISK_FULL _wd_my_cmd _wd_varlist _wd_longest
  unset -v _wd_msg _wd_res _wd_start_time _wd_pid _wd_timeout
  unset -v _wd_bytes_complete _wd_state _wd_curr_time _wd_elapsed _wd_elapsed_time
  unset -v _wd_rate _wd_stats _wd_DSTATE _wd_INT _wd_offset
  return $wd_ret
}
# @usage wipe_device <_wd_target_device>
# @vars _wd_target_device (global)
# @rule _wd_target_device must evaluate as a valid block device
# @warn this will destroy all data on the target device
# @ret 0: success, 1: general failure, 100(+): shell error
# @deps sys-apps/util-linux (blockdev)
# @deps linux kernel (/dev/urandom)
# @deps linux kernel (/proc/$pid/status and /proc/$pid/io)
# @deps sys-apps/coreutils (dd, date, sleep, grep, awk, bc)


#!/bin/sh
PN="${0##*/}"

_get_luks_edev_name()  # (POSIX) determine an edev name to use with this device
{ glen_ret=0; _glen_dev_name="$1"
  [ -z "$_glen_dev_name" ] && { j_msg -3 "error: no arg; _get_luks_edev_name requires \$1
 device"; return 1; }
  # reject device if the right-most character of the device name is not an iteger (i.e. n
ot a partition)
  if ! isint "${_glen_dev_name##"${_glen_dev_name%?}"}"; then
    die "rejecting device [$_glen_dev_name] which does not appear to be a partition"
  fi
  # ( e.g. sda2->eda2 nvme0n1p2->ev012 mmcblk0p2->em02 )
  _glen_bn="${_glen_dev_name##*/}" # equivalent to $(basename ${_glen_dev_name})
  j_msg -7n "Determining edev name ..."
  case ${_glen_bn} in
      "sd"* ) # scsi device (sda2 -> eda2)
        # strip s w/ parameter expansion, and replace w/ e
        _glen_myedev="e${_glen_bn#s}"
        ;;
      "nv"* ) # nvme device (nvme0n1p2 -> ev012)
        # strip 'nvme'w/ parameter expansion, then delete the n and the p w/ printf | tr
        _glen_tmp="${_glen_bn#nvme}"
        _glen_myedev="ev$(printf '%s' "$_glen_tmp" | tr -d 'np')"
        ;;
      "mm"* ) # mmc device (mmcblk0p2 -> em02)
         # strip 'mmcblk'w/ parameter expansion, then delete the p w/ printf | tr
        _glen_tmp="${_glen_bn#mmcblk}"
        _glen_myedev="em$(printf '%s' "$_glen_tmp" | tr -d 'p')"
        ;;
      *    ) # tbd (conditional newline; error die)
        j_msg -p; j_msg -3 "" ; die "bad luks device name [${_glen_bn}]"
        ;;
    esac
    _glen_result=$?
    handle_result $_glen_result "edev: ${_glen_myedev}" "" "$debug"
    export edev=${_glen_myedev}

    unset -v _glen_bn _glen_myedev _glen_tmp _glen_result _glen_dev_name
    return $glen_ret
}
# @usage _get_luks_edev_name <device>
# @ret 0 on success; calls die on unrecognized device patterns
# @vars _glen_dev_name (required global) edev (exported)
# @deps j_msg right_status tr
# @rule maps physical devices to 'e' prefixed names: sd* -> ed*, nvme* -> ev*, mmcblk* ->
 em*
# @rule uses parameter expansion and tr to sanitize device strings for LUKS mapper
# @note Note: _glen_dev_name must be globally defined
# @note example: nvme0n1p2 -> ev012; sda2 -> eda2; mmcblk0p2 -> em02
# @note operates at DEBUG severity
```

```
_unlock_device() {
    ud_ret=0
    # for version 4.x+, this is a complete re-write of the unlocking procedure.
    # The unlock method has already been determined, and the script now calls _unlock_dev
ice
    # to unlock one device, employing _get_luks_edev_name() as a helper
    [ -z "$1" ] && die "error: no arg; unlock_device requires \$1 unlock_method"
    [ -z "$2" ] && die "error: no arg; unlock_device requires \$2 device"
    _ud_method=$1; _ud_device="$2"

    j_msg -6 "calling _get_luks_edev_name \"$_ud_device\""
    # use _get_luks_edev_name in script_header_joetoo to id each _ud_device /dev/sdxn wit
h corresponding name edxn
    _get_luks_edev_name "$_ud_device"
    j_msg -6n "Done with _get_luks_edev_name; result:"
    handle_result $? "edev: $edev" "failed to determine edev name for device [$_ud_device
]" "$info"
    [ -z "$edev" ] && die "error: _get_luks_edev_name did not return a value for device [
$_ud_device]"

    case "$_ud_method" in
        "prompt")
            j_msg -6 "About to run cryptsetup luksOpen ${_ud_device} ${edev} with passphr
ase provided ..."
#           this is no longer a f-ing "indented" heredoc '-' in <<-EOF tells it to ignore
 leading TABs (not spaces)
#           cryptsetup luksOpen "${_ud_device}" "${_ud_edev}" <<-EOF  # can only be inden
ted with tab characters
cryptsetup luksOpen "${_ud_device}" "${edev}" <<EOF
$_ulds_pass
EOF
            _ud_result=$?
            handle_result "$_ud_result" \
              "Unlocked ${_ud_device} ${edev} with passphrase provided" "mycmd failed. Dr
opping to a shell ..."  "$info"

            if [ ! "$_ud_result" -eq 0 ]; then
                ud_ret=1
                [ ! "$QUIET" ] && d_do verbosity=7 sh_countdown $(( crypt_wait * 5 )) "$d
ebug"
                [ ! "${init_lock_down}" ] && rescue_shell
            fi;
            ;;
        "key")
            # passdevice already mounted,
            # Use cryptsetup option for key file -d <mount_point><keyfile> and -S <key sl
ot #>
            # unmount the passdevice when we are done with it
            _ud_keyfile="${crypt_keydev_mountpoint%/}/${crypt_keyfile#/}"  # ensure exact
ly one slash between mount point and keyfile
            j_msg -6 "About to run cryptsetup -d ${_ud_keyfile} -S 1 luksOpen ${_ud_devic
e} ${edev} "
            cryptsetup -d "${_ud_keyfile}" -S 1 luksOpen "${_ud_device}" "${edev}"
            _ud_result=$?
            j_msg -6n "unlock result:"
            handle_result $_ud_result "" "" "$info"
            if [ "$_ud_result" -eq 0 ]; then
                j_msg -6 "Unlocked ${_ud_device} ${edev} with key found at ${crypt_keydev
_mountpoint}${crypt_keyfile}"
            else  # cryptsetup luksOpen with key failed
                ud_ret=1
                _ud_msg="${BRon}Error: Could not unlock ${BYon}${_ud_device} ${edev}"
                _ud_msg="${_ud_msg} ${BRon}with key found at ${BMon}${_ud_keyfile}${Boff}
"
                j_msg -3 "${_ud_msg}"
                if [ ! "${init_lock_down}" ]; then
                    j_msg -3 "${BRon}Dropping to a shell${Boff}"
                    sh_countdown $(( crypt_wait * 5 ))
                    rescue_shell
```

```
                fi
            fi
            ;;
    esac

    unset -v _ud_method _ud_pass _ud_keyfile _ud_result _ud_msg _ud_device _ud_edev
    return $ud_ret
}
# @usage _unlock_device <method> <device>
# @ret 0 on success; 1 on failure; may call rescue_shell on fatal errors
# @vars _ud_method _ud_device edev _ud_keyfile _ud_result ud_ret
# @deps _get_luks_edev_name j_msg -6 j_msg -6n handle_result
# @cont j_msg -3 die cryptsetup sh_countdown rescue_shell
# @rule determines edev via _get_luks_edev_name before unlocking
# @rule supports two unlock methods: "prompt" (heredoc passphrase)
# @cont and "key" (keyfile via -d and slot via -S)
# @rule heredoc for passphrase must not be indented with spaces
# @rule on failure, may delay via sh_countdown and drop to rescue_shell
# @cont unless init_lock_down is set
# @note requires crypt_keydev_mountpoint and crypt_keyfile for key mode
# @note exports no globals except edev (via helper); cleans up locals
# @note prints INFO/ERROR messages using messaging-layer wrappers
# @note operates at INFO severity except for fatal die() paths

_get_luks_keyfile() # (POSIX) find and mount keydev and confirm it has key
{ glk_ret=0
  # Note: crypt_keydev_mountpoint must be globally defined by the calling script
  [ $# -lt 1 ] && { j_msg -3 "please specify crypt_keydev as arg \$1"; return 1 ; }
  _glk_crypt_keydev="$1"
  # ensure trailing slashes are removed for a clean grep match
  _glk_mnt_fixed="${crypt_keydev_mountpoint%/}"
  # determine keydev type
  # if it starts with "/" it is a device name
  # if it contains "-" it may be a UUID
  # if it does not, it may be a LABEL
  j_msg -7n "examining _glk_crypt_keydev [ ${_glk_crypt_keydev} ] ..."
  # determine keydev type (POSIX case is faster than grep)
  case "${_glk_crypt_keydev}" in
    /*)   _glk_crypt_keydev_type="devicename" ;;
    *-*)  _glk_crypt_keydev_type="UUID" ;;
    *)    _glk_crypt_keydev_type="LABEL" ;;
  esac
  # handle all cases as valid until examined further below
  handle_result 0 "${Boff}${Mon}${_glk_crypt_keydev_type}" "" "$debug"

  # check if there actually is such a device in the system
  case "$_glk_crypt_keydev_type" in
    "devicename" ) [ -b "$_glk_crypt_keydev" ] ; _glk_result=$? ;;
    "UUID"       ) blkid | grep -q "UUID=\"$_glk_crypt_keydev\"" ; _glk_result=$? ;;
    "LABEL"      ) blkid | grep -q "LABEL=\"$_glk_crypt_keydev\""; _glk_result=$? ;;
  esac
  if [ "$_glk_result" -eq 0 ]; then
    j_msg -7 "crypt_keydev [$_glk_crypt_keydev] is of type [$_glk_crypt_keydev_type] and
exists"
  else
    j_msg -7 "crypt_keydev [$_glk_crypt_keydev] is of type [$_glk_crypt_keydev_type] but
there is no such device"
    return 2
  fi
  # if we got here, it does exist in the system, so if _glk_crypt_keydev is not mounted,
try to mount it
  # safe grep: match space-padded " /path " exactly to avoid partial directory false-matc
hes
  # check /proc/mounts directly as the source of truth
  j_msg -7n "checking if ${_glk_crypt_keydev} is mounted"
  if grep -q " ${_glk_mnt_fixed} " /proc/mounts; then
    handle_result 0 "mounted" "" "$debug"
  else
    j_msg -7np " (${BYon}not mounted${Boff})"
    j_msg -7n "trying to mount (${_glk_crypt_keydev_type}) [${_glk_crypt_keydev}] ..."
```

```
    case "$_glk_crypt_keydev_type" in
      "devicename" ) mount "${_glk_crypt_keydev}" "${_glk_mnt_fixed}"; _glk_result=$? ;;
      "UUID"       ) mount UUID="${_glk_crypt_keydev}" "${_glk_mnt_fixed}"; _glk_result=$
? ;;
      "LABEL"      ) mount LABEL="${_glk_crypt_keydev}" "${_glk_mnt_fixed}"; _glk_result=
$? ;;
    esac
    case "$_glk_result" in
      0 ) handle_result 0 "mounted" "" "$debug";;
      1 ) handle_result 1 "" "mount failure (device missing, wrong FS, permission, etc)"
"$debug";;
      2 ) handle_result 2 "" "mount usage/syntax error" "$debug";;
      32 ) handle_result 32 "" "filesystem helper error (only for helper-based FS types)"
 "$debug";;
      * ) handle_result 99 "" "mount failure of unknown type" "$debug";;
    esac
    if [ ! "$_glk_result" -eq 0 ]; then
        unset -v _glk_mnt_fixed _glk_crypt_keydev _glk_crypt_keydev_type _glk_result
        return 3    # mount failure
    fi  # mount failure
  fi  # mounted ?
  j_msg -7n "looking for keyfile"
  [ -f "${_glk_mnt_fixed}/crypt/dat" ]
  handle_result $? "found" "not found" "$debug" || return 4    # has key check failure
  # set keyfile pathname
  export keyfile="${_glk_mnt_fixed}/crypt/dat"
  unset -v _glk_mnt_fixed _glk_crypt_keydev _glk_crypt_keydev_type _glk_result
  return $glk_ret
}
# @usage _get_luks_keyfile <key_device>
# @args $1: the device containing the key (dev path, UUID, or LABEL)
# @vars crypt_keydev_mountpoint (required global), keyfile (exported)
# @deps mount grep right_status j_msg -5n die
# @rule identifies device type (dev/UUID/LABEL) using POSIX case patterns
# @rule verifies mount status via /proc/mounts with space-padding to prevent false matche
s
# @ret 0: success; 1: invalid type; 2: mount fail; 3: has key fail
# @note Note: crypt_keydev_mountpoint must be globally defined by the calling script
# @note exports the $keyfile variable for use by cryptsetup or other LUKS tools
# @note operates at debug severity

_maybe_select_keying_device() {
    # go through the list of devices presented as argv -- select the first one that
    #   has a valid type, actually exists, mounts successfully, and contains a key,
    #   else fail upon exhaustion of the list
    # If select_keying_device successfully validates a keying device and keyfile,
    #   it effectively transitions the system from a "searching" state
    #   to an "authorized" state for LUKS decryption
    # assign global unlock_method
    #
    mskd_ret=0
    _mskd_HAS_KEY=$FALSE
    # set the input string (list of candidate keying devices) so we can loop thru them sa
fely
    # shellcheck disable=SC2048
    # shellcheck disable=SC2086
    # Note: we want word splitting here to loop through the list of single-word devices,
    #   so we disable the SC2048/SC2086 warnings about this
    set -- $*
    _mskd_candidate_key_device=""

    # careful: this loop runs for every candidate in the list - it can take a long time w
ith several second delays in each pass
    _mskd_count=1
    while [ $# -gt 0 ]; do
        _mskd_candidate_key_device="$1"
        # use get_luks_keyfile to find and mount the device if possible
        # it returns 0: success; 1: invalid; 2: mount failed; 3: has key failed
        # it reads global crypt_keydev_mountpoint and exports keyfile
```

```
        j_msg -7 "Calling _get_luks_keyfile ${_mskd_candidate_key_device}"
        # call get_luks_keyfile to examine this device
        _get_luks_keyfile "${_mskd_candidate_key_device}"
        _mskd_result=$?
        j_msg -7n "Done with _get_luks_keyfile; result:"
        handle_result $_mskd_result "keyfile: $keyfile" "failed with exit code [$_mskd_re
sult]" "$debug"
        d_do '
          case "$_mskd_result" in
            0 ) j_msg -7 "(debug) exit code [$_mskd_result] means \"success\"" ;;
            1 ) j_msg -7 "(debug) exit code [$_mskd_result] means \"no device identified
in call\"" ;;
            2 ) j_msg -7 "(debug) exit code [$_mskd_result] means \"device does not exist
 in the system\"" ;;
            3 ) j_msg -7 "(debug) exit code [$_mskd_result] means \"device exists, but mo
unt failed\"" ;;
            4 ) j_msg -7 "(debug) exit code [$_mskd_result] means \"device exists, mounte
d, but did not have key\"" ;;
            * ) j_msg -7 "(debug) exit code [$_mskd_result] means \"invalid exit code\""
;;
          esac
        ' "$debug"
        # check for the actual keyfile
        if [ $_mskd_result -eq 0 ]; then
          _mskd_HAS_KEY=$TRUE
          break # done - we have what we came for
        else
          # un-mount if it got mounted but did not have the key
          if [ "$_mskd_result" -eq 4 ]; then
            j_msg -7n "un-mounting candidate that did not have keyfile"
            umount "${crypt_keydev_mountpoint}" >/dev/null 2>&1
            handle_result $? "un-mounted" "failed" "$debug" || die "failed to umount ${cr
ypt_keydev_mountpoint}"
          fi  # mounted but no key
        fi  # get_luks_keyfile result
        shift  # go to next candidate device in list
    done
    # to leave the loop above, we either --
    #    (a) found a valid extant device, mounted it and confirmed it contains a key (_ms
kd_HAS_KEY is now "true"), or
    #    (b) exhausted the crypt_keydev_list of devices authorized to be used and found n
one of them
    #        on this system (in which case _mskd_HAS_KEY is still "false")
    if [ "$_mskd_HAS_KEY" ]; then
        j_msg -7 "Keystore: [$keyfile] exists"
        mskd_ret=0
    else
        j_msg -3 "${BRon}Keystore: [$keyfile] does not exist${Boff}"
        mskd_ret=1
    fi
    unset -v _mskd_HAS_KEY _mskd_candidate_key_device _mskd_result
    return $mskd_ret
}
# @usage _maybe_select_keying_device <candidate-device-list>
# @args <candidate-device-list> - list of candiate devices to examine
# @rule goes through the list of devices presented as argv -- selects the first one that
# @cont has a valid type, actually exists, mounts successfully, and contains a key,
# @cont else fails upon exhaustion of the list
# @note if _maybe_select_keying_device successfully validates a keying device and keyfile
,
# @cont it effectively transitions the system from a "searching" state
# @cont to an "authorized" state for LUKS decryption
# @note operates at debug severity

_get_unlock_method()
{   gum_ret=0
    j_msg -6n "Checking if crypt_keydev_list was specified ..."
    if [ ! -z "$crypt_keydev_list" ]; then _gum_PDL=$TRUE; else _gum_PDL=$FALSE; fi
    [ "$_gum_PDL" ] ; handle_result "$?" "yes" "no" "$info" || \
        { export unlock_method="prompt"; return 0; }
```

```
    j_msg -6n "Checking if crypt_keydev_mountpoint was specified ..."
    if [ ! -z "$crypt_keydev_mountpoint" ]; then _gum_PDMP=$TRUE; else _gum_PDMP=$FALSE;
fi
    [ "$_gum_PDMP" ] ; handle_result "$?" "yes" "no" "$info" || \
        { export unlock_method="prompt"; return 0; }

    j_msg -6n "Checking if crypt_keyfile was specified ..."
    if [ ! -z "$crypt_keyfile" ]; then _gum_KF=$TRUE; else _gum_KF=$FALSE; fi
    [ "$_gum_KF" ] ; handle_result "$?" "yes" "no" "$info" || \
        { export unlock_method="prompt"; return 0; }

    # if all tree are specified, call _maybe_select_keying_device with the global crypt_k
eydev_list as arg
    j_msg -6 "Calling _maybe_select_keying_device"
    _maybe_select_keying_device "$crypt_keydev_list" ; _gum_result=$?
    # if a valid keying device was identified and assigned, then assign unlock_method=key
    # otherwise assign unlock_method=prompt
    j_msg -6n "Done with _maybe_select_keying_device; result:"
    handle_result $_gum_result "keyfile: $keyfile" "no valid keyfile found" "$info"
    if [ "$_gum_result" -eq 0 ] && [ ! -z "$keyfile" ] ; then
        export unlock_method="key"
    else
        export unlock_method="prompt"
    fi

    unset -v _gum_PDL _gum_PDMP _gum_KF
    return $gum_ret
}
# @usage _get_unlock_method
# @args none
# ret emit set global unlock_method; return 0: success; 1: error
# @vars crypt_keydev_list crypt_keydev_mountpoint crypt_keyfile
# @rule if any of the global vars above is null use prompt method
# @rule if no candidate in crypt_keydev_list that actaully has a key, use prompt method
# @note operates at INFO severity

unlock_luks_devices()
{   ulds_ret=0; _ulds_device=""
    separator "${PN}-${BUILD}" "(unlock_luks_devices)"
    # guard: reject empty device list
    [ $# -eq 0 ] && \
        { j_msg -3 "unlock_luks_devices requires a non-emply device list"; return 1; }

    j_msg -7 "(debug) in unlock_luks_devices with \$2: [$@]"

    # determine unlock_method
    j_msg -5 "Determining unlock method"
    _get_unlock_method
    j_msg -5n "Done determining unlock method:"
    handle_result $? "method: $unlock_method" ""

    # if unlock_method is prompt, then prompt for passphrase so it can be used for all un
locks
    # securely read the passphrase with POSIX ask_pass function if needed
    if [ "$unlock_method" = "prompt" ]; then
        _ulds_pass=$(ask_pass "Enter LUKS passphrase: ")
        export _ulds_pass
    fi

    # call _unlock_device for each device in the list (passed as argv to this function)
    while [ $# -gt 0 ]; do
        _ulds_device="$1"
        j_msg -5 "Unlocking device: $_ulds_device"
        j_msg -7 "(debug) unlock_method: $unlock_method"
        j_msg -7 "(debug) _ulds_device: $_ulds_device"
        _unlock_device "$unlock_method" "$_ulds_device" ; _ulds_result=$?
        j_msg -5n "Done unlocking device; result:"
        handle_result $_ulds_result "" ""
        shift 1
```

```
    done

    # if unlock_method is prompt, then erase the exported value, now that we are done wit
h it
    if [ "$unlock_method" = "prompt" ]; then export _ulds_pass=""; fi

    unset -v _ulds_device_list _ulds_device _ulds_result
    return $ulds_ret
}
# @usage unlock_luks_devices <device_list>
# @args <device_list> - list of device(s) to unlock
# @vars crypt_keydev_list crypt_keydev_mountpoint crypt_keyfile
# @rule if global var above is null use prompt method
# @rule if no candidate in crypt_keydev_list that actaully has a key, use prompt method
# @note operates at NOTICE severity

# @note =====[ Section 8: Command Line Interface Framework ]============================
=====================================

# NOTE: varlist variables are initialized by app_configure_posix() and displayed by show_
config();
# the basic set of varlist variables is defined in script_header_joetoo and can be extend
ed
# either in the using script or in its .conf files.
## note: all varlist variables (including logFile and verbosity) get anulled by initializ
e_vars,
# so they must be explicitly reassigned (as app_configure_posix does for this set).
# the easiest way to ensure custom additions to varlist are properly initialized and then
 reassigned
# prior to main script use is to place the extending definitions in .conf file in the
# /etc/<package_name> configuration directory ($config_dir).  A commented out copy of thi
s
# information is provided in the template config file to exemplify
varlist="PN BUILD user logFile status_file config_dir config_files verbosity loglevel"
varlist="${varlist} num_cmds starting_step stopping_step BREAK"
varlist="${varlist} bool.INTERACTIVE bool.RESUME bool.LOGGING bool.QUIET bool.DEBUG"
varlist="${varlist} bool.EMOJI_CAPABLE bool.EMOJI_DISABLE"
varlist="${varlist} bool.handle_local_arg_EXISTS bool.handle_local_compound_arg_EXISTS"
varlist="${varlist} bool.handle_local_usage_EXISTS"
varlist="${varlist} app_customization_root local_cmdline_arg_handler"

app_configure() # (POSIX) initialize, set defaults, load config files, define cmd sequenc
e
{
  separator "$PN" "(configuring)"
  _c_FLAGGED=$FALSE

  # sets all variables named in varlist to null/$FALSE/No
  if [ ! "$DEBUG" ] && [ "$verbosity" -lt "$info" ] ; then
    # provide anounce/act/report-result summary report for initialize_vars
    # which operates at info severity
    j_msg -5n "initializing variables ..."
    initialize_vars $varlist; right_status $?
  else
    initialize_vars $varlist  # at verbosity -ge "$info" it will report for itself, in de
tail
  fi

  # tier 0 - prerequisites (note: can’t log until logging is set up below
  # initialize package name for this script
  j_msg -6n "assigning PN"
  PN="${0##*/}"   # equivalent to PN=$(basename "$0") but w/o subshell
  handle_result $? "$PN" ’’ "$info" || _c_FLAGGED=$TRUE

  # identify and assign user name
  j_msg -6n "assigning user"
  user=$(whoami)
  handle_result $? "$user" ’’ "$info" || _c_FLAGGED=$TRUE

  # assign and/or check logFile (may have been set by config
```

```
  # (do this after $user is assigned but before starting logging
  j_msg -6 "draft logFile name: /var/log/${PN}.log"
  j_msg -6n "assigninging logFile"
  if [ -z "$logFile" ]; then
    logFile="/var/log/${PN}.log"
    handle_result $? "now assigned: $logFile" "error assigning logFile" "$info" || _c_FLA
GGED=$TRUE
  else
    d_do '
      printf '%b' " (${BYon}already assigned: ${Gon}$logFile${Boff})"
      right_status 0
    ' "$info"
  fi
  # if it exists already, rotate it (with sudo, in a subshell)
  if [ -f "$logFile" ]; then
    # shift existing archives (delete the oldest, move others up)
    # use sudo sh -c "..." so we only need to call sudo once if needed for $user
    # (note: with " outside, the ' inside is just a literal single-quote
    #  char - interpreted AFTER expansion)
    j_msg -6n "rotating existing logFile(s)"
    sudo sh -c "
      [ -f '${logFile}.2' ] && mv -f '${logFile}.2' '${logFile}.3'
      [ -f '${logFile}.1' ] && mv -f '${logFile}.1' '${logFile}.2'
      mv -f '$logFile' '${logFile}.1'
      touch '$logFile'
      chown '${user}:${user}' '$logFile'
    " >/dev/null 2>&1
    handle_result $? "rotated" "error rotating logFile" "$info" || _c_FLAGGED=$TRUE
  else
    # create it and set permissoins
    j_msg -6n "creating logFile"
    sudo sh -c "
      touch '$logFile'
      chown '${user}:${user}' '$logFile'
    " >/dev/null 2>&1
    handle_result $? "created" "error rotating logFile" "$info"|| _c_FLAGGED=$TRUE
  fi

  # NOTE: *** Initialize LOGGING, INTERACTIVE, and verbosity BEFORE sourcing config, so t
hat
  # the config file can override these defaults ***

  # set QUIET to default $FALSE only if unset; - can be overridden by sourced config belo
w (and cmdline)
  j_msg -6n "assigning QUIET if unset; result:"
  : "${QUIET:=$FALSE}";   export QUIET
  handle_result $? "$(TrueFalse $QUIET)" '' "$info" || _c_FLAGGED=$TRUE

  # set DEBUG to default $FALSE only if unset; - can be overridden by sourced config belo
w (and cmdline)
  j_msg -6n "assigning DEBUG if unset; result"
  : "${DEBUG:=$FALSE}";   export DEBUG
  handle_result $? "$(TrueFalse $DEBUG)" '' "$info" || _c_FLAGGED=$TRUE

  # initialize LOGGING - can be overridden by sourced config below (and cmdline)
  j_msg -6n "assigning default LOGGING"
  export LOGGING="$TRUE"
  handle_result $? "$(TrueFalse $LOGGING)" '' "$info" || _c_FLAGGED=$TRUE

  # initialize INTERACTIVE - can be overridden by sourced config below (and cmdline)
  j_msg -6n "assigning default INTERACTIVE"
  export INTERACTIVE=$TRUE
  handle_result $? "$(TrueFalse $INTERACTIVE)" '' "$info" || _c_FLAGGED=$TRUE

  # set default verbosity if not set externally
  j_msg -6n "maybe assigning verbosity"
  if [ -z "$verbosity" ] ; then
    verbosity=5
    handle_result $? "now set: $verbosity" '' "$info" || _c_FLAGGED=$TRUE
  else
```

```
    handle_result 0 "already set: $verbosity" ’’ "$info"
  fi

  # set default loglevel if not set externally
  j_msg -6n "maybe assigning loglevel"
  if [ -z "$loglevel" ] ; then
    loglevel=5
    handle_result $? "now set: $loglevel" ’’ "$info" || _c_FLAGGED=$TRUE
  else
    handle_result 0 "already set: $loglevel" ’’ "$info"
  fi

  # initialize EMOJI_CAPABLE - is set automatically, redo that after initialization
  j_msg -6n "running is_utf8_terminal to assign EMOJI_CAPABLE"
  is_utf8_terminal && export EMOJI_CAPABLE="$TRUE"
  handle_result $? "$(TrueFalse $EMOJI_CAPABLE)" ’’ "$info" || _c_FLAGGED=$TRUE

  # initialize EMOJI_DISABLE - can be overridden by sourced config below (and cmdline)
  j_msg -6n "assigning defalut user setting for EMOJI_DISABLE"
  export EMOJI_DISABLE="$FALSE"
  handle_result $? "$(TrueFalse $EMOJI_DISABLE)" ’’ "$info" || _c_FLAGGED=$TRUE

  # tier 1 - global configuration
  # validate package config directory
  j_msg -6n "assigning config_dir"
  config_dir="/etc/${PN}"
  handle_result $? "$config_dir" "error assigning config_dir" "$info" || _c_FLAGGED=$TRUE
  validate_dir "$config_dir"   # does its own announce/act/report-results

  # BUILD="string" should be located in config_dir
  j_msg -6n "sourcing BUILD variable assignment"
  . "${config_dir}/BUILD"
  handle_result $? "$BUILD" ’’ "$info" || _c_FLAGGED=$TRUE

  # initialize RESUME - can be overridden by sourced config below (and cmdline)
  j_msg -6n "assigning default RESUME"
  export RESUME=$FALSE
  handle_result $? "$(TrueFalse $RESUME)" ’’ "$info" || _c_FLAGGED=$TRUE

  # if the package has a config file at /etc/$PN configure it here
  j_msg -6 "looking for config file(s) in /etc/${PN}"
  config_files=""
  validate_dir "/etc/${PN}"
  # match config files or links in the directory only (no subdirectories)

  # Start with the base directory
  _c_find_args="/etc/${PN}"                             # search target
  _c_find_args="$_c_find_args -maxdepth 1"                  # top level only (no subdirec
tories)
  _c_find_args="$_c_find_args -mindepth 1"                  # exclude the dir itself
  _c_find_args="$_c_find_args ’(’"                           # start type group
  _c_find_args="$_c_find_args -type f"                      # look for files
  _c_find_args="$_c_find_args -o"                           # OR
  _c_find_args="$_c_find_args -type l"                      # look for symlinks
  _c_find_args="$_c_find_args ’)’"                           # end type group
  _c_find_args="$_c_find_args -name ’*.conf’"               # matching *.conf
  # this find command produces output that is <newline>-delimited lines

#  d_do ’
#    j_msg -7 "about to run ${Gon}find $_c_find_args${Boff}"
#    j_msg -7 "(these are the find results that should get sourced) --"
#    eval "find $_c_find_args"
#  ’ "$debug"

  # read output of find with a while loop (Do NOT quote $_c_find_args here so it splits i
nto arguments)
  # IFS= means do NOT separate the output line into words (guards against file names with
 whitespace)
  # also - read with heredoc technique rather than pipe to while because | creates a subs
hell and the
```

```
  # config_files list and any variables set by sourcing _c_file would lost when the subsh
ell exits
  while IFS= read -r _c_file; do
    [ -z "$_c_file" ] && continue   # just ignore any blank lines in output
    # if _c_file is a symlink, make sure it is readable
    if [ -r "$_c_file" ]; then
      # recursion-check the list of config files to avoid sourcing it again (in case it t
ries to source itself)
      case " $config_files " in *" $_c_file "*)
          j_msg -4 "Recursion detected! Skipping already-sourced file: $_c_file"
          continue ;;  # go on to the next item in the while loop
      esac
      # accumulate found files in config_files, prepending a space only if it is not the
first entry
      config_files="${config_files:+ }$_c_file"
      j_msg -6n "sourcing $_c_file"
      . "$_c_file"
      handle_result $? "$_c_file" "" "$info" || _c_FLAGGED=$TRUE
    else
      j_msg -4 "skipping unreadable or broken symlink [$_c_file]"
    fi
  done <<EOF
$(eval "find $_c_find_args")
EOF
  j_msg -6 "config_files: [$config_files]"

  # tier 2 - application configuration
  # assign and validate app_customization_root
  # this is normally /etc/${PN}
  # (user can override in .conf file(s) defined above - making it ! -z)
  j_msg -6n "assigning app_customization_root variable"
  [ -z "$app_customization_root" ] && app_customization_root="/etc/${PN}/"
   handle_result $? "$app_customization_root" "error assigning app_customization_root" "$
info" || _c_FLAGGED=$TRUE
  j_msg -6 "validating app_customization_root [${Mon}$app_customization_root${Boff}]"
  validate_dir "$app_customization_root"

  # assign and validate status_file
  # this is NOT the app_customization_root, it is normally in the user's $HOME/.config/
  # (user can override in .conf file(s) defined above - making it ! -z)
  j_msg -6n "assigning status_file variable"
  [ -z "$status_file" ] && status_file="${HOME}/.config/${PN}.status"
   handle_result $? "$status_file" '' "$info" || _c_FLAGGED=$TRUE
  validate_file "$status_file"   # note this also validates the directory

  # tier 3 - application customization and extension
  # this should reside in app_customization_root directory
  # contains handle_local_arg(), handle_local_compound_arg(), and handle_local_usage
  # find (validates) *_local.cmdline_arguments; assign first match
  j_msg -6n "looking for *local.cmdline_arg_handler "
  _c_findcmd="find \"${app_customization_root%/}/\" -maxdepth 1 "     # look in app_custo
mization_root, no subdirectories
  _c_findcmd="${_c_findcmd}-name \"*local.cmdline_arg_handler\" | "   # for *local.cmdlin
e_arg_handler
  _c_findcmd="${_c_findcmd}head -n 1 "                                # take only the fir
st match
  local_cmdline_arg_handler=$(eval "${_c_findcmd}"); _c_result=$?
  _c_handler="${local_cmdline_arg_handler:-"none found"}"
   # =${var##*/} is =$(basename $var) but w/o subshell and function call
   handle_result $_c_result "${_c_handler##*/}" "error" "$info" || _c_FLAGGED=$TRUE
  if [ -f "$local_cmdline_arg_handler" ]; then
    j_msg -6n "sourcing local_cmdline_arg_handler"
    . "$local_cmdline_arg_handler"
    handle_result $? "sourced" "error sourcing local_cmdline_arg_handler" "$info" || _c_F
LAGGED=$TRUE
  fi

  # use POSIX method of indirect variable assignment to display the existence of local ha
ndlers
  for x in handle_local_arg handle_local_compound_arg handle_local_usage; do
```

```
    j_msg -6n "setting variable to display whether $x EXISTS"
    _c_extester="${x}_EXISTS"
    command -v "$x" >/dev/null 2>/dev/null ; _c_result=$?
    if [ "$_c_result" -eq 0 ] ; then eval "${_c_extester}=$TRUE"; else eval "${_c_exteste
r}=$FALSE"; fi
    d_do '
      printf " (%b%s%b)" "$(eval status_color \$$_c_extester)" "$(eval TrueFalse \$$_c_ex
tester)" "${Boff}" "$info"
      right_status "$_c_result"
    ' "$info"
  done

  # user's application script (which sources this header file)
  #   should have a initialize_command_sequence() function
  #   (MUST, if it uses run_sequence)
  # so, check for it and call it if it exists
  if command -v initialize_command_sequence >/dev/null 2>&1; then
    initialize_command_sequence; _c_result=$?
    j_msg -6n "command_sequence initialization status"
    handle_result $_c_result '' '' "$info" || _c_FLAGGED=$TRUE
  fi

  # final sanity checks
  if [ "$INTERACTIVE" ] && [ "$verbosity" -lt "$notice" ]; then
    verbosity="$notice"
    j_msg -6 "${BYon}process_cmdline clamped verbosity to [${Mon}${verbosity}${Boff}] in
INTERACTIVE mode"
  fi

  [ "$_c_FLAGGED" ] && c_ret=1
  unset -v _c_FLAGGED _c_file _c_result _c_handler _c_find_args
  return $c_ret
}
# @usage app_configure
# @vars PN, BUILD, user, logFile, status_file, config_dir, verbosity, INTERACTIVE, RESUME
# @vars num_cmds, starting_step, stopping_step, varlist, FALSE, TRUE
# @deps separator, initialize_vars, j_msg -5, j_msg -5n, handle_result, j_msg -5, j_msg -
5n
# @deps handle_result, sourcing, check_dir, check_file, process_cmdline, initialize_comma
nd_sequence
# @rule initializes and validates the entire script environment before execution
# @ret 0 on success; 1 if _c_FLAGGED is set during configuration
# @rule PN is package name and should be assigned = $(basename "$0") or ${0##*/} to ident
ify the calling script
# @rule POSIX assignment 'BUILD="string"' should reside in "${app_customization_root%/}/B
UILD"
# @rule app_customization_root may contain *local.cmdline_arg_handler extension file
# @cont which can contain handle_local_arg(), handle_local_compound_arg(), and handle_loc
al_usage()

usage()                # explain default usage; mod with local "usage module"
{
  N="${num_cmds}"
  separator "${PN}-${BUILD}" "(usage)"
  j_msg -3 "${BRon}Usage: ${BGon}${PN} [-[options]] ${Boff}"
  j_msg -5 "${BYon}valid commandline options --${Boff}"
  j_msg -5 "  -h | --help.............: print this usage message"
  j_msg -5 "  -I | --interactive......: run supervised; confirm execution of each step"
  j_msg -5 "  -N | --noninteractive...: run un-supervised; automatically do each step"
  j_msg -5 "  -s | --status....: return status (next step, step_number)"
  j_msg -5 "  -r | --resume....: resume at next step in status_file"
  j_msg -5 "  -l | --logging...: turn on logging to logFile"
  j_msg -5 "  -v | --verbose...: increase verbosity"
  j_msg -5 "  -q | --quiet.....: decrease verbosity"
  j_msg -5 "  -V | --VERBOSE | --DEBUG ..: force DEBUG output, ignoring verbosity, unless
 silenced"
  j_msg -5 "  -Q | --QUIET | --SILENCE ..: set QUIET (silent) output, ignoring verbosity
and DEBUG"
  j_msg -5 "  -[1-${N}]..........: save N to status file and resume at step N"
  j_msg -5 "  ${BYon}*${Boff} Single-character options may be combined."
```

```
  j_msg -5 "    e.g. ${BGon}${PN} --verbose -nqr8${Boff} would resume non-interactively"
  j_msg -5 "    (automatic, unsupervised) at step 8 with normal verbosity"
  j_msg -5 "${BMon}Caveat:${Boff}"
  j_msg -5 "   -I (interactive/supervised) is on by default"
  j_msg -5 "   -r (resume) sets starting_step to # in [ $status_file ]"
  j_msg -5 "   -[1-${N}] sets starting_step (default 1 otherwise)"
  # source user-script specific usage-module which should be built in the same format
  j_msg -5 "${BMon}additional ${app_customization_root} - commandline options:${Boff}"


  if command -v handle_local_usage >/dev/null 2>&1; then
     handle_local_usage  || j_msg -3 "error in handle_local_usage"
  fi
  j_msg -5p ""  # conditionalized newline, respecting messaging policy
  # can only display workflow (command sequence) if there is one, so look for $cmd_seq_fi
le
  if [ -f "$cmd_seq_file" ] ; then
    j_msg -5 "${BYon}${PN} workflow sequence (steps):${Boff}"
    for _u_step in $(seq 1 $num_cmds); do
      _u_entry=$(sed -n "${_u_step}p" "$cmd_seq_file") # extract the specific line corres
ponding to the current step counter
      _u_desc="${_u_entry#*${US}}"                      # human readable description
      j_msg -5  "${BBon}[${Boff}${Yon}${_u_step}${BBon}]${Boff} ${Con}${_u_desc}${Boff}"
    done
  fi
  unset -v _u_step _u_entry _u_desc
  leave "normal exit from usage"
}
# @usage usage
# @vars PN, BUILD, num_cmds, status_file, app_customization_root, cmd_seq_file, US
# @deps separator, j_msg -3, j_msg -5, die, sed, seq, printf
# @rule displays standard options, sources local.usage if present, and prints workflow
# @ret calls die, resulting in exit 1 (non-zero) from script
# @note uses _u_ prefix for internal loop variables; unsets before termination

validate_dir()  # validate or create a directory $1
{   vd_ret=0
    [ $# -ne 1 ] && { j_msg -3 "Error: must specify dir_to_validate" ; return 1 ; }
    _vd_dir_to_validate=$1
    j_msg -6n "validating dir ${_vd_dir_to_validate}"
    if [ -d "${_vd_dir_to_validate}" ] ; then
        d_do 'printf "%b%s%b" "${BGon}" " (valid)" "${Boff}"; right_status $TRUE' "$info"
    else
        # use sudo in case current user is not root
        d_do echo_n_msg "${BYon}" " (creating)" "${Boff}" "$info"  # conditionalize echo_
n_msg
        sudo sh -c " mkdir -p '${_vd_dir_to_validate}' >/dev/null 2>&1 "
        handle_result $? "created" "failed to create $_vd_dir_to_validate" "$info" || \
          { unset -v _vd_dir_to_validate; return 1 ; }
    fi
    # set ownership either way, using sudo in case current user is not root
    # (note: chown -R to give all contents to user)
    j_msg -6n "running chown -R ${user}:${user} ${_vd_dir_to_validate}"
    sudo sh -c " chown -R '${user}:${user}' '${_vd_dir_to_validate}' >/dev/null 2>&1 "
    handle_result $? "ownership set" "failed to set ownership" "$info" || \
      { unset -v _vd_dir_to_validate; return 1 ; }
    [ -w "${_vd_dir_to_validate}" ] ; vd_ret=$?     # final validation (writable by user)
    unset -v _vd_dir_to_validate
    return $vd_ret
}
# @usage validate_dir <path>
# @args $1 (string) path to the directory to validate or create
# @deps j_msg -3, j_msg -5n, right_status, handle_result, sudo, mkdir
# @rule ensures a directory exists, creating it with sudo -p if necessary
# @ret 0 if directory exists or was successfully created; 1 otherwise

validate_file()  # validate or create command sequence status file $1
{ vf_ret=0
  [ $# -ne 1 ] && { j_msg -3 "Error: must specify file_to_validate" ; return 1 ; }
  _vf_file_to_validate=$1
```

```
  _vf_dir_to_validate="${_vf_file_to_validate%/*}"   #equivalent to dirname, but w/o subs
hell function call
  # just in case the file is in the current directory (name passed w/o path)
  [ "$_vf_dir_to_validate" = "$_vf_file_to_validate" ] && _vf_dir_to_validate="."
  validate_dir "$_vf_dir_to_validate"
  j_msg -6n "validating file $_vf_file_to_validate"
  if [ -w "${_vf_file_to_validate}" ] ; then
    d_do 'printf "%b%s%b" "${BGon}" " (valid)" "${Boff}"; right_status $TRUE' "$info"
  else
    # use sudo in case current user is not root
    d_do printf " (%b%s%b)" "${BYon}" "creating/fixing" "${Boff}" "$info"
    sudo sh -c "
      touch '${_vf_file_to_validate}' && chown '${user}:${user}' '${_vf_file_to_validate}
'
    " >/dev/null 2>&1
    handle_result $? "file ready for owner" "failed to fix/set ownership of ${_vf_file_to
_validate}" "$info" || return 1
  fi
  # final validation
  [ -f "${_vf_file_to_validate}" ] ; vf_ret=$?
  unset -v _vf_file_to_validate _vf_dir_to_validate
  return $vf_ret
}
# @usage validate_file <path>
# @args $1 (string) path to the status file to validate or create
# @vars  user
# @deps j_msg -3, validate_dir, j_msg -5n, right_status, handle_result, sudo, dirname
# @rule ensures file exists; initializes with '1' and sets ownership to $user if created
# @ret 0 if file exists or was successfully created; 1 otherwise

process_cmdline() # (POSIX) process command line arguments
{ pc_ret=0
  _pc_arglist="$@"
  # pre-define a dynamic macro for adjusting verbosity
  # usage example: _pc_operator="+"; eval "$_pc_adjust_verbosity"
  _pc_adjust_verbosity="verbosity=\$(( verbosity \${_pc_operator} 1 ))"

  _pc_last=""
  j_msg -5 "processing command line with [ ${_pc_arglist} ]"
  # shift each argument into position $1 and examine it
  #   process the argument or process it with its own arguments
  while [ ! -z "$1" ]; do
    j_msg -6 "arg1 = [ $1 ]"
    # if arg begins with a single dash, process it alone
    case $1 in

      -- )     # end of options (possible future use to care for the rest of $*
        j_msg -6 "$1 means \"end of options\" (remainder of \@ is arguments"
        shift; break   # drop the solo '--' with 'shift' and the rest is args 'future to
do'
        ;;

      -[!-]* ) # short option try processing alone, but it could be a clustered compound
argument
        j_msg -6 "processing [ $1 ] alone as single-dash argument"
        process_argument "$1" "$2"   # incl $2 in case $1 uses an operand
        _pc_result=$?
        case "$_pc_result" in
          0|1 ) pc_ret="$_pc_result" ;;  # regular success/fail; pass it on
          6 ) shift; pc_ret=0  ;;        # process_argument consumed an operand, so 'shif
t' extra
          7 ) pc_ret="$_pc_result"; j_msg -3 "pc: arg not found [$1]" usage ;;   # not fo
und
          * ) j_msg -3 "invalid result code from process_argument [$_pc_result]"; pc_ret=
7 ;;    # invalid result code
        esac
        shift  # normal 'shift' to next argument
        ;;

      --* )    # if arg begins with a double dash, process it alone
```

```
        j_msg -6 "processing [ $1 ] alone as double-dash argument"
        process_argument "$1" "$2"  # incl $2 in case $1 uses an operand
        _pc_result=$?
        case "$_pc_result" in
          0|1 ) pc_ret="$_pc_result" ;;  # regular success/fail; pass it on
          6 ) shift; pc_ret=0  ;;        # process_argument consumed an operand, so 'shif
t' extra
          7 ) pc_ret="$_pc_result"; j_msg -3 "pc: arg not found [$1]" usage ;;   # not fo
und
          * ) j_msg -3 "invalid result code from process_argument [$_pc_result]"; pc_ret=
7 ;;    # invalid result code
        esac
        shift  # normal 'shift' to next argument
        ;;

      * )      # invalid
        j_msg -3 "pc: $1 does not start with - or --"
        usage  # usage will exit or return status 1
        ;;
    esac
  done
  if [ "$INTERACTIVE" ] && [ "$verbosity" -lt "$notice" ]; then
    verbosity="$notice"
    j_msg -6 "${BYon}process_cmdline clamped verbosity to [${Mon}${verbosity}${Boff}] in
INTERACTIVE mode"
  fi
  j_msg -6 "done with process_command_line"
  j_msg -4 "remaining arguments: [$*]"

  unset -v _pc_last _pc_arglist _pc_result _pc_adjust_verbosity _pc_operator
  return $pc_ret
}
# @usage process_cmdline "$@"
# @args $@: positional parameters from the command line
# @vars verbosity, INTERACTIVE, _pc_result
# @deps j_msg -5, process_compound_arg, shift
# @rule parses short (-q, -v) and long (--noninteractive) options
# @ret 0 on completion
# @note uses _pc_result to track loop status and prevent arithmetic errors

_get_entry() # (POSIX) get human readable description for command sequence step number
{ ge_ret=0; if ! isint "$1"; then j_msg -3 "error: zero args; _get_desc requires $1 step
number"; return 1; fi
  _ge_entry=$(sed -n "${1}p" "$cmd_seq_file") # extract the specific line corresponding t
o the current step counter
  _ge_desc="${_ge_entry#*${US}}"  # human readable description
  _ge_raw_cmd="${_ge_entry%%${US}*}"  # the actual command
  _ge_clean_cmd=$(printf -- '%b' "$_pa_cmd" | strip_ansi | _translate_escapes)
#  j_msg -7 "_ge_entry: $_ge_entry"
#  j_msg -7 "_ge_desc: $_ge_desc"
  case $2 in
    1 ) printf '%s' "$_ge_desc" ;;
    2 ) printf '%s' "$_ge_clean_cmd" ;;
    3 ) printf '%s' "$_ge_raw_cmd" ;;
    * ) j_msg -3 "invalid format [$2]"; ge_ret=1
  esac
  unset -v _ge_entry _ge_desc _ge_raw_cmd _ge_clean_cmd
  return $ge_ret
}
# @usage _get_entry <step_number> <format_type>
# @args $1 (int) the 1-indexed step number to retrieve from the command sequence
# @args $2 (int) format type: 1=description, 2=cleaned command, 3=raw command
# @vars cmd_seq_file (global) path to the temporary command sequence file
# @vars US (global) Unit Separator character used as the delimiter
# @vars ret (global) initialized to 0 for exit status tracking
# @deps isint, j_msg -3, sed, printf, strip_ansi, _translate_escapes
# @rule extracts specific lines from the command sequence file and parses by $US delimite
r
# @ret 0 on success (prints requested string to stdout); 1 on invalid arguments
# @note uses _ge_ prefix for local variables; unsets all before returning
```

```
# @note format 2 uses strip_ansi and _translate_escapes for log-safe output

_pc_action_status() # (POSIX) emit status of run_sequenct (next step to run)(last complet
e + 1)
{  _pas_val=1 # initialize
  if [ -f "$status_file" ]; then
    j_msg -6 "${BYon}reading status file: [ ${BWon}${status_file}${BYon}  ]${Boff}"
    read _pas_val < "$status_file"
    # if null/non-int or 0 bump to 1; if already complete, wrap to 1
    if ! isint "$_pas_val"  || [ "$_pas_val" -lt 1 ] || [ "$_pas_val" -ge "$num_cmds" ] ;
 then _pas_val=1; fi
  fi
#  j_msg -7 "_pas_val: $_pas_val"
  _pas_desc=$(_get_entry "$_pas_val" 1)
#  j_msg -7 "_pas_desc: $_pas_desc"

  _pas_msg="${BYon}Status: ${BWon}next step would be [ ${Boff}${Mon}${_pas_val}${BWon} ]"
  _pas_msg="${_pas_msg} ( ${BGon}${_pas_desc}${BWon} )${Boff}"
  j_msg -6 "${_pas_msg}"
  unset -v _pas_val _pas_desc _pas_msg
  # that is it; we are done - do not return from here; use leave() [ more polite than die
() ]
  # to exit/return depending on whether $- contains an "i" (interactive)
  leave
}
# @usage _pc_action_status
# @vars status_file (global) path to the command sequence status file
# @deps j_msg -6
# @rule reads the current step from the status file and calculates the previous step
# @ret emits number of last completed step, returns 0 on completion

_pc_action_resume() # (POSIX) process resume flag
{ par_ret=0
  if [ -f "$status_file" ]; then
    j_msg -6 "${BYon}reading status file: [ ${BWon}${status_file}${BYon}  ]${Boff}"
    read _par_val < "$status_file"
    # if null/non-int or 0 bump to 1; if already complete, wrap to 1
    if ! isint "$_par_val"  || [ "$_par_val" -lt 1 ] || [ "$_par_val" -ge "$num_cmds" ] ;
 then _par_val=1; fi
  else
    # for resume, if the status file does not exist, we can’t continue
    die "_pc_action_resume finds no status file exists"
  fi
  # resume at stored step
  export RESUME=$TRUE
  export starting_step="$_par_val"
  _par_desc=$(_get_entry "$starting_step" 1)
  _par_msg="${BYon}Resuming at step [ ${Mon}$starting_step${BYon} ]"
  _par_msg="${_par_msg} ( ${BGon}${_par_desc}${BYon} )${Boff}"
  j_msg -6 "${_par_msg}"
  unset -v _par_msg _par_desc
  return $par_ret
}
# @usage _pc_action_resume
# @vars RESUME, status_file (global) path to the command sequence status file
# @deps j_msg -6
# @rule reads the current step from the status file and calculates the previous step
# @ret sets global starting_step, and returns 0 on completion

_pc_action_number() # (POSIX) process resume flag
{ pan_ret=0 _pan_arg="$1"
  # if int and in range, save to status_file and resume at this step
  j_msg -7 "pc_an with $1"
  if isint "$_pan_arg" && [ "$_pan_arg" -ge 1 ] && [ "$_pan_arg" -le "$num_cmds" ] ; then
      export RESUME=$TRUE
      export starting_step="$_pan_arg";
      # dont write this step number to the status file, b/c a subsequent -r needs to be a
ble
      # to read the old stored value and override this earlier number;
      # also - the standard is to write the next number when the step is complete (so don
```

```
't write just now)
  else
      j_msg -3 "invalid starting_step [${starting_step}]"
      usage  # does not return here
  fi
  _pan_desc=$(_get_entry "$starting_step" 1)
  _pan_msg="${BYon}Resuming at step [ ${Mon}$starting_step${BYon} ]"
  _pan_msg="${_pan_msg} ( ${BGon}${_pan_desc}${BYon} )${Boff}"
  j_msg -6 "${_pan_msg}"
  unset -v _pan_desc _pan_msg _pan_arg
  return $pan_ret
}
# @usage _pc_action_number
# @vars RESUME, status_file (global) path to the command sequence status file
# @deps j_msg -6
# @rule reads the current step from the status file and calculates the previous step
# @ret sets global starting_step, and returns 0 on completion

_pc_action_verbosity() # (POSIX) increase or decrease verbosity using predefined macro
{ pav_ret=0
  # _pc_adjust_verbosity is a dynamic macro defined at the top of process_cmdline
  # adjust according to operator, if in range, else set-clamp (in case environment had it
 out of range)
  # as of ver 0.3.6, verbosity is aligned with RFC 5424 severity levels from 0: emerg to
7: debug
  j_msg -7 "in _pc_action_verbosity with _pc_operator: [$_pc_operator]"
  j_msg -7 "_pc_adjust_verbosity=[$_pc_adjust_verbosity]"
  case "$_pc_operator" in
    "+" ) if [ "$verbosity" -lt 7 ]; then _pv_action="increased"; else _pv_action="clampe
d"; verbosity=7; fi ;;
    "-" )
      if [ "$INTERACTIVE" ]; then
        if [ "$verbosity" -gt "$notice" ]; then _pv_action="decreased"; else _pv_action="
clamped"; verbosity="$notice"; fi
      else
        if [ "$verbosity" -gt 0 ]; then _pv_action="decreased"; else _pv_action="clamped"
; verbosity=0; fi
      fi
      ;;
    *   ) j_msg -3 "invalid operator [$_pc_operator]; verbosity unchanged: ${Boff}${Con}$
{verbosity}${Boff}"; pav_ret=1 ;;
  esac
  [ ! "$_pv_action" = "clamped" ] && { eval "$_pc_adjust_verbosity"; pav_ret=$? ; }
  j_msg -6 "${BYon}verbosity ${_pv_action}: ${BBon}[${Boff}${Mon}${verbosity}${BBon}]${Bo
ff}"
  unset -v _pv_action
  return $pav_ret
}

_pc_action_quiet() # (POSIX) process QUIET flag
{ paq_ret=0
  if [ "$INTERACTIVE" ]; then        # INTERACTIVE overrides QUIET for cli framework
    export QUIET=$FALSE;
    j_msg -4 "${BYon}(INTERACTIVE override) set QUIET: $(status_color ${QUIET})$(TrueFals
e ${QUIET})${Boff}"
  else
    export QUIET=$TRUE; paq_ret=$?
    j_msg -6 "${BYon}set QUIET: $(status_color ${QUIET})$(TrueFalse ${QUIET})${Boff}"
  fi
  return $paq_ret
}

process_argument() # (POSIX) process one argument $! (considering the next $2 a potential
 operand)
{ pa_ret=0
  j_msg -6 "pa about to process [ $* ]"
  j_msg -6 "1: [ $1 ], 2: [ $2 ]"
  # process command line argument (must be one of the following)
  [ -z "$1" ] && { j_msg -3 "empty arg1 [$1]"; return 1; }
  case "$1" in
```

```
    "-"[s] | "--status"         )      # display status
      _pc_action_status
      pa_ret=$?
      ;;
    "-"[h] | "--help"           )      # help
      usage
      ;;
    "-"[r] | "--resume"         )      # resume at stored step
      j_msg -7 "pa -[rR]): [running _pc_action_resume]"
      _pc_action_resume
      pa_ret=$?
      ;;
    -*[0-9]*  )
      j_msg -7 "pa 0-9)  with $1"
      # what we know: first char is '-' and there is at least one '0-9' in $1;
      # so (for process_argument) if the whole thing is int, in range action it, else han
d off compound
      #  b/c it could be a multi-char arg w letters as well as numbers
      _pa_arg="${1#?}"  # drop the leading "-" (strip shortest match of any single char '
?')
      j_msg -7 "pa 0-9) sriipped 1st char; _pa_arg: $_pa_arg"

      # if the second char is - (e.g. --24) strip that, too
      [ "${_pa_arg%${_pa_arg#?}}" = "-" ] && _pa_arg="${1#?}"
      # if we now have an int action it, else hand off compound
      if isint "$_pa_arg" ; then
        _pc_action_number "$_pa_arg"  # this will include checking if it is in range
        pa_ret=$?
      else
        # it may be a combination of numbers and letters - hand off to process_compound_a
rg
        # (in case we started with --, dont pass $1, put a single '-' back on _pa_arg (b/
c process_compound_arg expects that)
        j_msg -7 "pa case -*[0-9]*): running [process_compound_arg $1]"
        process_compound_arg "-${_pa_arg}"
        pa_ret=$?
      fi
      ;;
    "-"[I] | "--interactive"    )      # interactive
      j_msg -7 "pa -[iI]): [running export INTERACTIVE=$TRUE]"
      export INTERACTIVE=$TRUE;
      pa_ret=$?
      j_msg -6 "${BYon}setting INTERACTIVE: $(status_color ${INTERACTIVE})$(TrueFalse ${I
NTERACTIVE})${Boff}"
      ;;
    "-"[N] | "--noninteractive" )      # non-interactive
      export INTERACTIVE=$FALSE
      pa_ret=$?
      j_msg -6 "${BYon}setting INTERACTIVE: $(status_color ${INTERACTIVE})$(TrueFalse ${I
NTERACTIVE})${Boff}"
      ;;
    "-"[l] | "--logging" )             # logging
      export LOGGING="$TRUE"
      pa_ret=$?
      j_msg -6 "${BYon}setting LOGGING: $(status_color ${LOGGING})$(TrueFalse ${LOGGING})
${Boff}"
      ;;
    "-"[q] | "--quiet"          )       # decrease verbosity
      _pc_operator="-"; _pc_action_verbosity; pa_ret=$?
      ;;
    "-"[v] | "--verbose"        )       # increase verbosity
      _pc_operator="+"; _pc_action_verbosity; pa_ret=$?
      ;;
    "-"[Q] | "--QUIET" | "--SILENCE" )  # set $QUIET
      _pc_action_quiet; pa_ret=$?
      ;;
    "-"[V] | "--VERBOSE" | "--DEBUG" )  # set DEBUG
      export DEBUG=$TRUE; pa_ret=$?
      j_msg -6 "${BYon}setting DEBUG: $(status_color ${DEBUG})$(TrueFalse ${DEBUG})${Boff
}"
```

```
      ;;

    # note: unrecgnized args should fall through to this default
    *                             )
      # run local arg module if it exists (must end with "* ) process_compound_arg $1 ;;"
)
      j_msg -7 "pa case *): w/ \$1: $1"
      # command -v locates a command returns 0 if found (built-in or in PATH) non-zero if
 not found
      if command -v handle_local_arg >/dev/null; then
        j_msg -7 "pa case *): [running handle_local_arg $1 $2]"
        handle_local_arg "$1" "$2"
        # handle_local_arg returns 0: success; 1: error; 6: consumed operand; 7: not foun
d
        pa_ret=$?
        # if not found, give process_compound_arg a chance
        if [ "$pa_ret" -eq 7 ]; then
          j_msg -7 "pa case *): [handle_local_arg did not find \$1; running process_compo
und_arg $1]"
          process_compound_arg "$1"; pa_ret=$?
        fi
      else
        # it still may be a combination of single char args and numbers
        j_msg -7 "pa case *): running [process_compound_arg $1]"
        process_compound_arg "$1"
        pa_ret=$?
      fi
      ;;
  esac
  j_msg -6 "done with process_argument, returning exit status [$pa_ret]"

  unset -v _pa_desc _pa_msg _pa_arg
  return $pa_ret
}
# @usage process_argument <arg> [potential_operand]
# @args $1 (string) the flag or argument to process
# @args $2 (string) the potential_operand following the flag
# @vars status_file, starting_step, num_cmds, verbosity, INTERACTIVE, RESUME
# @deps j_msg -6, j_msg -6, j_msg -3, _get_entry, isint, process_compound_arg
# @deps status_color, TrueFalse
# @rule parses flags (-s, -r, -I, -N, -q, -v, -Q, -V) and numeric resume points (-[0-9])
# @note unsets _pa_ prefixed local variables before return
# @ret return 0 if success and processed no operand
# @ret return 6 if success and processed an operand $2
# @ret return 7 if not found
# @ret return 1 if error

process_compound_arg()    # process a compount argument
{ pca_ret=0
  j_msg -6 "about to process compound [ $* ]"
  # must begin with a single dash | "${var%${var#?}}" = 1st char, or "left_most()" but sa
ve a function call
  if [ ! "${1%${1#?}}" = "-" ]; then j_msg -3 "${E_BAD_ARGS}"; usage; j_msg -3 "exiting p
rocess [$$]"; return 1; fi
  # must not begin with two dashes (would have been picked in process_argument) | "${var%
${var#??}}" = 1st 2 chars
  if [ "${1%${1#??}}" = "--" ]; then j_msg -3 "${E_BAD_ARGS}"; usage; j_msg -3 "exiting p
rocess [$$]"; return 1; fi
  # strip leading dash(es)
  _pca_args="${1##-}"   # strip longest match to - from left
  # handle remaining characters in sequence --
  # ( ***one char at at time except possibly double-digit numbers below *** )
  while [ ! -z "${_pca_args}" ]; do
    j_msg -7 "top of _pca w/ _pca_args: $_pca_args"
    j_msg -7 "left_most: $(left_most "$_pca_args")"
    #handle one character at at time, from the left (last loop action, after esac, is to
strip the first char to "shift"
    case "${_pca_args}" in
      [s]* )  # display status
        _pc_action_status
```

```
        pca_ret=$?
        ;;
      [h]* )      # help
        usage
        ;;
      [r]* )  # resume at stored step
        _pc_action_resume
        pca_ret=$?
        ;;
      [0-9]* ) # resume at this step
        # what we know: 1st char was -, and was stripped; current 1st char is not (guards
 1, 2, 3 above enforce this)
        # we also know 1st char is now a num '0-9' since $_pca_args matched this case
        # since there may be > 10 (i.e. multi-digit) steps, then append whatever consecut
ive digits are there
        _pca_num="${_pca_args%%[!0-9]*}"     # extract ALL consecutive digits from the le
ft
        _pca_args="_${_pca_args#$_pca_num}"  # prepended dummy '_' to remainder, protecti
ng it from final loop-strip below
        # action the result
        _pc_action_number "$_pca_num"  # this will include checking if it is in range
        pca_ret=$?
        ;;
      [I]* )  # interactive
        export INTERACTIVE=$TRUE
        pca_ret=$?
        j_msg -6 "${BYon}setting INTERACTIVE: $(status_color ${INTERACTIVE})$(TrueFalse $
{INTERACTIVE})${Boff}"
        ;;
      [N]* )  # non-interactive
        export INTERACTIVE=$FALSE
        pca_ret=$?
        j_msg -6 "${BYon}setting INTERACTIVE: $(status_color ${INTERACTIVE})$(TrueFalse $
{INTERACTIVE})${Boff}"
        ;;
      [l]* )  # logging
        export LOGGING="$TRUE"
        pca_ret=$?
        j_msg -6 "${BYon}setting LOGGING: $(status_color ${LOGGING})$(TrueFalse ${LOGGING
})${Boff}"
        ;;
      [q]* )  # decrease verbosity
        _pc_operator="-"; _pc_action_verbosity; pca_ret=$?
        ;;
      [v]* )  # increase verbosity
        _pc_operator="+"; _pc_action_verbosity; pca_ret=$?
        ;;
      [Q]* )  # set $QUIET
        _pc_action_quiet; pa_ret=$?
        ;;
      [V]* )  # set DEBUG
        export DEBUG=$TRUE; pa_ret=$?
        j_msg -6 "${BYon}setting DEBUG: $(status_color ${DEBUG})$(TrueFalse ${DEBUG})${Bo
ff}"
      ;;
      *   )    # note: unrecognized args should fall through to this default
        # run local arg module if it exists (must end with "* ) j_msg -3 "${E_BAD_ARGS}"
&& usage ;;")
        j_msg -7 "_pca \*) w/ local_cmdline_compound_arguments: $local_cmdline_compound_a
rguments"
        j_msg -7 "_pca \*) w/ _pca_args: $_pca_args"
        if command -v handle_local_compound_arg >/dev/null; then
          _pca_char="${_pca_args%${_pca_args#?}}"  # peel the first character off
          j_msg -7 "pa case \*): running [handle_local_compound_arg $_pca_char]"
          handle_local_compound_arg "$_pca_char"   # hand off this problematic character
to handle_local_compound_arg
          pca_ret=$?   # 0: success; 6: 1: error
          [ "$pca_ret" -eq 7 ] && { j_msg -3 "${E_BAD_ARGS} [-$_pca_char]"; usage; } # ca
ll usage if not found
        else
```

```
          j_msg -3 "${E_BAD_ARGS}"
          usage
        fi
        ;;
    esac
    #strip first char from _pca_args (i.e. "shift" one character)
    _pca_args="${_pca_args#?}"   # strip 1st char
  done
  j_msg -5 "done with process_compount_arg; returning status code [$pca_ret]"

  unset -v _pca_msg _pca_args _pca_num
  return $pca_ret
}
# @usage process_compound_arg "string"
# @args $1: the compound argument string (e.g., "qv")
# @vars verbosity, _pa_len, _pa_i, _pa_char
# @deps j_msg -6, j_msg -5
# @rule iterates through each character of a compound argument to apply settings
# @ret return 0 if success and processed no operand
# @ret return 7 if not found
# @ret return 1 if error

run_sequence() # (POSIX) run command sequence; $1=status_file, $2=start, $3=stop
{ rs_ret=0
  # guard 1 - minimum arguments check (status, start, stop, + at least 1 cmd)
  [ $# -lt 3 ] && { j_msg -3 "usage: run_sequence <status_file> <start> <stop>"; return 1
; }
  _rs_status_file="$1"           # assign the status file
  _rs_start="${2:-1}"            # guard 2 - start step (default to zero but ensure it is
 an integer in range)
  ! isint "$_rs_start" && { j_msg -3 "run_sequence: start_step '$2' must be an integer 1
- ${num_cmds}"; return 1; }
  if [ "$_rs_start" -lt 1 ] || [ "$_rs_start" -gt "$num_cmds" ]; then
     j_msg -3 "run_sequence: start_step '$2' must be an integer 1 - ${num_cmds}"; return
1; fi
  _rs_stop="${3:-$num_cmds}" # guard 3 - stop step (default to num cmds but ensure it is
an integer in range)
  ! isint "$_rs_stop" && { j_msg -3 "run_sequence: stop_step '$3' must be '' or an intege
r 1 - ${num_cmds}"; return 1; }
  if [ "$_rs_stop" -lt 1 ] || [ "$_rs_stop" -gt "$num_cmds" ]; then
    j_msg -3 "run_sequence: stop_step '$3' must be '' or an integer 1 - ${num_cmds}"; ret
urn 1; fi
  # initialize both starting step (command) and loop counter
  _rs_step="$_rs_start"   # points to the number of the actual command that is "up" next
(gets wrapped to 1 after num_cmds)
  _rs_count="$_rs_start"  # points to current step + 1 (does NOT wrap at num_cmds; so loo
p can detect completion)
  while [ "$_rs_count" -le "$_rs_stop" ]; do        # enforce upper bound
    if [ "$_rs_step" -lt "$_rs_start" ]; then       # enforce lower bound (shift into ran
ge)
      _rs_count=$(( _rs_count + 1 )); _rs_step=$(( _rs_step + 1 )); continue; fi
    # assign commadn and description from input, delimitted by unit separator ($US)
    _rs_entry=$(sed -n "${_rs_step}p" "$cmd_seq_file") # extract the specific line corres
ponding to the current step counter
    _rs_cmd="${_rs_entry%%${US}*}"  # the actual command
    _rs_desc="${_rs_entry#*${US}}"  # human readable description
    _rs_clean_entry=$(printf -- '%b' "$_rs_entry" | strip_ansi | _translate_escapes)
    _rs_clean_cmd=$(printf -- '%b' "$_rs_cmd" | strip_ansi | _translate_escapes)

#    j_msg -7 "_rs_start: $_rs_start"
#    j_msg -7 "_rs_stop: $_rs_stop"
#    j_msg -7 "_rs_step: $_rs_step"
#    j_msg -7 "_rs_clean_entry: $_rs_clean_entry"
#    j_msg -7 "_rs_clean_cmd: $_rs_clean_cmd"
#    j_msg -7 "_rs_desc: $_rs_desc"
#    j_msg -7 "termwidth: $(termwidth)"

    # if INTERACTIVE, confirm before executing the step
    if [ "$INTERACTIVE" ]; then
      _rs_msg="${BYon}Are you ready to [${_data_color}${_rs_step}${BYon}]"
```

```
      _rs_msg="${_rs_msg} (${_op_color}${_rs_desc}${BYon}) ? ${Boff}"
      yns_prompt "$_rs_msg"
      _rs_response="$response"
    else
      _rs_response="yes"
    fi  # INTERACTIVE?
    # execute according to response
    case "$_rs_response" in
      [Yy]* )
        # start with a separator for every step
        separator "$PN" "(step [$_rs_step]: $_rs_desc)"
        j_msg -5 "${BCon}Executing step [${Mon}$_rs_step${BCon}]: ${BYon}${_rs_clean_cmd}
${Boff}"
        # eval expands variables inside the string
        eval "$_rs_cmd"; _rs_result=$?
        ### don't die - let the calling script decide what to do based on return
        ### handle_result $_rs_result || die "failed to execute step [$_rs_step]: $_rs_cl
ean_cmd"
        # display/log result and set bit in rs_ret status binary bitmask
        j_msg -5n "${BWon}step [${Mon}$_rs_step${BWon}] result:"
        handle_result $_rs_result '' "failed with exit code [$_rs_result]" || \
            rs_ret=$(( rs_ret | 1 << _rs_step )) ;;
      [Ss]* )
        _rs_msg="${BYon}Skipping step [${_data_color}${_rs_step}${BYon}]:"
        _rs_msg="${_rs_msg} ${_op_color}${_rs_desc}${BYon} as instructed${Boff}"
        j_msg -5 "$_rs_msg" ;;
      * )
        j_msg -5 "${BRon}Aborting sequence, as instructed${Boff}"
        rs_ret=1; break ;;
    esac
    # use 1-indexed modular increment to wrap _rs_step back to 1 at num_cmds
    # but just increment _rs_count w/o wrap, so loop can detect completion
    _rs_step=$(( (_rs_step % num_cmds) + 1 ))
    _rs_count=$(( _rs_count + 1 ))
    # store the pointer to the command that is actually "up" next (not the loop counter)
    j_msg -6n "writing step (next up) to status_file =>"
    printf '%s\n' "$_rs_step" > "$status_file"
    # demote handle_result to info severity with d_do ... "$info"
    handle_result $? "$_rs_step" "failed to write step [$_rs_step] to status_file [$statu
s_file]" "$info" || rs_ret=1
  done
  unset -v _rs_status_file _rs_step _rs_cmd _rs_result _rs_entry _rs_desc
  unset -v _rs_response _rs_clean_cmd _rs_start _rs_stop _rs_msg _rs_result
  return $rs_ret
}
# @usage: run_sequence <status_file> <start> <stop>
# @args: $1 (string) path to sequence status file
# @args: $2 (int) starting step (1-indexed)
# @rule: $2 start must be '' or integer 1(default) - num_cmds
# @args: $3 (int) stopping step (1-indexed)
# @rule: $3 stop must be '' or integer 1 - num_cmds(default)
# @vars: INTERACTIVE, logFile, PN, US, _data_color, _op_color
# @ret: 0 on success, 1 on abort, or bitmask of failed steps
# @deps: isint, yns_prompt, separator, j_msg -5, handle_result, strip_ansi, _translate_es
capes

linear_search() # (POSIX) linear seart for needle $1 in haystack (global cmd_seq_file)
 { linear_ret=0
  _ls_needle=$1
  # find line number where command (before US) matches needle
  _ls_idx=$(grep -n "^${_ls_needle}${US}" "$cmd_seq_file" | cut -d: -f1)
  printf '%s' "$_ls_idx"
  [ -z "$_ls_idx" ] && linear_ret=1
  unset -v _ls_needle _ls_idx
  return $linear_ret
}
# @usage linear_search <target_string> <file_path>
# @args $1 (string) the command or key to search for
# @args $2 (string) the path to the command sequence file (e.g., $cmd_seq_file)
# @vars US (global) the Unit Separator used as a delimiter in the sequence file
```

```
# @vars ret (global) exit status variable; initialized to 0, not localized
# @deps grep, cut, printf
# @rule searches a file for a record starting with the target string followed by $US
# @ret 0 and prints the line number (1-indexed) if found; 1 if not found
# @note replaces the BASH array-based search from the extended header
# @note uses _ls_ prefix for local variables and unsets before return


# @note =====[ Section 9: Summary & Demonstration Suite ]================================
============================

_sh_render_buffered_tag() {
    [ -z "$_sh_buffer" ] && return   # guard: just return if the buffer is empty
    # if targeting a specific function and this isn't it, kill the buffer
    if [ -n "$_sh_target_fn" ] && [ ! "$_sh_FOUND_TARGET" ]; then
        _sh_buffer=""
        return
    fi
    # map tag to label (the expanded list)
    # note: there is no append tag here because this is the function
    # that finishes what one of these tags started and that
    # append has been acccumulating more of...
    case "$_sh_current_tag" in
        usage) _sh_label="usage" ;;
        opts)  _sh_label="options" ;;
        args)  _sh_label="arguments" ;;
        vars)  _sh_label="variables" ;;
        ret)   _sh_label="returns" ;;
        deps)  _sh_label="dependencies" ;;
        req)   _sh_label="requirements" ;;
        rule)  _sh_label="rules" ;;
        note)  _sh_label="notes" ;;
        warn)  _sh_label="warning" ;;
        ex)    _sh_label="example" ;;
        *)     _sh_label="unknown" ;;
    esac

    # print the buffered block
    printf "    %b%s%b: " "${_data_color}" "$_sh_label" "${Boff}"
    pad "$_sh_label" ': ' "2" "${_data_color}" '.' "${_sh_label_col_width}"
    printf "  "
    smart_wrap_n "$_sh_label_col" "$_sh_label_indent" "" "$_sh_buffer"
    printf "\n"
    _sh_buffer="" # state reset: buffer is now empty
}


summarize_header() # (POSIX) list functions and notes in installed headers
{ _sh_ret=0; _sh_dir="/usr/sbin"
  _sh_QUIET="$FALSE"; _sh_target_fn=""; _sh_filter=""; _sh_buffer=""
  # parse "command line" options
  while [ $# -gt 0 ]; do
    case "$1" in
      -h|--help)
        printf '%s\n' "${BRon}Usage:${Boff} summarize_header [options] [filter]"
        printf '%s\n' "${BYon}Options:${Boff}"
        printf '  -q, --quiet        Functions and top-level descriptions only\n'
        printf '  -f, --func <name>  Deep dive into specific function metadata\n'
        printf '  -h, --help         Show this help\n\n'
        printf '%s\n' "${BMon}Filters:${Boff}"
        printf "  (none)             Default: primary header only\n"
        printf "  '*'                All headers (quote the asterisk)\n"
        printf '  <name>             Specific module (e.g., "unicode")\n'
        return 0
        ;;
      -q|--quiet) _sh_QUIET="$TRUE" ;;
      -f|--func)  shift; _sh_target_fn="$1" ;;  # note shift to get arg
      *)          _sh_filter="$1" ;;
    esac
    shift  # shift to get next cmdline opt/arg
```

```
  done
  # Note: script_header_installed_dir may be defined in/prior to function call
  #   to enable testing development versions
  # define some regex search patterns --
  # _sh_FUNCTION_REGEX: matches any legal POSIX filename
  # ( must start w alpha char or "_", and may include only alpha, numeric, or "_" prior t
o () )
  _sh_FUNCTION_REGEX="^[_[:alpha:]][_[:alnum:]]*\(\)"
  # _sh_METADATA_REGEX: matches summarize_headers() convention for supplemental comments
for users
  # match our new metadata tag format "# @tagname decriptivetext"
  # so look for zero+ whitespace, followed by one+ #,
  # followed by one+ whitespce, followed by @, followed by one+ alphanum
  _sh_METADATA_REGEX="^${W0}#+${W1}@${A1}"
  # default to all script_header_joetoo modules installed in /usr/sbin/
  if [ -d "$script_header_installed_dir" ] ; then
    _sh_dir="${script_header_installed_dir%/}"
  else
    _sh_dir="/usr/sbin"
  fi
  [ ! -d "$_sh_dir" ] && { j_msg -3 "invalid _sh_dir [$_sh_dir]"; return 1 ; }

  # define glob logic (user's proposal)
  if [ -z "$_sh_filter" ]; then
    _sh_header_glob="${_sh_dir}/script_header_joetoo"      # no arg = just base header
  elif [ "$_sh_filter" = "*" ]; then
    _sh_header_glob="${_sh_dir}/script_header_joetoo*"     # literal '*' = all headers
  else
    _sh_header_glob="${_sh_dir}/script_header_joetoo_${_sh_filter}*"    # specific filter
 among _headernames
  fi
  j_msg -5 "looking for headers in $_sh_header_glob"
  # expand the glob into positional parameters
  # to protect against the remote possibility of filename with whitespace
  set -- ${_sh_header_glob}
  # check if the glob actually found files
  if [ ! -f "$1" ]; then j_msg -3 "No header sources found in ${_sh_dir}"; return 1; fi
  # begin output with separator
  separator "summarize_header-${BUILD}" "(content summary)"
  # identify and process all matching installed header files
  for _sh_filespec in "$@"; do
    if [ ! -f "$_sh_filespec" ] ;then
      j_msg -3 "$_sh_filespec not found; continuing"
      continue  # if grep id'd a bad filename, skip to next in for loop
    fi
    # generate a separator for each module
#    separator "${_sh_PN}-${BUILD}" "($(basename ${_sh_filespec}))"
    separator "${_sh_PN}-${BUILD}" "(${_sh_filespec##*/})"   # equivalent to basename but
 w/o subshell function call

    # make two passes through the selected header file, use regex-parsing to select all l
ines
    #   that are either a function definition or a metadata tag/paylod
    # in the first pass, calculate the max length of function names and metadata lables u
sed
    #   so as to be able (in the second pass) to determine starting column and indent int
ended
    #     for both function names and metadata
    # in the second pass through the selected header file, use that info and the length o
f
    #   the current function name/metadata label to calculate the dot-padding required to
    #   provide the desired visual alignment, and generate the appropriately colorized ou
tput

    #-----[ FIRST PASS ]-------------------------------------------------------------
    # measure longest function name in this header file (to determine printf column width
)
    # (extract only the function name to a list, and then calculate the longest)
    if [ -n "$_sh_target_fn" ]; then
      # only measure the target function
```

```
      _sh_fn_list="$_sh_target_fn"
    else
      # measure all functions in the file
      _sh_fn_list=$(grep -E "$_sh_FUNCTION_REGEX" "$_sh_filespec" | cut -d'(' -f1)
    fi
    # FUNCTIONs
    # with the list, calculate the length of the longest function name
    #   add 2 for "()" to get true max displayed length
    _sh_fn_max_len=$(( $(get_longest $_sh_fn_list) + 2 ))
    j_msg -7 "_sh_fn_max_len: $_sh_fn_max_len"
    # assign the internal separator to appear between function name and padding
    _sh_fn_sep=" "
    # calculate length of the column (between the gutters) to be passed to pad()-->smart_
wrap_n
    _sh_fn_col_width=$(( _sh_fn_max_len + ${#_sh_fn_sep} ))
    # determine the offset to the function description column that accomodates
    # the entire column and the gutters - this is where the cursor
    #   will be when smart_wrap_n is called to smart-wrap the description
    #   Math: _sh_fn_max_len + 2c.Lpad + 2c.dots(min) + 2c.Rpad
    _sh_fn_col=$((_sh_fn_col_width + 2 + 2 + 2 ))
    j_msg -7 "_sh_fn_col: $_sh_fn_col"
    # assign the number of the column to which to indent any subseqent lines of the descr
iption
    _sh_fn_indent="${_sh_fn_col}"   # for now, wrap/indent to align directly under start
point

    # METADATA
    # calculate the length of the longest metadata lable to determine the prinf column wi
dth
    #   for metadata (these are known a priori, but change)
    _sh_label_list="usage arguments variables dependencies requirements"
    _sh_label_list="${_sh_label_list} rules notes warning unknown"
    _sh_label_max_len=$(get_longest $_sh_label_list)
    j_msg -7 "_sh_label_list: $_sh_label_list"
    j_msg -7 "_sh_label_max_len: $_sh_label_max_len"
    # assign the internal separator to appear between metadata label and padding
    _sh_label_sep=": "
    # calculate the length of the column (between the gutters) to be passed to pad()
    _sh_label_col_width=$(( _sh_label_max_len + ${#_sh_label_sep} ))
    # determine the offset to the function metadata payload column that accomodates
    # the entire column and the gutters - this is where the cursor
    #   will be when smart_wrap_n is called to smart-wrap the metadata payload
    #   Math: _sh_label_max_len + 4c.Lpad + 2c.dots(min) + 2c.Rpad
    _sh_label_col=$((_sh_label_col_width + 4 + 2 + 2 ))
    j_msg -7 "_sh_label_col: $_sh_label_col"
    # assign the number of the column to which to indent any subsequent lines of metadata
 payload
    _sh_label_indent="${_sh_label_col}"   # for now, wrap/indent to align directly under
start point

    #-----[ SECOND PASS ]-------------------------------------------------------------
    # regex-parse for both '^function()  # comment' and/or '^# *()* comment'
    # ( pick up both function definitions and tagged metadata )
    # ( see regex definitions above )( pipe grep output to a while read loop )
    # process the file line-by-line (aware of state - function/tag)
    while IFS= read -r _sh_line || [ -n "$_sh_line" ]; do
      case "$_sh_line" in
        [a-zA-Z_]*'() '*)        # FUNCTION state - matches POSIX function e.g "func_name
() # comment"
            _sh_render_buffered_tag     # new function, so flush the buffer if there is o
ne
            _sh_tmp="${_sh_line%%'('*}"
            _sh_fn_name="${_sh_tmp% }" # trim trailing space
            _sh_fn_name="${_sh_fn_name}()"
            _sh_comment="${_sh_line#*#}"
            [ "$_sh_comment" = "$_sh_line" ] && _sh_comment=""
            # reset tag state for new function
            _sh_last_label=""
            # if a specific function is targeted, see if it was found and set flag accord
ingly
```

```
            if [ -n "$_sh_target_fn" ]; then
              if [ "$_sh_fn_name" = "${_sh_target_fn}()" ] || [ "$_sh_fn_name" = "$_sh_ta
rget_fn" ]; then
                _sh_FOUND_TARGET="$TRUE"
              else
                _sh_FOUND_TARGET="$FALSE"
                continue
              fi
            fi
            printf '  %b%s%b ' "${_func_color}" "$_sh_fn_name" "${Boff}"
            pad "$_sh_fn_name" ' ' "2" "${_func_color}" '.' "${_sh_fn_col_width}"
            printf "  "
            smart_wrap_n "$_sh_fn_col" "$_sh_fn_indent" "" "$_sh_comment"
            printf "\n"
            ;;

        # match metadata tags: # @tag content
        '# @'[a-z]*' '*)         # TAG state - matches  e.g. '# @note This is a note'
            # strip shortest match for *@ from the left - leaving tag name as first word
            _sh_tag_part="${_sh_line#*@}"      # "note This is a note"
            # strip longest match for ' *' from the right - leaving only tagname
            _sh_tag="${_sh_tag_part%% *}"      # "note"
            # strip shortest match for '* ' from left - remove tag; leave payload
            _sh_payload="${_sh_tag_part#* }"   # "This is a note"
            # skip metadata if we are in quiet mode (unless deep-diving into a specific f
unction)
            [ "$_sh_QUIET" ] && [ -z "$_sh_target_fn" ] && continue
            # skip metadata if we are targeting a specific function and haven't found it
yet
            [ -n "$_sh_target_fn" ] && [ ! "$_sh_FOUND_TARGET" ] && continue
            case "$_sh_tag" in
              append)
                # no flush! just add payload to the existing buffer
                _sh_buffer="${_sh_buffer} ${_sh_payload}"
                ;;
              cont)
                # manual break: flush the bucket first, then print this line solo
                _sh_render_buffered_tag
                printf "%${_sh_label_col}s" ""
                smart_wrap_n "$_sh_label_col" "$_sh_label_indent" "" "$_sh_payload"
                printf "\n"
                ;;
              *)
                # new primary tag: flush the old bucket, start a new one
                _sh_render_buffered_tag
                _sh_current_tag="$_sh_tag"
                _sh_buffer="$_sh_payload"
                ;;
            esac   # tag case
            ;;
      esac   # line cse
    done < "$_sh_filespec"   # while
    _sh_render_buffered_tag  # done pass 2, so flush the buffer in case there is somethin
g "pending"
  done  # outer for
  echo
  j_msg -5 "To run functional tests, use: ${BGon}demonstrate_header${Boff}"
  separator "$(hostname)" "(complete)"
  unset -v _sh_PN _sh_msg _sh_filespec _sh_line _sh_fn_name _sh_comment _sh_dir _sh_heade
r_glob
  unset -v _sh_fn_list _sh_fn_max_len _sh_FUNCTION_REGEX _sh_fn_col _sh_METADATA_REGEX
  unset -v _sh_label_list _sh_label_max_len _sh_tag _sh_label_col _sh_label_indent _sh_la
bel _sh_label_payload
  unset -v _sh_tmp _sh_fn_sep _sh_fn_indent _sh_fn_col_width _sh_label_sep _sh_label_col_
width
  unset -v _sh_buffer
  return 0
}
# @usage summarize_header [filter_pattern]
# @args $1: optional regex/string to filter function names (e.g., "info_")
```

```
# @deps sed grep termwidth
# @rule processes the script source line-by-line to extract function names and @tags
# @rule filters output based on $1 to show specific function families
# @note this is the engine for the 'toc' command and header self-documentation
# @note future: will be extracted to a stand-alone package for broader script analysis
# @append this is a tesst of adding a really long line of additional information actually
 several lines
# @append each with its own append tag, to see them aggregated into one paragraph that ap
pends
# @append to the previous tag, whatever that was
# @rule this is a test to see if I can append text to a rule tag.
# @append this is the appended text

#-----[ Demonstration Sub-function Stubs ]-----------------------------------------------
----------------------

_dh_show_capabilities()   # demonstrate classes of capability in this header file
{ # @args none
  # @rule called by demonstrate_header

# preview the command syntax
  # should print two lines like this but theme-colored --
  # j_msg -5 "Description of header capabilities..."
  # printf "   "; j_msg -5n "; smart_wrap_n 7 7 7 "${BCon}Section: content..."
  # this requires a complex sequence of commands, so build it by line and parts
  # line 1 - j_msg -5 "Description of header capabilities..."
  _dh_cmd="${_func_color}j_msg -5${_Boff} ${_quote_color}\"Description of header capabili
ties...\"${_Boff}\n"
  # line 2 part 1 - printf "   ";    # note ${_Boff} ${_quote_color} in case not bold
  _dh_cmd="${_dh_cmd} ${_cmd_color}printf${_Boff} ${_quote_color}\"    \"${_Boff};"
  # line 2 part 2 - j_msg -5n "     # j_msg -5n displayed as a function
  _dh_cmd="${_dh_cmd} ${_func_color}j_msg -5n${_Boff} ${_quote_color}\"\"${_Boff};"
  # line 2 part 3 - smart_wrap_n 3 3 3  # ${_Boff} ${_data_color} in case not bold
  _dh_cmd="${_dh_cmd}  ${_func_color}smart_wrap_n${_Boff} ${_data_color}3 3 3${_Boff}"
  # line 2 part 4 - "${BCon}Section: content... (followed by each section running at 7 7
7)"
  _dh_cmd="${_dh_cmd} ${_quote_color}\"\${BCon}Section: content...(followed by each secti
on running at 7 7 7)\"${_Boff}"

  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"; printf "\n"
  # execute the actual demonstration
  _dh_msg="This script header is a robust ${BYon}POSIX-compliant utility suite${Boff}. It
s core capabilities include:"
  j_msg -5n ""; smart_wrap_n 3 3 3 "${_dh_msg}"
  printf "\n"

  # Variable Definitions
  _dh_msg="${BCon}Variable Definitions:${Boff} Constants for error messages, pseudo-boole
ans "
  _dh_msg="${_dh_msg}(${BYon}TRUE=0/FALSE=\"\"${Boff}), and terminal state defaults."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"

  # Validation Functions
  _dh_msg="${BCon}Validation Functions:${Boff} Logic to verify specific numeric types lik
e integers, "
  _dh_msg="${_dh_msg}hex values, and floats (${BYon}isint, ishexint, isfloat_posix${Boff}
), consolidated in ${BYon}isnumeric${Boff}."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"

  # System and Shell Checks
  _dh_msg="${BCon}System and Shell Checks:${Boff} Tools to verify root status (${BYon}che
ckroot${Boff}), "
  _dh_msg="${_dh_msg}specific mount points (e.g., ${BYon}checkboot${Boff}), and shell int
eractivity."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"

  # ANSI Escape Sequences
  _dh_msg="${BCon}ANSI Escape Sequences:${Boff} A comprehensive set of variables for text
 styling "
  _dh_msg="${_dh_msg}(colors, bold, underline, reverse) and cursor manipulation (${BYon}H
```

```
CU/SCU, SCP/RCP, CUP${Boff})."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"

  # Messaging tier
  _dh_msg="${BCon}Messaging tier:${Boff} A categorized system including standard (${BYon}
j_msg -5${Boff}), "
  _dh_msg="${_dh_msg}warning (${BYon}j_msg -4${Boff}), and error (${BYon}j_msg -3${Boff})
 outputs with visual color-coded distinction."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"

  # Box Drawing
  _dh_msg="${BCon}Box Drawing:${Boff} Mappings for DEC Special Graphics characters (e.g.,
 "
  _dh_msg="${_dh_msg}${BYon}H_wall='q', V_wall='x', corners${Boff}) and state control (${
BYon}ESCon, SO, SI${Boff})."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"

  # Debugging and Logging
  _dh_msg="${BCon}Debugging and Logging:${Boff} Conditional execution/messaging based on
verbosity "
  _dh_msg="${_dh_msg}levels and timestamped file logging functions (e.g., ${BYon}j_msg -5
${Boff})."
  printf "    "; j_msg -5n ""; smart_wrap_n 7 7 7 "${_dh_msg}"; printf "\n"
  printf "\n"
  unset -v _dh_cmd _dh_msg
  return 0
}
# @usage _dh_show_capabilities
# @args none
# @rule called by demonstrate_header

_dh_demo_verbosity() # (POSIX) use status_color, TrueFalse, etc/to display IAW system ver
bosity
{  demo_verb_ret=0
  separator "demonstrate_header-${BUILD}" "(verbosity)"
  if [ "$verbosity" -gt 0 ]; then _dv_VERBOSE=$TRUE; else _dv_VERBOSE=$FALSE; fi

  # preview the command syntax
  # Construct the "About to run" preview using the theme engine. it should like like this
, but theme-colored:
  #   j_msg -5 "${BYon}_dv_VERBOSE is currently $(status_color $_dv_VERBOSE)$(TrueFalse $
_dv_VERBOSE)${Boff}
  # this involves complex nested expansions, so do it in three pieces we can maintain --
  #   part 1 - j_msg -5 "${BYon}_dv_VERBOSE is currently "
  _dh_cmd="${_func_color}j_msg -5${_Boff} ${_quote_color}\"${_var_color}\${BYon}${_quote_
color}_dv_VERBOSE is currently "
  #   part 2 - $(status_color $_dv_VERBOSE)
  _dh_cmd="${_dh_cmd}\$(${_Boff}${_func_color}status_color${_Boff} ${_var_color}\$_dv_VER
BOSE${_Boff}${_quote_color})${_Boff}"
  #   part 3 $(TrueFalse $_dv_VERBOSE)${Boff}
  _dh_cmd="${_dh_cmd}${_quote_color}\$(${_Boff}${_func_color}TrueFalse${_Boff}"   # note
\$ is for subshell
  _dh_cmd="${_dh_cmd} ${_var_color}\$_dv_VERBOSE${_Boff}${_quote_color})${_Boff}"     # n
ote ${_Boff}${_quote_color}in case not bold
  _dh_cmd="${_dh_cmd}${_var_color}\${Boff}${_quote_color}\"${_Boff}"              # note
\$_dv_VERBOSE, \${Boff} displayed as a variables
  #   also add administrative comment
  _dh_cmd="${_dh_cmd}  ${_cmt_color}# using two-pass column alignment and pad() utility${
_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"; printf "\n"

  # execution logic - store current state to avoid side effects in the main script
  _dh_oldv=$verbosity
  # Set state for demonstration
  _dv_VERBOSE=$TRUE; verbosity=3
  j_msg -5 "  ${BYon}_dv_VERBOSE is currently $(status_color $_dv_VERBOSE)$(TrueFalse $_d
v_VERBOSE)${Boff}"
  demo_verb_ret=$?
  # Restore state
  verbosity=$_dh_oldv
```

```
  printf "\n"

  unset -v _dh_cmd _dh_oldV _dh_oldv _dv_VERBOSE
  return $demo_verb_ret
}
# @usage _dh_demo_verbosity
# @args none
# @rule called by demonstrate_header

_dh_demo_versions() # (POSIX) demonstrate version comparison
{ demo_vers_ret=0
  separator "demonstrate_header-${BUILD}" "(version comparison)"
  _dh_FAILED=$FALSE

# preview the command syntax
  _dh_cmd="${_ctl_color}for${_Boff} ${_var_color}v${_Boff} ${_ctl_color}in${_Boff} ${_quo
te_color}\"1 1\" \"2 1\" \"1 2\" ...${_Boff}; ${_ctl_color}do${_Boff}\n"
  _dh_cmd="${_dh_cmd}   ${_func_color}vercomp${_Boff} ${_data_color}\$v1 \$v2${_Boff}; ${
_func_color}show_result${_Boff} ${_data_color}\$?${_Boff}\n"
  _dh_cmd="${_dh_cmd} ${_ctl_color}done${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"; printf "\n"

  # execute the actual demonstration
  for _dh_vtest in "1 1" "2 1" "1 2" "1.2.0 1.2.1" "1.2.0-r1 1.2.0"; do
    _dh_v1=${_dh_vtest% *}  # equiv =$( printf '%s' "${_dh_vtest}" | cut -d' ' -f1) (but
w/o call to cut)
    _dh_v2=${_dh_vtest#* }  # equiv =$( printf '%s' "${_dh_vtest}" | cut -d' ' -f2) (but
w/o call to cut)

    j_msg -5n "${BYon}Compare version ${BWon}${_dh_v1} ${BYon}to ${BWon}${_dh_v2}${Boff}
 "
    vercomp "$_dh_v1" "$_dh_v2"
    _dh_vercomp_result=$?
    printf '%b\n' "$(show_result $_dh_vercomp_result)"
    show_result $_dh_vercomp_result >/dev/null 2>&1
    _dh_show_result_exit=$?
    # if any single comparison fails (returns non-zero), mark the module as failed
    [ "$_dh_show_result_exit" -eq 0 ] || _dh_FAILED=$TRUE
  done
  printf "\n"

  [ "$_dh_FAILED" ] && demo_vers_ret=1
  unset -v _dh_cmd _dh_vtest _dh_v1 _dh_v2 _dh_vercomp_result _dh_show_result_exit _dh_FA
ILED
  return "$demo_vers_ret"
}
# @usage _dh_demo_versions
# @args none
# @rule called by demonstrate_header

_dh_demo_numeric() # (POSIX) demonstrate numeric validation utilities
{ separator "demonstrate_header-${BUILD}" "(numeric validation)"
  # preview the command syntax
  _dh_cmd="${_func_color}isnumeric${_Boff} ${_quote_color}\"123.45\"${_Boff}${_op_color};
${_Boff} ${_func_color}isint${_Boff} ${_quote_color}\"123\"${_Boff}   "
  _dh_cmd="${_dh_cmd} ${_cmt_color}# using two-pass column alignment and pad() utility${_
Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"; printf "\n"

  j_msg -5 "${BYon}Demonstrating Numeric Validation Utilities --${Boff}"

  _dh_tests="isint|123|0 isint|abc|1 isfloat_posix|3.14|0 isfloat_posix|123|1 ishexint|0x
AF|0 ishexint|123|1 isnumeric|42.5|0 isnumeric|0xCC|0 isnumeric|xyz|1"
  _dh_sep=" "; _dh_sep_len=${#_dh_sep}; _dh_min=2
  _dh_FAILED=$FALSE
  demo_num_ret=0

  # notes on pad() usage: pad "cur" "sep" "min" "color" "marker" "col"
  # (  need to make 2 passes to parse the test data
  #     - one to calculate fixed targets for padding
```

```
  #     - second to output each line, properly formatted
  # pad() api --
  # req all 6 args must be supplied; see below how some '' vals can default
  # note where . str cur = the current entry in fixed width column
  # note ....... str sep = internal separator (e.g. ': ')
  # note ....... int min = the minimum number of pad chars ...... (default 2)
  # note ..... str color = ANSI escape sequence (e.g. '\033[32m')
  # note ... char marker = the character w which to construct pad (default '.')
  # note ....... int col = the columns total fixed width
  # note marker may be colorized w ANSI sequence in quoted arg
  # warn failure to properly quote all args may result in unexpected behavior

  # first pass: calculate max lengths
  _dh_max_test_len=0; _dh_max_val_len=0
  for _dh_t in ${_dh_tests}; do
    _dh_test=$(echo "${_dh_t}" | cut -d'|' -f1)
    [ "${#_dh_test}" -gt "$_dh_max_test_len" ] && _dh_max_test_len=${#_dh_test}
    _dh_val=$(echo "${_dh_t}" | cut -d'|' -f2)
    [ "${#_dh_val}" -gt "$_dh_max_val_len" ] && _dh_max_val_len=${#_dh_val}
  done

#  j_msg -5 "_dh_max_test_len:  $_dh_max_test_len" 4
#  j_msg -5 "_dh_max_val_len: $_dh_max_val_len" 4

  # set fixed widths
  _dh_opening="Testing"; _dh_open_len=${#_dh_opening}
#  j_msg -5 "_dh_open_len: $_dh_open_len" 4
#  j_msg -5 "_dh_sep_len: $_dh_sep_len" 4
  _dh_test_total_sep=$(( _dh_sep_len * 2 ))
#  j_msg -5 "_dh_test_total_sep: $_dh_test_total_sep" 4
  _dh_col_test=$(( $_dh_open_len + $_dh_max_test_len + $_dh_test_total_sep + $_dh_min ))
 #  "Testing isnumeric "
  _dh_col_val=$(( $_dh_max_val_len + $_dh_sep_len + $_dh_min ))  #  "'0xAF' "
  _dh_col_TF=9    #  "[False] " - permanently known emprically ([+F+a+l+s+e+]+.+.+)
#  j_msg -5 "_dh_col_test: $_dh_col_test" 4
#  j_msg -5 "_dh_col_val: $_dh_col_val" 4
#  j_msg -5 "_dh_col_TF: $_dh_col_TF" 4

  # second pass: execution and aligned table print
  for _dh_t in ${_dh_tests}; do
    _dh_test=$(echo "${_dh_t}" | cut -d'|' -f1)
    _dh_val=$(echo "${_dh_t}" | cut -d'|' -f2)
    _dh_exp=$(echo "${_dh_t}" | cut -d'|' -f3)

    ${_dh_test} "${_dh_val}" >/dev/null 2>&1
    _dh_result=$?

    # determine True/False state for visual coloring
    case "$_dh_result" in
        0 ) _dh_RESULT_BOOL=$TRUE ;;
        * ) _dh_RESULT_BOOL=$FALSE ;;
    esac
    _dh_bool_str="$(TrueFalse $_dh_RESULT_BOOL)"
    _dh_bool_color="$(status_color $_dh_RESULT_BOOL)"

    # determine if actual result matches expected result
    if [ "${_dh_result}" -ne "${_dh_exp}" ]; then
       _dh_FAILED=$TRUE
       _dh_pass_fail="${BRon}FAIL${Boff}"
    else
       _dh_pass_fail="${BGon}PASS${Boff}"
    fi

    # column 1: test function name
    _dh_col1_text="Testing $_dh_test "
    j_msg -5n "Testing ${BCon}${_dh_test}${Boff} "
    # force min pad 0 on this column
    pad "${_dh_col1_text} " "$((_dh_sep_len * 3))" "0" "$BCon" "." "$_dh_col_test"

    # column 2: value segment
```

```
    _dh_v_label="'${_dh_val}'"
    printf '%s' "'${BMon}${_dh_val}${Boff}' "
    pad "$_dh_v_label" "$_dh_sep_len" "$_dh_min" "$BCon" "." "$_dh_col_val"

    # column 3: status segment (coloring bracketed string correctly)
    printf '%s' " [${_dh_bool_color}${_dh_bool_str}${Boff}] "
    pad " [${_dh_bool_str}]" "$_dh_sep_len" "$_dh_min" "$BCon" "." "$_dh_col_TF"

    # column 4: raw return and final judgement
    printf '%s' " (ret:${_dh_bool_color}${_dh_result}${Boff}│${BWon}${_dh_exp}${Boff}:exp
) -> "
    printf '%s\n' "${_dh_pass_fail}"
  done
  printf "\n"

  [ "$_dh_FAILED" ] && demo_num_ret=1
  unset -v _dh_cmd _dh_tests _dh_sep _dh_sep_len _dh_min _dh_max_test_len _dh_max_val_len
  unset -v _dh_opening _dh_open_len _dh_test_total_sep _dh_col_test _dh_col_val _dh_col_T
F
  unset -v _dh_t _dh_test _dh_val _dh_exp _dh_result _dh_RESULT_BOOL _dh_bool_str
  unset -v _dh_bool_color _dh_pass_fail _dh_v_label _dh_col1_text _dh_FAILED

  return "$demo_num_ret"
}
# @usage _dh_demo_numeric
# @args none
# @rule called by demonstrate_header


_dh_demo_debug_utils() # (POSIX) demonstrate debugging (verbosity-conditional execution a
nd messaging)
{ _dh_FAILED=$FALSE
  demo_debug_ret=0
  if [ "$verbosity" -gt 0 ]; then _dh_VERBOSE=$TRUE; else _dh_VERBOSE=$FALSE; fi

  separator "demonstrate_header-${BUILD}" "(debugging utilities)"

  # temporarily set system verbosity for demonstration
  _dh_oldv=$verbosity
  _dh_VERBOSE=$TRUE; verbosity="${notice}"
  j_msg -5 "${_BYon}_dh_VERBOSE is $(status_color $_dh_VERBOSE)$(TrueFalse $_dh_VERBOSE)$
{_Boff}"
  j_msg -5 "${_BYon}system verbosity is ${Boff}[${_data_color}${verbosity}${_Boff}]"
  j_msg -5 "${_BYon}this corresponds to severity level ${_Boff}[${_data_color}$(get_sever
ity ${verbosity})${_Boff}]"

  # demonstrate verbosity and severity linkage with get_severity and globally defined sev
erity levels
  # --- Preview Construction for line 1 ---
  # Goal: printf '%s\n' " crit is severity [$(echo "$crit")]"
  _dh_cmd="  ${_func_color}printf${_Boff} ${_quote_color}'%s\\\n' "
  _dh_cmd="${_dh_cmd}\" crit is severity ${_Boff}${_ctl_color}[\$(${_Boff}${_cmd_color}ec
ho${_Boff}"
  _dh_cmd="${_dh_cmd} \"${_var_color}\$crit${_Boff}\"${_ctl_color})]${_quote_color}\"${_B
off}\n"

  # --- Preview Construction for line 2 ---
  # Goal: echo "severity 2 is [$(get_severity 2)]"
  _dh_cmd="${_dh_cmd}  ${_cmd_color}echo${_Boff} "
  _dh_cmd="${_dh_cmd}${_quote_color}\" severity 2 is ${_Boff}${_ctl_color}[\$(${_func_col
or}get_severity${_Boff}"
  _dh_cmd="${_dh_cmd} ${_data_color}2${_Boff}${_ctl_color})]${_quote_color}\"${_Boff}\n"

  # --- Display ---
  _dh_cmt="${_cmt_color}# demonstrating nested expansion and function calls${Boff}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}  ${_dh_cmt}${_dh_cmd}"

  # --- Logic Execution ---
  printf '%s\n' " crit is severity [$(echo "$crit")]"
  echo "severity 2 is [$(get_severity 2)]"
```

```
  # demonstrate j_msg -p - debugging message (output if verbosity >= level)
  _dh_cmd="${_func_color}j_msg -7p${_Boff} ${_quote_color}\"Raw debug data: \${_var}\"${_
Boff} \"${_var_color}\${debug}${_Boff}\""
  _dh_cmt="${_cmt_color}# if verbosity >= \${debug} (it is not, so expect no output)${Bof
f}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}  ${_dh_cmt} ${_dh_cmd}"
  j_msg -7p "  Raw debug data: example_value" "${debug}"
  # logic execution - and failure check
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  # demonstrate j_msg -5 - debugging message (output if verbosity >= level)
  _dh_cmd="${_func_color}j_msg -5${_Boff} ${_quote_color}\"Debug message at level \${noti
ce}\"${_Boff} \"${_var_color}\${notice}${_Boff}\""
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}  ${_cmt_color}# if verbosity >= \${n
otice} (it is)${_Boff}\n ${_dh_cmd}"
  # logic execution - and failure check
  j_msg -5 "Debug message at level ${notice}"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  # demonstrate j_msg -5n; logic test; right_status $? (conditional status linkage)
  # case A: looking for a file that does not exist
  _dh_file_bogus="/usr/sbin/script_header_joetoo_bogus"
  # construct line 1 of the commands to be displayed
  _dh_cmd=" ${_var_color}_dh_header${_Boff}=${_quote_color}\"${_dh_file_bogus}\"${_Boff}\
n"
  # construct line 2 of the commands to be displayed (single-quote the line with escaped
quotes
  _dh_cmd="${_dh_cmd} ${_func_color}j_msg -5n${_Boff}"
  _dh_cmd="${_dh_cmd} ${_quote_color}\"looking for \${_dh_header}\"${_Boff}"
  _dh_cmd="${_dh_cmd} \"${_var_color}\${crit}${_Boff}\"\n"
  # construct line 3 of the commands to be displayed - starting with the [ test
  _dh_cmd="${_dh_cmd} ${_ctl_color}[${_Boff} ${_op_color}"
  # isolate the  '!' - copy/paste to test this block will choke otherwise
  _dh_cmd="${_dh_cmd}"'!'
  _dh_cmd="${_dh_cmd}${_Boff} ${_opt_color}-z${_Boff} "
  # Add the find subshell
  _dh_cmd="${_dh_cmd}${_quote_color}\"\$(${_cmd_color}find${_Boff} \${_dh_header}"
  _dh_cmd="${_dh_cmd} ${_data_color}2${_Boff}${_op_color}>/dev/null)${_quote_color}\"${_B
off}"
  # Close bracket and add right_status
  _dh_cmd="${_dh_cmd} ${_ctl_color}]${_Boff} ${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}right_status${_Boff} ${_var_color}\$?${_Boff} "
  _dh_cmd="${_dh_cmd}\"${_var_color}\${crit}${_Boff}\""

  _dh_cmt="${_cmt_color}# if verbosity >= \"\$crit\" (it is)${Boff}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}  ${_dh_cmt}${_dh_cmd}"
  # logic execution - and failure check
  _dh_header="${_dh_file_bogus}"; j_msg -5n "looking for ${_dh_header}"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE
  [ ! -z "$(find ${_dh_header} 2>/dev/null)" ]; right_status $?
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  # case B: Looking for a file that does exist
  _dh_file_real="/usr/sbin/script_header_joetoo"
  # construct line 1 of the commands to be displayed
  _dh_cmd="${_var_color}_dh_header${_Boff}=${_quote_color}\"${_dh_file_real}\"${_Boff}\n"
  # construct line 2 of the commands to be displayed
  _dh_cmd="${_dh_cmd} ${_func_color}j_msg -5n${_Boff} "
  _dh_cmd="${_dh_cmd}${_quote_color}\"looking for \${_dh_header}\"${_Boff} "
  _dh_cmd="${_dh_cmd}\"${_var_color}\${crit}${_Boff}\"\n"
  # construct line 3 of the commands to be displayed - starting with the [ test
  _dh_cmd="${_dh_cmd} ${_ctl_color}[${_Boff} ${_op_color}!${_Boff} ${_opt_color}-z${_Boff
} "
  # Add the find subshell
  _dh_cmd="${_dh_cmd}${_quote_color}\"\$(${_cmd_color}find${_Boff} \${_dh_header} "
  _dh_cmd="${_dh_cmd}${_data_color}2${_Boff}${_op_color}>/dev/null)${_quote_color}\" "
  # Close bracket and add right_status (note: j_msg -5 will add a newline)
  _dh_cmd="${_dh_cmd}${_ctl_color}]${_Boff}${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}right_status${_Boff} ${_var_color}\$?${_Boff} "
```

```
  _dh_cmd="${_dh_cmd}\"${_var_color}\${crit}${_Boff}\""
  _dh_cmt="${_cmt_color}# if verbosity >= \${crit}  (it is)${Boff}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}  ${_dh_cmt}${_dh_cmd}"
  # logic execution - and failure check
  _dh_header="${_dh_file_real}"; j_msg -5n "looking for ${_dh_header}"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE
  [ ! -z "$(find ${_dh_header} 2>/dev/null)" ];   right_status $?
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  # demonstrate d_do - case A: simple command (execution if verbosity >= level)
  # build command - d_do "printf '${BGon}Executed${Boff}'" "$crit" (theme-colored)
  _dh_cmd="${_func_color} d_do${_Boff} ${_quote_color}\"printf '\${BGon}Executed\${Boff}'
\"${_Boff} ${_var_color}\${crit}${_Boff}\""
  _dh_cmt="${_cmt_color}# if verbosity >= \"\$crit\" (it is)${_Boff}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}  ${_dh_cmt}${_dh_cmd}"
  # logic execution - and failure check
  d_do "printf '${BGon} * Executed${Boff}\n'" "$crit"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  # demonstrate d_do - case B: complex multi-part command (execution if verbosity >= leve
l)
  # build command d_do 'echo "      line 1.1"; echo "      line 1.2"' "$notice" (theme co
lored)
  _dh_cmd="${_func_color} d_do${_Boff} ${_quote_color}'echo \"      line 1.1\"; echo \""
  _dh_cmd="${_dh_cmd}      line 1.2\"'${_Boff} \"${_var_color}\${crit}${_Boff}\""
  _dh_cmt="${_cmt_color}# if verbosity >= \"\$crit\" (it is)${Boff}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_dh_cmt}${_dh_cmd}"
  # logic execution - and failure check
  d_do 'echo "   line 1.1"; echo "   line 1.2"' "$notice"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  [ "$_dh_FAILED" = "$TRUE" ] && demo_debug_ret=1
  # restore previous system verbosity
  verbosity=$_dh_oldv
  printf "\n"

  unset -v _dh_cmd _dh_cmt _dh_FAILED _dh_oldV _dh_oldv _dh_file_bogus _dh_file_real _dh_
header _dh_VERBOSE
  return $demo_debug_ret
}
# @usage _dh_demo_debug_utils
# @args none
# @rule demonstrating conditional messaging and status utilities

_dh_demo_boolean() # (POSIX) demonstrate bool truth validation with a variety of data typ
es
{ demo_bool_ret=0
  _dh_FAILED=$FALSE

  separator "demonstrate_header-${BUILD}" "(boolean utilities)"

  # data format: label|value|expectation (0=True, 1=False, 2=unset)
  _dh_tests="\$TRUE|$TRUE|0 \$FALSE|$FALSE|1 y|y|0 Y|Y|0"
  _dh_tests="${_dh_tests} n|n|1 N|N|1 t|t|0 T|T|0 f|f|1 F|F|1"
  _dh_tests="${_dh_tests}  up|up|0 down|down|1 0|0|0 1|1|2 z|z|2 ''|''|1"

  # preview the command syntax for each of the commands to be run (4 lines, like this)
  #  for x in "$TRUE $FALSE y ... '' $TRUE $FALSE"; do
  #    j_msg -5n "testing: [${x}] is $(status_color ${x})$(TrueFalse ${x})"
  #    if [ "$(TrueFalse ${x})" = "True" ]; then right_status 0
  #    else right_status 1; fi
  #  done
  # line 1 part 1 - for x in
  _dh_cmd="${_ctl_color}  for${_Boff} ${_var_color}x${_Boff} ${_ctl_color}in${_Boff} "
  # line 1 part 2 - "y n 0 1 z ... '' $TRUE $FALSE"; do
  _dh_cmd="${_dh_cmd}${_quote_color}\"\$TRUE \$FALSE y n ... ''\"${_Boff}${_op_color};${_
Boff} ${_ctl_color}do${_Boff}\n"
  # line 2 part 1 -  j_msg -5n "testing: [${x}] is
  _dh_cmd="${_dh_cmd}     ${_func_color}j_msg -5n${_Boff} ${_quote_color}\"testing: [\${x
}] is \$(${_Boff}"
```

```
  # line 2 part 2 - $(status_color ${x})
  _dh_cmd="${_dh_cmd}${_func_color}status_color${_Boff} ${_var_color}\${x}${_Boff}${_quot
e_color})\$(${_Boff}"
  # line 2 part 3 - $(TrueFalse ${x})"
  _dh_cmd="${_dh_cmd}${_func_color}TrueFalse${_Boff} ${_var_color}\${x}${_Boff}${_quote_c
olor})\"${_Boff}\n"
  # line 3 part 1 - if [
  _dh_cmd="${_dh_cmd}     ${_ctl_color}if [${_Boff}"
  # line 3 part 2 - "$(TrueFalse ${x})"
  _dh_cmd="${_dh_cmd}  ${_quote_color}\"\$(${_Boff}${_func_color}TrueFalse${_Boff} ${_var
_color}\${x}${_Boff}${_quote_color})\"${_Boff} "
  # line 3 part 3 - = "True" ];
  _dh_cmd="${_dh_cmd}${_op_color}=${_Boff} ${_quote_color}\"True\"${_Boff} ${_ctl_color}]
${_Boff}${_op_color};${_Boff} "
  # line 3 part 4 - then right_status 0
  _dh_cmd="${_dh_cmd}${_ctl_color}then${_Boff} ${_func_color}right_status${_Boff} ${_var_
color}\$TRUE${_Boff}\n"
  # line 4 part 1 - else right_status 1;
  _dh_cmd="${_dh_cmd}   ${_ctl_color}else${_Boff} ${_func_color}right_status${_Boff} ${_d
ata_color}1${_Boff}${_op_color};${_Boff} "
  # line 4 part 2 - fi
  _dh_cmd="${_dh_cmd}${_ctl_color}fi${_Boff}\n"
  # line 5 - done
  _dh_cmd="${_dh_cmd}${_ctl_color}  done${_Boff}"

  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"
  printf "\n"
  j_msg -5 "  ${BYon}Summary of T/F tests with ${BGon}status_color${Boff}() and ${BGon}Tr
ueFalse${Boff}() --"

  # logic execution
  for _dh_t in ${_dh_tests}; do
    _dh_label=$(printf '%s' "${_dh_t}" | cut -d'|' -f1)
    _dh_val=$(printf '%s' "${_dh_t}" | cut -d'|' -f2)
    _dh_exp=$(printf '%s' "${_dh_t}" | cut -d'|' -f3)

    # explicit translation of the literal '' placeholder to a true null value
    [ "${_dh_val}" = "''" ] && _dh_val=""
    # get the formatted output
    _dh_out=$(TrueFalse "${_dh_val}")
    # get the exit status from TrueFalse
    _dh_status=$?

    j_msg -5n "    ${BWon}testing: [${BCon}${_dh_label}${Boff}${BWon}] is $(status_color
"${_dh_val}")${_dh_out}${Boff}"

    # validate that the exit status matches our expectation for this value
    if [ "$_dh_status" -eq "$_dh_exp" ] ; then
      right_status 0
    else
      right_status 1
      _dh_FAILED=$TRUE
    fi
  done

  printf "\n"
  j_msg -5n ""
  smart_wrap_n 3 3 3 "" "Note: most shells evaluate null ('') as ${BRon}FALSE${Boff} and
non-null as ${BGon}TRUE${Boff}"
  printf "\n"

  [ "$_dh_FAILED" = "$TRUE" ] && demo_bool_ret=1

  unset -v _dh_cmd _dh_FAILED _dh_t _dh_label _dh_val _dh_exp _dh_out _dh_status _dh_test
s _dh_PN
  return $demo_bool_ret
}
# @usage _dh_demo_boolean
# @args none
# @rule demonstrating truth-testing and conversion helpers
```

```
_dh_demo_terminal_io() # (POSIX) demonstrate terminal io, animation, box drawing
{ demo_term_ret=0
  _dh_FAILED=$FALSE
  _dh_cock="${BRon}-*-${Boff}"    # define a 3-char shuttlecock (stationary)
  _dh_r_cock="${BRon}--*${Boff}"  # define a 3-char shuttlecock (right bound)
  _dh_l_cock="${BRon}*--${Boff}"  # define a 3-char shuttlecock (left bound)
  _dh_wait="0.1"                  # define the loop wait time (seconds)
  _dh_wait="25"                   # define the loop wait time (milliseconds)
  separator "demonstrate_header-${BUILD}" "(terminal i/o & graphics)"

  #-----[ part 1: shuttle animation ]-------------------------------------
  # ( including hide/show cursor )
  # preview the command syntax for each of the commands to be run (4 lines, like this)
  # construct the command to be displayed - 2 lines
  #    j_msg -5n "Shuttle: [                    ]"; CUB 21
  #    # Loop: CUF/CUB to shift '-*-' shuttlecock; HCU/SCU to hide/show cursor
  # line 1 part 1 -    j_msg -5n "Shuttle:
  _dh_cmd="${_func_color}j_msg -5n${_Boff} ${_quote_color}\"Shuttle:"
  # line 1 part 2  [                    ]";
  _dh_cmd="${_dh_cmd} [                    ]\"${_Boff}${_op_color};${_Boff}"
  # line 1 part3 -  CUB 21
  _dh_cmd="${_dh_cmd} ${_func_color}CUB${_Boff} ${_data_color}21${_Boff}\n"
  # line 2 - # Loop: CUF/CUB to shift '-*-' shuttlecock; HCU/SCU to hide/show cursor
  _dh_cmd="${_dh_cmd} ${_cmt_color}# Loop: CUF/CUB to shift '-=-' shuttlecock; HCU/SCU to
 hide/show cursor${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# 20-char window, 2 cyc
les${_Boff}\n ${_dh_cmd}"

  # print the 20 char shuttle field bounded by braces [ ], (22 char total), and return to
 start of field
  # create room in case the cursor is at the bottom of the screen (newline twice then up
1)
  printf "\n\n"; CUU 1

  HCU   # hide the cursor during animation (very distracting otherwise)
  # complete 2 cycles of move to the right and back to the left
  _dh_cycle=0
#  while [ $_dh_cycle -lt 2 ]; do
  while [ "$_dh_cycle" -lt 1 ]; do    # decided to speed things up
    # return to left margin (resets every thing (more important in second+ cycle)
    printf "\r"
    j_msg -5n "shuttle zone: [                    ]"
    # return to start of shuttle-zone
    CUB 21
    #draw the shuttlecock
    printf '%s' "${_dh_cock}"; milli_sleep "${_dh_wait}"; CUB 3
    # note: starting cursor location (col the NEXT print goes into) is [ + 4
    _dh_i=0
    while [ "$_dh_i" -lt 17 ]; do     # 21 - 4 = 17 is collision with ]
    # shift shuttle right
      printf '%s' " ${_dh_r_cock}"; milli_sleep "${_dh_wait}"; CUB 3
       _dh_i=$((_dh_i + 1))
    done
    # note: cursor location was on top of ] after last shuttlecock print,
    #    but we then did CUB 3 as in every iteration of the prior loop
    #    so - to initialize properly for the next loop we need to more
    #    to the position each of its iterations will end at, and
    #    because its first act is CUB 5, we need to CUF 4 to get to ]+1
    CUF 4
    while [ "$_dh_i" -gt 0 ]; do     # should be same # of steps to get back
    # shift shuttle right
      CUB 5; printf '%s' "${_dh_l_cock} "; milli_sleep "${_dh_wait}"
       _dh_i=$((_dh_i - 1))
    done
    _dh_cycle=$((_dh_cycle + 1))
  done
  printf "\r"; j_msg -5n "shuttle zone: [                    ]"      # clear the zone aga
in (done)
  SCU   # show the cursor again after animation is over
```

```
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE
  printf "\n"

  #-----[ part 2a: DEC Graphics (Box Drawing) ]---------------------------------
  _dh_text="DEC Graphics (SO/SI Method)"
  # build command string for preview
  _dh_cmd="${_func_color}vt_init${_Boff}${_op_color};${_Boff}"
  _dh_cmd="${_dh_cmd} ${_func_color}box_shift${_Boff} ${_quote_color}\"${_dh_text}\"${_Bo
ff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# High-fidelity SO/SI B
ox${_Boff}\n ${_dh_cmd}"
  # logic execution
  vt_init; box_shift "${_dh_text}"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  #-----[ part 2b: DEC graphics Explicit G0 Switch ]----------------------------
  _dh_text="Explicit G0 (ESCon/B)"
  # build command string for preview
  _dh_cmd="${_func_color}box_esca${_Boff} ${_quote_color}\"${_dh_text}\"${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# High-fidelity ESCon/B
 Box${_Boff}\n ${_dh_cmd}"
  # logic execution
  box_esca "${_dh_text}"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  [ "$_dh_FAILED" = "$TRUE" ] && demo_term_ret=1
  unset -v _dh_cmd _dh_FAILED _dh_i _dh_cycle _dh_cock _dh_wait
  return $demo_term_ret
}
# @usage _dh_demo_terminal_io
# @args none
# @rule demonstrating cursor state and DEC graphics

_dh_demo_environment() # (POSIX) demonstrate environment checks (isroot, checknotroot, ch
eckboot, etc)
{ demo_env_ret=0
  _dh_FAILED=$FALSE

  separator "demonstrate_header-${BUILD}" "(environment & privileges)"

  #-----[ part 1: isroot - silent root privilege check ]---------------------------------
----------------------------------
  # construct the command preview string - isroot;  ... handle_result
  # part 1 - isroot; ...
  _dh_cmd="${_op_color}(${_Boff} ${_func_color}isroot${_Boff} ${_op_color});${_Boff} ...
"
  # [art 2 - handle_result
  _dh_cmd="${_dh_cmd}${_func_color}handle_result${_Boff}${_Boff} "
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# silent predicate chec
k${_Boff}\n ${_dh_cmd}"
  j_msg -5n "  Testing isroot (silent): "
  # logic execution
  isroot; _dh_result=$?
  if [ "$(id -u)" -eq 0 ]; then  ## we really are root (basis for test of isroot()
    [ "$_dh_result" -eq 0 ]
    handle_result $? "yes root and returned 0" "yes root but did not return 0"
  else
    # If we ARE NOT root, isroot should have returned 1 (Failure)
    [ "$_dh_result" -eq 1 ]
    handle_result $? "not root and returned 1" "not root but did not return 1"
  fi
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  #-----[ part 2: checkroot - gatekeeper root privilege check ]--------------------------
----------------------------------------
  # construct the command preview string - ( checkroot ); ... handle_result
  # part 1 - ( checkroot ); ...
  _dh_cmd="${_op_color}(${_Boff} ${_func_color}checkroot${_Boff} ${_op_color});${_Boff} .
.. "
  # part 2 - handle_result
```

```
  _dh_cmd="${_dh_cmd}${_func_color}handle_result${_Boff}${_Boff} "
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# root gatekeeper in su
bshell (exits if not root)${_Boff}\n ${_dh_cmd}"
  j_msg -5n "  Testing checkroot gatekeeper "
  # logic execution
  ( checkroot ) >/dev/null 2>&1 ; _dh_result=$?
  if isroot; then  ## we really are root (basis for test of checkroot()
    [ "$_dh_result" -eq 0 ]
    handle_result $? "yes root and returned 0" "yes root but did not return 0"
  else
    # If we ARE NOT root, isroot should have returned 1 (Failure)
    [ "$_dh_result" -eq 1 ]
    handle_result $? "not root and *exited* 1" "not root but did not exit 1"
  fi
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  #-----[ part 3: checknotroot - gatekeeper root privilege check ]-----------------------
--------------------------------------------
  # construct the command preview string - ( checknotroot ); ... handle_result
  # part 1 - ( checknotroot ); ...
  _dh_cmd="${_op_color}(${_Boff} ${_func_color}checknotroot${_Boff} ${_op_color});${_Boff
} ... "
  # part 2 - handle_result
  _dh_cmd="${_dh_cmd}${_func_color}handle_result${_Boff}${_Boff} "
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# user gatekeeper in su
bshell (exits if root)${_Boff}\n ${_dh_cmd}"
  j_msg -5n "  Testing checknotroot gatekeeper: "
  # logic execution
  ( checknotroot ) >/dev/null 2>&1 ; _dh_result=$?
  if ! isroot; then  ## we really are not root (basis for test of checknotroot()
    [ "$_dh_result" -eq 0 ]
    handle_result $? "not root and returned 0" "not root but did not return 0"
  else
    # If we ARE root, checknotroot should have returned 1 (Failure)
    [ "$_dh_result" -eq 1 ]
    handle_result $? "yes root and *exited* 1" "yes root but did not exit 1"
  fi
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE
  # Note: In a real script, checkroot might exit; here we just observe.

  #-----[ part 4: checkboot - boot mount check ]-----------------------------------------
------------------------------
  # construct the command preview string - ( checknotroot ); ... handle_result
  # part 1 - ( checkboot ); ...
  _dh_cmd="${_op_color}(${_Boff} ${_func_color}checkboot${_Boff} ${_op_color});${_Boff} .
.. "
  # part 2 - handle_result
  _dh_cmd="${_dh_cmd}${_func_color}handle_result${_Boff}${_Boff} "
  _dh_cmt="${_cmt_color}# verifies mounted status of boot/efi targets via bitmask exit st
atus${_Boff}\n"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_dh_cmt} ${_dh_cmd}"
  j_msg -5n "  Testing checkboot gatekeeper: "
  # logic execution - run checkboot; if it says fail, double-check
  _dh_OK_msg="All defined boot targets are mounted"
  _dh_FAIL_msg="One or more targets that should be is not mounted (Code: $_dh_result)"

  ( checkboot ) >/dev/null 2>&1 ; _dh_result=$?
  handle_result "$_dh_result" "${_dh_OK_msg}" "${_dh_FAIL_msg}"
  if [ ! "$_dh_result" -eq 0 ]; then
    # explain the failure code(s) using bitwise math
    _dh_i=0
    _dh_targets="/boot /efi /boot/efi"   # bitmask represents these in LSB->MSB order
    for _dh_tgt in $_dh_targets; do
      _dh_MOUNTPOINT=$FALSE
      # check if bit _dh_i is set in the result mask
      # (shift i bits right and check the LSB by bitwise AND w 1)
      if [ "$(( (_dh_result >> _dh_i) & 1 ))" -eq 1 ]; then
        # double-check checkboot's result using sys-util/util-linux's mountpoint
        # (checks whether a directory or file is a mountpoint. -q = silently)
        j_msg -5n "checkboot: ${BYon}${_dh_tgt}${Boff} should be but is ${BRon}not${Boff}
```

```
 mounted"
        mountpoint -q "${_dh_tgt}" && _dh_MOUNTPOINT=$TRUE
        if [ $_dh_MOUNTPOINT ] ; then
            printf '%s' "(${BRon}it is${Boff})"
            _dh_FAILED=$TRUE
            right_status 1
        else
            printf '%s' "(${BGn}it is not${Boff})"
            right_status 0
        fi  # actually is mounted
      fi  # bit i is checkboot failure report
      _dh_i=$((_dh_i + 1))
    done  # for _dh_tgt
  fi  # check boot reports error?
  # Note: we are NOT double-checking if checkboot says all ok

  [ "$_dh_FAILED" ] && demo_env_ret=1
  unset -v _dh_cmd _dh_cmt _dh_FAILED _dh_OK_msg _dh_FAIL_msg
  unset -v _dh_result _dh_i _dh_targets _dh_tgt _dh_MOUNTPOINT
  unset -v _dh_BOOT_REALITY
  return $demo_env_ret
}
# @usage _dh_demo_environment
# @args none
# @rule demonstrating environment and privilege checks

_dh_demo_widgets() # (POSIX) demonstrate separator, progress bar, countdown)
{ demo_widg_ret=0
  _dh_FAILED=$FALSE
  _dh_result=0

  _dh_46="Donaldson Emmons Seward Seymour Panther Couchsachraga Santanoni"
  _dh_46="${_dh_46} Allen Cliff Redfield Colden Marshall Wright Algonquin"
  _dh_46="${_dh_46} Iroquois Street Nye Phelps TableTop Gray Skylight"
  _dh_46="${_dh_46} Marcy Haystack Basin BigSlide LowerWolfJaw UpperWolfJaw"
  _dh_46="${_dh_46} Armstrong Gothics Saddleback Giant RockyPeak Sawteeth"
  _dh_46="${_dh_46} Colvin Blake Dial Nippletop Dix Hough SouthDix"
  _dh_46="${_dh_46} GracePeak Macomb Porter Cascade Esther Whiteface"

  #-----[ part 1: separator - horizontal categorization ]--------------------------------
-------------------------------
  # construct the command preview string - separator "DEMO" "(ui organization)"
  # line 1 part 1 - separator "DEMO"
  _dh_cmd="${_func_color}separator${_Boff} ${_quote_color}\"DEMO\"${_Boff} "
  # line 1 part 2 - "(ui organization)"
  _dh_cmd="${_dh_cmd}${_quote_color}\"(ui organization)\"${_Boff}${_op_color};${_Boff} "
  # line 1 part 3 - handle_result $?
  _dh_cmd="${_dh_cmd}${_func_color}handle_result${_Boff} ${_var_color}\$?${_Boff}\n"
  # line 2 - # horizontal rule with title
  _dh_cmd="${_dh_cmd} ${_cmt_color}# horizontal rule with title${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"
  # Logic execution
  separator "DEMO" "(ui organization)"
  [ "$?" -eq 0 ]
  handle_result $? "" "separator widget failed"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE

  #-----[ part 2: progress_inline - task status ]----------------------------------------
-------------------------------
  # construct the command preview string - for loop below
  # line 1 - _dh_i=1; HCU; _dh_offset=65;
  _dh_cmd="   ${_var_color}_dh_i${_Boff}${_op_color}=${_Boff}${_data_color}1${_Boff}${_op
_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}HCU${_Boff}${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_var_color}_dh_offset${_Boff}${_op_color}=${_Boff}${_data_color}65
${_Boff}${_op_color};${_Boff}\n"
  # line 2 -    for _dh_peak in $_dh_46; do
  _dh_cmd="${_dh_cmd}   ${_ctl_color}for${_Boff} ${_var_color}_dh_peak${_Boff} ${_ctl_col
or}in${_Boff} "
  _dh_cmd="${_dh_cmd}${_var_color}\$_dh_46${_Boff}${_op_color};${_Boff} ${_ctl_color}do${
```

```
_Boff}\n"
  # line 3 -      progress_inline $_dh_i 47;
  _dh_cmd="${_dh_cmd}     ${_func_color}progress_inline${_Boff} ${_var_color}\$_dh_i${_Bo
ff} ${_data_color}47${_Boff}${_op_color};${_Boff}\n"
  # line 4 -      SCP; printf "\r";   ## note careful backslashes required (particularly
for \r -> \\\r)
  _dh_cmd="${_dh_cmd}     ${_func_color}SCP${_Boff}${_op_color};${_Boff} ${_func_color}pr
intf${_Boff} "
  _dh_cmd="${_dh_cmd}${_quote_color}\"\\\r\"${_Boff}${_op_color};${_Boff}\n"
  # line 5 -      CUF $_dh_offset; printf ...; EL; RCP;
  _dh_cmd="${_dh_cmd}     ${_func_color}CUF${_Boff} ${_var_color}\$_dh_offset${_Boff}${_o
p_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}printf${_Boff} ${_data_color}...${_Boff}${_op_color};$
{_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}EL${_Boff}${_op_color};${_Boff} ${_func_color}RCP${_Bo
ff}${_op_color};${_Boff}\n"
  # line 6 -    done; SCU;
  _dh_cmd="${_dh_cmd}   ${_ctl_color}done${_Boff}${_op_color};${_Boff} ${_func_color}SCU$
{_Boff}${_op_color};${_Boff}\n"
  # line 7 -    progress_inline 46 46; handle_result $?
  _dh_cmd="${_dh_cmd}   ${_func_color}progress_inline${_Boff} ${_data_color}46 46${_Boff}
${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}handle_result${_Boff} ${_var_color}\$?${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"

  # Logic execution
  _dh_longest=$(get_longest ${_dh_46})
  _dh_offset=65   # empirically determined from progress_inline output (+10 after migrati
on for unk reason)
  _dh_i=1         # index from first peak, not 0
  _dh_wait=80     # millisecond loop wait time
  HCU  # hide the cursor until done tracking progress (distracting white block otherwise)
  for _dh_peak in $_dh_46; do
    # update progress bar
    # "lie" about progress target total to suppress final newline and get last peak name
on the right line
    progress_inline $_dh_i 47
    # save cursor position; return to left margin (absolute reference)
    SCP; printf "\r"
    # advance to position to the right of probress bar
    CUF $_dh_offset
    # print name of peak in fixed widht [] bounded "box",
    #   then erase the rest of the line to ensure it is clear, and restore cursor positio
n
    printf "${BCon}[${BMon}%-${_dh_longest}s${BCon}]${Boff}" "$_dh_peak"; EL; RCP
    _dh_i=$((_dh_i + 1))
    milli_sleep $_dh_wait
  done
  SCU  # cursor again (otherwise invisible and confusing on cli)
  progress_inline 46 46  # now that we finished w/o final newline, update the accurate %
progress
  [ "$?" -eq 0 ]; handle_result $? "reached 46 peaks" "progress_inline widget failed"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE
  printf "\n"

  #-----[ part 3: u_box_msg ]-----------------------------------------------------------
--------------------------------
  # construct the command preview string -
  # line 1 -    u_box_msg "$longone"  $(( 8 * $(termwidth) / 10)) ’’ "${BCon}"
  _dh_cmd="   ${_func_color}u_box_msg${_Boff} ${_quote_color}\"${_var_color}\$longone${_q
uote_color}\""
  _dh_cmd="${_dh_cmd} ${_op_color}\$(( ${_data_color}8 ${_op_color}* ${_op_color}\$(${_fu
nc_color}termwidth${_op_color})"
  _dh_cmd="${_dh_cmd} / ${_data_color}10${_op_color} ))${_Boff}"
  _dh_cmd="${_dh_cmd} ${_op_color}’’ ${_quote_color}\"${_var_color}\${BCon}${_quote_color
}\" ${_data_color}4${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"
  u_box_msg "$longone" $(( 8 * $(termwidth) / 10 )) ’’ "${BCon}" 4;   _dh_result=$?
  j_msg -5n "done running u_box_msg; result:"
  handle_result $_dh_result "" "" || _dh_FAILED=$TRUE
```

```
  #-----[ part 4: sh_countdown - timed safety pause ]-----------------------------------
-------------------------------
  # construct the command preview string -
  # line 1 -    sh_countdown 3 "Safety pause before deployment";
  _dh_cmd="   ${_func_color}sh_countdown${_Boff} ${_data_color}3${_Boff} "
  _dh_cmd="${_dh_cmd}${_quote_color}\"Safety pause before deployment\"${_Boff}${_op_color
};${_Boff}\n"
  # line 2 -    handle_result $?
  _dh_cmd="${_dh_cmd}   ${_func_color}handle_result${_Boff} ${_var_color}\$?${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# interactive timer${_B
off}\n ${_dh_cmd}"
  # Logic execution
  sh_countdown 3 "Safety pause before deployment"
  [ "$?" -eq 0 ]; handle_result $? "pause complete" "countdown widget failed"
  [ "$?" -eq 0 ] || _dh_FAILED=$TRUE
  printf "\n"

  [ "$_dh_FAILED" ] && demo_widg_ret=1
  unset -v _dh_cmd _dh_FAILED _dh_46 _dh_longest _dh_offset _dh_i _dh_wait _dh_peak
  return $demo_widg_ret
}
# @usage _dh_demo_widgets
# @args none
# @rule demonstrating established decorative and informative UI elements

_dh_demo_emojis() # (POSIX) demonstrate emoji policy, emoji_safe, and right_status
{ demo_emoji_ret=0
  _dh_FAILED=$FALSE
  _dh_result=0

  #-----[ part 1: show emoji policy state ]---------------------------------------------
-----------
  # construct the command preview string -
  # line 1 - printf 'EMOJI_CAPABLE=[ %s ] EMOJI_DISABLE=[ %s ]\n' "${EMOJI_CAPABLE}" "${E
MOJI_DISABLE}"
  _dh_cmd="   ${_func_color}printf${_Boff} ${_quote_color}\"EMOJI_CAPABLE=[ %s ]  EMOJI_D
ISABLE=[ %s ]\\\\n\"${_Boff} "
  _dh_cmd="${_dh_cmd}${_var_color}\"\${EMOJI_CAPABLE}\" \"\${EMOJI_DISABLE}\"${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"

  # Logic execution
  printf 'EMOJI_CAPABLE=[ %s ]  EMOJI_DISABLE=[ %s ]\n' "${EMOJI_CAPABLE:-}" "${EMOJI_DIS
ABLE:-}"
  printf "\n"

  #-----[ part 2: basic emoji_safe() + right_status ]-----------------------------------
----------
  # construct the command preview string -
  # line 1 - emoji_safe "$check_mark_button"; right_status $?
  _dh_cmd="   ${_func_color}emoji_safe${_Boff} ${_quote_color}\"\${check_mark_button}\"${
_Boff}${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}right_status${_Boff} ${_var_color}\$?${_Boff}\n"
  # line 2 - emoji_safe "$cross_mark"; right_status $?
  _dh_cmd="${_dh_cmd}   ${_func_color}emoji_safe${_Boff} ${_quote_color}\"\${cross_mark}\
"${_Boff}${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}right_status${_Boff} ${_var_color}\$?${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"

  # Logic execution
  emoji_safe "$check_mark_button"; _dh_result=$?
  right_status $_dh_result
  printf "\n"
  emoji_safe "$cross_mark"; _dh_result=$?
  right_status $_dh_result
  printf "\n"

  #-----[ part 3: emoji_safe() inside j_msg -5 ]----------------------------------------
--------
  # construct the command preview string -
```

```
  # line 1 - j_msg -5 "$(emoji_safe "$hammer") Running a work-like task..."
  _dh_cmd="   ${_func_color}j_msg -5${_Boff} "
  _dh_cmd="${_dh_cmd}${_quote_color}\"\$(${_func_color}emoji_safe${_Boff} ${_quote_color}
\"\${hammer}\"${_quote_color}${_Boff})"
  _dh_cmd="${_dh_cmd} Running a work-like task...\"${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff}\n ${_dh_cmd}"

  # Logic execution
  j_msg -5 "$(emoji_safe "$hammer") Running a work-like task..."
  _dh_result=$?
  [ "$_dh_result" -eq 0 ] || _dh_FAILED=$TRUE
  printf "\n"

  #-----[ part 4: demonstrate EMOJI_DISABLE override ]-----------------------------------
----------
  # construct the command preview string -
  # line 1 - EMOJI_DISABLE="$TRUE"; emoji_safe "$check_mark_button"; right_status $?
  _dh_cmd="   ${_var_color}EMOJI_DISABLE${_Boff}${_op_color}=${_Boff}${_quote_color}\"\${
TRUE}\"${_Boff}${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}emoji_safe${_Boff} ${_quote_color}\"\${check_mark_butt
on}\"${_Boff}${_op_color};${_Boff} "
  _dh_cmd="${_dh_cmd}${_func_color}right_status${_Boff} ${_var_color}\$?${_Boff}\n"
  # line 2 - EMOJI_DISABLE="$FALSE"
  _dh_cmd="${_dh_cmd}   ${_var_color}EMOJI_DISABLE${_Boff}${_op_color}=${_Boff}${_quote_c
olor}\"\${FALSE}\"${_Boff}"
  printf "\n"; j_msg -5 "${BYon}About to run:${Boff} ${_cmt_color}# EMOJI_DISABLE overrid
e${_Boff}\n ${_dh_cmd}"

  # Logic execution
  EMOJI_DISABLE="$TRUE"
  emoji_safe "$check_mark_button"; _dh_result=$?
  right_status $_dh_result
  printf "\n"
  EMOJI_DISABLE="$FALSE"

  [ "$_dh_FAILED" ] && demo_emoji_ret=1
  unset -v _dh_cmd _dh_FAILED _dh_result
  return $demo_emoji_ret
}
# @usage _dh_demo_emojis
# @args none
# @rule demonstrates emoji policy, emoji_safe(), and right_status integration


demonstrate_header() # (POSIX) describe, demonstrate, and test capabilities of this heade
r
{ dh_final_status=0   # clear all bits of this status "bitmask" variable

  # begin output with separator
  separator "demonstrate_header-${BUILD}" "(demonstration)"

  # execute sub-modules ( use a bitmask status "byte" to track overall status, indetail )
  # var|(1<<i) means: var = (binary shift a 1 i bits to the left) then bitwise OR that nu
mber with var
  # i.e. SET the ith most significant bit
  _dh_show_capabilities        || dh_final_status=$(( dh_final_status | ( 1 << 0 ) ))
  _dh_demo_verbosity           || dh_final_status=$(( dh_final_status | ( 1 << 1 ) ))
  _dh_demo_versions            || dh_final_status=$(( dh_final_status | ( 1 << 2 ) ))
  _dh_demo_numeric             || dh_final_status=$(( dh_final_status | ( 1 << 3 ) ))
  _dh_demo_debug_utils         || dh_final_status=$(( dh_final_status | ( 1 << 4 ) ))
  _dh_demo_boolean             || dh_final_status=$(( dh_final_status | ( 1 << 5 ) ))
  _dh_demo_terminal_io         || dh_final_status=$(( dh_final_status | ( 1 << 6 ) ))
  _dh_demo_environment         || dh_final_status=$(( dh_final_status | ( 1 << 7 ) ))
  _dh_demo_widgets             || dh_final_status=$(( dh_final_status | ( 1 << 8 ) ))

  #-----[ results summary ]-------------------------------------------------------------
-----------------------
  # preview the command syntax
  # line 1 - j_msg -5n "${BCon}Finishing with status of summarization${Boff}"
  cmd="  ${_func_color}j_msg -5n${_Boff} ${_quote_color}\"\${BCon}Finishing with status o
```

```
f summarization\${Boff}\"${_Boff}\n"
  # line 2 - right_status $dh_final_status
  cmd="${cmd}   ${_func_color}right_status${_Boff} ${_var_color}\$dh_final_status${_Boff}
"
  # demonstrate j_msg -5n and right_status
  j_msg -5 "${BYon}About to run:${Boff}\n ${cmd}"
  j_msg -5n "${BCon}Finishing with status of summarization${Boff}"
  # also capture the final status in the bitmask
  _dh_result=$?; [ "$_dh_result" -eq 0 ] || dh_final_status=$((dh_final_status|(1<<9) ))
  printf " (exit status: %b%s%b) --->" " $(status_color $dh_final_status)" "${dh_final_st
atus}" "$Boff"
  right_status $dh_final_status
  printf "\n"

  #-----[ results summary ]-----------------------------------------------------------
----------------------
  separator "RESULTS" "(diagnostic summary)"

  # list functions in order from LSB (bit 0) to MSB
  _dh_tests="_dh_show_capabilities"               # bit 0
  _dh_tests="${_dh_tests} _dh_demo_verbosity"     # bit 1
  _dh_tests="${_dh_tests} _dh_demo_versions"      # bit 2
  _dh_tests="${_dh_tests} _dh_demo_numeric"       # bit 3
  _dh_tests="${_dh_tests} _dh_demo_debug_utils"   # bit 4
  _dh_tests="${_dh_tests} _dh_demo_boolean"       # bit 5
  _dh_tests="${_dh_tests} _dh_demo_terminal_io"   # bit 6
  _dh_tests="${_dh_tests} _dh_demo_environment"   # bit 7
  _dh_tests="${_dh_tests} _dh_demo_widgets"       # bit 8
  _dh_tests="${_dh_tests} _dh_summarization"      # bit 9
  _dh_longest=$(get_longest $_dh_tests)
  _dh_fw=$((_dh_longest + 1))
  _dh_i=0
  for _dh_test_name in $_dh_tests; do
    # read the ith most significant bit
    # (binary shift binary value right by i bits) (binary bitwise AND with 1)
    # check if the ith bit is set: bit=$(( (mask >> i) & 1 ))
    _dh_bit=$(( (dh_final_status >> _dh_i) & 1 ))
    # output Test i: testname
    printf " Test %02d: [${BMon}%-${_dh_fw}s${Boff}]" "$((_dh_i + 1))" "${_dh_test_name}"
    # render status using the handle_result library function
    # passing status bit, "passed" ok_msg, and "failed" error msg
    printf "(%b%1d%b)" " $(status_color $_dh_bit)" "$_dh_bit" "$Boff"
    handle_result $_dh_bit "passed" "failed"
    _dh_i=$((_dh_i + 1))
  done
  printf "\n"

  # clean up local variables for POSIX hygiene
  unset -v _sh_FUNCTION_REGEX _sh_METADATA_REGEX _sh_dir _sh_header_glob
  unset -v _sh_msg _sh_filespec _sh_PN _sh_fn_list _sh_fn_max_len
  unset -v _sh_fn_sep _sh_fn_col_width _sh_fn_col _sh_fn_indent
  unset -v _sh_label_list _sh_label_max_len _sh_label_sep
  unset -v _sh_label_col_width _sh_label_col _sh_label_indent
  unset -v _sh_line _sh_tmp _sh_fn_name _sh_meta _sh_tag _sh_payload
  unset -v _dh_result cmd _dh_tests _dh_longest
  unset -v  _dh_fw _dh_i _dh_test_name _dh_bit
  return $dh_final_status
}
# @usage demonstrate_header
# @deps _dh_test_render j_msg -5 separator
# @rule iterates through core UI components (colors, cursor, math, messaging)
# @rule provides a "live" visual verification of POSIX terminal compatibility
# @ret exit 0 if all tests pass; 1 if any core logic fails
# @note use this to verify terminal behavior on new or minimal POSIX environments
# @note
# @note POSIX Argument Expansion Reference:
# @note "$@" (Quoted) -> THE GOLD STANDARD. Preserves arguments exactly as passed
# @cont Example: [ "arg one" "arg two" ] stays [ "arg one" "arg two" ]
# @cont Use for: d_do, wrappers, and passing arguments to other functions
# @note
```

```
# @note "$*" (Quoted) -> THE STRING BUILDER. Flattens all arguments into ONE string
# @cont Example: [ "arg one" "arg two" ] becomes [ "arg one arg two" ]
# @cont Use for: j_msg -5, echo, and strip_ansi (treats message as a single blob)
# @note
# @note $@ or $* (Unquoted) -> THE WORD SPLITTER. Dangerous/Rarely used
# @cont Triggers "Internal Field Separation" (IFS). Spaces break into new arguments
# @cont Example: [ "arg one" ] becomes [ "arg" "one" ]
# @cont Use for: for-loops where you explicitly want to iterate over words
# @rule Wrapper Rule: Always use "$@" to pass data to workers to prevent mangling
# @rule Logging Rule: Use "$*" when piping to sed/strip_ansi for stream processing

# load the compatibility header (wrappers to redirect calls to
#    legacy functions to new capability
. ${script_header_installed_dir%/}/script_header_joetoo_compat
```